

Onderzoeksontwerp en Beschrijvende Statistiek

Taalwetenschap

2026-09-01

Inhoudsopgave

Welkom	10
Suggesties voor verbeteringen	10
Verantwoording	10
1 Inleiding	11
1.1 Hoe dit dictaat te gebruiken	11
1.2 Generatieve AI	12
2 R en RStudio	14
2.1 Werken met R en RStudio	14
2.2 Functies	16
2.3 Packages	19
Verder leren	21
3 Data in R	22
3.1 Datatabellen en vectoren	22
3.2 Data inlezen	28
3.3 Voorbeeldscript	41
3.4 Omgaan met errors	42
Verder leren	44
4 Data Bewerken	45
4.1 de <i>pipe</i> -operator	46
4.2 Observaties uitlichten	48
4.3 Variabelen selecteren	54
4.4 Missing values	57
4.5 Simpele tellingen	59
Verder leren	65
5 Grafieken maken	66
5.1 De anatomie van een <code>ggplot</code>	67
5.2 Soorten grafieken	83
Verder leren	96

6	Categorische variabelen	97
	De data in dit hoofdstuk	98
6.1	Eigenaardigheden van barplots	101
6.2	Herordenen van levels	106
6.3	Levels hernoemen of samenvoegen	114
6.4	Meerdere categorische variabelen	117
6.5	Kleuren	128
6.6	Een level highlighten	130
	Verder leren	132
7	Numerieke variabelen	133
	De data in dit hoofdstuk	134
7.1	Distributies zichtbaar maken	135
7.2	Beschrijvende statistieken berekenen	153
7.3	Centrummaten zichtbaar maken	158
7.4	De data zichtbaar maken	160
	Verder leren	162
I	Practica {#sec-practica .unnumbered,}	163
	Hoe haal je het meeste uit een practicum?	164
	Wat heb je nodig?	165
	Aan de slag	166
	Practicum 1	168
	Werken met data in R	169
	Leerdoelen	169
	Vorbereiding	169
	Introductie	169
	Het inlezen van de data	171
	7.5 Opdracht	171
	7.6 Hint	171
	7.7 Code	171
	Het verkennen van de data	172
	Eerste zes rijen zichtbaar maken	172
	7.8 Opdracht	172
	7.9 Hint	172
	7.10 Code	172
	Aantal rijen en kolommen	173

7.11 Opdracht	173
7.12 Hint	173
7.13 Code	173
De functie <code>View()</code>	173
7.14 Opdracht	173
7.15 Code	174
Namen van de variabelen	174
7.16 Opdracht	174
7.17 Hint	174
7.18 Antwoord	174
7.19 Opdracht	174
7.20 Hint	174
7.21 Code	175
Meetniveaus van de variabelen	175
7.22 Opdracht	175
7.23 Hint	175
Details van variabelen opvragen	175
7.24 Opdracht	175
7.25 Hint	176
7.26 Code	176
Bewerken van de data	177
Aanpassen van het type van een variabele	177
7.27 Opdracht	177
7.28 Hint	177
7.29 Code	177
Toevoegen namen van levels	178
7.30 Opdracht	178
7.31 Hint	178
7.32 Code	178
Nieuwe variabele <code>proportie_correct_voormeting</code> toevoegen	179
7.33 Opdracht	179
7.34 Hint	179
7.35 Code	180
Nieuwe variabele <code>verschil_metingen</code> toevoegen . . .	180
7.36 Opdracht	180
7.37 Hint	180
7.38 Code	181
Nieuwe variabele <code>leeftijd</code> toevoegen	181
7.39 Opdracht	181

7.40 Hint	181
7.41 Code	181
Een grafiek maken	181
7.42 Opdracht	182
7.43 Hint	182
7.44 Code	182
Practicum 2	184
Bewerken van data	185
Leerdoelen	185
Vorbereiding	185
Introductie	185
Het inlezen van de data	188
Databestand inlezen	188
7.45 Opdracht	188
7.46 Hint	188
7.47 Code	189
7.48 Opdracht	189
7.49 Hint	189
7.50 Code	190
Aantal observaties en variabelen	190
7.51 Opdracht	190
7.52 Hint	190
7.53 Code	190
Meetniveaus van de variabelen	192
7.54 Opdracht	192
Details van variabelen opvragen	192
7.55 Opdracht	192
7.56 Hint	192
7.57 Code	192
Aanpassen van het type van een variabele	193
7.58 Opdracht	193
7.59 Hint	193
7.60 Code	194
Tibbles bewerken	196
Observaties behouden op basis van waarden	196
7.61 Opdracht	196
7.62 Hint	196

7.63 Code	197
Selecteren van variabelen	200
7.64 Opdracht	200
7.65 Hint	200
7.66 Code	200
Toevoegen van een nieuwe variabele	201
7.67 Opdracht	201
7.68 Hint	201
7.69 Code	202
Variabelen van plaats verwisselen	202
7.70 Opdracht	202
7.71 Hint	202
7.72 Code	203
De pipe	203
7.73 Opdracht	203
7.74 Hint	204
7.75 Code	204
Ordenen van een dataset	204
7.76 Opdracht	204
7.77 Hint	205
7.78 Code	205
Datasets samenvoegen en opslaan	206
Inlezen van databestand <code>hulpwerkwoorden.txt</code>	206
7.79 Opdracht	206
7.80 Hint	206
7.81 Code	207
Samenvoegen van de datatabellen	208
7.82 Opdracht	208
7.83 Hint	208
7.84 Code	209
Opslaan van de nieuwe datatabel	209
7.85 Opdracht	209
7.86 Hint	210
7.87 Code	210
Numerieke samenvattingen maken	211
Tellen van voorkomens	211
7.88 Opdracht	211
7.89 Hint	211
7.90 Code	211

Een telling per groep	212
7.91 Opdracht	212
7.92 Hint	212
7.93 Code	212
Gemiddelde waarde per groep berekenen	214
7.94 Opdracht	214
7.95 Hint	214
7.96 Code	214
7.97 Opdracht	214
7.98 Hint	215
7.99 Code	215
Tellingen en proporties	215
7.100Opdracht	215
7.101Hint	216
7.102Code	216
Sampling	218
Data zichtbaar maken	218
7.103Opdracht	218
7.104Hint	218
7.105Code	218
Een observatie met maximale of minimale waarde zichtbaar maken	219
7.106Opdracht	219
7.107Hint	219
7.108Code	220
Een simple random sample maken	220
7.109Opdracht	220
7.110Hint	220
7.111Code	221
Een stratified sample maken	221
7.112Opdracht A	221
7.113Hint	222
7.114Code	222
7.115Opdracht B	223
7.116Hint	223
7.117Code	223
Practicum 3	224

Grafieken maken met ggplot2	225
Leerdoelen	225
Vorbereiding	225
Introductie	225
Opdracht 1: Internet use and life expectancy	227
Het inlezen van de data	227
7.118Opdracht	228
7.119Hint	228
7.120Code	228
ggplot2-code beter begrijpen	229
7.121Opdracht	229
7.122Hint	230
7.123Code	230
Interpreteren van een waarschuwing	231
7.124Opdracht	231
7.125Hint	231
7.126Code	232
Kleur van datapunten aanpassen	233
7.127Opdracht	233
7.128Hint	233
7.129Code	233
Datapunten doorzichtig maken	234
7.130Opdracht	234
7.131Hint	234
7.132Code	235
De kleur van datapunten variëren op basis van een variabele	236
7.133Opdracht	236
7.134Hint	236
7.135Code	236
Een trendlijn toevoegen	237
7.136Opdracht	237
7.137Hint	237
7.138Code	238
Opdracht 2: UK baby names - Fiona	240
Het zichtbaar maken van de datapunten	240
7.139Opdracht	240
7.140Hint	241
7.141Code	241

Lijnen toevoegen per groep	243
7.142Opdracht	243
7.143Hint	244
7.144Code	244
De kleur en lijnsoort aanpassen per groep	245
7.145Opdracht	245
7.146Hint	245
7.147Code	245
Code annoteren	247
7.148Opdracht	247
7.149Hint	247
7.150Code	247
Meer data zichtbaar maken	248
7.151Opdracht	248
7.152Hint	248
7.153Code	248
Opdracht 3: Nu helemaal zelf: tijden op de 1500m	
hardlopen	250
7.154Opdracht	250
7.155Hint	251
7.156Code	251
Deel 2: My first Quarto	253
Practicum 4	255
Grafieken en tabellen voor categorische data	256
Leerdoelen	256
Vorbereiding	256
Quarto	256
Talen van de wereld	257
Vorbereiding van de data	258
Opdrachten	259

Welkom

Dit is het dictaat bij periode 1 van de practica van de B1-cursus *Onderzoeksontworp en Beschrijvende Statistiek*. Dit document geeft een beknopte inleiding in het werken met **R** en **RStudio** aan de hand van taalkundige voorbeelden. Aan het eind van elk hoofdstuk vind je onder het kopje *Verder leren* informatie waar je je kennis verder kunt verdiepen. De practica bij de cursus vind je onder het kopje *Practica* in de menubalk.

Suggesties voor verbeteringen

Dit document is in ontwikkeling. Kom je dingen tegen die niet werken, niet kloppen of die onduidelijk zijn of heb je andere suggesties voor verbeteringen, laat het weten. Dit kan via het forum op Brightspace of direct bij de docent tijdens de practicumuren. Op deze manier zorgen we samen dat dit document een steeds beter leermiddel wordt.

Verantwoording

Het materiaal in dit dictaat is beïnvloed door een groot aantal inspiratiebronnen die ik door de afgelopen jaren heb bestudeerd. Het is niet altijd eenduidig aan te wijzen waar uitleg of voorbeelden op gebaseerd zijn, daarom hieronder een alfabetische lijst van de meest inspirerende bronnen.

1 Inleiding

De taalwetenschap heeft zich de afgelopen decennia ontwikkeld tot een sterk empirisch wetenschapsgebied waarin theorievorming meer en meer wordt gebaseerd op inzichten verkregen uit corpora (grote verzamelingen van taaldata) en experimenten. Om zinnige informatie uit die empirische data te kunnen halen heb je kennis nodig van onderzoeksontwerp, data-analyse en statistische analyse. Aan de ene kant als *consument* van wetenschappelijke literatuur waarin dergelijke technieken worden toegepast, om deze literatuur goed te kunnen begrijpen en te kunnen evalueren. Aan de andere kant als *producent* van nieuwe wetenschappelijke kennis wanneer je zelf taalwetenschappelijk onderzoek gaat doen, bijvoorbeeld tijdens je scriptie. De cursus *Onderzoeksontwerp en Beschrijvende Statistiek* (OBS) biedt je een eerste inleiding in de noodzakelijke onderzoeksmethoden die nodig zijn om je rol als consument en producent zo goed mogelijk te kunnen vervullen.

Dit dictaat biedt een inleiding in het werken met de programmeertaal R. R is het standaard analyse-instrument in de taalwetenschap. Het wordt gebruikt voor het organiseren van je data, het maken van grafieken en het doen van statistische analyses. Binnen de opleiding Taalwetenschap werk je in verschillende cursussen met R. We gebruiken hierbij de variant *tidyverse*, een dialect binnen R.

1.1 Hoe dit dictaat te gebruiken

Mogelijk is het werken met een programmeertaal nieuw voor je. Dat is helemaal geen probleem. We beginnen vanaf het begin en veronderstellen geen voorkennis op dit gebied. Voor sommigen zal het leren programmeren (of beter gezegd *coderen*) soepel

verlopen, maar misschien ben je wel iemand die er meer moeite mee heeft. Daar hoeft je je geen zorgen over te maken. De leercurve kan best steil zijn. Vergelijk het met het leren van een nieuwe taal. Dat kost nu eenmaal wat tijd en kan lastig zijn. Het is belangrijk dat je hierbij actief met de stof aan de gang gaat. Maak voor jezelf een overzicht van het leermateriaal. Laat je niet te snel uit het veld slaan en stel vragen.

Als je fit wilt worden, dan moet je zelf aan de slag in de sportschool of op het sportveld. Voor coderen geldt hetzelfde. Als je wilt leren coderen, moet je zelf aan de slag. Het alleen doorlezen van dit dictaat is hierbij niet voldoende. Probeer voorbeeldcode te ontcijferen, zodat je snapt wat elk onderdeel doet. Het helpt daarbij om de code zelf uit te voeren in R. Op die manier kun je precies zien wat er gebeurt in elke stap. Alle bestanden die in het dictaat gebruikt worden, kun je downloaden vanaf de Brightspace-pagina van deze cursus. Je kunt alle analyses dus op je eigen computer nadoen. Download hiervoor het bestand `OBS_practica.zip` van Brightspace en pak dat uit op je eigen computer. Zet deze map op een handige locatie op je computer (bijv. in de map voor de cursus OBS). In het volgende hoofdstuk leer je hoe je aan de slag kunt met de bestanden in deze map.

Dit dictaat biedt slechts een inleiding. Er is veel meer te leren en vertellen over R. Aan het eind van elk hoofdstuk vind je onder het kopje *Verder leren* informatie waar je meer achtergrondkennis kunt opdoen of je kennis verder kunt verdiepen. Maak daar gebruik van. De leerdoelen aan het begin van ieder hoofdstuk kun je gebruiken om voor jezelf vast te stellen of je de stof uit het hoofdstuk voldoende onder de knie hebt.

1.2 Generatieve AI

Generatieve AI biedt allerlei mogelijkheden. Het genereren van code is er daar een van. Het probleem is alleen dat je de gegenereerde code alleen kunt begrijpen als je zelf al iets weet over coderen. Dit noemen we ook wel de *expert-paradox*: je hebt (basis)kennis nodig om generatieve AI goed te kunnen

gebruiken. En dan nog: ervaren coders geven aan dat wanneer je nieuwe problemen wilt analyseren generatieve AI je niet zoveel te bieden heeft. Des te meer reden dus om zelf kennis op te doen. Bovendien is het natuurlijk ook veel leuker en interessanter als je aan het eind van je opleiding naar eerlijkheid kunt zeggen dat je daadwerkelijk iets geleerd hebt en daadwerkelijk zelf iets kunt. We maken in deze cursus dus geen gebruik van generatieve AI. Misschien dat je later met de opgedane kennis ervan kunt profiteren, maar op dit moment doen we alles op eigen kracht.

2 R en RStudio

In deze cursus leer je om data-analyses uit te voeren met R. R is een programmeertaal om data-analyse (datavisualisaties, statistische analyses) mee uit te voeren. In feite is het een grote verzameling functies die je kunt inzetten om data te analyseren. Net als elke taal kent ook R bepaalde regels (de grammatica van de taal). Leren werken met R betekent dus dat je je die regels eigen moet maken.

Na afloop van dit hoofdstuk:

- ben je bekend met de opbouw van het programma RStudio;
- kun je bestaande functies in R toepassen;
- kun je packages installeren en aanroepen zodat je de functies in de packages kunt gebruiken.

2.1 Werken met R en RStudio

Het softwarepakket R is tegenwoordig de standaard voor data-analyse in taalkundig onderzoek. Het grote voordeel van R is dat veel professionele onderzoekers steeds weer nieuwe pakketten maken voor specifieke statistische of data-analytische functies.¹ Daarnaast is het gratis te downloaden voor verschillende computerplatforms. Je kunt R downloaden als programma op je computer. Je kunt direct in het programma R werken. Als je het opent, krijg je een scherm te zien, de *console*, waarin je code kunt typen. Wij gebruiken echter het programma RStudio, dat een wat fijnere gebruikersinterface levert dan R zelf, en dus wat vriendelijker is voor (beginnende) gebruikers.

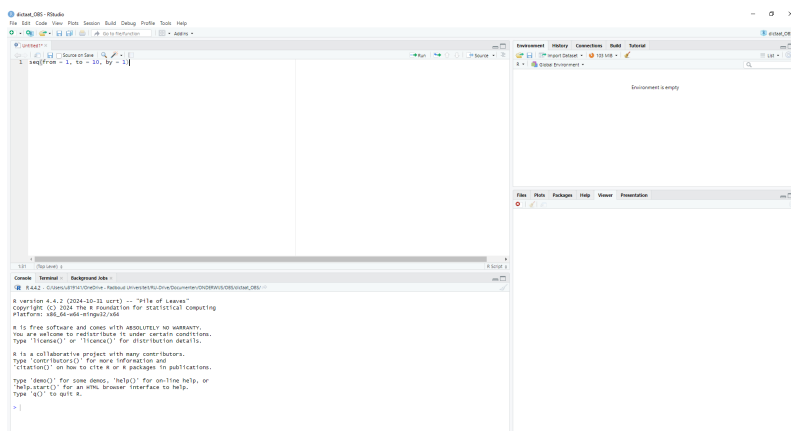
Op de computers in het ESC horen R en RStudio al aanwezig te zijn. Om het op je eigen computer te installeren: ga naar

¹ Bodo Winter geeft in sectie 0.2 van Winter (2019) nog een aantal belangrijke voordelen.

Winter, B. (2019). *Statistics for Linguists: An Introduction Using R*. Routledge. <https://doi.org/10.4324/9781315165547>

<https://docs.posit.co/ide/user/#rstudio-ide-oss-downloads> en kies het installatieprogramma dat bij jouw computer past. Als je R nog niet hebt, installeer dat dan ook, vanaf die pagina of anders via <https://cran.rstudio.com> of <http://cran.r-project.org>.

Wanneer je RStudio opent, opent zich een programma met een aantal vensters (3 of 4), zoals je kunt zien in Figuur 2.1. Linksonder vind je de console van R (wanneer je vanuit het programma R zou werken, krijg je alleen dit venster te zien). Dat is waar alle code die je gaat maken zal worden uitgevoerd. Linksboven zie je een script met daarin 1 regel code. In het script kun je als in een kladblok code schrijven en die vervolgens uitproberen door deze naar de console te sturen. Dit doe je door de relevante code te selecteren en op **ctrl + Enter** (Cmd + Enter op een Mac) te drukken. Wanneer je je cursor ergens op een regel hebt staan en je gebruikt **ctrl + Enter** (Cmd + Enter op een Mac) dan wordt alle code op die regel (en bijbehorende regels) uitgevoerd.



Figuur 2.1: De vensters van het programma *RStudio*

Het venster rechtsboven geeft een overzicht van alle data (datatables en vectoren, zie het volgende hoofdstuk) die voor R beschikbaar zijn om bewerkingen op uit te voeren. Dit venster is nu nog leeg. Hetzelfde geldt voor het venster rechtsonder. Dit venster kun je gebruiken om gemaakte grafieken (*plots*) te bekijken, inzicht te krijgen in je bestanden en ook de packages die je hebt geïnstalleerd (zie hieronder). Niet onbelangrijk: hier vind je ook de help-functie.

Omdat het waarschijnlijker makkelijker is om een overzicht te krijgen van RStudio met bewegend beeld kun je deze video <https://youtu.be/FIrsOBy5k58> bekijken voor een korte rondleiding.

2.2 Functies

Zoals gezegd is R in feite een grote verzameling functies die je kunt inzetten om data te analyseren. **Functies** koppelen een bepaalde *input* aan een bepaalde *output*. Neem bijvoorbeeld de functie `seq()`: gegeven een beginpunt, een eindpunt en een stapgrootte geeft deze functie je een reeks getallen. In onderstaand voorbeeld is het beginpunt gegeven als 1, het eindpunt als 10 en de stapgrootte als 1. Het resultaat is een reeks gehele getallen van 1 tot 10.

```
seq(from = 1, to = 10, by = 1)
```

```
[1] 1 2 3 4 5 6 7 8 9 10
```

Wanneer we de stapgrootte veranderen naar 2, krijgen we niet 10 getallen maar 5 (alle oneven getallen onder de 10).

```
seq(from = 1, to = 10, by = 2)
```

```
[1] 1 3 5 7 9
```

Veranderen we vervolgens ook het beginpunt naar 2 dan krijgen we 5 even getallen:

```
seq(from = 2, to = 10, by = 2)
```

```
[1] 2 4 6 8 10
```

Deze voorbeelden laten zien dat de uitkomst (*output*) van de functie dus volledig wordt bepaald door wat je erin stopt (de *input*). De functie `seq()` vraagt om drie argumenten `from`, `to` en `by`. Ieder argument heeft in principe een vaste plek in de functie. Wanneer we die vaste volgorde aanhouden kunnen we de namen van de argumenten weglaten:

```
seq(1, 10, 1)
```

```
[1] 1 2 3 4 5 6 7 8 9 10
```

Wanneer je bekend raakt met bepaalde functies, zul je er op gegeven moment waarschijnlijk voor kiezen om de namen van de argumenten weg te laten. Dit scheelt namelijk veel typewerk. Het maakt je code mogelijk wel minder leesbaar voor buitenstaanders (en voor jezelf op een later moment). In dit dictaat zullen we in de meeste gevallen bij het eerste gebruik van een functie de argumenten bij naam noemen, om ze bij vervolgd gebruik weg te laten.

Wanneer we de namen wel gebruiken, kunnen we er ook voor kiezen om de argumenten in een andere volgorde neer te zetten. Door de namen weet R wat we bedoelen.

```
seq(by = 1, from = 1, to = 10)
```

```
[1] 1 2 3 4 5 6 7 8 9 10
```

Laten we de namen weg en gebruiken we een andere volgorde, krijgen we een andere output:

```
seq(1, 1, 10)
```

```
[1] 1
```

In dit geval interpreteert R de argumenten in de standaardvolgorde (`from`, `to`, `by`).

Het is voor elke functie dus belangrijk om te weten wat de benodigde argumenten zijn en in welke volgorde ze worden verwacht. De argumenten staan altijd tussen () achter de functienaam en worden gescheiden door een komma. Tussen de naam van het argument en de waarde die je wilt meegeven zet je een =-teken.

Tabel 2.1: De anatomie van een functie

functienaam(argument1, argument2, argument3)
seq(from = , to = , by =)

Wanneer je wilt achterhalen hoe de structuur van een functie in elkaar zit, kun je de help-functie in R gebruiken. Door een vraagteken voor de functienaam te zetten, verschijnt in RStudio rechtsonder informatie over de functie.

```
?seq()
```

Je zult zien dat veel functies naast de verplichte argumenten ook *default* argumenten hebben. Dit zijn argumenten die een standaardwaarde hebben en dus niet door de gebruiker hoeven te worden gegeven, maar die je eventueel wel zou kunnen aanpassen.

Niet alle functies hebben argumenten met een naam. De functie `c()` waarmee we dingen kunnen samenvoegen tot een geheel is bijvoorbeeld een veelgebruikte functie zonder argumentnamen:

```
c("Nederland", 1988, "plek 1")
```

```
[1] "Nederland" "1988"      "plek 1"
```

Deze functie `c()` kun je ook gebruiken om meerdere waarden als argument van een functie mee te geven. Neem als voorbeeld de functie `round()`. Deze functie rondt getallen af op een aangegeven aantal decimalen (*digits*). In onderstaand voorbeeld wordt er afgerond op twee decimalen:

```
round(4.246, digits = 2)
```

```
[1] 4.25
```

Met `c()` is het mogelijk om meerdere getallen tegelijk te laten afronden (in dit geval op 1 decimaal):

```
round(c(4.246, 5.628, 9.463), 1)
```

```
[1] 4.2 5.6 9.5
```

codeboek

In deze cursus leer je werken met zo'n 100 veelgebruikte functies in R. Het is belangrijk om voor jezelf een goed overzicht te houden van deze functies. Werk hier actief aan door een eigen codeboek aan te maken waarin je alle functies met een korte beschrijving van de werking opneemt.

2.3 Packages

R komt standaard met een hele verzameling aan functies die je kunt gebruiken. Dit noemen we ook wel **base R**. Soms wil je echter meer. In dat geval kun je gebruik maken van zogenaamde aanvullende **packages**. Een package is een verzameling functies die meestal gemaakt zijn om één bepaalde taak goed uit te kunnen voeren, bijv. het maken van grafieken of het analyseren van teksten. Om deze te kunnen gebruiken moet je deze packages eerst installeren op je computer en vervolgens ook aanroepen in je R-sessie. Tijdens deze cursus zullen we veelvuldig gebruik maken van het package **tidyverse**. Dat is eigenlijk een verzameling packages die helpen bij verschillende onderdelen van data-analyse. Het **tidyverse** is inmiddels zo succesvol geworden dat het een soort dialect binnen de programmeertaal R is geworden.

Wanneer je het package `tidyverse` installeert en aanroept dan roep je meteen alle packages die daaronder vallen ook aan.

Packages installeer je met behulp van de functie `install.packages()`. Tussen de haken zet je de naam van het package tussen aanhalingstekens.

```
install.packages("tidyverse")
```

N.B. je hoeft een package maar één keer op je computer te installeren. Zet daarom, nadat je de installatie hebt voltooid, een `#` voor de regel met `install.packages()` of verwijder de regel met deze code. Als je de regel laat staan kan dat later problemen opleveren. Door de `#` zal R de regel negeren wanneer je het script nog een keer uitvoert. In RStudio kun je in het window rechtsonder onder het tabblad *packages* zien welke packages er geïnstalleerd zijn op jouw computer. Je vindt daar ook een optie om packages handmatig te installeren.

Een package installeer je dus maar één keer, maar om de functies in het package te kunnen gebruiken moet je het package wel elke R-sessie opnieuw activeren. Dit doe je met behulp van de functie `library()` waarin je de naam van het te gebruiken package invult zonder aanhalingstekens. Dit moet je dus elke keer uitvoeren aan het begin van een nieuwe R-sessie, anders zal R klagen de functies uit het package niet te kennen. Om de functies uit het `tidyverse` te kunnen gebruiken, type je dus:

```
library(tidyverse)
```

Je kunt deze functie voor meerdere packages achter elkaar uitvoeren. Let hierbij wel op dat de volgorde waarin je de packages aanroept een effect kan hebben. Wanneer er in verschillende packages een functie met dezelfde naam zit, dan zal de R de functie gebruiken zoals die in het laatst aangeroepen package gedefinieerd is. De functie in het eerder aangeroepen package zal dan *gemaskeerd* worden. R geeft je hier ook een waarschuwing over. Wanneer je uit een geïnstalleerd package maar één specifieke functie wilt gebruiken, dan kun je dit probleem omzeilen door het package niet aan te roepen met

`library()`, maar door de notatie `packagenaam::functienaam` te gebruiken:

```
janitor::clean_names()
```

In het bovenstaande voorbeeld kun je de functie `clean_names` uit het package `janitor` gebruiken zonder dat alle andere functies uit het package actief worden en mogelijk functies uit andere packages maskeren. Je moet het package `janitor` hiervoor natuurlijk wel geïnstalleerd hebben op je computer.

Het is verstandig om aan het begin van elk R-script een blokje te maken waarin je alle benodigde packages aanroept met `library()`. Dit is vooral ook handig voor lezers van je script. Die weten dan meteen wat ze kunnen verwachten en welke packages ze eventueel nog moeten installeren.

Verder leren

- In Hoofdstuk 1 van SfL (Winter, 2019) wordt de basis van R (base R) verder uitgelegd. Ter voorbereiding op het eerste practicum is het prettig om vast wat commando's te proberen uit dit hoofdstuk. Maak hiervoor een nieuw script aan in RStudio (via het menu **File > New File > R Script** of de sneltoetsen **Ctrl + Shift + N** (**Cmd + Shift + N** op een Mac)).² Werk om een gevoel te krijgen bij hoe R werkt, nu hoofdstuk 1 door t/m sectie 1.4. Typ de commando's in de grijze vakken over en voer ze uit. In sectie 1.2 doe je dit direct in de R-console, vanaf sectie 1.3 (en in alle volgende practica!) in een script. Zie sectie 1.15 voor handige shortcuts die je hierbij kunt gebruiken. Controleer steeds of je dezelfde output krijgt als in het boek. Toegegeven, het letterlijk overtypen van de commando's uit het boek voelt misschien een beetje raar, maar het is dé manier om gevoel te krijgen bij hoe R werkt en waar je op moet letten (zie ook sectie 1.16 voor belangrijke adviezen op dit gebied). In de practica krijg je nog tijd genoeg om het allemaal zelf uit te vinden.

Winter, B. (2019). *Statistics for Linguists: An Introduction Using R*. Routledge. <https://doi.org/10.4324/9781315165547>

² Je kunt eventueel ook <https://rdr.io/snippets> gebruiken voor het uitvoeren van R-code, mocht de installatie op je eigen computer niet werken. We kunnen dan tijdens een practicum proberen R en RStudio werkend te krijgen op je computer.

3 Data in R

Voor data-analyse hebben we data nodig. Datasets bevinden zich soms in packages, maar vaker moet je ze inlezen vanuit een extern databestand. In dit hoofdstuk bekijken welke soorten dataformaten we in R tegenkomen en hoe we data in R kunnen inlezen.

Na afloop van dit hoofdstuk:

- kun je werken met twee prominente dataformaten in R: datatabellen (`tibble`) en vectoren;
- kun je data uit externe databestanden inlezen als een datatabel in R;
- kun je de naam van een variabele aanpassen;
- kun je het datatype van een variabele vaststellen en aanpassen;
- kun je code en een codescript goed vormgeven;
- kun je op een relaxte manier omgaan met errors.

codeboek

In deze cursus leer je werken met zo'n 100 veelgebruikte functies in R. Het is belangrijk om voor jezelf een goed overzicht te houden van deze functies. Werk hier actief aan door een eigen codeboek aan te maken. Voeg tijdens of na afloop van het bestuderen van dit hoofdstuk de codes toe aan jouw codeboek.

3.1 Datatabellen en vectoren

We kunnen data in R in verschillende formaten tegenkomen. Tijdens deze cursus krijgen we vooral te maken met *datatabellen*

en *vectoren*. Een **datatabel** wordt in **tidyverse** een **tibble** genoemd en gaat in R ook wel door het leven onder de naam **data.frame**.³ Een datatabel bestaat uit rijen (de observaties) en kolommen (de variabelen) en doet daarmee erg denken aan een gewone tabel of spreadsheet zoals je dat misschien wel kent uit bijvoorbeeld Excel.

Hieronder vind je een voorbeeld van een tibble met data uit het package **wordbankr**. Dit package geeft toegang tot *Wordbank* (Frank et al., 2017) een grote database met daarin informatie over hoe de woordenschat van kinderen zich ontwikkelt in verschillende talen.⁴

De data in deze database zijn verzameld aan de hand van de *MacArthur-Bates Communicative Development Inventory* (MB-CDI), een vragenlijst waarin ouders aangeven welke woorden hun kind op welke leeftijd kent. Informatie over alle beschikbare versies van deze vragenlijst in verschillende talen hebben we opgeslagen in de **tibble** met de naam **wordbank_datasets**.

```
wordbank_datasets
```

```
# A tibble: 89 x 8
  instrument_id language          form form_type age_min age_max has_grammar
      <dbl> <chr>                <chr> <chr>      <dbl>  <dbl>      <dbl>
1             1 British Sign Langu- WG    WG          8     36          0
2             2 Cantonese          WS    WS         16     30          0
3             3 Croatian           WG    WG          8     16          0
4             4 Croatian           WS    WS         16     30          0
5             5 Danish             WG    WG          8     20          0
6             6 Danish             WS    WS         16     36          1
7             7 English (American) WG    WG          8     18          0
8             8 English (American) WS    WS         16     30          1
9             9 French (Quebecois) WG    WG          8     16          0
10            10 French (Quebecois) WS    WS         16     30          1
# i 79 more rows
# i 1 more variable: unilemma_coverage <dbl>
```

Je kunt de inhoud van een datatabel bekijken door de naam van de tabel uit te laten voeren in de console (zie Hoofdstuk 2). Bij

³ Het **tidyverse** is voor een groot deel ontwikkeld door de Nieuw-Zeelandse Hadley Wickham. Het woord **tibble** schijnt te verwijzen naar de manier waarop in Nieuw-Zeeland **table** wordt uitgesproken.

Frank, M. C., Braginsky, M., Yurovsky, D., & Marchman, V. A. (2017). Wordbank: An open repository for developmental vocabulary data. *Journal of Child Language*, 44, 677–694. <https://doi.org/10.1017/S0305000916000209>

⁴ Voor een uitgebreide beschrijving van deze data, zie de website van het *Wordbank Project*: <https://wordbank.stanford.edu/>

een `tibble` krijg je standaard de eerste tien rijen te zien. Het aantal kolommen (variabelen) dat je te zien krijgt is afhankelijk van de grootte van je scherm. Bovenaan de `tibble` kun je zien om hoeveel rijen en kolommen het gaat (hier 89 x 8). Onderaan wordt je verteld hoeveel rijen en welke variabelen er niet getoond zijn (hier bijvoorbeeld 79 rijen). Je kunt het aantal rijen en variabelen ook zichtbaar maken met de functies `nrow()` en `ncol()` respectievelijk.

```
nrow(wordbank_datasets)
```

```
[1] 89
```

```
ncol(wordbank_datasets)
```

```
[1] 8
```

Wil je de gehele datatabel zien (afgeraden bij heel grote bestanden) dan kun je de functie `View()` gebruiken. Er zal dan een tabblad geopend worden in RStudio waarin je de datatabel kunt bekijken.

```
View(wordbank_datasets)
```

De weergave van de `tibble` in R vertelt je ook iets over de manier waarop R de variabelen uit de `tibble` ziet. Onder de namen van de variabelen (de bovenste regel) vind je namelijk tussen vishaken `<>` het type dat R heeft toegekend op basis van de datapunten in de variabele. Zo heeft de variabele `language` het type `chr` voor *character* (tekst) toegewezen gekregen en de variabele `age_min` het type `int` voor *integer* (geheel getal). Tabel 3.1 geeft een overzicht van een aantal veel voorkomende datatypes van variabelen in R.

Tabel 3.1: een aantal veel voorkomende datatypes in R

soort	naam in R	afkorting in R	voorbeeld	meetniveau
tekst, string	character	char, chr	“a”, “woord”	nominaal
boolean, waarheidswaarde	logical	logi, lgl	TRUE, FALSE	nominaal
factor	factor	fct	Duits, Engels	nominaal
geordende factor	ordered factor	ord	laag, hoog	ordinaal
getal	numeric, double	num, dbl	1, 2.5, 3.14	numeriek
geheel getal	integer	int	1, 2, 100	numeriek

Naast de datatabel zijn **vectoren** een veel voorkomende dataformaat in R. Een vector is een geordend rijtje van elementen van hetzelfde type. Een variabele uit een datatabel kunnen we bijvoorbeeld omzetten naar een vector met de functie `pull()`.

```
languages <- pull(wordbank_datasets, language)
```

Bovenstaande code wijst door het gebruik van de operator `<-` de inhoud van de variabele `language` uit `wordbank_datasets` toe aan een vector met de naam `languages`. Je kunt voor de vector zelf vrijwel elke naam kiezen. Het is verstandig om namen te kiezen die betekenisvol zijn voor jezelf (ook nog als je je code een tijd later weer leest) of voor iemand anders die je code bekijkt. Namen als `dataset1`, `datasetdef` en `datasetechtdefinitief` zijn dus niet zo handig. Daarnaast is het voor de leesbaarheid fijn om consistent te zijn: gebruik alleen kleine letters (*snake case*) en scheid inhoudswoorden met een underscore `_`, dus bijv. `wordbank_datasets`.

Het resultaat van het uitvoeren van `pull()` is een vector met de naam `languages` met daarin de 89 namen van talen. Deze vector zal in RStudio in het venster rechtsboven onder het tabblad *Environment* verschijnen onder het kopje *Values*. In de *Environment* vind je een overzicht van alle datatabellen en

vectoren die voor R toegankelijk zijn. Je kunt de namen van deze databronnen gebruiken in de code die je schrijft. Zo kunnen we de naam `languages` nu gebruiken als argument van functies. Laten we eerst kijken wat er in de vector zit. Hiervoor typen we alleen de naam van de vector en voeren de code uit:

```
languages
```

```
[1] "British Sign Language" "Cantonese"           "Croatian"
[4] "Croatian"             "Danish"               "Danish"
[7] "English (American)"   "English (American)"   "French (Quebecois)"
[10] "French (Quebecois)"    "German"               "Hebrew"
[13] "Hebrew"               "Italian"              "Italian"
[16] "Mandarin (Beijing)"   "Mandarin (Beijing)"   "Mandarin (Beijing)"
[19] "Norwegian"            "Norwegian"            "Russian"
[22] "Russian"              "Slovak"               "Slovak"
[25] "Spanish (Mexican)"    "Spanish (Mexican)"     "Swedish"
[28] "Swedish"              "Turkish"              "Turkish"
[31] "English (British)"    "English (British)"     "American Sign Language"
[34] "American Sign Language" "American Sign Language" "American Sign Language"
[37] "Greek (Cypriot)"      "Kigiriama"            "Kigiriama"
[40] "Kiswahili"            "Kiswahili"            "Czech"
[43] "English (Australian)" "English (British)"     "Latvian"
[46] "Latvian"              "Korean"               "Korean"
[49] "French (French)"      "French (French)"       "Spanish (European)"
[52] "Spanish (European)"   "Portuguese (European)" "Portuguese (European)"
[55] "Mandarin (Taiwanese)" "Mandarin (Taiwanese)"   "English (Irish)"
[58] "Irish"                "Finnish"              "Dutch"
[61] "Dutch"                "Dutch"                "Dutch"
[64] "Dutch"                "Dutch"                "Dutch"
[67] "Hungarian"            "Spanish (Argentinian)" "American Sign Language"
[70] "Spanish (Chilean)"     "Spanish (Peruvian)"    "Spanish (Peruvian)"
[73] "Persian"              "Persian"              "Finnish"
[76] "English (American)"   "English (American)"    "English (British)"
[79] "Arabic (Saudi)"       "Arabic (Saudi)"        "Arabic (Saudi)"
[82] "Estonian"             "Catalan"              "Korean"
[85] "Finnish"              "Finnish"              "Catalan"
[88] "Japanese"             "Japanese"
```

Het gaat hier om een opsomming van alle talen die voorkomen

in de dataset `wordbank_datasets`. Het aantal elementen in de vector kunnen we vaststellen met de functie `length()` en het datatype van de vector met de functie `class()`

```
length(languages)
```

```
[1] 89
```

```
class(languages)
```

```
[1] "character"
```

Het gaat hier dus om een character vector die bestaat uit 89 elementen. Het type (de *class*) van de vector wordt toegepast op alle elementen in de vector ook als er oorspronkelijk elementen in zijn gestopt van een verschillend type (bijv. woorden en getallen). Dit zien we bijvoorbeeld als we aan R vragen wat het type is van onze eerder in Hoofdstuk 2 met de functie `c()` gemaakte vector.

```
ek_1988 <- c("Nederland", 1988, "plek 1")  
class(ek_1988)
```

```
[1] "character"
```

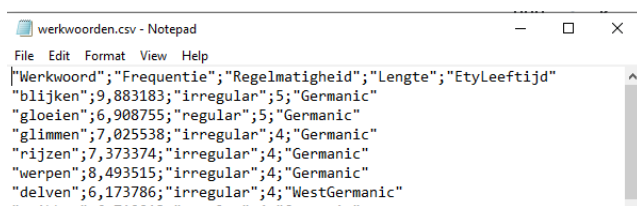
Ondanks dat we een getal hebben toegevoegd aan onze vector is het type van de vector als geheel toch *character*. Wanneer we de vector laten zien, dan valt ook op dat er rond het oorspronkelijk getal 1988 nu aanhalingstekens staan. Het wordt door R dus gezien als een woord.

```
ek_1988
```

```
[1] "Nederland" "1988"      "plek 1"
```

3.2 Data inlezen

Data om te analyseren zijn vaak opgeslagen in databestanden. Deze bestanden kunnen een verschillend formaat (extensie) hebben. Veel voorkomende formaten (in deze cursus) zijn tekstbestanden (`.txt`) en csv-bestanden (`.csv`). CSV staat voor ‘comma separated values’ en geeft aan dat de waarden van variabelen (‘de cellen’ in een spreadsheet) van elkaar gescheiden zijn door scheidingstekens. Zoals de naam doet vermoeden is de komma hier het te verwachten scheidingsteken, maar verwarrend genoeg kan dit ook een puntkomma zijn (zie Figuur 3.1). Welk scheidingsteken wordt gebruikt hangt onder anderen af van de instellingen van je computer. Het is dus van belang om goed te kijken met wat voor een bestand je te maken hebt voor je het gaat inlezen en ook na het inlezen goed te controleren of dit wel goed is gegaan.



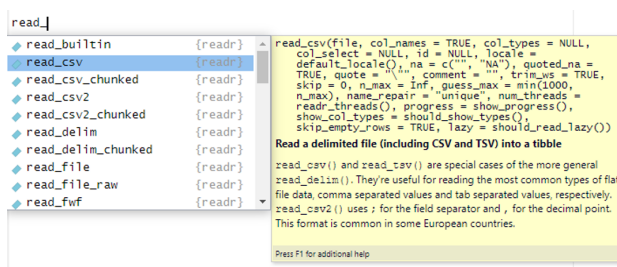
```
werkwoorden.csv - Notepad
File Edit Format View Help
["Werkwoord";"Frequentie";"Regelmatigheid";"Lengte";"EtyLeeftijd"
"blijken";9,883183;"irregular";5;"Germanic"
"gloeien";6,908755;"regular";5;"Germanic"
"glimmen";7,025538;"irregular";4;"Germanic"
"rijzen";7,373374;"irregular";4;"Germanic"
"werpen";8,493515;"irregular";4;"Germanic"
"delven";6,173786;"irregular";4;"WestGermanic"
".....";6,746013;".....";4;"....."]
```

Figuur 3.1: CSV-bestand met puntkomma als scheidingsteken

Externe databestanden zul je eerst moeten inlezen in R voor je ermee kunt gaan werken. Hiervoor bestaan gelukkig verschillende functies in R, gespecificeerd voor het soort bestand waarmee je werkt. Die functies beginnen doorgaans met `read`. Binnen het `tidyverse` gebruiken we `read`-functies gevolgd door een underscore (`read_`). Deze functies lezen een specifiek soort databestand in als een `tibble`. Zo is er voor csv-bestanden een functie `read_csv` voor bestanden met een komma als scheidingsteken en een `read_csv2` voor bestanden met een puntkomma als scheidingsteken. De functie `read_delim()` (voor *read in delimited data file*) is de algemene functie om databestanden in te lezen, waaronder tekstbestanden. De functies `read_csv()` en `read_csv2()` zijn specifieke versies van `read_delim()` waarbij het type *delimiter* (het scheidingsteken) is vastgelegd (dus `read_csv()` is onderliggend `read_delim(file = , delim = ",")`). Type maar eens ‘`read_`’ in je R-script en RStudio zal automatisch alle opties geven zowel binnen `baseR` als binnen het `tidyverse`

(zie Figuur 3.2). Aan jou de taak om de juiste functie te vinden bij je databestand. Wanneer je werkt in een R-project zal de `read_()`-functie naar het databestand gaan zoeken in de map van het R-project (zie box *Waar staat je databestand?* verderop).

Soms is het handig om een in R-gecreëerde dataset op te slaan naar een databestand, zodat je het later of in een ander programma kunt gebruiken. Het opslaan van databestanden in R is eenvoudig. Er bestaat voor elke `read_`-functie namelijk een tegenhanger die begint met `write_`, bijv. `write_csv`. Wanneer je werkt in een R-project, hoeft je de locatie wederom niet te specificeren. RStudio slaat het bestand automatisch op in de map van je project.



Figuur 3.2: automatische aanvullingen in RStudio

Het is belangrijk om altijd goed te controleren of het inlezen van je databestand is gelukt. Heb je inderdaad het aantal observaties en kolommen (variabelen) dat je verwacht. Zo niet, dan is er iets misgegaan. In onderstaand voorbeeld lezen we bijvoorbeeld het databestand `wordbank_datasets.csv` in met de functie `read_csv2()`, terwijl de datapunten niet gescheiden zijn door een puntkomma, maar door een komma (en dus `read_csv()` nodig is). Het resultaat laat zien dat we in plaats van 8 variabelen (en dus 8 kolommen), maar 1 variabele (1 kolom) hebben waarin alle informatie is samengepropt.

```
wordbank_datasets <- read_csv2(file = "_datasets/wordbank_datasets.csv")
```

i Using "','" as decimal and "'.'" as grouping mark. Use ``read_delim()`` for more control.

```
Rows: 89 Columns: 1
```

```
-- Column specification -----
```

```

Delimiter: ";"
chr (1): instrument_id,language,form,form_type,age_min,age_max,has_grammar,u...

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.

```

```
wordbank_datasets
```

```

# A tibble: 89 x 1
  instrument_id,language,form,form_type,age_min,age_max,has_grammar,unilemma_~1
  <chr>
1 1,British Sign Language,WG,WG,8,36,0,0.95
2 2,Cantonese,WS,WS,16,30,0,0.95
3 3,Croatian,WG,WG,8,16,0,1
4 4,Croatian,WS,WS,16,30,0,1
5 5,Danish,WG,WG,8,20,0,1
6 6,Danish,WS,WS,16,36,1,0.93
7 7,English (American),WG,WG,8,18,0,1
8 8,English (American),WS,WS,16,30,1,1
9 9,French (Quebecois),WG,WG,8,16,0,1
10 10,French (Quebecois),WS,WS,16,30,1,0.99
# i 79 more rows
# i abbreviated name:
# 1: `instrument_id,language,form,form_type,age_min,age_max,has_grammar,unilemma_coverage`

```

Dit moeten we herstellen voordat we verdergaan met de data-analyse. Bezuinig niet op de controle van deze stap. Je loopt anders geheid vast.

```
wordbank_datasets <- read_csv(file = "_datasets/wordbank_datasets.csv")
```

```

Rows: 89 Columns: 8
-- Column specification -----
Delimiter: ","
chr (3): language, form, form_type
dbl (5): instrument_id, age_min, age_max, has_grammar, unilemma_coverage

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.

```

wordbank_datasets

```
# A tibble: 89 x 8
  instrument_id language      form form_type age_min age_max has_grammar
      <dbl> <chr>      <chr> <chr>      <dbl>   <dbl>     <dbl>
1             1 British Sign Langu~ WG     WG           8     36         0
2             2 Cantonese          WS     WS          16     30         0
3             3 Croatian          WG     WG           8     16         0
4             4 Croatian          WS     WS          16     30         0
5             5 Danish            WG     WG           8     20         0
6             6 Danish            WS     WS          16     36         1
7             7 English (American) WG     WG           8     18         0
8             8 English (American) WS     WS          16     30         1
9             9 French (Quebecois) WG     WG           8     16         0
10            10 French (Quebecois) WS     WS          16     30         1
# i 79 more rows
# i 1 more variable: unilemma_coverage <dbl>
```

Waar staat je databestand?

Om databestanden te kunnen inlezen in R, moet je normaal gesproken eerst de computer vertellen waar deze te vinden zijn. Wanneer de databestanden onderdeel zijn van een R-project, weet RStudio al waar het die bestanden moet vinden. RStudio kijkt namelijk automatisch in de map van je project. Dat is een groot voordeel van werken met een R-project. Je kunt dan werken met een *relatief pad*. In bovenstaand voorbeeld staan de databestanden binnen de map van het R-project in een map `_datasets` en door dit voor de naam van het bestand te plakken (`_datasets/wordbank_datasets.csv`), gaat R in de map van het R-project zoeken naar bestanden in de map `_datasets`. Wanneer je problemen ondervindt met relatieve paden kun je ook kiezen voor een absoluut pad. Hierbij geeft je de hele naam van de locatie op je computer, bijv. `C://Peter/OBS/_datasets/wordbank_datasets.csv`. Dit laatste is meer typewerk, maar zal in principe altijd goed gaan. Een nadeel van deze methode is echter dat

het je code minder makkelijk overdraagbaar maakt naar een andere computer.

3.2.1 Variabelennamen aanpassen

Wanneer je het databestand goed hebt ingelezen is het eerste wat je moet doen de namen van de variabelen controleren. Zoals eerder gemeld, hanteren we voor variabelennamen *snake_case*: alleen kleine letters en scheiding van inhoudswoorden met een underscore `_`, dus bijv. `wordbank_datasets`. Dit is echter niet de enige manier waarop variabelennamen voorkomen. In de praktijk kun je echt vanalles verwachten. Zie bijvoorbeeld hieronder, waar we de namen van twee variabelen uit onze datatabel `wordbank_datasets` ter illustratie hebben aangepast. We gebruiken hier de functie `head()` om alleen een deel van de rijen zichtbaar te maken (hier maar 1):

```
head(wordbank_datasets_slechte_namen, n = 1)
```

```
# A tibble: 1 x 8
```

	InstrumentID	language	form	`form type`	age_min	age_max	has_grammar
	<dbl>	<chr>	<chr>	<chr>	<dbl>	<dbl>	<dbl>
1	1	British Sign Language	WG	WG	8	36	0

```
# i 1 more variable: unilemma_coverage <dbl>
```

We kunnen ook de functie `names()` gebruiken:

```
names(wordbank_datasets_slechte_namen)
```

```
[1] "InstrumentID"      "language"          "form"
[4] "form type"        "age_min"           "age_max"
[7] "has_grammar"      "unilemma_coverage"
```

We zien twee variabelen die een afwijkende naam hebben. `InstrumentID` gebruikt hoofdletters en geen `_` om inhoudswoorden (*instrument* en *id*) te scheiden en `form type` heeft een spatie i.p.v. een underscore. Dat laatste is zeker iets om te vermijden,

want dat kan allerlei ongemak opleveren. We kunnen deze namen handmatig aanpassen met de functie `rename()`:

```
rename(.data = wordbank_datasets_slechte_namen,
       instrument_id = InstrumentID,
       form_type = `form type`)
```

```
# A tibble: 89 x 8
  instrument_id language          form form_type age_min age_max has_grammar
      <dbl> <chr>          <chr> <chr>      <dbl>  <dbl>      <dbl>
1           1 British Sign Langu~ WG     WG          8     36          0
2           2 Cantonese          WS     WS         16     30          0
3           3 Croatian          WG     WG          8     16          0
4           4 Croatian          WS     WS         16     30          0
5           5 Danish            WG     WG          8     20          0
6           6 Danish            WS     WS         16     36          1
7           7 English (American) WG     WG          8     18          0
8           8 English (American) WS     WS         16     30          1
9           9 French (Quebecois) WG     WG          8     16          0
10          10 French (Quebecois) WS     WS         16     30          1
# i 79 more rows
# i 1 more variable: unilemma_coverage <dbl>
```

Bij `rename()` geven we als eerste argument `.data` de naam van de datatabel mee waarin we namen willen veranderen en vervolgens, gescheiden door komma's de namen van de variabelen die we willen hernoemen. Hierbij geef je (verwarrend genoeg) eerst de nieuwe naam van de variabele en dan de oude naam. Variabelennamen met een spatie erin moet je tussen *backticks* ‘ ‘ zetten.

We zijn er nu nog niet. Met bovenstaande code hebben we de namen nog niet blijvend veranderd. Ze zijn alleen veranderd in de print out die je te zien krijgt in R. Dit kunnen we zien als we de datatabel nogmaals naar het scherm printen:

```
wordbank_datasets_slechte_namen
```

```
# A tibble: 89 x 8
```

	InstrumentID	language	form	`form type`	age_min	age_max	has_grammar
	<dbl>	<chr>	<chr>	<chr>	<dbl>	<dbl>	<dbl>
1	1	British Sign Lang~	WG	WG	8	36	0
2	2	Cantonese	WS	WS	16	30	0
3	3	Croatian	WG	WG	8	16	0
4	4	Croatian	WS	WS	16	30	0
5	5	Danish	WG	WG	8	20	0
6	6	Danish	WS	WS	16	36	1
7	7	English (American)	WG	WG	8	18	0
8	8	English (American)	WS	WS	16	30	1
9	9	French (Quebecois)	WG	WG	8	16	0
10	10	French (Quebecois)	WS	WS	16	30	1

```
# i 79 more rows
# i 1 more variable: unilemma_coverage <dbl>
```

Hier zien we nog steeds de oude namen. Om de namen blijvend te veranderen moeten we de verandering toewijzen aan de datatabel zelf met de operator <- die we eerder hebben gezien:

```
wordbank_datasets_slechte_namen <- rename(.data = wordbank_datasets_slechte_namen,
  instrument_id = InstrumentID,
  form_type = `form type`)

names(wordbank_datasets_slechte_namen)
```

```
[1] "instrument_id"      "language"           "form"
[4] "form_type"         "age_min"            "age_max"
[7] "has_grammar"       "unilemma_coverage"
```

De functie `rename()` is goed te gebruiken als je maar een beperkt aantal namen moet aanpassen. Wanneer er echter veel (meer dan 3) namen moeten worden aangepast is het een beetje omslachtig. In dat geval kun je beter de functie `clean_names()` uit het package `janitor` gebruiken. Je kunt hiervoor het package als geheel laden of de functie aanroepen met `janitor::clean_names()`. Ook hier geldt dat je de output van deze functie weer moet toewijzen aan een datatabel om de verandering blijvend door te voeren. In onderstaand

voorbeeld hebben we gekozen om de oorspronkelijk naam voor de datatabel (`wordbank_datasets`) weer te gebruiken, aangezien de slechte namen zijn aangepast.

```
wordbank_datasets <- janitor::clean_names(wordbank_datasets_slechte_namen)

names(wordbank_datasets)
```

```
[1] "instrument_id"      "language"           "form"
[4] "form_type"         "age_min"            "age_max"
[7] "has_grammar"       "unilemma_coverage"
```

3.2.2 Type van variabelen aanpassen

Nu de variabelen allemaal een eigen kolom en een goede naam hebben, moeten we nog controleren of ze ook van het juiste type zijn. Bij het inlezen van de data doet R een *educated guess* naar het type van de variabelen op basis van de data die in de variabelen zitten. Zitten er getallen in dan zal het gaan voor `dbl` (of `num`, `int`) en wanneer er alleen tekst in zit dan voor `char`. In de meeste gevallen gaat dit goed, maar niet altijd.

Er is één ding om goed op te letten en dat is het feit dat we in statistische analyse categorische variabelen (zie [sectie 1.2](#) uit IMS) doorgaans **factoren** noemen. Factoren zijn categorische variabelen die een beperkt aantal terugkerende waarden kunnen aannemen. Die waarden noemen we **levels**. In R krijgen factoren het datatype `factor`.

Neem als voorbeeld de variabele `language` uit onze datatabel `wordbank_datasets`, die we eerder ook als vector `languages` hebben opgeslagen. In deze variabelen staan de talen opgesomd van alle vragenlijsten die in de wordbank zitten. We zien dat sommige talen meerdere keren voorkomen. Bij de eerste 10 observaties komen 4 talen 2 keer voor (Croatian, Danish, English (American) en French (Quebecois)):

```
wordbank_datasets
```

```
# A tibble: 89 x 8
  instrument_id language      form form_type age_min age_max has_grammar
      <dbl> <chr>      <chr> <chr>      <dbl> <dbl>      <dbl>
1             1 British Sign Langu~ WG    WG          8     36          0
2             2 Cantonese          WS    WS         16     30          0
3             3 Croatian          WG    WG          8     16          0
4             4 Croatian          WS    WS         16     30          0
5             5 Danish            WG    WG          8     20          0
6             6 Danish            WS    WS         16     36          1
7             7 English (American) WG    WG          8     18          0
8             8 English (American) WS    WS         16     30          1
9             9 French (Quebecois) WG    WG          8     16          0
10            10 French (Quebecois) WS    WS         16     30          1
# i 79 more rows
# i 1 more variable: unilemma_coverage <dbl>
```

Met de functie `dinstinct()` voor datatabellen of `unique()` voor vectoren kunnen we laten zien dat er in de 89 observaties in de data 42 verschillende talen voorkomen:

```
distinct(.data = wordbank_datasets, language)
```

```
# A tibble: 42 x 1
  language
  <chr>
1 British Sign Language
2 Cantonese
3 Croatian
4 Danish
5 English (American)
6 French (Quebecois)
7 German
8 Hebrew
9 Italian
10 Mandarin (Beijing)
# i 32 more rows
```

```
unique(languages)
```

[1] "British Sign Language"	"Cantonese"	"Croatian"
[4] "Danish"	"English (American)"	"French (Quebecois)"
[7] "German"	"Hebrew"	"Italian"
[10] "Mandarin (Beijing)"	"Norwegian"	"Russian"
[13] "Slovak"	"Spanish (Mexican)"	"Swedish"
[16] "Turkish"	"English (British)"	"American Sign Language"
[19] "Greek (Cypriot)"	"Kigiriana"	"Kiswahili"
[22] "Czech"	"English (Australian)"	"Latvian"
[25] "Korean"	"French (French)"	"Spanish (European)"
[28] "Portuguese (European)"	"Mandarin (Taiwanese)"	"English (Irish)"
[31] "Irish"	"Finnish"	"Dutch"
[34] "Hungarian"	"Spanish (Argentinian)"	"Spanish (Chilean)"
[37] "Spanish (Peruvian)"	"Persian"	"Arabic (Saudi)"
[40] "Estonian"	"Catalan"	"Japanese"

Wanneer de variable `language` als type `character` wordt gezien dan missen we de overeenkomsten tussen deze terugkerende talen. Door er een *factor* van te maken waarin elke taal een *level* is maken we dit juist zichtbaar. Dit biedt later allerlei voordelen bij het maken van grafieken, zoals het makkelijker kunnen herordenen van levels, en het doen van statistische analyses. Het is dus verstandig om categorische variabelen om te zetten naar type *factor*.

Je kunt een variabele omzetten naar een factor met de functie `as.factor()`.⁵ Deze zal de levels ordenen op alfabet. Om deze aanpassing te kunnen doen gebruiken we de veelgebruikte functie `mutate()` waarbij we weer de naam van de variabele eerst noemen en na de `=` wat we erin willen stoppen. Ook hier geldt weer: om de verandering blijvend te maken moeten we deze toewijzen aan de datatabel met `<-`.

```
wordbank_datasets <-
  mutate(.data = wordbank_datasets,
         language = as.factor(language))
```

Om een snelle blik te werpen op hoe R de variabelen in onze datatabel ziet, kunnen we de functie `str()` voor *structure* gebruiken (het *tidyverse*-alternatief is `glimpse()`).⁶

⁵ Er bestaat ook een *tidyverse*-variant `as_factor()`. Deze functie zet variabelen ook om naar een factor, maar gebruikt de eerste observatie in de data als het eerste level.

⁶ De functie `str()` gebruikt types uit base R (dus bijv. `num`), terwijl `glimpse()` types uit het *tidyverse* gebruikt (dus `<dbl>`).

```
str(wordbank_datasets)
```

```
tibble [89 x 8] (S3: tbl_df/tbl/data.frame)
 $ instrument_id      : num [1:89] 1 2 3 4 5 6 7 8 9 10 ...
 $ language           : Factor w/ 42 levels "American Sign Language",...: 3 4 6 6 8 8 10 10 17 17
 $ form              : chr [1:89] "WG" "WS" "WG" "WS" ...
 $ form_type         : chr [1:89] "WG" "WS" "WG" "WS" ...
 $ age_min           : num [1:89] 8 16 8 16 8 16 8 16 8 16 ...
 $ age_max           : num [1:89] 36 30 16 30 20 36 18 30 16 30 ...
 $ has_grammar       : num [1:89] 0 0 0 0 0 1 0 1 0 1 ...
 $ unilemma_coverage: num [1:89] 0.95 0.95 1 1 1 0.93 1 1 1 0.99 ...
```

We zien dat `language` nu een factor is met 42 levels (zoveel talen als we hadden geteld) en het eerste level is *American Sign Language*, de naam die als eerste in het alfabet komt. Deze omzetting naar een factor zouden we ook moeten uitvoeren voor de andere variabelen van type `character` (`form` en `form_type`). Ook de variabele `instrument_id` komt hiervoor in aanmerking, want de getallen die hier zijn gebruikt als id voor de vragenlijst moeten we niet zien als getallen, maar als namen. Deze omzettingen kunnen we in een keer doen binnen de functie `mutate()` door de variabelen te scheiden met een komma. Voor de overzichtelijkheid van de code zetten we iedere omzetting op een eigen regel.

```
wordbank_datasets <-
  mutate(.data = wordbank_datasets,
         form = as.factor(form),
         form_type = as.factor(form_type),
         instrument_id = as.factor(instrument_id)
  )

str(wordbank_datasets)
```

```
tibble [89 x 8] (S3: tbl_df/tbl/data.frame)
 $ instrument_id      : Factor w/ 89 levels "1","2","3","4",...: 1 2 3 4 5 6 7 8 9 10 ...
 $ language           : Factor w/ 42 levels "American Sign Language",...: 3 4 6 6 8 8 10 10 17 17
 $ form              : Factor w/ 24 levels "CDITwo","FormA",...: 17 22 17 22 17 22 17 22 17 22 .
 $ form_type         : Factor w/ 2 levels "WG","WS": 1 2 1 2 1 2 1 2 1 2 ...
```

```
$ age_min      : num [1:89] 8 16 8 16 8 16 8 16 8 16 ...
$ age_max      : num [1:89] 36 30 16 30 20 36 18 30 16 30 ...
$ has_grammar  : num [1:89] 0 0 0 0 0 1 0 1 0 1 ...
$ unilemma_coverage: num [1:89] 0.95 0.95 1 1 1 0.93 1 1 1 0.99 ...
```

Let dus goed op: getallen zijn niet altijd getallen, maar kunnen ook als namen voor levels van een categorische variabele zijn gebruikt. Stel we maken een datatabel `woordfrequenties` aan met daarin voor een aantal woorden de waarden van de variabele `frequentie`. De woordfrequentie is door de onderzoeker gecodeerd als 1, 2 en 3 wat staat voor respectievelijk ‘laag frequent’, ‘midden frequent’ en ‘hoog frequent’.

```
woordfrequenties <- tibble(woord = c("woord1", "woord2", "woord3", "woord4", "woord5"),
                             frequentie = c(1,3,1,1,2))

woordfrequenties
```

```
# A tibble: 5 x 2
  woord frequentie
  <chr>      <dbl>
1 woord1          1
2 woord2          3
3 woord3          1
4 woord4          1
5 woord5          2
```

Omdat de waarden van de variabele `frequentie` getallen zijn, denkt R dat we hier te maken hebben met een numerieke variabele. We kunnen dit weer omzetten met de functie `as.factor()` (dit doen we ook meteen voor de variabele `woord`), vervolgens kunnen we met de functie `fct_recode()` de levels van deze factor expliciete namen geven. Wanneer we de datatabel nu weer bekijken dan zien we in plaats van getallen de namen van de levels. Dat maakt het inzichtelijker voor gebruikers van de data.

```

woordfrequenties <- mutate(woordfrequenties,
                           woord = as.factor(woord),
                           frequentie = as.factor(frequentie),
                           frequentie = fct_recode(frequentie,
                                                    laag = "1",
                                                    midden = "2",
                                                    hoog = "3")
                           )

woordfrequenties

```

```

# A tibble: 5 x 2
  woord frequentie
  <fct>   <fct>
1 woord1 laag
2 woord2 hoog
3 woord3 laag
4 woord4 laag
5 woord5 midden

```

De functie `mutate()` komen we nog vaak tegen. We kunnen deze bijvoorbeeld ook gebruiken om een nieuwe variabele aan te maken uit bestaande variabelen. Zo zouden we de variabele `age_range` kunnen aanmaken die het leeftijdsbereik uitdrukt als het verschil tussen de minimumleeftijd (`age_min`) en de maximumleeftijd (`age_max`):

```

wordbank_datasets <-
  mutate(wordbank_datasets,
         age_range = age_max - age_min)

relocate(.data = wordbank_datasets,
         age_range,
         .after = age_max)

```

```

# A tibble: 89 x 9
  instrument_id language form form_type age_min age_max age_range has_grammar
  <fct>          <fct>   <fct> <fct>      <dbl>   <dbl>   <dbl>      <dbl>
1 1              British ~ WG    WG          8      36      28          0

```

```

2 2      Cantonese WS   WS      16      30      14      0
3 3      Croatian  WG   WG       8      16       8      0
4 4      Croatian  WS   WS      16      30      14      0
5 5      Danish    WG   WG       8      20      12      0
6 6      Danish    WS   WS      16      36      20      1
7 7      English ~ WG   WG       8      18      10      0
8 8      English ~ WS   WS      16      30      14      1
9 9      French (~ WG   WG       8      16       8      0
10 10     French (~ WS   WS      16      30      14      1
# i 79 more rows
# i 1 more variable: unilemma_coverage <dbl>

```

We maken hier met `relocate()` de nieuwe variabele zichtbaar direct achter de variabele `age_max` in de datatabel. Wanneer we dit niet doen dan zou deze achteraan komen. Je kunt `.after` (of `.before`) ook direct als argument aan `mutate()` meegeven.

3.3 Voorbeeldscript

Wanneer je begint met coderen is het vooral belangrijk dat je code werkt. Hoe je code precies tot het resultaat komt is nog van ondergeschikt belang. Het kan altijd eleganter of sneller. Toch is het verstandig om je vanaf het begin af aan aan een aantal zaken aan te leren die je code leesbaarder maken. Hier een aantal richtlijnen:

Goede code gebruikt:

- **duidelijke namen** voor datatabellen en vectoren, waarbij we gebruik maken van *snake_case* (dus `age_range` en niet `AgeRange`);
- **spaties** om operatoren, zoals `=` en `,` (dus `x = y` en `x, y` i.p.v. `x=y` en `x,y`);
- **nieuwe regels** om de lengte van code te beperken, bijvoorbeeld door na een `,` of `<-` een return te geven;
- **inspringing** om aan te geven dat code op de huidige regel nog bij een vorige regel hoort en bijvoorbeeld argumenten van een functie op hetzelfde niveau te zetten.

Een goed script:

- heeft een **heldere opbouw** met eerst het aanroepen van packages, dan het inlezen van de data, het bewerken van variabelen en dan de overige dingen die je gaat doen;
- gebruikt **witregels** om codeblokken te maken van regels code die bij elkaar horen en deze te scheiden van andere code met een andere functie;
- heeft een **goede toelichting** op de werking van een blokje code, hierbij gebruik je **#** aan het begin van de regel. Je kunt **#** ook gebruiken om op een regel commentaar toe te voegen. R negeert alles wat na een **#** staat en probeert dit dus niet uit te voeren.

Hieronder een voorbeeldscript gebaseerd op de code uit dit hoofdstuk:

```
# Aanroepen benodigde packages
library(tidyverse)

# Bekijken dataset
wordbank_datasets

# Aanpassen type van variabelen
wordbank_datasets <-
  mutate(.data = wordbank_datasets,
         language = as.factor(language),
         form = as.factor(form),
         form_type = as.factor(form_type),
         instrument_id = as.factor(instrument_id)
  )

# Aanmaken aparte vector met alle talen in de dataset
languages <- pull(wordbank_datasets, language)
```

3.4 Omgaan met errors

R neemt alles wat je typt heel letterlijk. Je zult dus heel precies moeten werken, anders geeft R je een foutmelding. In onderstaand voorbeeld zijn we bijvoorbeeld de ‘s’ vergeten aan het eind van ‘dataset’. We vragen R dus om `wordbank_dataset`

te laten zien, terwijl we `wordbank_datasets` bedoelen. Het is een klein verschil, maar voor R voldoende om een error te geven.

```
wordbank_dataset
```

```
Error in eval(expr, envir, enclos): object 'wordbank_dataset' not found
```

Errors horen bij coderen. Ze kunnen ontstaan door typefouten of door het verkeerd gebruik van de regels van de taal (bijv. het vergeten van een argument dat nodig is om een functie uit te kunnen voeren). Errors kunnen heel frustrerend zijn en soms ook veel tijd kosten om op te lossen. Probeer rustig te blijven, haal diep adem en probeer uit te vinden waar R over klaagt. Vaak zit er in de error-melding wel een aanwijzing wat er mis is. Wanneer R meldt ‘object not found’ dan gaat het meestal of om een typefout in de naam van het object dat je wilt gebruiken of om een variabele in een datatabel, terwijl je R nog niet hebt verteld in een specifieke datatabel te zoeken.

Naast errors geeft R ook waarschuwingen (*warnings*). De ernst daarvan is vaak wat moeilijker te bepalen. Soms kun je die voorbij laten gaan, maar soms moet je er ook echt iets mee. Ook voor warnings geldt dat het verstandig is om ze goed te bekijken, want vaak staat er een aanwijzing in. In onderstaand voorbeeld proberen we met de functie `mean()` het gemiddelde te berekenen van een vector die niet uit getallen bestaat. De functie `mean()` vraagt echter om getallen. R wijst ons hierop in de *warning* en geeft ook aan wat het resultaat hiervan is: je krijgt ontbrekende data NAs (voor *not available*).

```
mean(languages)
```

```
Warning in mean.default(languages): argument is not numeric or logical:  
returning NA
```

```
[1] NA
```

Zoals gezegd, errors horen bij coderen. Je moet er simpelweg mee leren leven. Laat je niet gek maken en weet dat je niet alleen bent. De kans is groot dat iemand anders dezelfde error ook heeft gehad en er al een oplossing voor heeft gevonden. Stel dus vragen of kopieer je error-melding naar Google om uit te vinden wat het probleem nu eigenlijk is en welke oplossingen er bestaan. Websites als *stackoverflow* bieden vaak hulp.

Verder leren

- In Hoofdstuk 1 van SfL (Winter, 2019) wordt de basis van R (**base R**) verder uitgelegd. Ter voorbereiding op het eerste practicum is het prettig om vast wat commando's te proberen uit dit hoofdstuk. Werk verder in het script dat je in het vorige hoofdstuk hebt aangemaakt.⁷ Werk nu hoofdstuk 1 door t/m sectie 1.8. Typ de commando's in de grijze vakken over en voer ze uit. In sectie 1.2 doe je dit direct in de R-console, vanaf sectie 1.3 (en in alle volgende practica!) in een script. Zie sectie 1.15 voor handige shortcuts die je hierbij kunt gebruiken. Controleer steeds weer of je dezelfde output krijgt als in het boek.

Winter, B. (2019). *Statistics for Linguists: An Introduction Using R*. Routledge. <https://doi.org/10.4324/9781315165547>

⁷ Je kunt eventueel ook <https://rdr.io/snippets> gebruiken voor het uitvoeren van R-code, mocht de installatie op je eigen computer niet werken. We kunnen dan tijdens een practicum proberen R en RStudio werkend te krijgen op je computer.

Nu zou je in staat moeten zijn Practicum 1 te maken.

4 Data Bewerken

In het vorige hoofdstuk heb je geleerd hoe je databestanden kunt inlezen in R. Databestanden komen in een gegeven formaat. Wanneer je data-analyse gaat doen is het handig als je het databestand kunt aanpassen naar jouw wensen. Soms wil je bijvoorbeeld maar een deel van de data analyseren of nieuwe variabelen toevoegen. Je moet als het ware ‘leren spelen’ met data. In dit hoofdstuk maak je kennis met een aantal functies uit het **tidyverse** die je daarbij kunnen helpen.

Na afloop van dit hoofdstuk:

- kun je data filteren, splitsen en sorteren
- kun je datatabellen samenvoegen
- kun je simpele numerieke samenvattingen maken van datatabellen
- ben je bekend met de belangrijkste ‘werkwoorden’ uit het **tidyverse**: `filter()`, `arrange()`, `select()`, `mutate()`, `summarize()`, `group_by()` en de pipe operator `%>%`.

codeboek

In deze cursus leer je werken met zo’n 100 veelgebruikte functies in R. Het is belangrijk om voor jezelf een goed overzicht te houden van deze functies. Werk hier actief aan door een eigen codeboek aan te maken. Voeg tijdens of na afloop van het bestuderen van dit hoofdstuk de codes toe aan jouw codeboek.

4.1 de *pipe*-operator

In het *tidyverse* zit een hele verzameling aan werkwoorden die je kunt gebruiken om bewerkingen op je data uit te voeren. Deze werkwoorden hebben gemeenschappelijk dat ze altijd een bewerking uitvoeren op een datatabel (*tibble*) en (meestal) weer een nieuwe datatabel opleveren. In het vorige hoofdstuk hebben we bijvoorbeeld `distinct()` gezien. Dit ‘werkwoord’ geeft van een variabele in een datatabel de unieke waarden terug.

```
distinct(.data = wordbank_datasets, language)
```

```
# A tibble: 42 x 1
  language
  <fct>
1 British Sign Language
2 Cantonese
3 Croatian
4 Danish
5 English (American)
6 French (Quebecois)
7 German
8 Hebrew
9 Italian
10 Mandarin (Beijing)
# i 32 more rows
```

De datatabel waarop de bewerking wordt uitgevoerd (hier: `wordbank_datasets`) staat altijd als eerste argument van de functie genoemd. Er is echter ook een andere formulering mogelijk. Om de code overzichtelijker te maken en om het mogelijk te maken om meerdere operaties na elkaar uit te voeren bestaat de *pipe*-operator `%>%`. Deze operator maakt het mogelijk om de datatabel uit de functie-aanroep te halen en ervoor te zetten:

```
wordbank_datasets %>% distinct(language)
```

```
# A tibble: 42 x 1
  language
  <fct>
1 British Sign Language
2 Cantonese
3 Croatian
4 Danish
5 English (American)
6 French (Quebecois)
7 German
8 Hebrew
9 Italian
10 Mandarin (Beijing)
# i 32 more rows
```

Deze code doet hetzelfde als de eerdere variant waarbij we de datatabel nog op de eerste plek benoemden. Onderliggend is dit namelijk precies wat R voor je doet, wanneer je de pipe gebruikt:

de *pipe*-operator

`x %>% functie(y)` wordt geïnterpreteerd als `functie(.data = x, y)`

Het is even wennen maar de pipe operator `%>%` is een ontzettend handige toevoeging aan je gereedschapskist. Met deze operator kun je namelijk heel makkelijk verschillende operaties aan elkaar plakken zonder dat je data tussendoor een nieuwe naam hoeft te geven. Je kunt de pipe lezen als ‘pak datatabel *x* en pas functie *f* erop toe’. Aangezien de output van de functies doorgaans een nieuwe datatabel oplevert kun je de output van een functie met een pipe weer doorzetten naar een volgende. Je kunt de pipe dan lezen als **en doe dan dit**. De pipe maakt het eenvoudiger om je code van links naar rechts te lezen in plaats van binnen naar buiten.

```
data %>% functie1() %>% functie2()
```

Wij schrijven de pipe als `%>%`, omdat deze manier ook gehanteerd wordt in de overige literatuur die we gebruiken in

deze cursus. De nieuwste versie van `tidyverse` schrijft de pipe als `|>`. In ons geval zijn deze twee schrijfwijzen equivalent, maar wij gebruiken in het dictaat en uitwerkingen dus `%>%`.

4.2 Observaties uitlichten

4.2.1 `filter()`

Het komt vaak voor dat je maar een deel van de data wilt bekijken. De functie `filter()` kun je gebruiken om een selectie te maken van de observaties die je wilt gaan bekijken. In de datatabel `wordbank_datasets` komen bijvoorbeeld twee typen vragenlijsten voor (variabele: `form_type`): vragenlijsten van type `WG`, wat staat voor *Words & Gestures*, zijn afgenomen bij ouders van babies en vragenlijsten van type `WS`, wat staat voor *Words & Sentences*, zijn afgenomen bij ouders van peuters. Wanneer we onze analyse nu willen beperken tot alleen de vragenlijsten afgenomen bij babies kunnen we dit met `filter()` aangeven:

```
wordbank_datasets %>%  
  filter(form_type == "WS")
```

```
# A tibble: 48 x 9  
  instrument_id language      form form_type age_min age_max has_grammar  
    <fct>         <fct>    <fct> <fct>      <dbl>  <dbl>      <dbl>  
1 2            Cantonese    WS    WS         16     30         0  
2 4            Croatian     WS    WS         16     30         0  
3 6            Danish       WS    WS         16     36         1  
4 8            English (American) WS    WS         16     30         1  
5 10           French (Quebecois) WS    WS         16     30         1  
6 11           German       WS    WS         18     30         0  
7 13           Hebrew       WS    WS         25     36         1  
8 15           Italian       WS    WS         18     36         0  
9 17           Mandarin (Beijing) TC    WS         17     30         0  
10 18           Mandarin (Beijing) WS    WS         16     30         0  
# i 38 more rows  
# i 2 more variables: unilemma_coverage <dbl>, age_range <dbl>
```

De conditie `form_type == "WS"` binnen `filter()` moeten we lezen als ‘behoud de observaties waarbij `form_type` gelijk is aan `WS`’. We zien dat 48 van de 89 vragenlijsten in de oorspronkelijke dataset aan deze conditie voldoen. Wanneer we met alleen deze data verder willen werken dan zouden we ervoor kunnen kiezen om deze weg te schrijven naar een nieuwe datatabel (met de inmiddels bekende operator `<-`).

```
wordbank_WS <- wordbank_datasets %>%
  filter(form_type == "WS")
```

De functie `filter()` werkt met condities waaraan voldaan moet worden. Afhankelijk van het type variabele kunnen die een andere vorm aannemen. Bij factoren zal het doorgaans gaan om het al dan niet gelijk zijn aan een bepaald level (`==`: gelijk aan, `!=`: is niet gelijk aan) waarbij de naam van het level tussen aanhalingstekens moet staan. Bij numerieke variabelen kan er bijvoorbeeld gekeken worden of een getal groter (`>`) of kleiner (`<`) is dan een bepaalde waarde. Willen we bijvoorbeeld alleen kijken naar datasets waarin de babies maximaal 18 maanden oud zijn dan kunnen we de variabele `age_max` beperken met `<=` (‘kleiner dan of gelijk aan’):

```
wordbank_datasets %>%
  filter(age_max <= 18)
```

```
# A tibble: 31 x 9
  instrument_id language      form form_type age_min age_max has_grammar
  <fct>         <fct>      <fct> <fct>      <dbl>  <dbl>      <dbl>
1 3            Croatian    WG    WG          8     16          0
2 7            English (American) WG    WG          8     18          0
3 9            French (Quebecois) WG    WG          8     16          0
4 16           Mandarin (Beijing) IC    WG         12     16          0
5 21           Russian      WG    WG          8     18          0
6 23           Slovak       WG    WG          8     16          0
7 25           Spanish (Mexican) WG    WG          8     18          0
8 27           Swedish      WG    WG          8     16          0
9 29           Turkish      WG    WG          8     16          0
10 38          Kigiriyama    WG    WG          8     15          0
# i 21 more rows
```

```
# i 2 more variables: unilemma_coverage <dbl>, age_range <dbl>
```

De condities in `filter()` moeten een waarheidswaarde (`true`, `false`) opleveren. De operatoren die hierbij gebruikt kunnen worden zie je in Tabel 4.2. Dit worden ook wel **booleans** genoemd.

operator	te lezen als	voorbeeld
<code>==</code>	is gelijk aan	<code>form_type == "WS"</code>
<code>!=</code>	is niet gelijk aan	<code>form_type != "WS"</code>
<code>%in%</code>	komt voor in de vector	<code>language %in% c("Dutch", "German")</code>
<code><</code>	kleiner dan	<code>age_max < 19</code>
<code><=</code>	kleiner dan of gelijk aan	<code>age_max <= 18</code>
<code>></code>	groter dan	<code>age_min > 3</code>
<code>>=</code>	groter dan of gelijk aan	<code>age_min >= 2</code>
<code>between()</code>	tussen	<code>between(age_min, 2, 10)</code>
<code>is.na()</code>	is een missing value	<code>is.na(form_type)</code>
<code>!is.na()</code>	is geen missing value	<code>!is.na(form_type)</code>
<code>!</code>	niet (keert de waarheidswaarde om)	<code>!age_max <= 18</code>

Tabel 4.2: boolean operatoren in R

Het is ook mogelijk om meerdere condities te combineren. Dit kunnen condities zijn op dezelfde variabele of op verschillende variabelen. Voor het combineren van condities kun je de logische operatoren `&` ‘en (beide condities moeten waar zijn)’, `|` ‘of (een van beide of beiden condities moeten waar zijn)’ of `xor()` ‘exclusief of (een van beide maar niet beide condities moeten waar zijn)’. In plaats van `&` kun je de condities ook scheiden door een komma. Als we dus filteren op de WS-vragenlijsten van babies van maximaal 18 maanden, hebben we twee opties:

```
wordbank_datasets %>%
  filter(form_type == "WS" & age_max <= 18)
```

```
# A tibble: 2 x 9
  instrument_id language      form    form_type age_min age_max has_grammar
  <fct>         <fct>      <fct>   <fct>      <dbl>   <dbl>     <dbl>
1 85           Arabic (Saudi) WS      WS          8      16         1
2 86           Arabic (Saudi) WSOther WS          8      16         1
# i 2 more variables: unilemma_coverage <dbl>, age_range <dbl>
```

```
wordbank_datasets %>%
  filter(form_type == "WS",
         age_max <= 18)
```

```
# A tibble: 2 x 9
  instrument_id language      form    form_type age_min age_max has_grammar
  <fct>         <fct>      <fct>   <fct>      <dbl>   <dbl>     <dbl>
1 85           Arabic (Saudi) WS      WS          8      16         1
2 86           Arabic (Saudi) WSOther WS          8      16         1
# i 2 more variables: unilemma_coverage <dbl>, age_range <dbl>
```

4.2.2 slice()

Met de verschillende `slice`-functies kun je bepaalde rijen zichtbaar maken of selecteren op basis van hun positie in de datatabel of een minimale of maximale waarde op een variabele.

Wanneer we de eerste of laatste rijen zichtbaar willen maken gebruiken we `slice_head()` of `slice_tail()`. Het argument `n` geeft hierbij aan hoeveel rijen je zichtbaar wilt maken:

```
# eerste 3 rijen zichtbaar maken
wordbank_datasets %>%
  slice_head(n = 3)
```

```
# A tibble: 3 x 9
  instrument_id language          form form_type age_min age_max has_grammar
<fct>         <fct>          <fct> <fct>      <dbl>  <dbl>    <dbl>
1 1           British Sign Langua~ WG    WG          8    36          0
2 2           Cantonese           WS    WS         16    30          0
3 3           Croatian           WG    WG          8    16          0
# i 2 more variables: unilemma_coverage <dbl>, age_range <dbl>
```

```
# laatste 2 rijen zichtbaar maken
wordbank_datasets %>%
  slice_tail(n = 2)
```

```
# A tibble: 2 x 9
  instrument_id language form form_type age_min age_max has_grammar
<fct>         <fct>  <fct> <fct>      <dbl>  <dbl>    <dbl>
1 93           Japanese WG    WG          8    18          1
2 94           Japanese WS    WS         16    36          1
# i 2 more variables: unilemma_coverage <dbl>, age_range <dbl>
```

De functie `slice()` is een algemenere functie waarin je de nummers van de rijen in de datatabel als argument meegeeft:

```
# rijen 10 t/m 15 zichtbaar maken
wordbank_datasets %>%
  slice(10:15)
```

```
# A tibble: 6 x 9
  instrument_id language          form form_type age_min age_max has_grammar
<fct>         <fct>          <fct> <fct>      <dbl>  <dbl>    <dbl>
1 10           French (Quebecois) WS    WS         16    30          1
2 11           German           WS    WS         18    30          0
3 12           Hebrew           WG    WG         11    25          0
4 13           Hebrew           WS    WS         25    36          1
5 14           Italian           WG    WG          7    24          0
6 15           Italian           WS    WS         18    36          0
# i 2 more variables: unilemma_coverage <dbl>, age_range <dbl>
```

Met `slice_min()` en `slice_max()` kun je de rijen selecteren met de minimale of maximale waarde op een variabele. Ook hier

kun je met `n` aangeven hoeveel rijen je zichtbaar wilt maken (bijv. de rijen met de 3 hoogste waarden op een variabele). Wanneer meerdere rijen hier dezelfde waarde hebben kun je meer rijen krijgen dan het gevraagde aantal.

```
# rijen zichtbaar maken met de hoogste waarde op de variabele age_max
wordbank_datasets %>%
  slice_max(age_max)
```

```
# A tibble: 1 x 9
  instrument_id language      form form_type age_min age_max has_grammar
  <fct>         <fct>         <fct> <fct>      <dbl> <dbl>    <dbl>
1 32           English (British) TEDS Th~ WS      34     47          1
# i 2 more variables: unilemma_coverage <dbl>, age_range <dbl>
```

```
# rijen zichtbaar maken met de 3 laagste waarden op de variabele age_min
wordbank_datasets %>%
  slice_min(age_min,
            n = 3)
```

```
# A tibble: 41 x 9
  instrument_id language      form form_type age_min age_max has_grammar
  <fct>         <fct>         <fct> <fct>      <dbl> <dbl>    <dbl>
1 14           Italian      WG    WG          7     24          0
2 1           British Sign Langu~ WG    WG          8     36          0
3 3           Croatian      WG    WG          8     16          0
4 5           Danish        WG    WG          8     20          0
5 7           English (American) WG    WG          8     18          0
6 9           French (Quebecois) WG    WG          8     16          0
7 19          Norwegian      WG    WG          8     20          0
8 21          Russian        WG    WG          8     18          0
9 23          Slovak         WG    WG          8     16          0
10 25          Spanish (Mexican) WG    WG          8     18          0
# i 31 more rows
# i 2 more variables: unilemma_coverage <dbl>, age_range <dbl>
```

Wil je random een aantal observaties uit je datatabel halen dan kun je `slice_sample()` gebruiken. Deze functie zal

iedere keer een ander resultaat geven. Het is dan voor de reproduceerbaarheid verstandig om aan het begin van je script (of voordat je de functie aanroept) een *random seed* aan te maken met de functie `set.seed()`. Wanneer iemand anders deze functie met hetzelfde getal uitvoert, zal die dezelfde resultaten krijgen. Dit geldt dus ook voor jezelf, wanneer je het script meerdere keren of op een later moment uitvoert.

```
set.seed(25)

wordbank_datasets %>%
  slice_sample(n = 3)
```

```
# A tibble: 3 x 9
  instrument_id language      form form_type age_min age_max has_grammar
  <fct>         <fct>      <fct>  <fct>    <dbl>  <dbl>    <dbl>
1 7            English (American) WG     WG         8      18         0
2 24           Slovak          WS     WS        17     36         1
3 65           Dutch          Swingl~ WG         8      36         1
# i 2 more variables: unilemma_coverage <dbl>, age_range <dbl>
```

4.3 Variabelen selecteren

We hebben nu steeds gekeken hoe je observaties (rijen) kunt uitlichten uit een dataset. Hetzelfde kunnen we ook doen met variabelen. De functie `select()` is hierbij de belangrijkste functie. Aan deze functie geven we de variabelen mee die we willen behouden. Stel we willen in onze dataset `wordbank_datasets` alleen verder met de variabelen `instrument_id` en `language`. We voegen dan deze variabelen gescheiden door een `,` toe aan de functie `select()`:

```
wordbank_datasets %>%
  select(instrument_id, language)
```

```
# A tibble: 89 x 2
  instrument_id language
  <fct>         <fct>
```

```

1 1      British Sign Language
2 2      Cantonese
3 3      Croatian
4 4      Croatian
5 5      Danish
6 6      Danish
7 7      English (American)
8 8      English (American)
9 9      French (Quebecois)
10 10     French (Quebecois)
# i 79 more rows

```

Let erop dat je in dit geval deze verandering alleen tijdelijk hebt gemaakt. Het is nu nog niet zo dat je de variabele definitief hebt verwijderd uit de datatabel. Om dat voor elkaar te krijgen zul je de actie toe moeten schrijven aan een naam met de operator `<-`, zoals we al vaker hebben gezien. Wees altijd voorzichtig met het overschrijven van je oorspronkelijke datatabel.

Wanneer je meerdere opeenvolgende variabelen wilt toevoegen aan `select()` kun je gebruik maken van de operator `:`, die je moet lezen als ‘t/m’. Deze operator is te combineren met `,` als je daarnaast ook nog extra variabelen wilt toevoegen die verderop in de dataset voorkomen. Dit scheelt je typewerk.

```

wordbank_datasets %>%
  select(instrument_id, language, form_type:has_grammar)

```

```

# A tibble: 89 x 6
  instrument_id language      form_type age_min age_max has_grammar
  <fct>         <fct>         <fct>    <dbl>  <dbl>    <dbl>
1 1      British Sign Language WG         8      36         0
2 2      Cantonese          WS        16      30         0
3 3      Croatian          WG         8      16         0
4 4      Croatian          WS        16      30         0
5 5      Danish            WG         8      20         0
6 6      Danish            WS        16      36         1
7 7      English (American) WG         8      18         0
8 8      English (American) WS        16      30         1
9 9      French (Quebecois) WG         8      16         0

```

```
10 10          French (Quebecois)    WS          16          30          1
# i 79 more rows
```

Soms is het handiger om niet de variabelen die je wilt behouden, maar juist die variabelen die je niet wilt behouden te noemen. Hiervoor kun je de operator `-` gebruiken voor de naam van de variabele die je niet meer wilt gebruiken. Dus bijvoorbeeld om de variabele `form` uit te sluiten gebruiken we:

```
wordbank_datasets %>%
  select(- form)
```

```
# A tibble: 89 x 8
  instrument_id language          form_type age_min age_max has_grammar
  <fct>         <fct>          <fct>    <dbl>  <dbl>    <dbl>
1 1            British Sign Language WG          8     36          0
2 2            Cantonese          WS         16     30          0
3 3            Croatian           WG          8     16          0
4 4            Croatian           WS         16     30          0
5 5            Danish             WG          8     20          0
6 6            Danish             WS         16     36          1
7 7            English (American) WG          8     18          0
8 8            English (American) WS         16     30          1
9 9            French (Quebecois)  WG          8     16          0
10 10           French (Quebecois)  WS         16     30          1
# i 79 more rows
# i 2 more variables: unilemma_coverage <dbl>, age_range <dbl>
```

Door de pipe kunnen we `select()` combineren met de eerder geïntroduceerde werkwoorden om de dataset tegelijkertijd ook in te perken op basis van de waarde van observaties. Zo kunnen we `select()` combineren met `filter()`, bijvoorbeeld om een datatabel te maken met de eerste twee variabelen met alleen vragenlijsten van type `WS`:

```
wordbank_datasets %>%
  filter(form_type == "WS") %>% #alleen vragenlijsten van type WS
  select(instrument_id, language) #alleen de eerste twee variabelen behouden
```

```
# A tibble: 48 x 2
  instrument_id language
  <fct>         <fct>
1 2             Cantonese
2 4             Croatian
3 6             Danish
4 8             English (American)
5 10            French (Quebecois)
6 11            German
7 13            Hebrew
8 15            Italian
9 17            Mandarin (Beijing)
10 18           Mandarin (Beijing)
# i 38 more rows
```

In principe maakt het niet uit in welke volgorde we `select()` en `filter()` zette, maar in sommige gevallen kan dit wel cruciaal zijn. In ons voorbeeld is dat het geval. Als je wilt filteren op basis van een variabele (hier `form_type`), dan moet je natuurlijk niet eerst de dataset zo inperken dat die variabele niet meer aanwezig is, want dan krijg je een error. Denk dus altijd goed na welke gevolgen de volgorde van je functies kan hebben.

4.4 Missing values

Onze voorbeelddataset `wordbank_datasets` is een complete dataset. Voor elke observatie zijn er voor alle variabelen waarden aanwezig. Dit is lang niet altijd het geval. Vaak is het zo dat er datapunten ontbreken. Een waarde voor een bepaalde variabele kan niet beschikbaar zijn, bijvoorbeeld doordat deze niet te achterhalen was, niet is uitgevraagd of dat er iets mis is gegaan in de dataverwerking. Ontbrekende waarden noemen we ook wel *missing values*. Deze worden in R gemarkeerd als `NA` voor ‘not available’. Er is veel te vertellen over *missing values*. We beperken ons hier ertoe dat je je bewust moet zijn van het bestaan van *missing values*. Het is dus verstandig om bij elke dataset voor de relevante variabelen die je mee wilt nemen in je analyse na te gaan of er ontbrekende waarden aanwezig zijn

en deze eventueel te verwijderen. Op deze manier voorkom je vaak problemen die later kunnen ontstaan.

Ter illustratie hebben we een dataset `wordbank_dataset_NA` gemaakt waaraan observaties zijn toegevoegd met een *missing value* op de variabele `age_range`. We kunnen de functie `is.na()` gebruiken (zie Tabel 4.2) binnen `filter()` om deze observaties zichtbaar te maken:

```
# zichtbaar maken van observaties met waarde NA op age_range
wordbank_dataset_NA %>%
  filter(is.na(age_range))
```

```
# A tibble: 3 x 4
  instrument_id language form_type age_range
  <fct>         <fct>    <fct>    <dbl>
1 999          Irish     WS         NA
2 9999         Dutch     WS         NA
3 99999        Croatian WG         NA
```

We zien inderdaad in de kolom `age_range` drie keer NA verschijnen. De ontkenning `!is.na()` kunnen we gebruiken in combinatie met `filter()` om deze observaties uit de dataset te verwijderen.

```
# verwijderen van observaties met waarde NA op age_range
wordbank_dataset_NA %>%
  filter(!is.na(age_range))
```

Zoals eerder gezien bij andere functies, kunnen we ook hier meerdere variabelenamen invoeren gescheiden door een `,`. Een alternatieve functie om observaties met de waarde NA te verwijderen, is `drop_na()` die, zoals de naam als zegt, de observaties verwijdert die een ontbrekende waarde hebben op de variabele die als argument wordt meegegeven:

```
# verwijderen van observaties met waarde NA op age_range
wordbank_dataset_NA %>%
  drop_na(age_range)
```

Zoals altijd, om deze wijziging blijvend te maken moeten we de datatabel weer toeschrijven aan een naam met `<-`.

4.5 Simpele tellingen

Wanneer je begint te werken met een dataset wil je allereerst een goed beeld krijgen van de data. Nadat je eventueel de naam en het type van een of meerdere variabelen hebt aangepast (zie Hoofdstuk 3), is het handig om tellingen te maken van relevante variabelen. Dit geeft je een idee van wat je kan verwachten en of er bijvoorbeeld dubbelingen in de data zitten. We kunnen hiervoor de functie `count()` gebruiken. We geven als argument de naam van de variabele mee waarin we geïnteresseerd zijn, in dit geval `language`. Deze functie geeft een tabel (`tibble`) waarin het aantal voorkomens staat vermeld in de kolom `n`.

```
wordbank_dataset_NA %>%  
  count(language)
```

```
# A tibble: 42 x 2  
  language          n  
  <fct>          <int>  
1 American Sign Language 5  
2 Arabic (Saudi)         3  
3 British Sign Language  1  
4 Cantonese              1  
5 Catalan                2  
6 Croatian               3  
7 Czech                  1  
8 Danish                 2  
9 Dutch                  8  
10 English (American)    4  
# i 32 more rows
```

We kunnen de tellingen in `n` met `arrange()` ordenen van klein naar groot ...

```
# tellingen ordenen van klein naar groot
wordbank_dataset_NA %>%
  count(language) %>%
  arrange(n)
```

```
# A tibble: 42 x 2
```

	language	n
	<fct>	<int>
1	British Sign Language	1
2	Cantonese	1
3	Czech	1
4	English (Australian)	1
5	English (Irish)	1
6	Estonian	1
7	German	1
8	Greek (Cypriot)	1
9	Hungarian	1
10	Spanish (Argentinian)	1

```
# i 32 more rows
```

... of met toevoeging van desc() van groot naar klein:

```
# tellingen ordenen van groot naar klein
wordbank_dataset_NA %>%
  count(language) %>%
  arrange(desc(n))
```

```
# A tibble: 42 x 2
```

	language	n
	<fct>	<int>
1	Dutch	8
2	American Sign Language	5
3	English (American)	4
4	English (British)	4
5	Finnish	4
6	Arabic (Saudi)	3
7	Croatian	3
8	Korean	3
9	Mandarin (Beijing)	3

```
10 Catalan
# i 32 more rows
```

2

De functie `count()` is zelfs ook toe te passen op numerieke variabelen

```
wordbank_dataset_NA %>%
  count(age_range) %>%
  arrange(desc(n))
```

```
# A tibble: 16 x 2
  age_range      n
  <dbl> <int>
1      20     20
2       8     16
3      14     13
4      28      8
5      10      7
6       7      6
7      12      5
8      18      4
9      13      3
10     NA      3
11      19      2
12       4      1
13       9      1
14      11      1
15      15      1
16      17      1
```

We zien in bovenstaande output dat *missing values* ook worden geteld. Zoals eerder gezegd, het is dus verstandig deze eerst te verwijderen:

```
wordbank_dataset_NA %>%
  drop_na(age_range) %>%
  count(age_range) %>%
  arrange(desc(n))
```

```
# A tibble: 15 x 2
  age_range      n
  <dbl> <int>
1      20      20
2       8      16
3      14      13
4      28       8
5      10       7
6       7       6
7      12       5
8      18       4
9      13       3
10     19       2
11       4       1
12       9       1
13      11       1
14      15       1
15      17       1
```

Soms wil je al iets meer de diepte in met je analyse. Stel je wilt kijken naar voorkomens van een variabele binnen verschillende niveaus van een andere variabele. Je kunt dan gebruikmaken van `group_by()`. Wanneer we de variabele `form_type` meegeven als argument van deze functie dan krijgen we tellingen die weergeven hoe vaak elke taal voorkomt per type vragenlijst (WG en WS):

```
wordbank_dataset_NA %>%
  group_by(form_type) %>%
  count(language)
```

```
# A tibble: 72 x 3
# Groups:   form_type [2]
  form_type language      n
  <fct>      <fct>    <int>
1 WG        American Sign Language  2
2 WG        Arabic (Saudi)         1
3 WG        British Sign Language   1
4 WG        Catalan                 1
5 WG        Croatian                 2
```

```

6 WG      Danish      1
7 WG      Dutch       5
8 WG      English (American) 2
9 WG      English (British)  2
10 WG     Finnish     2
# i 62 more rows

```

Je kunt bovenstaand resultaat ook krijgen door beide variabelen mee te geven aan `group_by`:

```

wordbank_dataset_NA %>%
  group_by(form_type, language) %>%
  count()

```

```

# A tibble: 72 x 3
# Groups:   form_type, language [72]
  form_type language      n
  <fct>     <fct>     <int>
1 WG      American Sign Language 2
2 WG      Arabic (Saudi)         1
3 WG      British Sign Language  1
4 WG      Catalan                1
5 WG      Croatian               2
6 WG      Danish                 1
7 WG      Dutch                  5
8 WG      English (American)     2
9 WG      English (British)      2
10 WG     Finnish                2
# i 62 more rows

```

Wanneer je iets anders dan tellingen wilt uitvoeren in combinatie met `group_by()`, bijvoorbeeld een gemiddelde berekenen, dan moet je deze functie combineren met `summarize()` (of `summarise()`). Binnen `summarize()` geef je dan aan wat je wilt dat R voor je berekent. Als we bijvoorbeeld de gemiddelde `age_range` per type vragenlijst willen bepalen, kunnen we de volgende code gebruiken:

```
wordbank_dataset_NA %>%
  drop_na(age_range) %>% #NAs verwijderen
  group_by(form_type) %>% #resultaten weergeven per type vragenlijst
  summarize(mean_age_range = mean(age_range)) #gemiddelde berekenen
```

```
# A tibble: 2 x 2
  form_type mean_age_range
  <fct>      <dbl>
1 WG         11.8
2 WS         17.2
```

We gebruiken hiervoor de functie `mean()` om het gemiddelde te berekenen. Binnen `summarize()` geven we hierbij aan hoe we de variabele waarin het gemiddelde wordt opgeslagen willen noemen. Dit doen we voor het `=`-teken. Let op dat je wel eerst de *missing values* verwijdert, want anders geeft de functie `mean()` NAs als resultaat, aangezien er van een reeks getallen met een NA geen gemiddelde kan worden berekend.

```
wordbank_dataset_NA %>%
  group_by(form_type) %>%
  summarize(mean_age_range = mean(age_range))
```

```
# A tibble: 2 x 2
  form_type mean_age_range
  <fct>      <dbl>
1 WG         NA
2 WS         NA
```

Bij de functie `mean()` kun je overigens ook in de functie zelf aangeven dat je *missing values* niet mee wilt nemen in de berekening:

```
wordbank_dataset_NA %>%
  group_by(form_type) %>%
  summarize(mean_age_range = mean(age_range, na.rm = TRUE))
```

```
# A tibble: 2 x 2
  form_type mean_age_range
  <fct>      <dbl>
1 WG          11.8
2 WS          17.2
```

Door het argument `na.rm` de waarde `TRUE` (of afgekort `T`) te geven worden `NA`s niet meegenomen. Wanneer je binnen `summarize()` een telling wilt toevoegen dan maak je gebruik van `n()` i.p.v. `count()`.

```
wordbank_dataset_NA %>%
  group_by(form_type) %>%
  summarize(mean_age_range = mean(age_range, na.rm = TRUE),
            n = n())
```

```
# A tibble: 2 x 3
  form_type mean_age_range     n
  <fct>      <dbl> <int>
1 WG          11.8     42
2 WS          17.2     50
```

In Hoofdstuk 7 vind je meer voorbeelden van functies die je binnen `summarize()` kunt gebruiken.

Verder leren

- In hoofdstuk 2 (t/m 2.6) uit Winter (2019) worden de belangrijkste `tidyverse`-functies besproken met andere voorbeelden.
- In hoofdstuk 3 van Wickham et al. (2023) vind je een uitgebreidere beschrijving van de functies die in dit hoofdstuk aan bod zijn gekomen.
- het [dplyr cheatsheet](#) geeft een overzicht van de werking van de besproken functies en meer.

Winter, B. (2019). *Statistics for Linguists: An Introduction Using R*. Routledge. <https://doi.org/10.4324/9781315165547>

Wickham, H., Çetinkaya-Rundel, M., & Grolemund, G. (2023). *R for Data Science* (2de dr.). O'Reilly. <https://r4ds.hadley.nz/>

Nu zou je in staat moeten zijn Practicum 2 te maken.

5 Grafieken maken

De eerste stap in data-analyse is een indruk krijgen van de data. Dit doen we door de data te visualiseren in grafieken en/of tabellen. In dit hoofdstuk maak je kennis met het onderdeel `ggplot2` uit het `tidyverse`. Dit is een zeer krachtig package waarmee je oneindig veel grafieken kunt maken. Het is uitgegroeid tot de standaard in veel wetenschapsgebieden en daarbuiten. Het kost wel wat tijd om de logica van `ggplot2` te doorgronden. Hier zullen we de komende hoofdstukken mee bezig zijn. Let op: `ggplot2` is onderdeel van `tidyverse` je hoeft het dus niet apart te laden als je `tidyverse` hebt aangeroepen met `library()`.

Na afloop van dit hoofdstuk:

- heb je overzicht hoe grafieken gemaakt worden met `ggplot2`;
- kun je de code aanpassen van bestaande grafieken gemaakt met `ggplot2`;
- kun je op basis van nieuwe data zelf een grafiek maken met `ggplot2`;

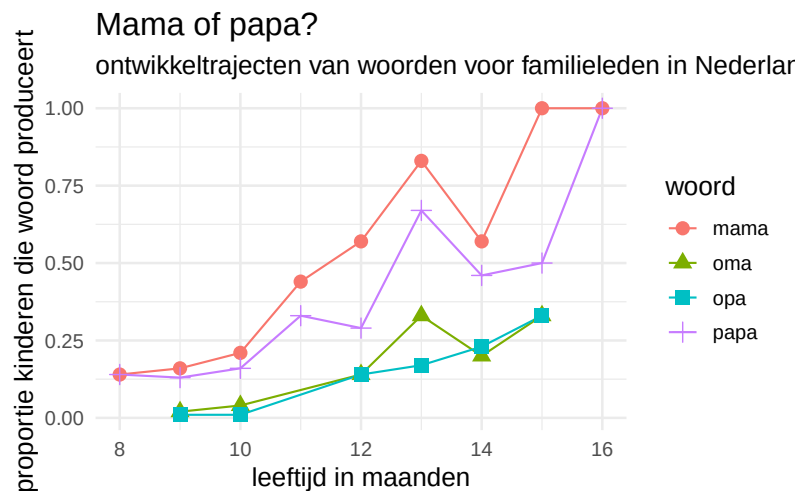
codeboek

In deze cursus leer je werken met zo'n 100 veelgebruikte functies in R. Het is belangrijk om voor jezelf een goed overzicht te houden van deze functies. Werk hier actief aan door een eigen codeboek aan te maken. Voeg tijdens of na afloop van het bestuderen van dit hoofdstuk de codes toe aan jouw codeboek.

5.1 De anatomie van een ggplot

Het doel van dit hoofdstuk is om de grafiek in Figuur 5.1 na te maken.⁸ Op deze manier leer je de logica van het maken van grafieken met `ggplot` hopelijk snel begrijpen. De doelgrafiek biedt een weergave van data van Nederlandse kinderen uit *Wordbank*. Het geeft voor vier woorden aan door hoeveel van de kinderen voor wie een vragenlijst is ingevuld ze gebruikt worden in de leeftijd van 8 t/m 16 maanden.

⁸ De opzet van dit hoofdstuk is gebaseerd op hoofdstuk 1 uit [R for Data Science](#).



Figuur 5.1: Ons doel voor dit hoofdstuk

5.1.1 de data

De data waar we onze grafiek op baseren staan in het databestand `wordbank_dutch_WG_items.csv` in de map `_datasets` in ons R-project. We gaan dit bestand eerst inlezen en kijken of we aanpassingen moeten doen aan de namen of het type van de variabelen. Zorg voordat we beginnen dat je het package `tidyverse` hebt geladen. Dit zorgt ervoor dat het package `ggplot2` ook actief is.

```
library(tidyverse)
```

```
#inlezen databestand
wordbank_dutch <- read_csv("_datasets/wordbank_dutch_WG_items.csv")

#data bekijken
wordbank_dutch
```

```
# A tibble: 3,969 x 5
  item_id item_definition category month proportion
  <dbl>   <chr>           <chr>   <dbl>     <dbl>
1       1 1 au/ai(pijn)      sounds     8         0
2       2 1 au/ai(pijn)      sounds     9         0
3       3 1 au/ai(pijn)      sounds    10         0
4       4 1 au/ai(pijn)      sounds    11        0.11
5       5 1 au/ai(pijn)      sounds    12        0.14
6       6 1 au/ai(pijn)      sounds    13        0.17
7       7 1 au/ai(pijn)      sounds    14        0.14
8       8 1 au/ai(pijn)      sounds    15        0.17
9       9 1 au/ai(pijn)      sounds    16         0.4
10      10 2 behbeh(schaap)      sounds     8         0
# i 3,959 more rows
```

De data zien er goed uit: elke variabele heeft zijn eigen kolom. Het inlezen is dus goed gelukt. De namen van de variabelen voldoen aan onze standaard (*snake_case*) dus daar hoeven we niets aan aan te passen. Laten we de data eens bekijken om te zien of we misschien het type van variabelen aan moeten passen. De dataset geeft informatie over op welke leeftijd woorden (in de kolom `item_definition`) door hoeveel kinderen worden geproduceerd. De eerste variabele `item_id` bestaat uit getallen en wordt door R als numeriek gezien. De `item_id` moeten we echter als een naam zien en het dus beter om deze om te zetten naar een factor. Hetzelfde geldt voor de variabelen `item_definition` en `category` die nu nog van type `character` zijn. We hebben hier te maken met de specifieke woorden waarvoor gegevens verzameld zijn (`item_definition`) en de categorie waartoe de woorden volgens de vragenlijst behoren (`category`). Dit zijn beide terugkerende namen (bijv. `au/ai(pijn)`) komt meerdere keren voor, net als `sounds`) en observaties willen we aan elkaar

kunnen linken. Dat lukt alleen als we ze als factor behandelen. Tot slot geeft de variabele `month` aan op welke leeftijd (in maanden) van de kinderen de vragenlijst is ingevuld en de variabele `proportion` geeft aan welke proportie kinderen (voor wie een vragenlijst is ingevuld) op die leeftijd het woord kon produceren. Het gaat hier in beide gevallen om getallen en de variabelen zijn door R ook als zodanig herkend (overigens is het ook mogelijk om `month` als een factor te beschouwen). Dit betekent dat we de eerste drie variabelen moeten omzetten naar factoren, aangezien we hier te maken hebben met namen van levels. Om een bewerking op meerdere kolommen tegelijk uit te voeren, maken we gebruik van de functie `across()`. Dit bespaart wat typewerk. Hiermee passen we de functie `as.factor()` toe op drie variabelen tegelijk. We gebruiken : tussen de namen van de variabelen om aan te geven dat we ook de tussenliggende variabele(n) willen aanpassen. Het is hierbij natuurlijk wel belangrijk dat de volgorde van de variabelen niet is aangepast.

```
wordbank_dutch <- wordbank_dutch %>%
  mutate(across(item_id:category, as.factor)) #variabelen item_id t/m category omzetten naar
wordbank_dutch
```

```
# A tibble: 3,969 x 5
```

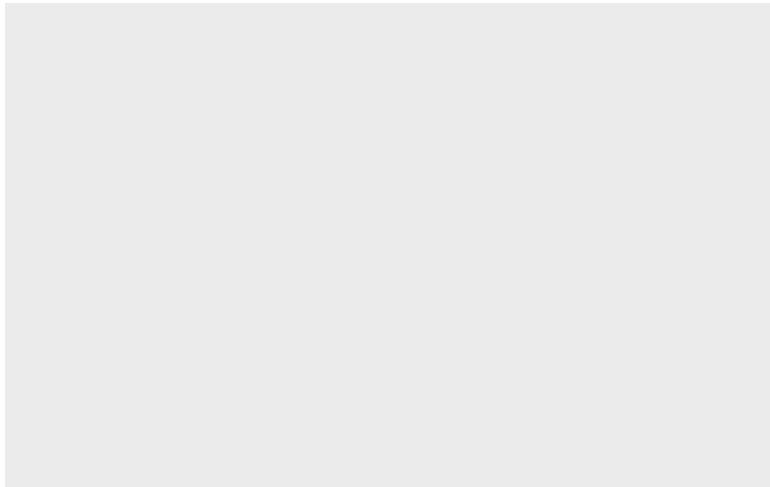
	item_id	item_definition	category	month	proportion
	<fct>	<fct>	<fct>	<dbl>	<dbl>
1	1	au/ai(pijn)	sounds	8	0
2	1	au/ai(pijn)	sounds	9	0
3	1	au/ai(pijn)	sounds	10	0
4	1	au/ai(pijn)	sounds	11	0.11
5	1	au/ai(pijn)	sounds	12	0.14
6	1	au/ai(pijn)	sounds	13	0.17
7	1	au/ai(pijn)	sounds	14	0.14
8	1	au/ai(pijn)	sounds	15	0.17
9	1	au/ai(pijn)	sounds	16	0.4
10	2	behbeh(schaap)	sounds	8	0

```
# i 3,959 more rows
```

5.1.2 De grafiek maken

De data staan er nu goed bij en we kunnen aan de slag met onze grafiek. Elke grafiek in `ggplot` begint met een aanroep van de functie `ggplot()` waarin we moeten aangeven op basis van welke datatabel we een grafiek willen maken.

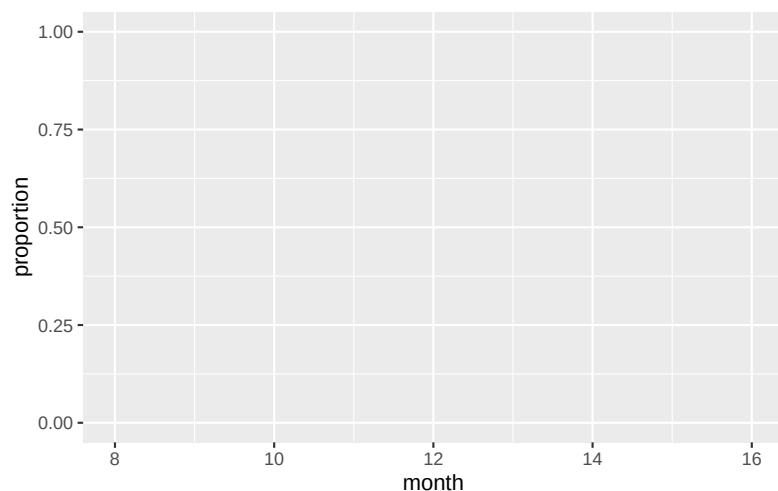
```
ggplot(data = wordbank_dutch)
```



Figuur 5.2: Een leeg canvas

Dit levert een leeg canvas op (Figuur 5.2). We hebben `ggplot` namelijk nog niet verteld welke variabelen uit de dataset we op welke manier willen weergeven in de grafiek. Dit laatste noemen we een **mapping** van variabelen naar eigenschappen van de grafiek. Die eigenschappen worden **aesthetics** genoemd (**aes** in het kort). Je moet hierbij denken aan de x-as, y-as, kleuren en vormen. Zo zien we in onze doelgrafiek dat de variabele `month` is afgebeeld op de x-as, de variabele `proportion` op de y-as en dat de variabele `item_definition` tot uitdrukking komt in de kleur en de vorm van symbolen. Laten we beginnen met de mapping van variabelen op de x-as en de y-as.

```
ggplot(data = wordbank_dutch,  
       mapping = aes(x = month, y = proportion))
```



Figuur 5.3: Mapping van variabelen op de x-as en de y-as

Nu zien we in [Figuur 5.3](#) de namen van de variabelen verschijnen op de assen en zien we dat ook het bereik van beide variabelen is weergegeven, 0-1 voor `proportion` op de y-as en 8-16 voor `month` op de x-as. Voor we verder gaan, versimpelen we de code eerst met de *pipe*-operator die we in het vorige hoofdstuk hebben leren kennen. Onderstaande code geeft je het standaard begin voor elke grafiek in `ggplot`: een dataset die naar `ggplot` wordt doorgezet en in `aes()` een specificatie van de mapping van variabelen die moeten worden afgebeeld.

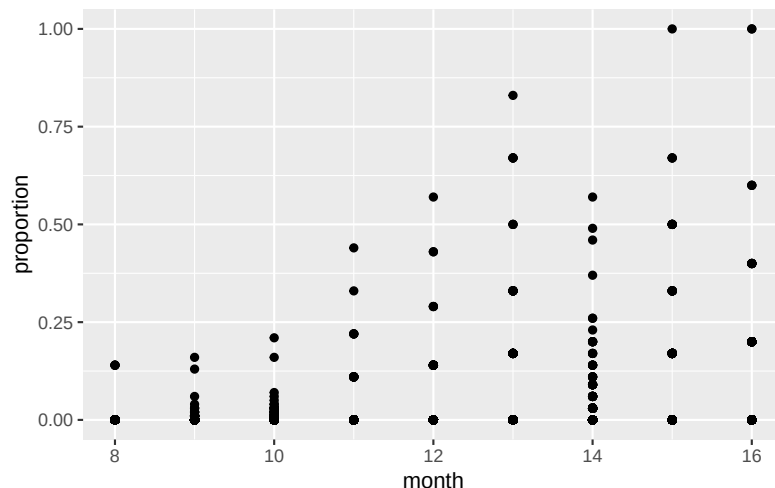
```
wordbank_dutch %>%  
  ggplot(aes(x = month, y = proportion))
```

We zien nu welke variabelen er afgebeeld gaan worden, maar we zien de data zelf nog niet. Om die zichtbaar te maken moeten we een **geom** toevoegen als een nieuwe laag aan onze grafiek. We hebben hier de keuze uit een groot aantal mogelijke type grafieken, denk aan lijnen, punten, staven. Elke dataset vraagt om zijn eigen aanpak. Dus het is noodzakelijk om vooraf goed na te denken over wat je precies wilt laten zien. Het beste kun je eerst op papier een schets maken voor jezelf.

In onze doelgrafiek wordt er gebruik gemaakt van zowel punten als lijnen. We beginnen met de punten. Hiervoor gebruiken we `geom_point()`. Let op: om een laag toe te voegen aan een

grafiek gebruiken we na de aanroep van `ggplot()` een `+`. Deze `+` moet aan het eind van de regel komen, anders krijg je een error.

```
wordbank_dutch %>%
  ggplot(aes(x = month, y = proportion)) +
  geom_point()
```

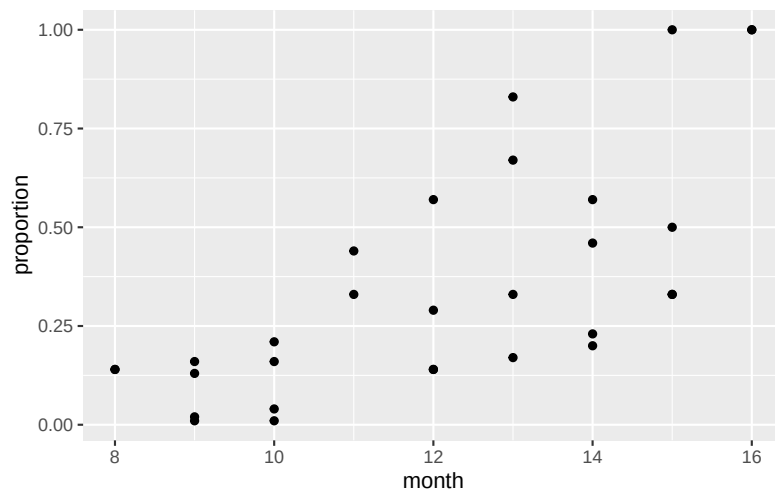


Figuur 5.4: De datapunten zijn zichtbaar geworden dankzij `geom_point()`.

We hebben in Figuur 5.4 veel meer stippen dan we in onze doelgrafiek zien. Dit komt, omdat we de data van alle woorden in de vragenlijst naar `ggplot()` hebben gestuurd terwijl we de gegevens van maar vier woorden willen laten zien. We moeten onze dataset dus eerst nog filteren voor we de grafiek gaan maken. We maken hiervoor eerst een vector `dutch_words` aan met de relevante woorden erin. We filteren ook proporties van 0 eruit om de data iets overzichtelijker te maken.

```
dutch_words <- c("mama", "papa", "oma", "opa")
```

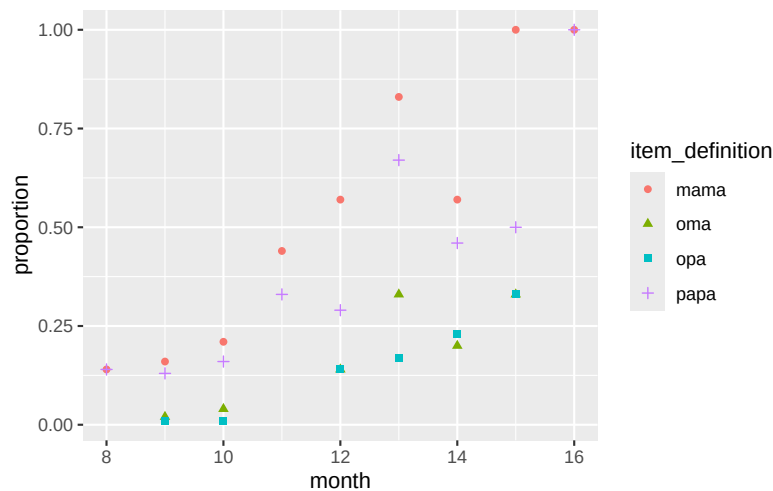
```
wordbank_dutch %>%
  filter(item_definition %in% dutch_words, #alleen de relevante woorden meenemen
         proportion > 0) %>% #alleen woorden met een proportie > 0 meenemen
  ggplot(aes(x = month, y = proportion)) +
  geom_point()
```



Figuur 5.5: Alleen de datapunten van de doelwoorden zijn zichtbaar.

We zien nu de datapunten (Figuur 5.5), maar nog niet bij welk woord ze horen. Alle punten hebben dezelfde vorm en kleur. Als we de vorm en kleur willen laten variëren op basis van het woord waar het datapunt bijhoort, dan moeten we een nieuwe mapping maken, namelijk van de variabele `item_definition` (het woord). Zoals hierboven gezien, wordt die mapping gemaakt binnen `aes()`. We kunnen dit op twee niveaus doen: (i) op het niveau van `ggplot()` en/of (ii) op het niveau van het `geom`. Ter illustratie laten we het hier op beide niveaus zien: de mapping naar vorm (`shape`) maken we op het niveau van `ggplot()` en de mapping naar kleur (`colo(u)r`) op het niveau van `geom_point()`.

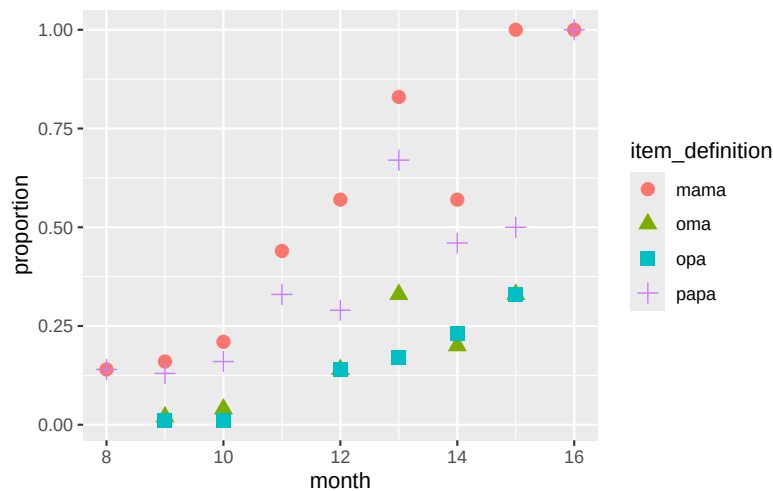
```
wordbank_dutch %>%
  filter(item_definition %in% dutch_words,
         proportion > 0) %>%
  ggplot(aes(x = month, y = proportion, shape = item_definition)) +
  geom_point(aes(color = item_definition))
```



Figuur 5.6: Dankzij een mapping naar **shape** en **color** is nu zichtbaar bij welke woorden de datapunten horen.

Dit lijkt er al meer op (Figuur 5.6). De grootte van de symbolen is alleen kleiner dan in onze doelgrafiek. Dit kunnen we aanpassen door het argument **size** toe te voegen aan **geom_point()** (Figuur 5.7). Omdat het hier niet gaat om de mapping van een variabele, maar om het toekennen van een constante waarde (de waarde '3'), voegen we dit argument toe buiten de **aes()** van het geom.

```
wordbank_dutch %>%
  filter(item_definition %in% dutch_words,
         proportion > 0) %>%
  ggplot(aes(x = month, y = proportion, shape = item_definition)) +
  geom_point(aes(color = item_definition), size = 3)
```

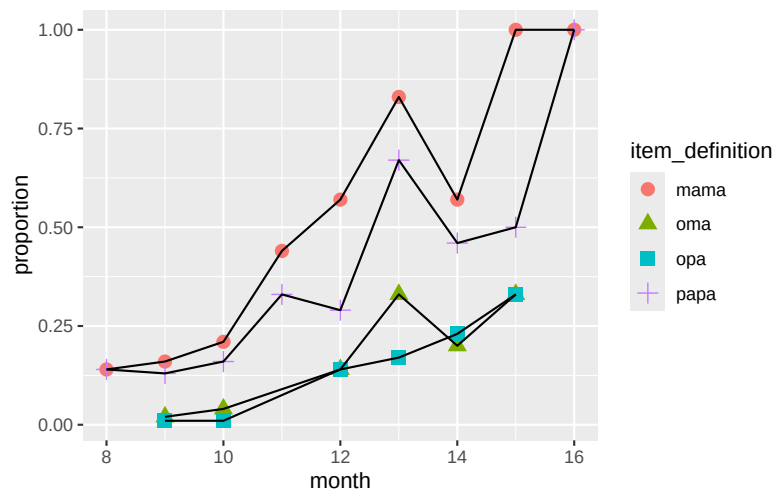


Figuur 5.7: De grootte van de datapunten is aangepast met `size`.

Let op: wanneer je een variabele meegeeft als argument buiten `aes()` dan gebeuren er andere dingen dan je verwacht (je zult waarschijnlijk een error krijgen). Bedenk dus altijd goed of je een eigenschap van de grafiek wil laten variëren op basis van een variabele: zo ja, dan moet dit binnen `aes()` gebeuren, zo niet, dan moet het buiten `aes()`.

Laten we nu de lijnen toevoegen, die ontbreken nog. Hiervoor gebruiken we het `geom_line()`. Ook deze voegen we met een `+` toe aan onze bestaande grafiek.

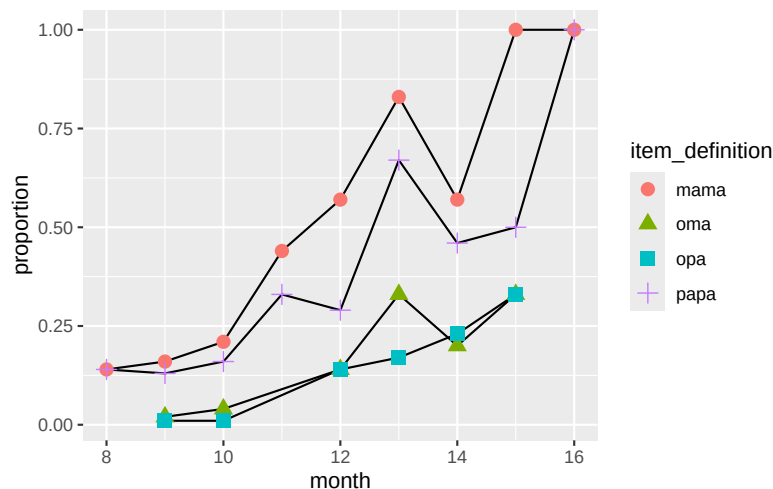
```
wordbank_dutch %>%
  filter(item_definition %in% dutch_words,
         proportion > 0) %>%
  ggplot(aes(x = month, y = proportion, shape = item_definition)) +
  geom_point(aes(color = item_definition), size = 3) +
  geom_line()
```



Figuur 5.8: Er zijn nu ook lijnen zichtbaar dankzij `geom_line()`

We zien lijnen in Figuur 5.8, maar er gaan twee dingen mis: (i) de lijnen hebben niet de juiste kleur, ze zijn allemaal zwart, en (ii) als je goed kijkt, zie je dat de lijnen over de punten heenlopen, dat ziet er minder fraai uit. Dit laatste is makkelijk op te lossen door de volgorde van de geoms om te draaien (Figuur 5.9).

```
wordbank_dutch %>%
  filter(item_definition %in% dutch_words,
         proportion > 0) %>%
  ggplot(aes(x = month, y = proportion, shape = item_definition)) +
  geom_line() +
  geom_point(aes(color = item_definition), size = 3)
```



Figuur 5.9: Door de volgorde van geoms om te draaien komen de punten boven de lijnen te liggen.

Dit laat zien dat je het maken van grafieken in `ggplot` (tot een bepaalde hoogte) kunt vergelijken met het schilderen op een doek (canvas): wat je het laatst toevoegt is het best zichtbaar. Een `ggplot` is letterlijk opgebouwd uit lagen (*layers*).

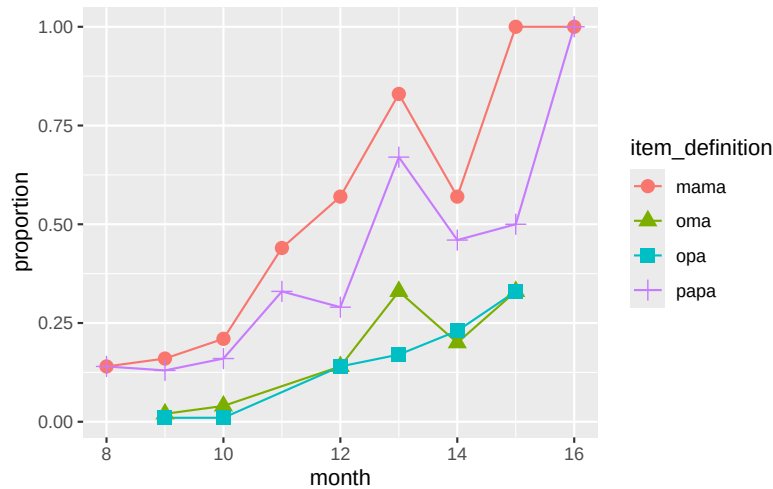
Het eerste probleem (dat de lijnen zwart zijn) laat iets heel fundamenteels zien over de mapping in `aes()`. Zoals we eerder hebben gezien, kun je deze mapping zowel op het niveau van `ggplot()` als op het niveau van het geom specificeren. Dit maakt een wezenlijk verschil. Mappings op het niveau van `ggplot()` worden overgeërfd door latere geoms, tenzij de mapping op het niveau van het geom wordt overschreven. Mappings op het niveau van een geom zijn alleen beschikbaar op dat niveau en dus niet toegankelijk voor andere geoms. Dat is waar het in onze grafiek misgaat. We hebben kleur gespecificeerd op het niveau van `geom_point()` en daardoor is deze mapping niet toegankelijk voor `geom_line()`. Wanneer we deze mapping verhuizen naar het hoogste niveau van `ggplot()` dan zien we dat de lijnen de juiste kleur krijgen (Figuur 5.10).

```
wordbank_dutch %>%
  filter(item_definition %in% dutch_words,
         proportion > 0) %>%
  ggplot(aes(x = month, y = proportion,
```

```

    color = item_definition, shape = item_definition)) + #mapping van kleur nu op d
geom_line() +
geom_point(size = 3)

```



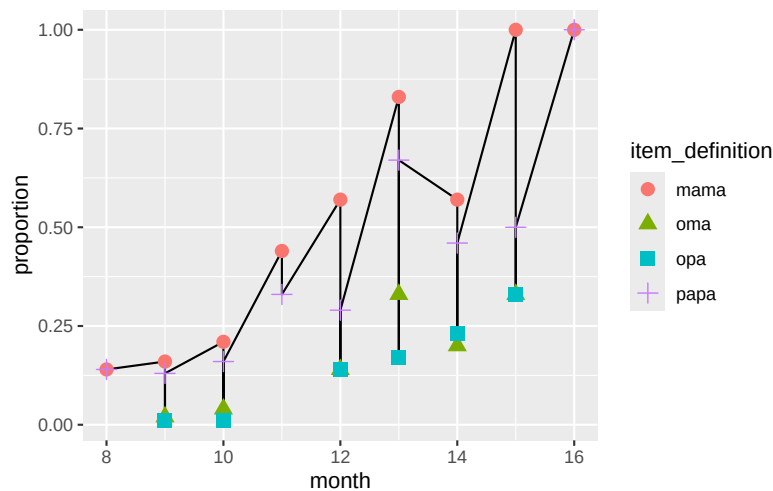
Figuur 5.10: Door de mapping van `color` op het niveau van de functie `ggplot()` te zetten, wordt deze toegankelijk voor alle volgende geoms.

Het is dus heel belangrijk om vooraf goed te bedenken waar je een mapping van een variabele naar een eigenschap van de grafiek (*aesthetics*) allemaal terug wilt zien. Om het belang van de goede plek van de mapping te onderstrepen, laten we zien wat er gebeurt als we beide mappings (`color` en `shape`) van `ggplot()` verhuizen naar het niveau van `geom_point()`.

```

wordbank_dutch %>%
  filter(item_definition %in% dutch_words,
         proportion > 0) %>%
  ggplot(aes(x = month, y = proportion)) +
  geom_line() +
  geom_point(aes(color = item_definition, shape = item_definition), size = 3)

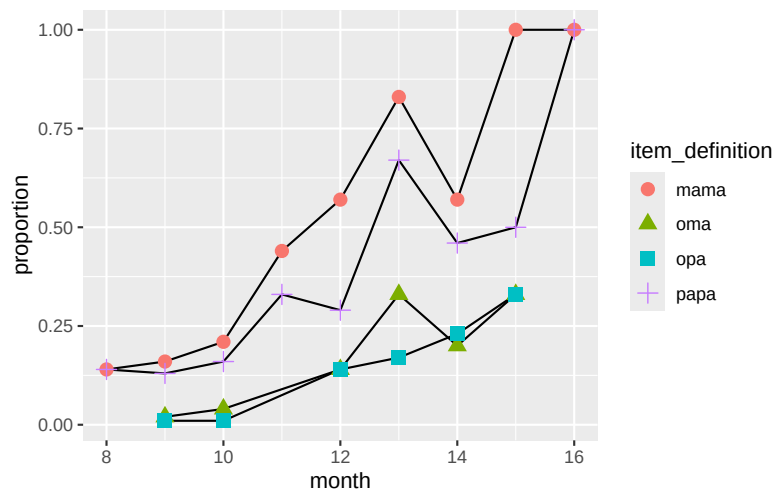
```



Figuur 5.11: Door verplaatsing van de mapping naar het laagste geom zijn de lijnen helemaal de weg kwijt.

De lijn verliest niet alleen zijn kleur, maar is nu ook helemaal de weg kwijt. Dit komt doordat `geom_line()` nu niet meer weet welke punten bij elkaar horen. Eerder kreeg het geom die informatie nog overgeërfd via de mapping van `shape` in `ggplot()`, maar die is nu verdwenen. Om dit op te lossen moeten we die mapping weer terugzetten of we moeten de groepering expliciet maken op het niveau van `ggplot()`. Dat laatste doen we door `item_definition` te mappen op `group` (Figuur 5.12).

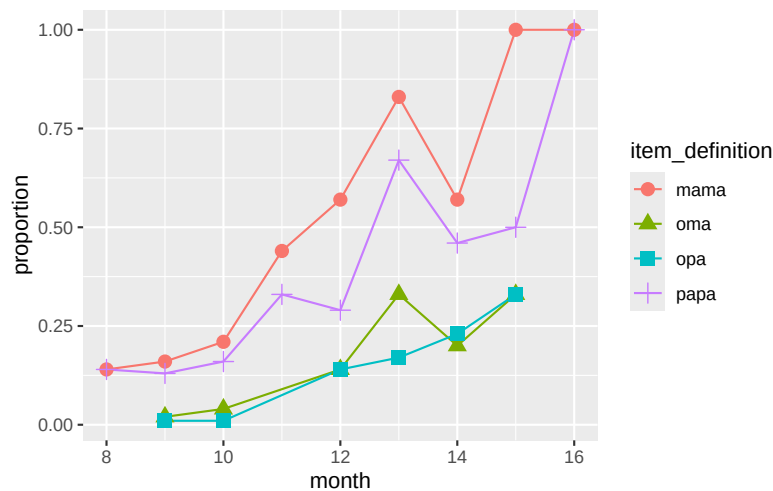
```
wordbank_dutch %>%
  filter(item_definition %in% dutch_words,
         proportion > 0) %>%
  ggplot(aes(x = month, y = proportion, group = item_definition)) +
  geom_line() +
  geom_point(aes(color = item_definition, shape = item_definition), size = 3)
```



Figuur 5.12: Het herstellen van de orde door mapping van **group**.

Dit laat zien dat er dus meerdere manieren zijn om tot een grafiek te komen. Stelregel is daarbij dat als je een bepaalde mapping van een variabele naar een eigenschap van de grafiek op meerdere plekken tot uitdrukking wilt laten komen, dat je die mapping dan zo hoog mogelijk moet specificeren (dus op het niveau van `ggplot()`). In ons geval is dit dus de beste oplossing:

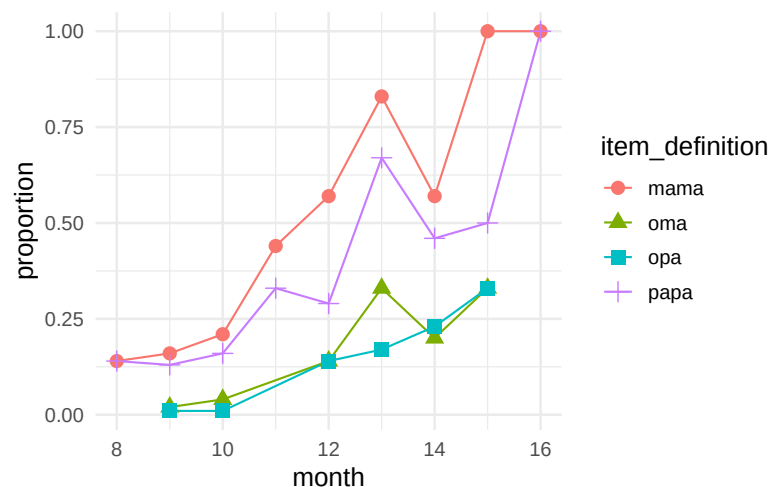
```
wordbank_dutch %>%
  filter(item_definition %in% dutch_words,
         proportion > 0) %>%
  ggplot(aes(x = month, y = proportion,
             color = item_definition, shape = item_definition)) +
  geom_line() +
  geom_point(size = 3)
```



Figuur 5.13: Mappings op de beste plek voor deze grafiek

De data in de grafiek in Figuur 5.13 zien er nu goed uit. We moeten alleen het uiterlijk van de grafiek nog aanpassen. We veranderen eerst de achtergrondkleur. Dit doen we door de *theme* aan te passen. Er zijn binnen `ggplot` meerdere *themes* beschikbaar. We gebruiken hier `theme_minimal()` dat een witte achtergrond met lichte rasterlijnen geeft (Figuur 5.14). Het getal (13) geeft de basisgrootte van de titels in de grafiek.

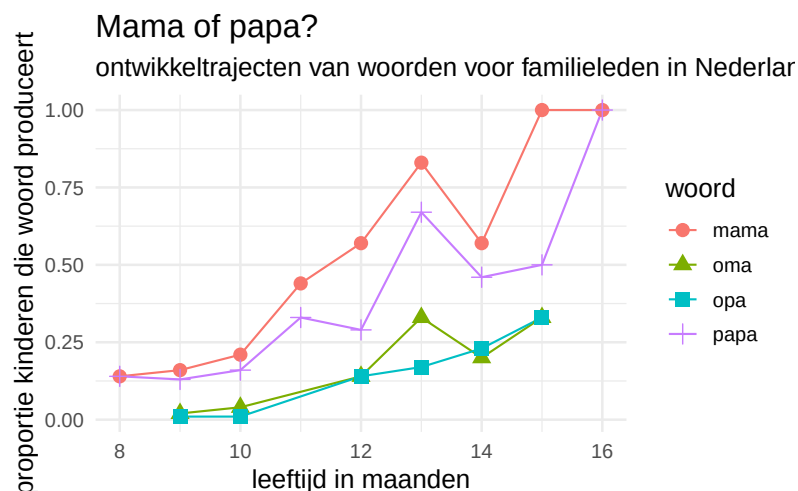
```
wordbank_dutch %>%
  filter(item_definition %in% dutch_words,
         proportion > 0) %>%
  ggplot(aes(x = month, y = proportion,
             color = item_definition, shape = item_definition)) +
  geom_line() +
  geom_point(size = 3) +
  theme_minimal(13)
```



Figuur 5.14: Met `theme_minimal()` krijgt de grafiek een witte achtergrond met lichte rasterlijnen.

De laatste stap is om de namen van de assen en de legenda aan te passen en een titel en ondertitel toe te voegen. Dit doen we via `labs()` (Figuur 5.15).

```
wordbank_dutch %>%
  filter(item_definition %in% dutch_words,
         proportion > 0) %>%
  ggplot(aes(x = month, y = proportion,
             color = item_definition, shape = item_definition)) +
  geom_line() +
  geom_point(size = 3) +
  theme_minimal(13) +
  labs(
    x = "leeftijd in maanden",
    y = "proportie kinderen die woord produceert",
    shape = "woord",
    color = "woord",
    title = "Mama of papa?",
    subtitle = "ontwikkeltrajecten van woorden voor familieleden in Nederlandse kindertaal"
  )
```



Figuur 5.15: Namen van assen en de legenda kun je aanpassen via `labs()`.

Hiermee is het ons stapsgewijs gelukt om onze doelgrafiek (Figuur 5.1) volledig na te maken. Voordat je verder leest: check of je elke regel code in het laatste codeblok voor jezelf uit kunt leggen. Zo niet, kijk dan eerst nog even terug in het hoofdstuk.

5.2 Soorten grafieken

Hieronder geven we alvast een voorbeeld van de verschillende grafieken die je gaat tegenkomen in het vervolg van de cursus en lichten we er in het voorbijgaan nog wat functionaliteiten van `ggplot2` uit. Dit gaat in vogelvlucht. Veel van de dingen die je hier ziet langskomen zullen we in latere hoofdstukken nog verder uitwerken. Probeer voor nu steeds voor jezelf helder te krijgen wat de code doet.

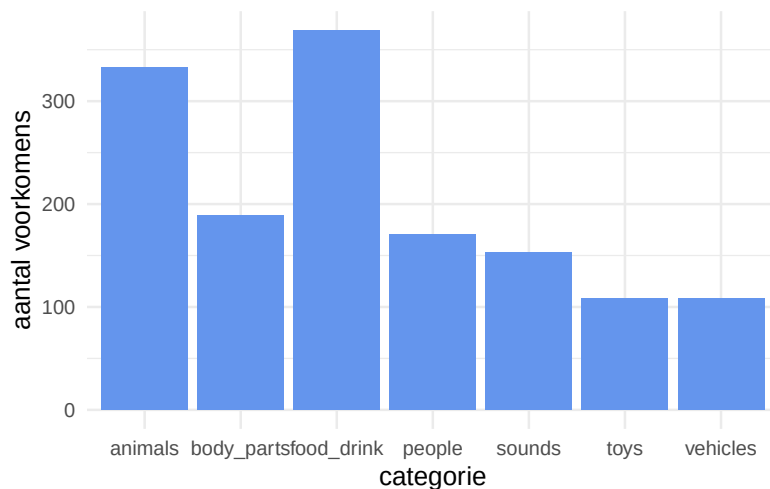
5.2.1 staafdiagrammen

Voor staafdiagrammen gebruiken we de geoms `geom_bar()` of `geom_col()`. Het verschil is dat de eerste zelf tellingen uitvoert op basis van de datatabel (Figuur 5.16) en de tweede (Figuur 5.17) tellingen gebruikt die je eerst zelf hebt aangemaakt. De kleur van een staaf pas je aan via

de *aesthetic* `fill` en `color` wordt gebruikt voor de rand om de staaf heen. In onderstaande figuur hebben we deze vastgezet op een bepaalde kleur (`cornflowerblue`). Dit is een van de standaardkleuren beschikbaar in R. Zie `colours()` voor een overzicht van kleurnamen die je kunt gebruiken. Zie Paragraaf 6.5 voor een uitgebreidere beschrijving van kleurgebruik in `ggplot2`.

```
# Aanmaken van subset van woordcategorieën die we willen gaan tellen
dutch_categories <- c("animals", "sounds", "people", "vehicles", "toys", "food_drink", "body_parts")
```

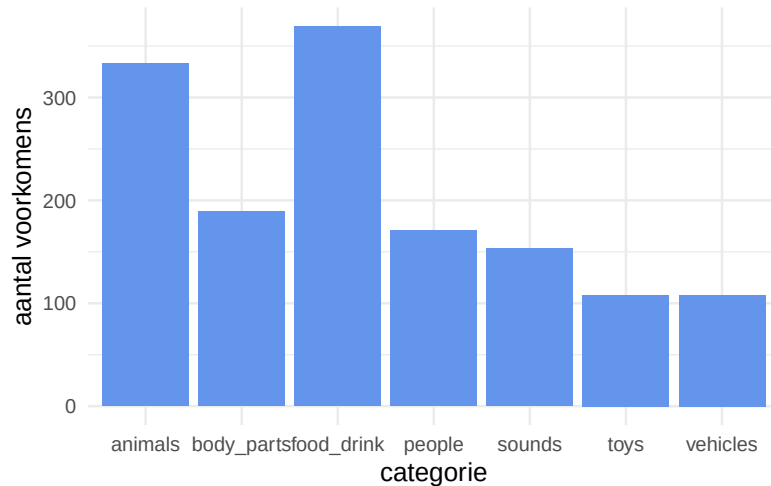
```
wordbank_dutch %>%
  filter(category %in% dutch_categories) %>%
  ggplot(aes(x = category)) +
  geom_bar(fill = "cornflowerblue") +
  theme_minimal(13) +
  labs(
    x = "categorie",
    y = "aantal voorkomens"
  )
```



Figuur 5.16: Een staafdiagram met `geom_bar()` waarbij R zelf de tellingen uitvoert

```
wordbank_dutch %>%
  filter(category %in% dutch_categories) %>%
  count(category) %>% # tellen hoe vaak elke categorie voorkomt in de data
```

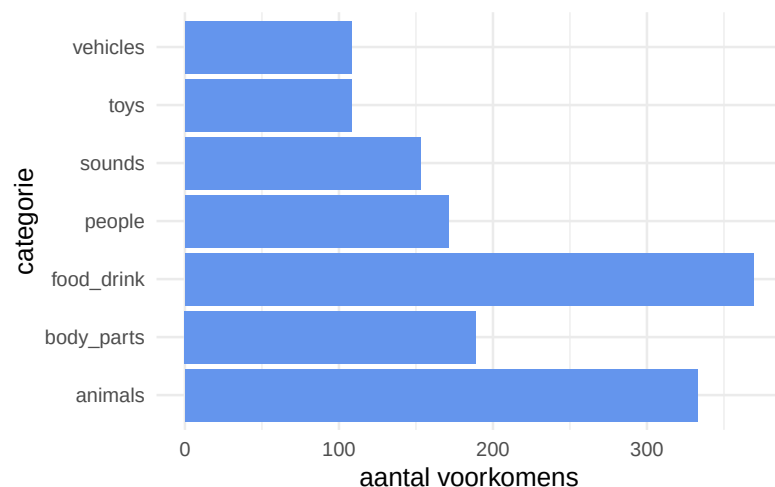
```
ggplot(aes(x = category, y = n)) +
  geom_col(fill = "cornflowerblue") +
  theme_minimal(13) +
  labs(
    x = "categorie",
    y = "aantal voorkomens"
  )
```



Figuur 5.17: Zelfde staafdiagram maar nu met `geom_col()` waarbij we eerst zelf moeten tellen.

Om een liggend staafdiagram te maken (Figuur 5.18), vaak beter leesbaar, kun je de functie `coord_flip()` toevoegen. Deze functie draait de assen om (x wordt y en y wordt x).

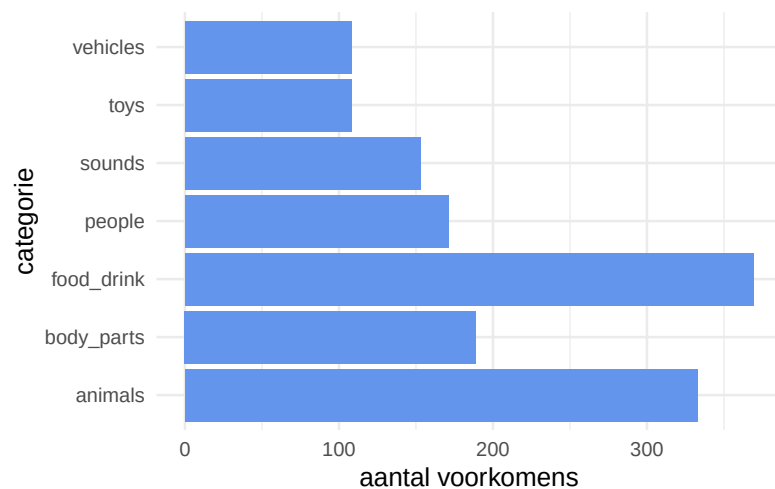
```
wordbank_dutch %>%
  filter(category %in% dutch_categories) %>%
  ggplot(aes(x = category)) +
  geom_bar(fill = "cornflowerblue") +
  coord_flip() + # omdraaien van de assen
  theme_minimal(13) +
  labs(
    x = "categorie",
    y = "aantal voorkomens"
  )
```



Figuur 5.18: Een liggend staafdiagram gemaakt met `coord_flip()`

Bij `geom_col()` (en andere geoms waarbij een `x` en/of `y` gespecificeerd kan worden) kun je voor hetzelfde resultaat ook de mapping van variabelen naar `x` en `y` omdraaien i.p.v. `coord_flip()` te gebruiken. Dit kan nuttig zijn als je ook een andere `coord_`-functie wilt gebruiken. Er is er namelijk maar een per grafiek toegestaan.

```
wordbank_dutch %>%
  filter(category %in% dutch_categories) %>%
  count(category) %>%
  ggplot(aes(x = n, y = category)) + # x en y zijn gewisseld t.o.v. eerder
  geom_col(fill = "cornflowerblue") +
  theme_minimal(13) +
  labs( #ook hier x en y gewisseld om de juiste namen op de assen te krijgen
    y = "categorie",
    x = "aantal voorkomens"
  )
```

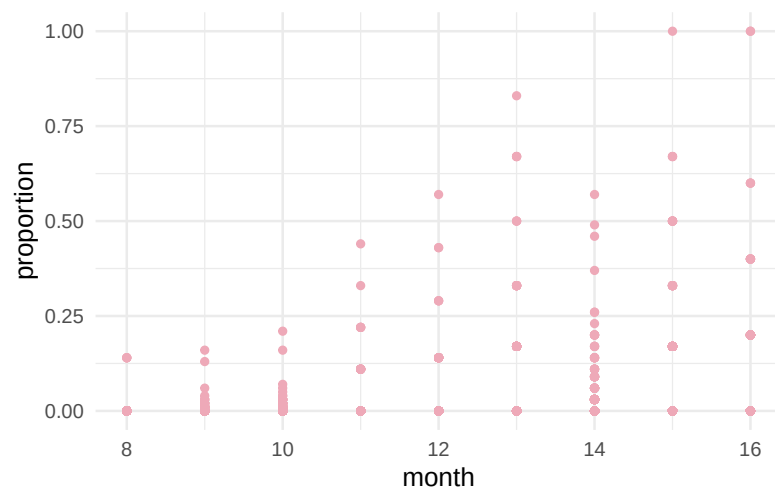


Figuur 5.19: Een liggend staafdiagram gemaakt door de mapping naar de assen te verwisselen

5.2.2 dotplots en scatterplots

Eerder in dit hoofdstuk hebben we gezien dat we `geom_point()` kunnen gebruiken om *dotplots* of *scatterplots* te maken. Hierbij geeft elk punt een observatie in de data weer. In Figuur 5.20 is voor elk kind in de dataset `wordbank_dutch` de proportie woorden die het kind kende toen de vragenlijst werd ingevuld uitgezet tegen de leeftijd in maanden op dat moment.

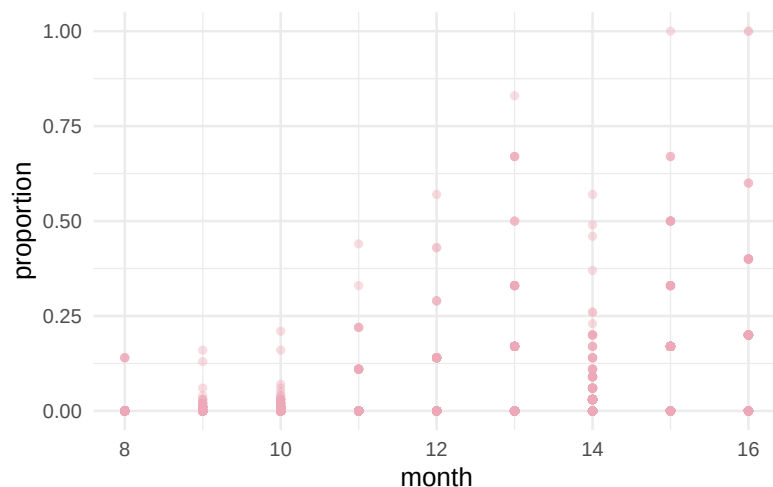
```
wordbank_dutch %>%
  ggplot(aes(x = month, y = proportion)) +
  geom_point(color = "pink2") +
  theme_minimal(13)
```



Figuur 5.20: Een scatterplot

Wanneer er veel observaties worden afgebeeld loop je bij dit soort grafieken het risico op *overplotting*, veel datapunten worden dan over elkaar heen afgebeeld. Hierdoor is het aantal observaties niet goed zichtbaar. Je ziet namelijk geen verschil tussen bijvoorbeeld 1 observatie of 10 observaties met dezelfde waarde. Om dit probleem (enigszins) tegen te gaan kun je de doorzichtigheid van punten veranderen. Wanneer je punten doorzichtiger maakt, zullen plekken waar zich veel observaties ophouden uiteindelijk donkerder worden dan plekken met weinig observaties. De doorzichtigheid van geoms kun je instellen via de *aesthetic* `alpha`. De `alpha`-waarde ligt tussen 0 (doorzichtig) en 1 (ondoorzichtig). In Figuur 5.21 hebben we die ingesteld op de waarde `.4`. Vergelijk dit figuur maar eens met Figuur 5.20.

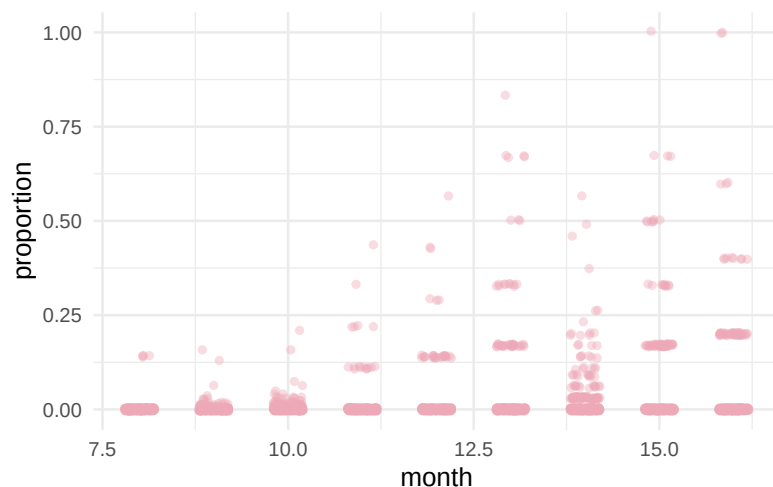
```
wordbank_dutch %>%
  ggplot(aes(x = month, y = proportion)) +
  geom_point(color = "pink2", alpha = .4) +
  theme_minimal(13)
```



Figuur 5.21: Door de **alpha**-waarde van de punten aan te passen wordt duidelijker waar zich meerdere observaties bevinden.

Een alternatief om overplotting tegen te gaan is door gebruik te maken van *jitter*. Hierbij wordt er wat *ruis* toegevoegd aan elke observatie en zal deze iets van positie veranderen. Je kunt hiervoor `geom_jitter()` gebruiken waarbij je met het argument `width` aangeeft hoeveel ruis je wilt toevoegen (tussen 0 (weinig) en 1 (veel)). Je kunt deze functie ook gebruiken in combinatie met het toevoegen van doorzichtigheid, zoals in Figuur 5.22. Voorzichtigheid is wel geboden bij dit geom, omdat het de data enigszins aanpast. Dat zie je bijvoorbeeld ook als je naar de assen in Figuur 5.22 kijkt. Die geven nu continue waarden weer, terwijl de variabele `month` discreet gemeten is.

```
wordbank_dutch %>%
  ggplot(aes(x = month, y = proportion)) +
  geom_jitter(color = "pink2", alpha = .4, width = .2) +
  theme_minimal(13)
```

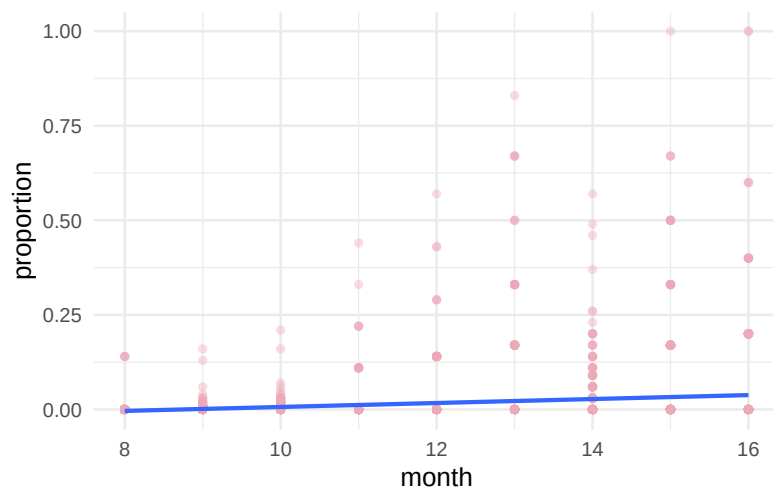


Figuur 5.22: Met `geom_jitter()` kun je de positie van de punten enigszins aanpassen. Hierdoor wordt duidelijker waar zich meerdere observaties bevinden.

Om het verband tussen twee variabelen in een scatterplot zichtbaar te maken kun je een *trendlijn* (of *regressielijn*) toevoegen. Dit doe je met het geom `geom_smooth()`. Door het argument `method = "lm"` mee te geven zorg je dat er een rechte lijn wordt getrokken door de datapunten (Figuur 5.23). We voegen hier argument `se = FALSE` toe om de onzekerheidsmarge rondom de lijn te onderdrukken.

```
wordbank_dutch %>%
  ggplot(aes(x = month, y = proportion)) +
  geom_point(color = "pink2", alpha = .4) +
  geom_smooth(method = "lm", se = FALSE) +
  theme_minimal(13)
```

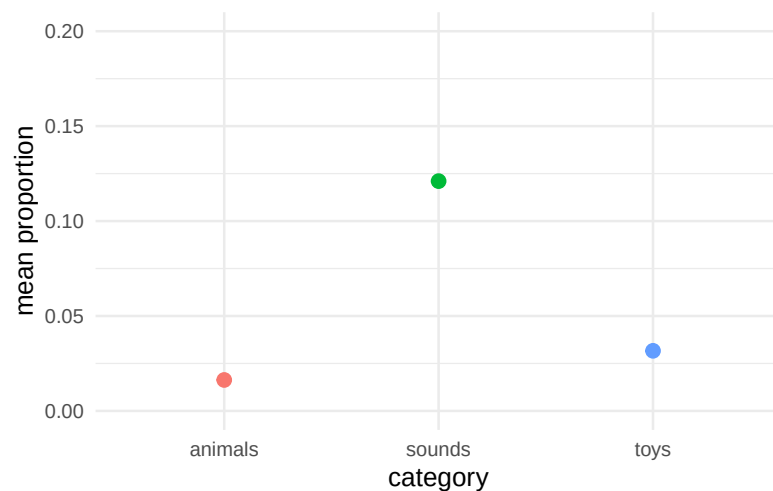
```
`geom_smooth()` using formula = 'y ~ x'
```



Figuur 5.23: Een trendlijn kun je toevoegen met `geom_smooth`.

Je kunt *dotplots* met `geom_point()` ook gebruiken om berekende statistieken, zoals het gemiddelde per categorie, weer te geven (Figuur 5.24). Hiervoor zul je wel eerst zelf deze statistieken moeten berekenen, bijv. met een combinatie van de functies `group_by()` en `summarize()`. Meer hierover in Hoofdstuk 7. Met de toevoeging `ylim()` hebben we in Figuur 5.24 het bereik van de y-as ingeperkt van 0 tot 0.2.

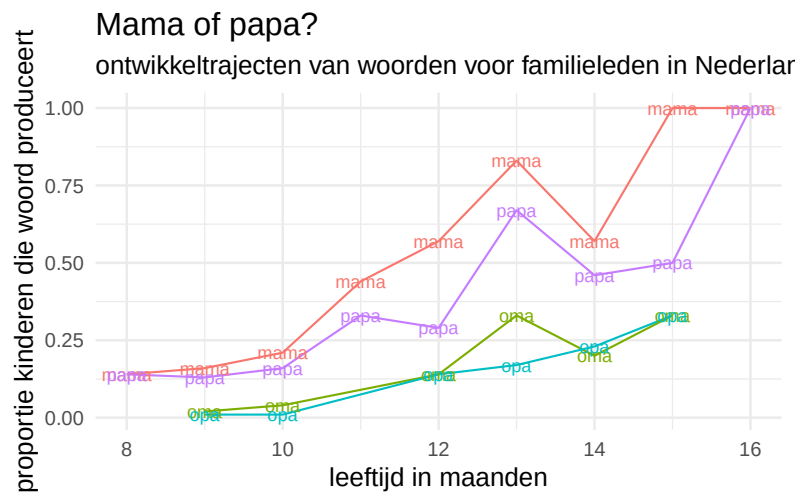
```
wordbank_dutch %>%
  filter(category %in% c("animals", "sounds", "toys")) %>%
  group_by(category) %>%
  summarize(mean_proportion = mean(proportion)) %>%
  ggplot(aes(x = category, y = mean_proportion)) +
  geom_point(aes(color = category), size = 3, show.legend = FALSE) +
  ylim(0, .2) + #inperken van bereik y-as
  labs(y = "mean proportion") +
  theme_minimal(13)
```



Figuur 5.24: Een dotplot met `geom_point()` waarbij een gemiddelde waarde per categorie wordt weergegeven

Tot slot, soms, en zeker in taalwetenschappelijk onderzoek, wil je in plaats van punten woorden zichtbaar maken. Het `geom_text()` geeft je die mogelijkheid. In Figuur 5.25 hebben we dit toegepast op onze doelgrafiek aan het begin van het hoofdstuk. Onder `theme()` hebben we hier bovendien toegevoegd dat we de legenda willen verwijderen, want met de woorden in de grafiek is deze overbodig geworden.

```
wordbank_dutch %>%
  filter(item_definition %in% dutch_words,
         proportion > 0) %>%
  ggplot(aes(x = month, y = proportion, color = item_definition)) +
  geom_line() +
  geom_text(aes(label = item_definition), size = 3) +
  theme_minimal(13) +
  labs(
    x = "leeftijd in maanden",
    y = "proportie kinderen die woord produceert",
    title = "Mama of papa?",
    subtitle = "ontwikkeltrajecten van woorden voor familieleden in Nederlandse kindertaal"
  ) +
  theme(legend.position = "none") # legenda verwijderen
```

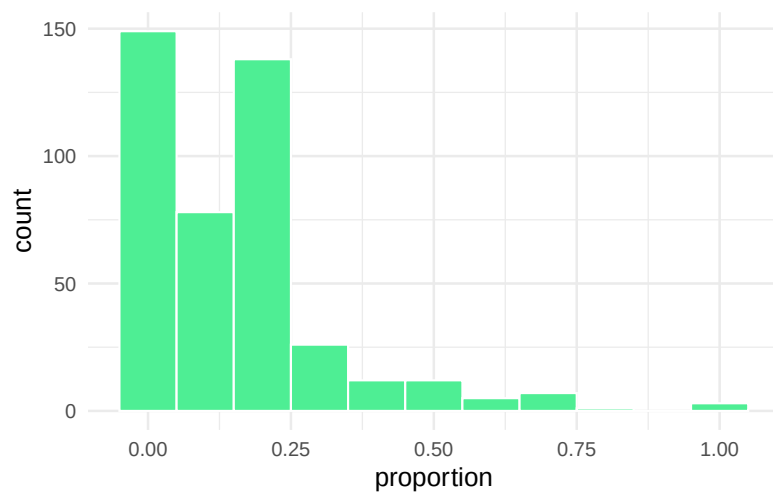


Figuur 5.25: Met `geom_text()` kun je woorden zichtbaar maken.

5.2.3 verdelingen zichtbaar maken

Een verdeling van een numerieke variabele kun je zichtbaar maken met een histogram via `geom_histogram()` (Figuur 5.26). Net als bij `geom_bar()` (en `geom_col()`) kun je hier de kleur van de staven en de rand aanpassen met respectievelijk `fill` en `color`.

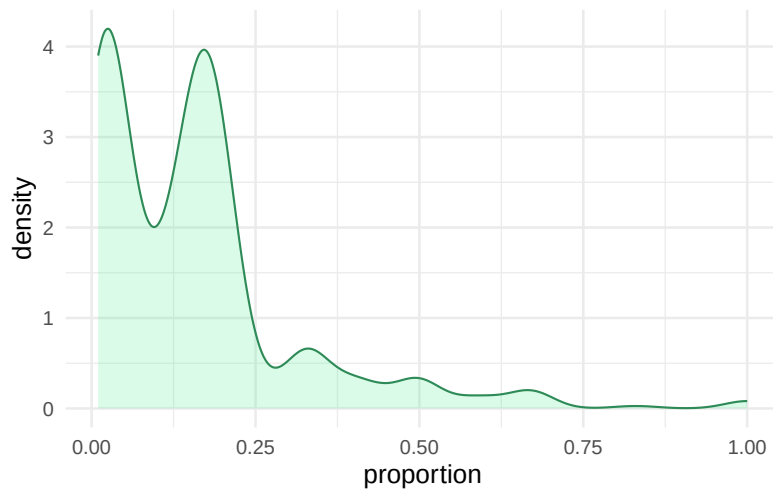
```
wordbank_dutch %>%
  filter(proportion > 0) %>%
  ggplot(aes(x = proportion)) +
  geom_histogram(binwidth = .1, fill = "seagreen2", color = "white") +
  theme_minimal(13)
```



Figuur 5.26: Een histogram gemaakt met `geom_histogram()`

Een alternatief is een *density plot* waarvoor `geom_density()` gebruikt wordt. Dit levert een wat vloeiendere variant op van een histogram (Figuur 5.27).

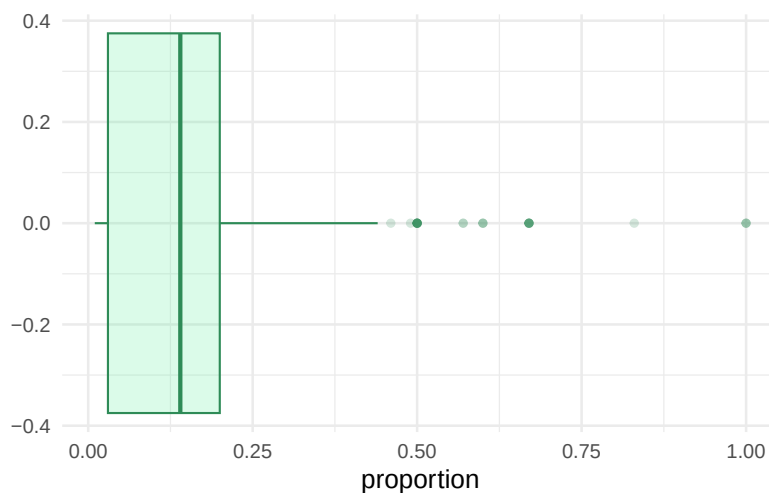
```
wordbank_dutch %>%
  filter(proportion > 0) %>%
  ggplot(aes(x = proportion)) +
  geom_density(color = "seagreen4", fill = "seagreen2", alpha = .2) +
  theme_minimal(13)
```



Figuur 5.27: Een density plot via `geom_density()`

Het laatste alternatief voor verdelingen van numerieke variabelen is het *boxplot* (Figuur 5.28). Zoals je misschien al kon raden, hebben we hiervoor `geom_boxplot()` tot onze beschikking. Boxplots zullen we uitgebreider bespreken in Hoofdstuk 7.

```
wordbank_dutch %>%
  filter(proportion > 0) %>%
  ggplot(aes(x = proportion)) +
  geom_boxplot(color = "seagreen4", fill = "seagreen2", alpha = .2) +
  theme_minimal(13)
```



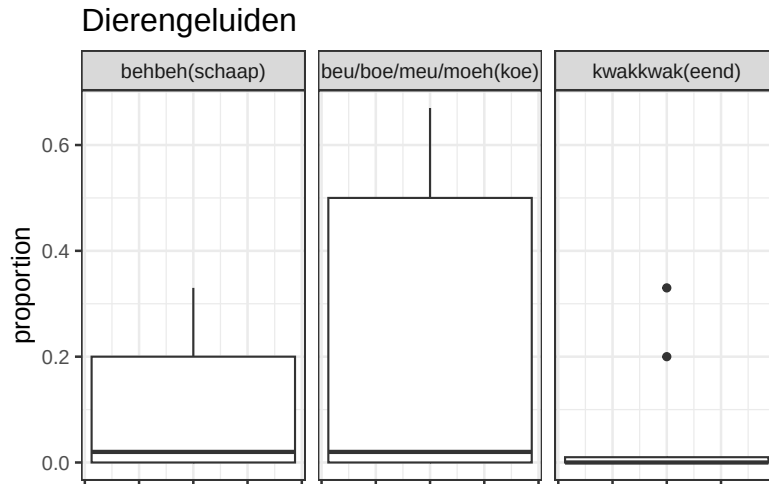
Figuur 5.28: Met `geom_boxplot()` kun je een boxplot maken.

Soms wil je dezelfde grafiek maken voor verschillende levels van een andere variabele. Hiervoor kun je `facet_wrap()` gebruiken. Het resultaat is een *trellis plot* waarbij ieder level zijn eigen grafiek krijgt. In Figuur 5.29 is dit gedaan om voor 3 verschillende dierengeluiden die kinderen zouden kunnen nadoen een boxplot te maken.

```
# Aanmaken van subset van dierengeluiden
animal_sounds <- c("behbeh(schaap)", "beu/boe/meu/moeh(koe)", "kwakkwak(eend)")

wordbank_dutch %>%
  filter(item_definition %in% animal_sounds) %>%
```

```
ggplot(aes(y = proportion)) +
  geom_boxplot() +
  facet_wrap(~ item_definition) +
  labs(title = "Dierengeluiden") +
  theme_bw(13) +
  theme(axis.text.x = element_blank()) # verwijderen getallen op de x-as
```



Figuur 5.29: De toevoeging `facet_wrap()` maakt het mogelijk om dezelfde grafiek voor verschillende levels van een variabele te produceren.

Verder leren

- In [hoofdstuk 1](#) van Wickham et al. (2023) vind je een andere uitwerking van veel functies die in dit hoofdstuk aan bod zijn gekomen.
- [Hoofdstuk 3](#) van Healy (2026) geeft een goede uiteenzetting van de anatomie van `ggplots`.
- het [ggplot2 cheatsheet](#) geeft een overzicht van de werking van `ggplot2`. Voor nu misschien nog wat overweldigend, maar later zeker de moeite waard.

Wickham, H., Çetinkaya-Rundel, M., & Grolemund, G. (2023). *R for Data Science* (2de dr.). O'Reilly. <https://r4ds.hadley.nz/>

Healy, K. (2026). *Data Visualisation: A Practical Introduction*. (2de dr.). Princeton University Press. <https://socviz.co/>

Nu zou je in staat moeten zijn Practicum 3 te maken.

6 Categorische variabelen

In dit hoofdstuk richten we ons op het inzichtelijk maken van data van categorische variabelen. De beschrijvende statistiek van categorische data is betrekkelijk eenvoudig: we kunnen de voorkomens van de categorieën tellen of de proportie die een categorie inneemt zichtbaar maken. Deze informatie kunnen we vervolgens zichtbaar maken in een grafiek. Uitdagingen zitten hem in het ordenen van de data in een grafiek. Hier leer je meer over de werking van factoren en hoe je die kunt manipuleren. We gebruiken dit hoofdstuk ook meteen om meer inzicht te krijgen in het gebruik van kleuren in je met `ggplot2` gemaakte grafieken.

Na afloop van dit hoofdstuk:

- kun je categorische variabelen (factoren) visualiseren met verschillende soorten barplots in `ggplot2`;
- kun je de levels van categorische variabelen (factoren) herordenen en van naam veranderen;
- kun je de kleuren van je grafieken naar wens aanpassen.

codeboek

In deze cursus leer je werken met zo'n 100 veelgebruikte functies in R. Het is belangrijk om voor jezelf een goed overzicht te houden van deze functies. Werk hier actief aan door een eigen codeboek aan te maken. Voeg tijdens of na afloop van het bestuderen van dit hoofdstuk de codes toe aan jouw codeboek.

De data in dit hoofdstuk

We beginnen met het inlezen van de data die we dit hoofdstuk gaan analyseren. We hebben een dataset `wordbank_WS_demographics.csv` met daarin demografische gegevens over de kinderen voor wie een vragenlijst over hun woordenschat is ingevuld. Naast welke taal ze spreken gaat het dan bijvoorbeeld om het hoeveelste kind ze in het gezin zijn en wat het opleidingsniveau van hun ouder/verzorger is. We lezen deze dataset eerst in:

```
wordbank_demo <- read_csv("_datasets/wordbank_WS_demographics.csv")
```

De belangrijkste informatie over de dataset maken we vervolgens zichtbaar met `glimpse()`:

```
glimpse(wordbank_demo)
```

```
Rows: 51,340
Columns: 22
$ downloaded      <date> 2025-07-14, 2025-07-14, 2025-07-14, 2025-07-14, ~
$ language        <chr> "Cantonese", "Cantonese", "Cantonese", "Cantonese~
$ form           <chr> "WS", "WS", "WS", "WS", "WS", "WS", "WS", "WS", "~
$ dataset_name    <chr> "Tardif", "Tardif", "Tardif", "Tardif", "Tardif",~
$ child_id        <dbl> 27998, 27999, 28000, 28001, 28002, 28003, 28004, ~
$ age             <dbl> 17, 29, 16, 16, 16, 16, 16, 16, 16, 16, 17, 17, 1~
$ comprehension  <dbl> 19, 600, 27, 29, 8, 4, 7, 36, 3, 74, 113, 61, 25,~
$ production      <dbl> 19, 600, 27, 29, 8, 4, 7, 36, 3, 74, 113, 61, 25,~
$ is_norming      <lgl> TRUE, TRUE, TRUE, TRUE, TRUE, TRUE, TRUE, TRUE, T~
$ birth_order     <chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ caregiver_education <chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ ethnicity       <chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ race            <chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ sex             <chr> "Male", "Female", "Male", "Male", "Male", "Female~
$ birth_weight    <lgl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ born_early_or_late <lgl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ gestational_age <lgl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ zygosity        <lgl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ language_exposures <chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ health_conditions <lgl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
```

```
$ monolingual          <lgl> TRUE, TRUE, TRUE, TRUE, TRUE, TRUE, TRUE, TRUE, T~
$ typically_developing <lgl> TRUE, TRUE, TRUE, TRUE, TRUE, TRUE, TRUE, TRUE, T~
```

De dataset heeft meer dan 50.000 observaties en telt 22 variabelen. Het overzicht laat ook meteen zien dat we voor veel variabelen missing datapunten (NAs) hebben. Om het allemaal wat handzamer te maken, perken we de dataset in. We gaan alleen kijken naar de Europese Romaanse talen Frans, Italiaans, Portugees en Spaans en daarbij richten we ons op de variabelen `language` (welke taal het kind leert), `age` (leeftijd in maanden), `birth_order` (hoeveelste kind in het gezin), `sex` (geslacht van het kind) en `caregiver_education` (het opleidingsniveau van de ouder/verzorger). We zetten ook meteen alle variabelen die als type `character` hebben om naar een `factor`:

```
wordbank_demo_romance <- wordbank_demo %>%
  select(language, age, birth_order, sex, caregiver_education) %>%
  filter(language %in% c("French (French)", " Italian", "Portuguese (European)", "Spanish (E
  mutate(across(where(is.character), as.factor))
```

Laten we nu eens tellen hoe vaak elke taal voorkomt in onze dataset:

```
wordbank_demo_romance %>%
  count(language)
```

```
# A tibble: 3 x 2
  language          n
  <fct>            <int>
1 French (French)  1233
2 Portuguese (European) 3012
3 Spanish (European)   593
```

Wat opvalt is dat het Italiaans niet in de dataset voor lijkt te komen. Als we echter de oorspronkelijke dataset bekijken dan blijken er wel degelijk observaties voor het Italiaans te zijn (752 in totaal):

```
wordbank_demo %>%
  filter(language == "Italian") %>%
  nrow()
```

```
[1] 752
```

Wat is hier fout gegaan? Als we de eerder gebruikte code goed bekijken dan zien we dat we een typefout hebben gemaakt. Er stond een extra spatie voor *Italian* " Italian". Zoals we weten, neemt R alles letterlijk en aangezien er geen talen met de naam " Italian" bestaan krijgen we geen resultaten. Een extra herinnering dat we precies moeten blijven werken.

We herstellen dit snel en krijgen nu wel de goede tellingen:

```
wordbank_demo_romance <- wordbank_demo %>%
  select(language, age, birth_order, sex, caregiver_education) %>%
  filter(language %in% c("French (French)", "Italian", "Portuguese (European)", "Spanish (European)"))
  mutate(across(where(is.character), as.factor))
```

```
wordbank_demo_romance %>%
  count(language)
```

```
# A tibble: 4 x 2
```

	language	n
	<fct>	<int>
1	French (French)	1233
2	Italian	752
3	Portuguese (European)	3012
4	Spanish (European)	593

Zoals gezegd bieden categorische variabelen niet veel opties qua beschrijvende statistiek. We kunnen de tellingen nog omzetten in proporties (en eventueel percentages). Dit doen we door voort te bouwen op onze datatabel met tellingen. Met de functie `sum()` kunnen we getallen bij elkaar optellen. In dit geval doen we dat met alle getallen in de kolom `n`. Dit levert ons het totaal aantal voorkomens van talen op. Wanneer we de variabele `n` nu

delen door dit totaal dan krijgen we de proportie die elke taal inneemt in onze dataset. We zien in de output dat R met één commando `n/sum(n)` voor elke waarde van `n` de bijbehorende proportie geeft. We hoeven dit dus niet voor elke taal apart te doen. We ronden de proporties tot slot af op twee decimalen met `round()`.

```
counts_romance <- wordbank_demo_romance %>%
  count(language) %>%
  mutate(proportion = n/sum(n),
         proportion = round(proportion, 2))

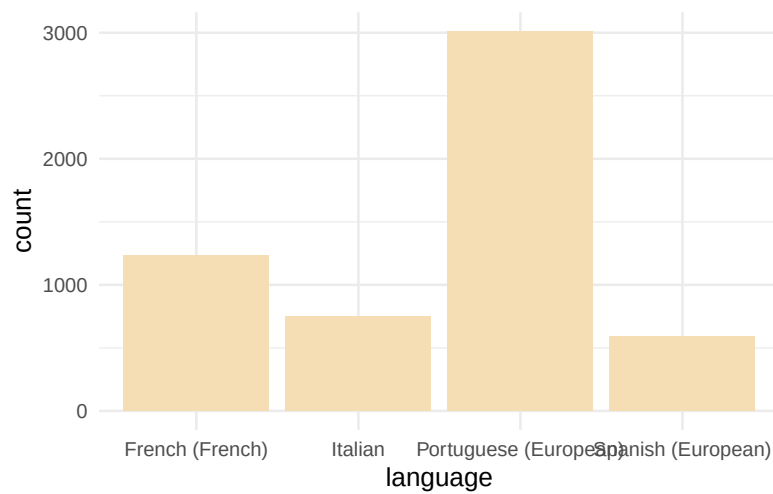
counts_romance
```

```
# A tibble: 4 x 3
  language          n proportion
  <fct>          <int>     <dbl>
1 French (French)   1233      0.22
2 Italian           752      0.13
3 Portuguese (European) 3012      0.54
4 Spanish (European)  593      0.11
```

6.1 Eigenaardigheden van barplots

De beschrijvende statistieken van categorische variabelen worden traditioneel zichtbaar gemaakt met staafdiagrammen, hoewel we hier ook punten zouden kunnen gebruiken. In `ggplot` gebruiken we hiervoor het `geom_bar()` of `geom_col()`. Als je de tellingen door R wilt laten uitvoeren tijdens het maken van de grafiek kies je voor `geom_bar()`.

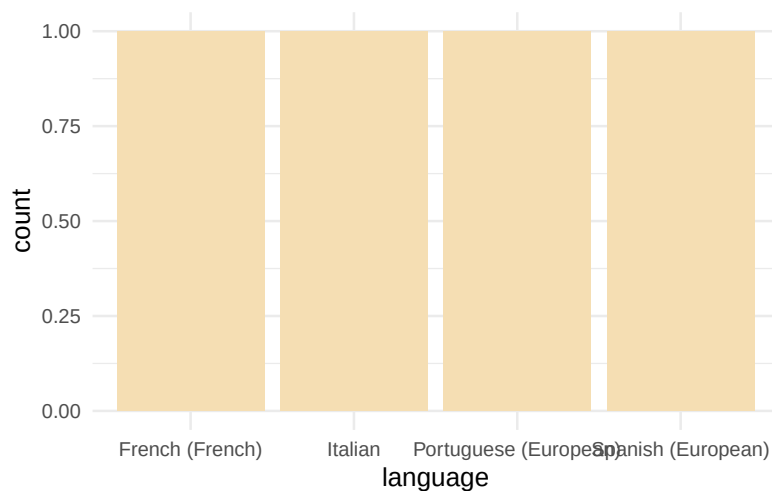
```
wordbank_demo_romance %>%
  ggplot(aes(x = language)) +
  geom_bar(fill = "wheat") + #kleur van de staven bepalen
  theme_minimal(13)
```



Figuur 6.1: Staafdiagram gemaakt met `geom_bar()`

We hebben in Figuur 6.1 alleen een variabele meegegeven voor op de x-as, `geom_bar` berekent achter de schermen zelf de aantallen per categorie voor op de y-as. De standaard is hier `stat(count)` en deze default zien we terug in de naam van de y-as. Dat `geom_bar()` puur het aantal voorkomens in de dataset telt, zien we duidelijk als we onze datatabel `counts_romance` waarin we zelf de aantallen hebben berekend als input meegegeven. We krijgen dan iets dat op het eerste gezicht mogelijk onverwacht is:

```
counts_romance %>%
  ggplot(aes(x = language)) +
  geom_bar(fill = "wheat") +
  theme_minimal(13)
```

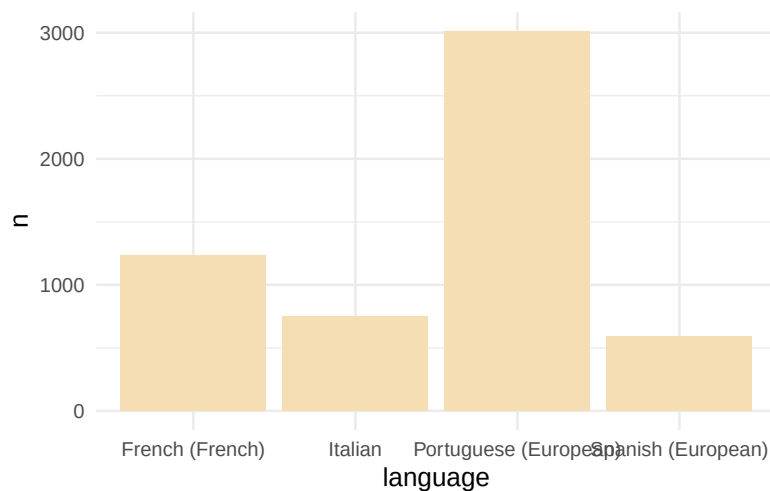


Figuur 6.2: Het geom `geom_bar()` telt het aantal voorkomens in een dataset. In de dataset `counts_romance` komt elke taal maar één keer voor.

Dit staafdiagram (Figuur 6.2) laat zien dat elke taal één keer voorkomt in de dataset. Dit klopt inderdaad, want de datatabel telt voor elke taal één regel. Er zijn twee opties om onze eigen telling te laten zien (i) de statistiek van `geom_bar()` veranderen, (ii) het geom `geom_col()` gebruiken.

Zoals gezegd rekent `geom_bar()` standaard met de statistiek `count`. Deze statistiek kun je veranderen in het geom door `stat =` te specificeren als `identity`. Het geom gaat nu niet meer zelf tellen, maar gebruikt de waarden uit de variabele `n` in onze eigen telling. We moeten dan wel bij de aesthetics mapping van `ggplot` aangeven dat we deze variabele `n` willen afbeelden op de y-as (zie Figuur 6.3):

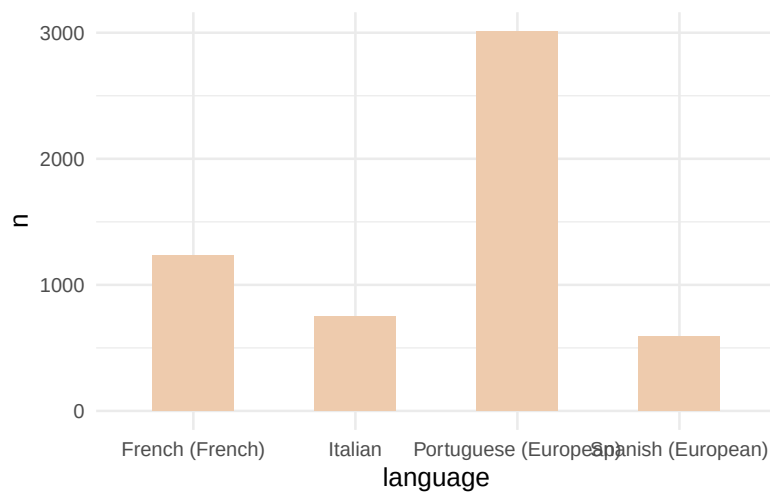
```
counts_romance %>%
  ggplot(aes(x = language, y = n)) +
  geom_bar(stat = "identity", fill = "wheat") +
  theme_minimal(13)
```



Figuur 6.3: Door `stat = "identity"` toe te voegen kunnen we `geom_bar()` dwingen om niet zelf te gaan tellen, maar een waarde van een variabele uit de dataset te gebruiken (hier de variabele `n`).

Het tweede alternatief `geom_col()` is eigenlijk een afkorting voor `geom_bar(stat = "identity")`. Om dit geom te gebruiken moet je dus altijd eerst zelf de tellingen uitvoeren. Die geef je mee als variabele. Dit geom neemt dus altijd twee variabelen als argument onder `aes()`. Het geom `geom_col()` geeft je volledige controle over welke getallen worden afgebeeld. Je moet ze alleen wel eerst zelf berekenen. We zien dat dit geom dezelfde grafiek oplevert (vergelijk Figuur 6.4 met Figuur 6.3). We hebben de kleur en de breedte van de staven aangepast om de twee varianten uit elkaar te houden.

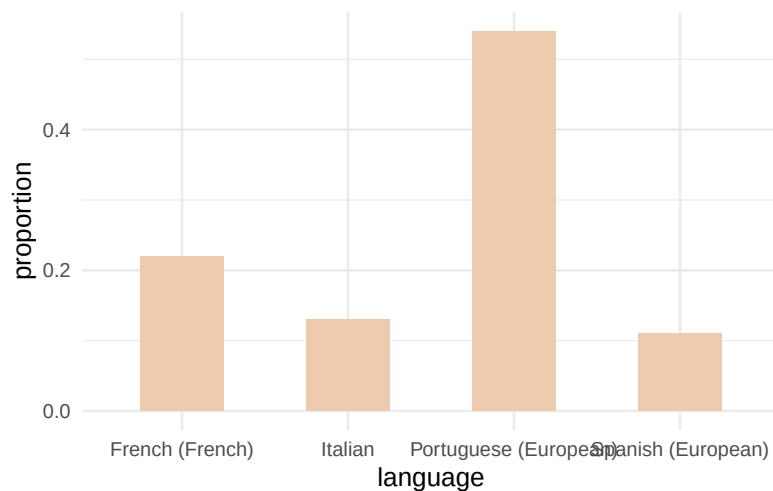
```
wordbank_demo_romance %>%
  count(language) %>%
  ggplot(aes(x = language, y = n)) +
  geom_col(fill = "peachpuff2", width = .5) + #met width pas je de breedte van de staven aan
  theme_minimal(13)
```



Figuur 6.4: Met `geom_col()` kun je zelf bepalen welke getallen worden afgebeeld. De breedte van de staven kun je aanpassen met het argument `width`.

Aangezien je `geom_col()` zelf de getallen die je wilt afbeelden moet aanleveren is het ook meteen te gebruiken voor proporties. Die hadden we al toegevoegd aan onze datatabel `counts_romance`. We veranderen hiervoor alleen de mapping van de y-variabele naar `y = proportion` in de `aes()` van `ggplot()`, zie Figuur 6.5:

```
counts_romance %>%
  ggplot(aes(x = language, y = proportion)) +
  geom_col(fill = "peachpuff2", width = .5) +
  theme_minimal(13)
```



Figuur 6.5: Andere berekende waarden, zoals hier proporties, zijn ook makkelijk zichtbaar te maken met `geom_col()`.

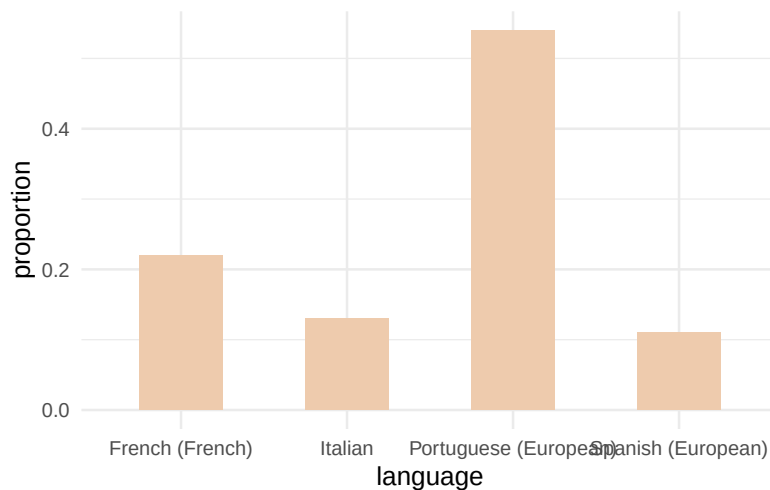
6.2 Herordenen van levels

In al onze grafieken zijn de talen op de x-as in alfabetische volgorde afgebeeld. Dit komt omdat R bij het aanmaken van een factor met `as.factor()` de levels van de factor alfabetisch ordent. De grafiek volgt dus precies de informatie in de factor. Dit is alleen niet wat we altijd willen. Wanneer er geen logische volgorde in een factor zit (wat bijvoorbeeld wel zo is als het om een geordende factor (of ordinale variabele) gaat, zie Paragraaf 6.2.1) is alfabetisch niet altijd de handigste optie om inzicht in de data te krijgen. Zeker wanneer er veel factorlevels zijn, kan dit ervoor zorgen dat de grafiek nogal chaotisch oogt. Om dit te verbeteren kunnen we de levels ordenen op basis van hun waarde (aantal keer dat het level voorkomt of de proportie). In andere woorden, we ordenen van groot naar klein (of van klein naar groot). Om dit voor elkaar te krijgen moeten we de levels anders ordenen. Het heeft dus geen zin de data in de datatabel in volgorde van groot naar klein te zetten met behulp van `arrange()`. Als we dat namelijk doen, dan krijgen we nog steeds dezelfde grafiek. Vergelijk Figuur 6.6 maar met Figuur 6.5.

```
counts_romance %>%
  arrange(n)
```

```
# A tibble: 4 x 3
  language          n proportion
  <fct>          <int>      <dbl>
1 Spanish (European)    593      0.11
2 Italian              752      0.13
3 French (French)     1233      0.22
4 Portuguese (European) 3012      0.54
```

```
counts_romance %>%
  arrange(n) %>%
  ggplot(aes(x = language, y = proportion)) +
  geom_col(fill = "peachpuff2", width = .5) +
  theme_minimal(13)
```

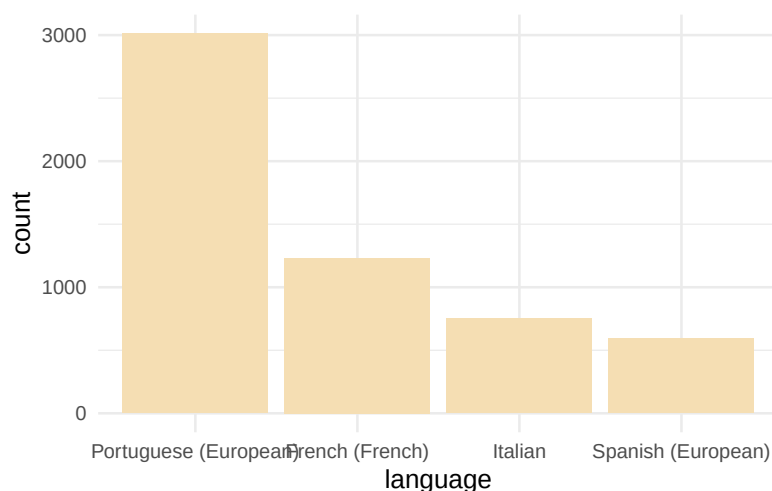


Figuur 6.6: Om de ordening van de staven aan te passen, helpt het niet om de ordening van de data zelf aan te passen. Je moet hiervoor de ordening van de levels veranderen.

Levels van factoren aanpassen kunnen we doen met functies uit het package `forcats`. Dit package is onderdeel van `tidyverse` en wordt dus meteen geladen als je `library(tidyverse)` aanroept (je hoeft het dus niet apart aan te roepen). Het package bevat een groot aantal functies waarmee je factoren kunt manipuleren. Deze functies beginnen allemaal met `fct_`. Zo ordent de functie `fct_infreq` de levels op basis van het aantal voorkomens in de ruwe data met het level dat het meest voorkomt eerst. Om deze ordening zichtbaar te maken moeten we deze functies toepassen binnen een

`mutate()` (zie Figuur 6.7). Let op: in onderstaande code veranderen we de ordening niet in de oorspronkelijke datatabel `wordbank_demo_romance`, maar alleen in de tabel die door middel van *pipes* naar de functie `ggplot()` wordt gestuurd. Willen we het blijvend veranderen dan moeten we de `mutate`-stap toewijzen aan de datatabel met `<-` en vervolgens deze datatabel naar `ggplot()` sturen.

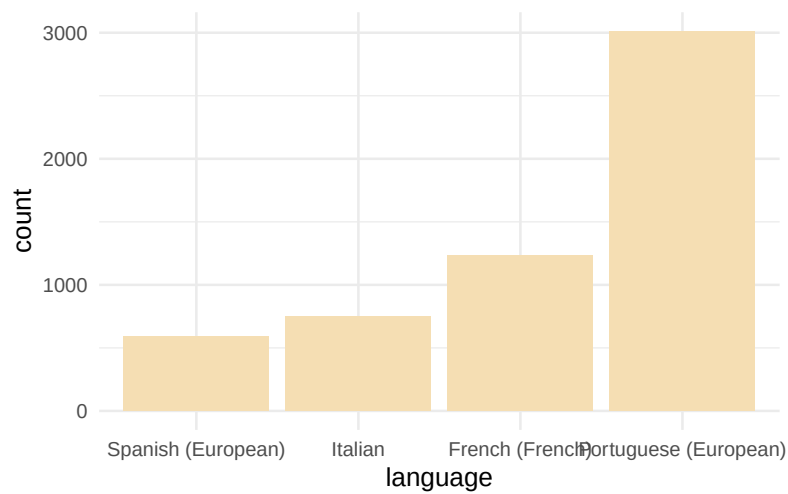
```
wordbank_demo_romance %>%
  mutate(language = fct_infreq(language)) %>%
  ggplot(aes(x = language)) +
  geom_bar(fill = "wheat") +
  theme_minimal(13)
```



Figuur 6.7: Door de levels aan te passen met een van de `fct_`-functies verandert de volgorde van de staven. Hier zijn ze met `fct_infreq()` geordend op basis van aantal voorkomens in de dataset.

Wil je de volgorde omdraaien naar van klein naar groot dan kun je `fct_rev()` gebruiken. We passen deze hier in een keer toe samen met `fct_infreq()` door de *pipe* te gebruiken binnen `mutate()`.

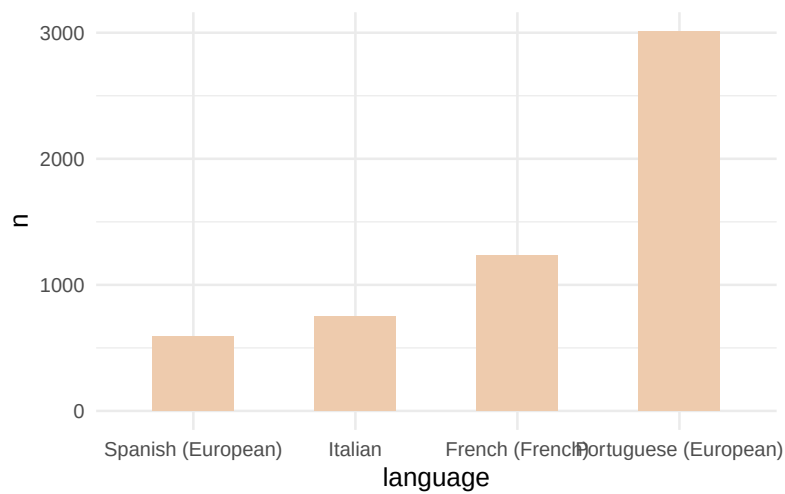
```
wordbank_demo_romance %>%
  mutate(language = language %>% fct_infreq() %>% fct_rev()) %>%
  ggplot(aes(x = language)) +
  geom_bar(fill = "wheat") +
  theme_minimal(13)
```



Figuur 6.8: Met `fct_rev()` draaien we de volgorde van de levels om.

Er is ook de functie `fct_reorder()` die deze twee stappen in een keer kan uitvoeren. Deze functie neemt twee argumenten: (i) de variabele waarvan je de levels wilt aanpassen en (ii) de variabele op basis waarvan je de levels wilt ordenen (hier `n`).

```
wordbank_demo_romance %>%
  count(language) %>%
  mutate(language = fct_reorder(language, n)) %>%
  ggplot(aes(x = language, y = n)) +
  geom_col(fill = "peachpuff2", width = .5) +
  theme_minimal(13)
```



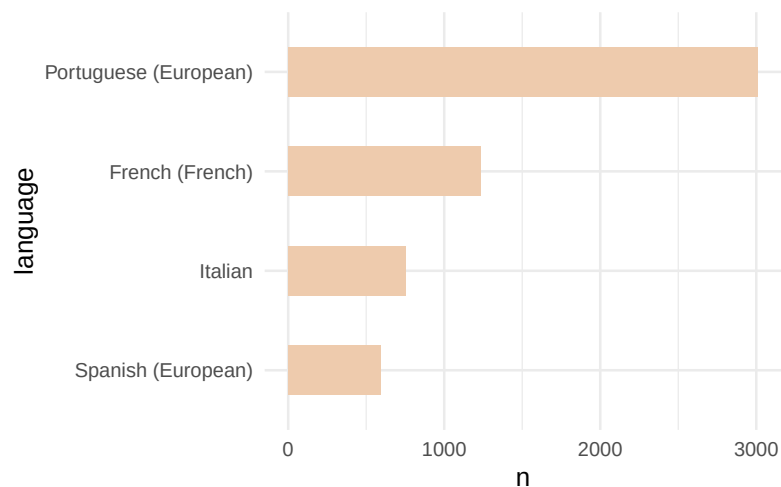
Figuur 6.9: Staafdiagram met de x-as geordend van klein naar groot met `fct_reorder()`.

Ook in dit geval kunnen we het eerste argument (`language`) buiten de functie zetten met een *pipe* (in regel 3). Door de mapping van `x` en `y` om te wisselen (regel 4) maken we er meteen een liggend staafdiagram van (Figuur 6.10). Deze optie maakt het vaak makkelijker om verschillen te zien, zeker als die niet erg groot zijn.

```

1 wordbank_demo_romance %>%
2   count(language) %>%
3   mutate(language = language %>% fct_reorder(n)) %>%
4   ggplot(aes(x = n, y = language)) +
5   geom_col(fill = "peachpuff2", width = .5) +
6   theme_minimal(13)

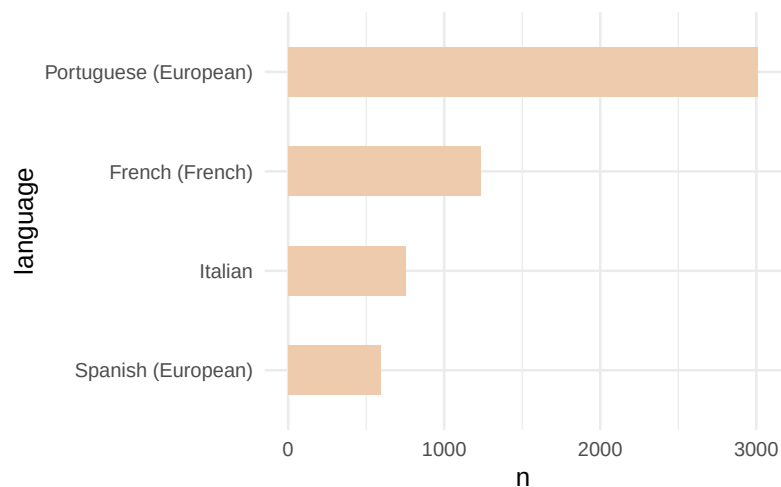
```



Figuur 6.10: Door de mapping van `x` en `y` om te wisselen krijgen we een liggend staafdiagram.

Het draaien van de grafiek kan ook met de functie `coord_flip()`, zie Figuur 6.11. Deze is alleen te gebruiken als je niet al een andere `coord_-`functie toepast in je grafiek. Wanneer je dat wel doet, zul je de variabelen moeten verwisselen in de `aes()` van `ggplot()` of het `geom`.

```
wordbank_demo_romance %>%
  count(language) %>%
  mutate(language = language %>% fct_reorder(n)) %>%
  ggplot(aes(x = language, y = n)) +
  geom_col(fill = "peachpuff2", width = .5) +
  coord_flip() +
  theme_minimal(13)
```

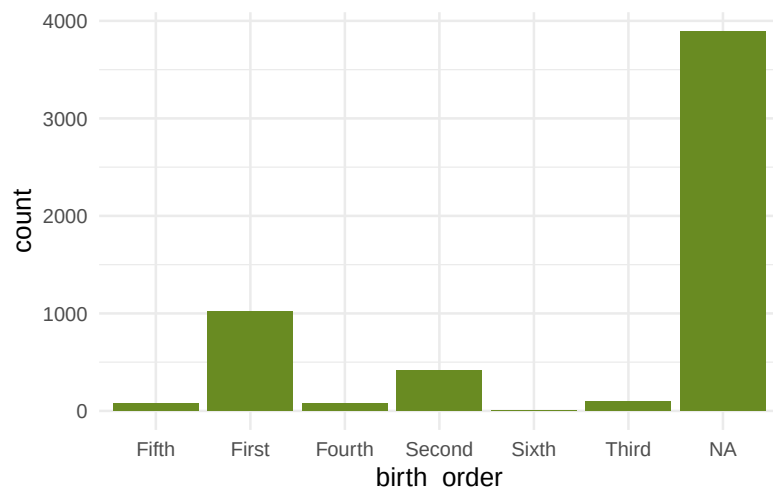


Figuur 6.11: Het draaien van de assen kan ook met de functie `coord_flip()`, maar alleen als je geen andere `coord_`-functie gebruikt.

6.2.1 Een geordende factor

Bij geordende factoren kun je niet vrijelijk de levels herordenen, omdat de ordening hier een betekenis heeft. Laten we dit eens bekijken aan de hand van de variabele `birth_order`. Zoals de naam al zegt, gaat het hier om een geordende factor. De variabele geeft namelijk de volgorde van geboorte in het gezin aan. We maken een eerste staafdiagram van deze variabele (Figuur 6.12):

```
wordbank_demo_romance %>%
  ggplot(aes(x = birth_order)) +
  geom_bar(fill = "olivedrab4") +
  theme_minimal(13)
```



Figuur 6.12: Het staafdiagram laat zien dat R de levels van de factor `birth_order` alfabetisch geordend heeft.

De hoge staaf bij NA geeft aan dat deze informatie voor veel kinderen in de dataset niet geregistreerd is. Daarnaast valt vooral op dat de levels alfabetisch zijn geordend met 'Fifth' als eerste en niet **First**. Dit weerspiegelt de manier waarop R de ordening van de levels heeft geregistreerd, nl. alfabetisch. We kunnen de ordening van de levels van een variabele zichtbaar maken met de functie `levels()`. We gebruiken hierbij de `$`-notatie om aan te geven welke variabele in de dataset `wordbank_demo_romance` we willen bekijken.

```
# zichtbaar maken van de ordening van de levels van een variabele
levels(wordbank_demo_romance$birth_order)
```

```
[1] "Fifth" "First" "Fourth" "Second" "Sixth" "Third"
```

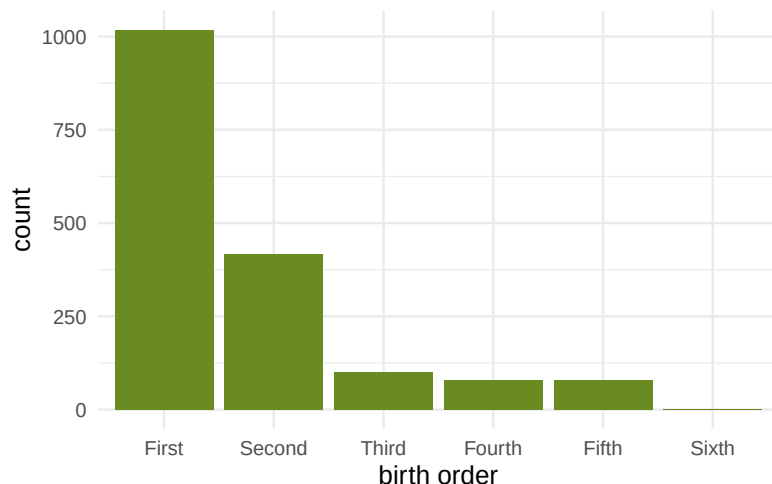
Dit gaan we aanpassen. Om onze code overzichtelijk te houden, maken we eerst een vector `birth_order_levels` aan met de juiste ordening:

```
birth_order_levels <- c("First", "Second", "Third", "Fourth", "Fifth", "Sixth")
```

Met `fct_relevel()` kunnen we deze ordening nu toepassen op de variabele `birth_order`. Deze functie neemt als eerste argument de naam van de aan te passen variabele (hier met de `%>%`) en als tweede argument de ordening die je

wilt toepassen op de levels. We hebben de ordening hier meegegeven door te verwijzen naar de eerder aangemaakte vector `birth_order_levels`. Je zou de ordening ook hier in de code handmatig kunnen specificeren. Dat maakt de code wel wat onoverzichtelijker. Nadat de ordening is toegepast sturen we de datatabel naar `ggplot()`. Zie het resultaat in Figuur 6.13.

```
wordbank_demo_romance %>%  
  filter(!is.na(birth_order)) %>%  
  mutate(birth_order = birth_order %>% fct_relevel(birth_order_levels)) %>%  
  ggplot(aes(x = birth_order)) +  
  geom_bar(fill = "olivedrab4") +  
  labs(x = "birth order") +  
  theme_minimal(13)
```



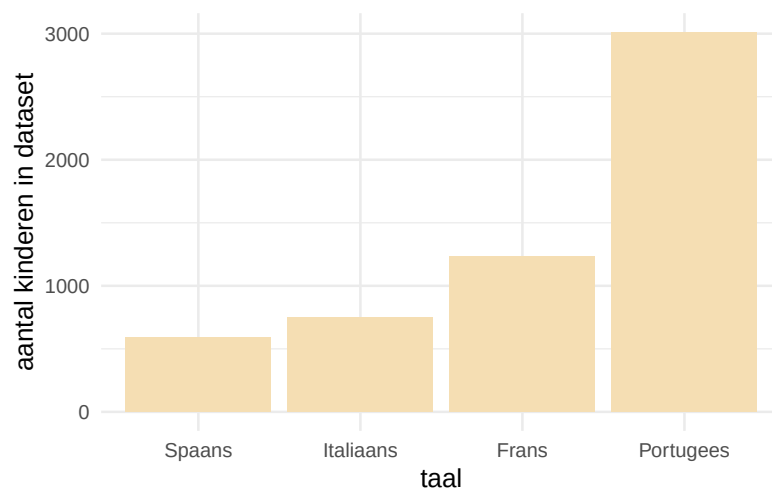
Figuur 6.13: Staafdiagram met correcte ordening van een geordende factor na toepassen van `fct_relevel()`.

6.3 Levels hernoemen of samenvoegen

Soms wil je meer dan alleen de bestaande levels herordenen. Stel dat je onze eerdere grafiek Figuur 6.9 wilt opnemen in een Nederlandstalig verslag. Je zult dan Nederlandse namen voor assen en levels willen gebruiken. Onze dataset bevat alleen Engelse namen. Het aanpassen van de namen van de assen kunnen we in `ggplot()` doen binnen `labs()`, zoals we al

eerder hebben gedaan. Het staat ons hier vrij om een naam te kiezen, dat kan dus ook een Nederlandse naam zijn. De namen van de levels van een variabele zullen we moeten hernoemen in de datatabel. Hiervoor gebruiken we `fct_recode()`, die we eerder al hebben gezien toen we namen aan levels toekenden die als cijfers waren gecodeerd (zie Paragraaf 3.2.2). We gebruiken `fct_recode()` in onderstaande code om elke taal een Nederlandse naam te geven (zie Figuur 6.14). Let op dat de nieuwe levelnaam voor = komt zonder aanhalingstekens en de oude naam erachter met aanhalingstekens.

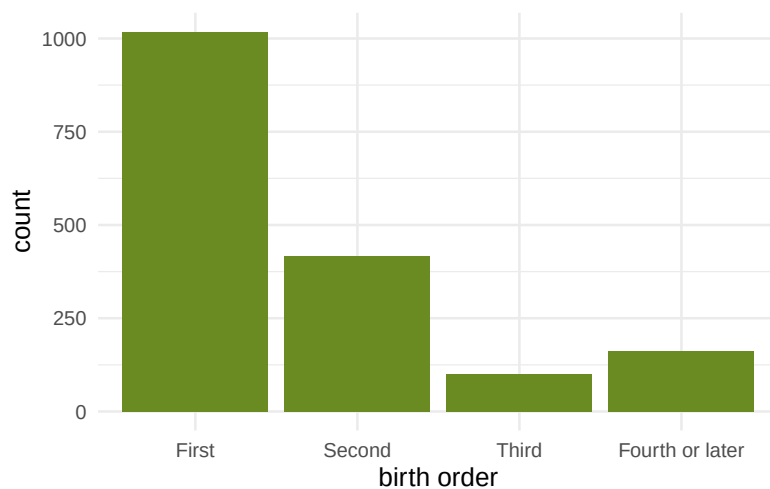
```
wordbank_demo_romance %>%  
  count(language) %>%  
  mutate(language = fct_reorder(language, n), #aanpassen van de ordening van de levels  
         language = language %>% fct_recode( #aanpassen van de namen van de levels  
           Frans = "French (French)",  
           Italiaans = "Italian",  
           Portugees = "Portuguese (European)",  
           Spaans = "Spanish (European)"  
         )) %>%  
  ggplot(aes(x = language, y = n)) +  
  geom_col(fill = "wheat") +  
  labs(x = "taal",  
       y = "aantal kinderen in dataset") +  
  theme_minimal(13)
```



Figuur 6.14: Met `fct_recode()` zijn de namen van de levels Nederlandstalig aangepast voordat de datatabel is doorgestuurd naar `ggplot()`.

Soms wil je het aantal levels reduceren, bijvoorbeeld om je grafiek overzichtelijker te maken of omdat bepaalde levels weinig voorkomen in de data. Met `fct_collapse()` kun je meerdere levels samenvoegen tot een nieuw level (soms is `fct_other()` handiger, want minder typewerk). In onderstaand voorbeeld voegen we de levels `Fourth`, `Fifth` en `Sixth` samen tot een nieuw level `Fourth or later` dat we hier met spaties schrijven, omdat dat mooier staat in de grafiek (Figuur 6.15). Om te zorgen dat R het als één naam herkent, zetten we enkele aanhalingstekens (*backticks*) om deze nieuwe naam heen (dus: `'Fourth or later'`). De levels die we willen samenvoegen geven we mee als argumenten aan de functie `c()`.

```
wordbank_demo_romance %>%
  filter(!is.na(birth_order)) %>%
  mutate(birth_order = birth_order %>% fct_relevel(birth_order_levels),
         birth_order = birth_order %>% fct_collapse(`Fourth or later` = c("Fourth", "Fifth",
ggplot(aes(x = birth_order)) +
  geom_bar(fill = "olivedrab4") +
  labs(x = "birth order") +
  theme_minimal(13)
```



Figuur 6.15: Door levels met weinig observaties samen te voegen via `fct_collapse()` ontstaat er een overzichtelijkere grafiek.

6.4 Meerdere categorische variabelen

We hebben ons tot nu toe gericht op grafieken met een enkele categorische variabele. De variabele `geslacht` is hier een voorbeeld van:

```
wordbank_demo_romance %>%  
  count(sex)  
  
# A tibble: 3 x 2  
  sex      n  
  <fct> <int>  
1 Female 2655  
2 Male   2785  
3 <NA>    150
```

Vaak willen we beschrijvende statistieken geven van meerdere categorische variabelen tegelijk. In onze dataset kunnen we bijvoorbeeld de verdeling van geslacht per taal bekijken. We halen wel eerst de missing values eruit met `!is.na()`.

```
wordbank_demo_romance %>%  
  filter(!is.na(sex)) %>%  
  count(language, sex)  
  
# A tibble: 8 x 3  
  language      sex      n  
  <fct>      <fct> <int>  
1 French (French) Female   558  
2 French (French) Male    525  
3 Italian      Female   377  
4 Italian      Male    375  
5 Portuguese (European) Female 1424  
6 Portuguese (European) Male   1588  
7 Spanish (European)  Female   296  
8 Spanish (European)  Male    297
```

Omdat we twee levels hebben van de variabele `sex` en vier van de variabele `language` krijgen we 2×4 getallen terug.

De volgorde waarin we de variabelen meegeven aan `count()` bepaalt de volgorde waarin R de data teruggeeft aan ons. Wanneer we eerst `sex` meegeven, dan worden de data geordend per level van die variabele, dus eerst alle waarden voor het eerste level `Female` en dan voor het tweede level `Male`. Binnen elk level van `sex` worden de levels van `language` steeds getoond in de levelvolgorde van die variabele met `French (French)` als eerste.

```
wordbank_demo_romance %>%
  filter(!is.na(sex)) %>%
  count(sex, language)
```

```
# A tibble: 8 x 3
  sex    language      n
  <fct> <fct>      <int>
1 Female French (French)    558
2 Female Italian           377
3 Female Portuguese (European) 1424
4 Female Spanish (European)    296
5 Male   French (French)    525
6 Male   Italian           375
7 Male   Portuguese (European) 1588
8 Male   Spanish (European)    297
```

De uitdaging ontstaat wanneer we proporties gaan berekenen. We moeten ons dan steeds de vraag stellen: proporties van wat? We hebben hier namelijk verschillende mogelijkheden en het aantal mogelijkheden wordt groter met het toenemen van het aantal variabelen. We kunnen bijvoorbeeld van elke combinatie van taal en geslacht de proportie binnen de dataset als geheel berekenen. Hiervoor bepalen we eerst het totaal aantal observaties (regel 4) en gebruiken dat als deler voor `n`.

```
1 wordbank_demo_romance %>%
2   filter(!is.na(sex)) %>%
3   count(language, sex) %>%
4   mutate(sum_total = sum(n),
```

```

5     proportion_total = n/sum_total,
6     proportion_total = round(proportion_total, 2))

# A tibble: 8 x 5
  language      sex      n sum_total proportion_total
  <fct>        <fct> <int>    <int>          <dbl>
1 French (French) Female  558    5440           0.1
2 French (French) Male    525    5440           0.1
3 Italian      Female  377    5440          0.07
4 Italian      Male    375    5440          0.07
5 Portuguese (European) Female 1424    5440          0.26
6 Portuguese (European) Male   1588    5440          0.29
7 Spanish (European) Female  296    5440          0.05
8 Spanish (European) Male    297    5440          0.05

```

De kolom `proportion_total` laat zien dat binnen elke taal de proportie per geslacht ongeveer een gelijk aandeel van het geheel heeft.

We kunnen de proporties ook per taal berekenen, dus bijvoorbeeld welk aandeel heeft het level `female` binnen het Frans? Hiervoor moeten we de totalen per taal berekenen. Dit doen we met de functie `group-by()`, die we al eerder zijn tegengekomen in Paragraaf 4.5. Deze functie biedt de mogelijkheid om een volgende functie uit te voeren per level van een (of meerdere) variabele(n).

`group_by()` en `ungroup()`

De groepering die je aanbrengt met `group_by()` blijft behouden in je datatabel. Dit kun je bijvoorbeeld zien als je de datatabel naar het scherm print. Bovenaan zal R je dan vertellen welke groepen er bestaan in de data. Deze groeperingen kunnen soms voor onverwachte resultaten zorgen in latere code. Het is daarom aan te raden om, wanneer je de groepering niet meer nodig hebt voor je bewerkingen, deze expliciet ongedaan te maken met de functie `ungroup()`.

```

datatabel %>%
  group_by(variabele) %>%
  summarize(aantal = n()) %>%
  ungroup()

```

```

wordbank_demo_romance %>%
  filter(!is.na(sex)) %>%
  count(language, sex) %>%
  mutate(sum_total = sum(n),
         proportion_total = n/sum_total,
         proportion_total = round(proportion_total, 2)) %>%
  group_by(language) %>%
  mutate(sum_language = sum(n)) %>% #berekenen totaal aantal voorkomens per taal
  ungroup()

```

A tibble: 8 x 6

	language	sex	n	sum_total	proportion_total	sum_language
	<fct>	<fct>	<int>	<int>	<dbl>	<int>
1	French (French)	Female	558	5440	0.1	1083
2	French (French)	Male	525	5440	0.1	1083
3	Italian	Female	377	5440	0.07	752
4	Italian	Male	375	5440	0.07	752
5	Portuguese (European)	Female	1424	5440	0.26	3012
6	Portuguese (European)	Male	1588	5440	0.29	3012
7	Spanish (European)	Female	296	5440	0.05	593
8	Spanish (European)	Male	297	5440	0.05	593

We gebruiken die totalen per taal om vervolgens het aantal observaties erdoor te delen:

```

wordbank_demo_romance %>%
  filter(!is.na(sex)) %>%
  count(language, sex) %>%
  mutate(sum_total = sum(n),
         proportion_total = n/sum_total,
         proportion_total = round(proportion_total, 2)) %>%
  group_by(language) %>%

```

```
mutate(sum_language = sum(n),
       proportion_language = n/sum_language,
       proportion_language = round(proportion_language, 2)) %>%
ungroup()
```

language	sex	n	sum_total	proportion_total	language	proportion_language
French	Female	558	5440	0.10	French	0.52
French	Male	525	5440	0.10	French	0.48
Italian	Female	377	5440	0.07	Italian	0.50
Italian	Male	375	5440	0.07	Italian	0.50
Portuguese	Female	424	5440	0.26	Portuguese	0.47
Portuguese	Male	1588	5440	0.29	Portuguese	0.53
Spanish	Female	296	5440	0.05	Spanish	0.50
Spanish	Male	297	5440	0.05	Spanish	0.50

We zien hier dat binnen elke taal de proporties voor beide geslachten rond de .50 ligt.

De laatste optie is per geslacht te berekenen welke proportie elke taal inneemt. Hiervoor berekenen we eerst het totaal aantal observaties per geslacht met `group_by(sex)`.

```
demo_romance <- wordbank_demo_romance %>%
  filter(!is.na(sex)) %>%
  count(language, sex) %>%
  mutate(sum_total = sum(n),
         proportion_total = n/sum_total,
         proportion_total = round(proportion_total, 2)) %>%
  group_by(language) %>%
  mutate(sum_language = sum(n),
         proportion_language = n/sum_language,
         proportion_language = round(proportion_language, 2)) %>%
  group_by(sex) %>%
```

```
mutate(sum_sex = sum(n),
       proportion_sex = n/sum_sex,
       proportion_sex = round(proportion_sex, 2)) %>%
ungroup()

demo_romance
```

language	sex	n	sum	total	proportion	sum	total	proportion	language	proportion_sex
French	Female	158	5440	0.10	1083	0.52	2655	0.21	French	
(French)										
French	Male	525	5440	0.10	1083	0.48	2785	0.19	French	
(French)										
Italian	Female	377	5440	0.07	752	0.50	2655	0.14	Italian	
Italian	Male	375	5440	0.07	752	0.50	2785	0.13	Italian	
Portuguese	Female	142	5440	0.26	3012	0.47	2655	0.54	Portuguese	
(European)										
Portuguese	Male	158	5440	0.29	3012	0.53	2785	0.57	Portuguese	
(European)										
Spanish	Female	296	5440	0.05	593	0.50	2655	0.11	Spanish	
(European)										
Spanish	Male	297	5440	0.05	593	0.50	2785	0.11	Spanish	
(European)										

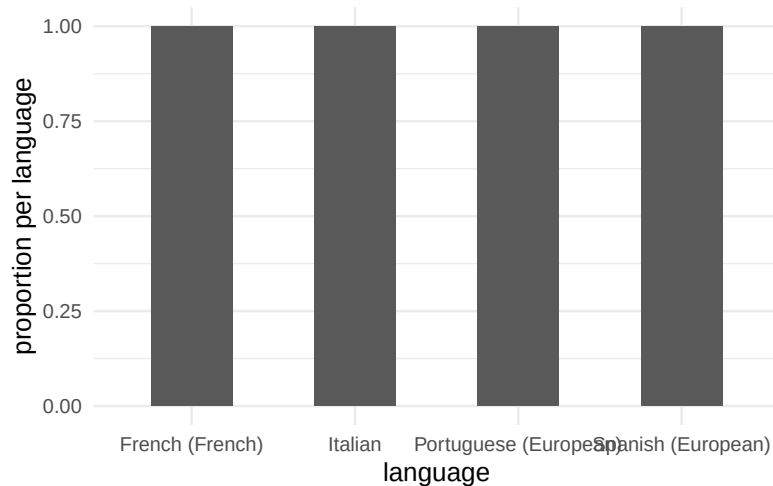
Dit voorbeeld laat zien dat je zelf als onderzoeker goed moet nadenken wat je wilt laten zien. Elke proportie vertelt een net wat ander verhaal.

Hoewel het mogelijk is om de proporties te laten bereken door `geom_bar()` levert dit al snel complicaties op. Wij kiezen er daarom voor om zelf proporties te berekenen en deze af te beelden met `geom_col()`. Op deze manier houdt je volledige controle over de data.

Laten we eerst eens kijken of we de proportie geslacht per taal kunnen afbeelden (Figuur 6.16).

```
demo_romance %>%
  ggplot(aes(x = language, y = proportion_language)) +
  geom_col(width = 0.5) +
```

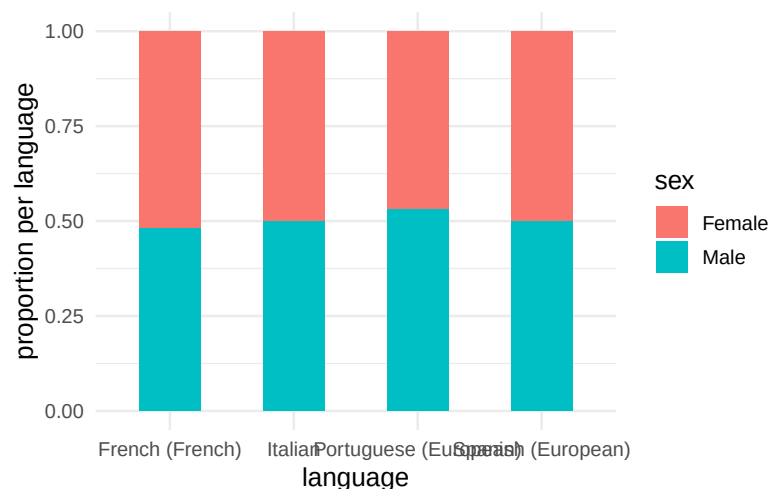
```
labs(y = "proportion per language") +
theme_minimal(13)
```



Figuur 6.16: Een mislukte poging om de proportie geslacht per taal weer te geven. De levels van de variabele **sex** zijn niet zichtbaar.

Deze grafiek geeft nog weinig inzicht. We zullen hiervoor de staven moet kleuren op basis van de variabele **sex**. We voegen deze toe in de **aes()** van **ggplot()**, waar we een mapping maken met **fill**. (N.B. **color** verandert de rand van de staaf niet de binnenkant.) Figuur 6.17 laat het resultaat zien.

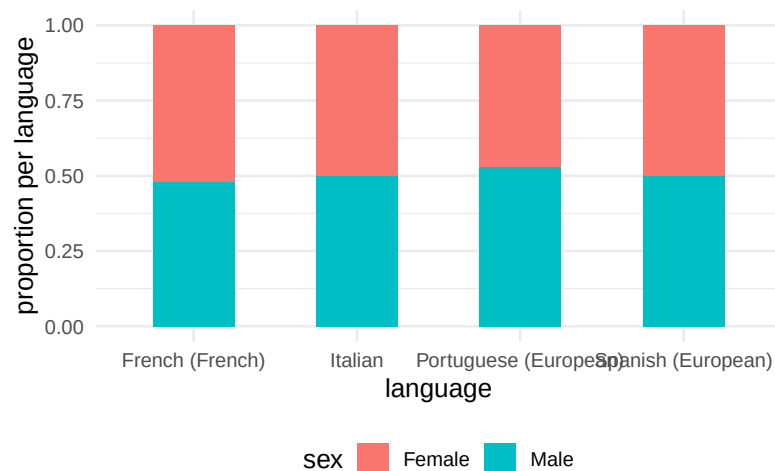
```
demo_romance %>%
  ggplot(aes(x = language, y = proportion_language, fill = sex )) +
  geom_col(width = 0.5) +
  labs(y = "proportion per language") +
  theme_minimal(13)
```



Figuur 6.17: Door een mapping van de variabele `sex` naar het attribuut `fill` zijn de levels zichtbaar geworden.

Dit laat meteen een probleem zien dat ontstaat door factoren met lange namen van levels. Door het toevoegen van kleur is er een legenda bij gekomen en hierdoor is er minder ruimte voor de staven. De namen van de levels overlappen. Dit probleem geeft ons de mogelijkheid om wat meer opties van `ggplot()` te laten zien. Er zijn namelijk meerdere manieren om dit op te lossen. We kunnen de grafiek draaien of we kunnen de positie van de legenda veranderen. Dit laatste doen we binnen de functie `theme()` met de optie `legend.position`. In Figuur 6.18 verplaatsen we de legenda naar de onderkant van de grafiek (andere opties zijn `left`, `right`, `top` en `none` die we al eerder hebben gezien om de legenda geheel te verwijderen).

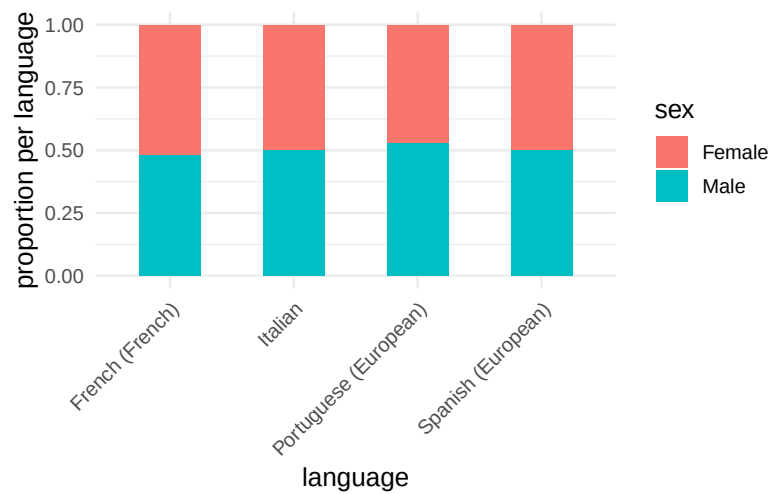
```
demo_romance %>%
  ggplot(aes(x = language, y = proportion_language, fill = sex )) +
  geom_col(width = 0.5) +
  labs(y = "proportion per language") +
  theme_minimal(13) +
  theme(legend.position = "bottom")
```



Figuur 6.18: Door de legenda onder de grafiek te plaatsen ontstaat er iets meer ruimte in de breedte.

We kunnen er ook voor kiezen om de labels van de x-as wat de draaien (Figuur 6.19). Ook dit specificeren we binnen `theme()`. Er zijn talloze opties om hierbinnen het uiterlijk van de grafiek aan te passen, gebruik `?theme()` voor een overzicht. Hier passen we een rotatie van 45 graden (`angle = 45`) toe op de text van de x-as `axis.text.x` en we verplaatsen de tekst ook wat in de hoogte en de breedte met `vjust` en `hjust`.

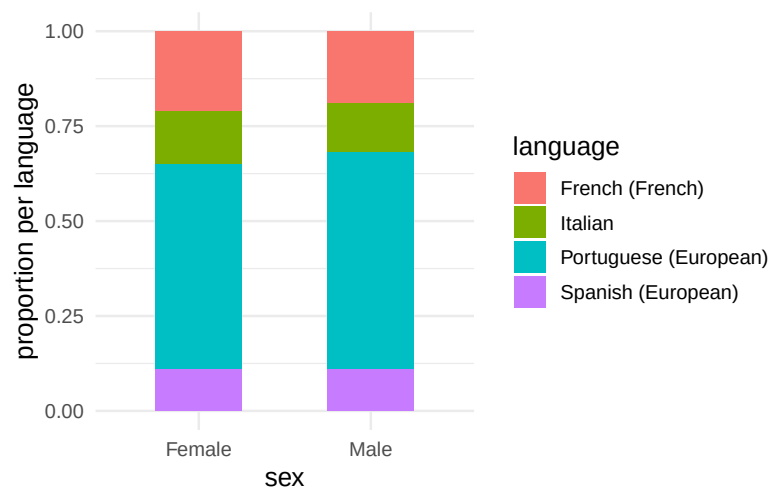
```
demo_romance %>%
  ggplot(aes(x = language, y = proportion_language, fill = sex )) +
  geom_col(width = 0.5) +
  labs(y = "proportion per language") +
  theme_minimal(13) +
  theme(axis.text.x = element_text(angle = 45, vjust = 1, hjust = 1))
```



Figuur 6.19: Door de labels op de x-as iets te draaien worden ze beter leesbaar.

Wanneer we de proporties voor meer dan twee levels willen laten zien dan geeft een gestapeld staafdiagram niet altijd de beste optie (zie Figuur 6.20). Het kan dan namelijk lastig worden om over staven heen gelijke levels te vergelijken, omdat die een ander beginpunt hebben.

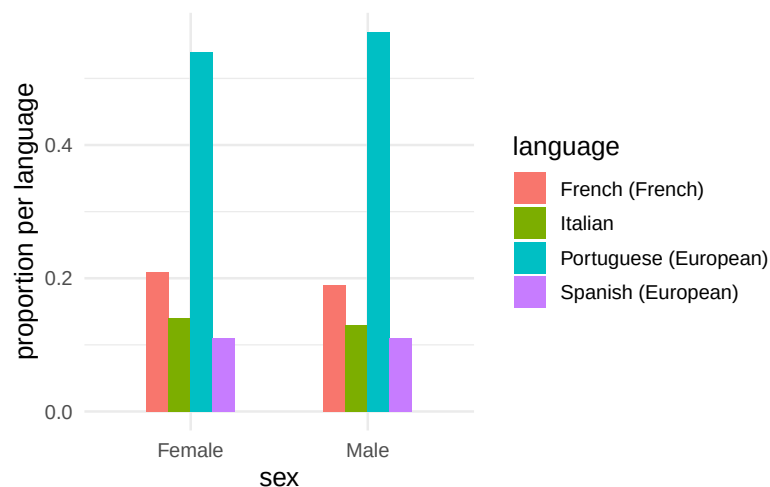
```
demo_romance %>%
  ggplot(aes(x = sex, y = proportion_sex, fill = language )) +
  geom_col(width = 0.5) +
  labs(y = "proportion per language") +
  theme_minimal(13)
```



Figuur 6.20: Bij gestapelde staafdiagrammen is het soms lastig om levels met elkaar te vergelijken, omdat ze een ander beginpunt hebben.

We kunnen er in dat geval voor kiezen om de elk level een eigen staaf te geven (Figuur 6.21). Dat doen we binnen `geom_col()` door het argument `position = "dodge"` toe te voegen.

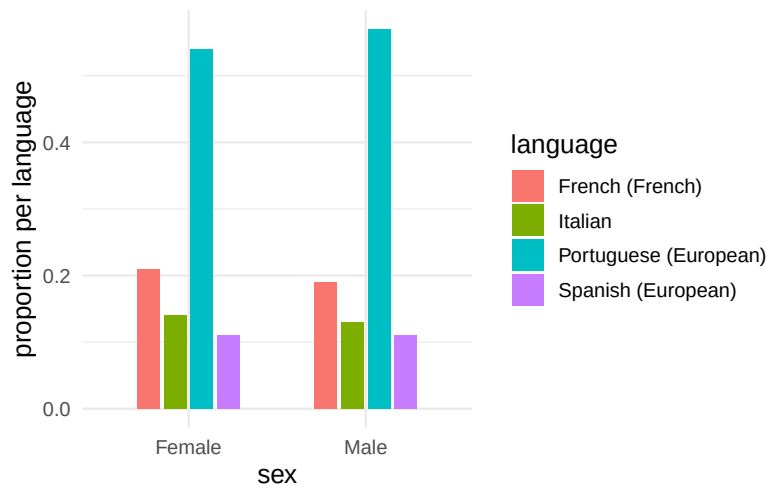
```
demo_romance %>%
  ggplot(aes(x = sex, y = proportion_sex, fill = language)) +
  geom_col(position = "dodge", width = 0.5) +
  labs(y = "proportion per language") +
  theme_minimal(13)
```



Figuur 6.21: Elke level krijgt een eigen staaf met de optie ‘position = “dodge”’)

Dit is een verkorte schrijfwijze voor `position = position_dodge()` dat je kunt gebruiken om de afstand tussen de staven aan te passen:

```
demo_romance %>%  
  ggplot(aes(x = sex, y = proportion_sex, fill = language )) +  
  geom_col(position = position_dodge(.6), width = 0.5) +  
  labs(y = "proportion per language") +  
  theme_minimal(13)
```



Figuur 6.22: Staafdiagram met iets grotere afstand tussen de staven.

6.5 Kleuren

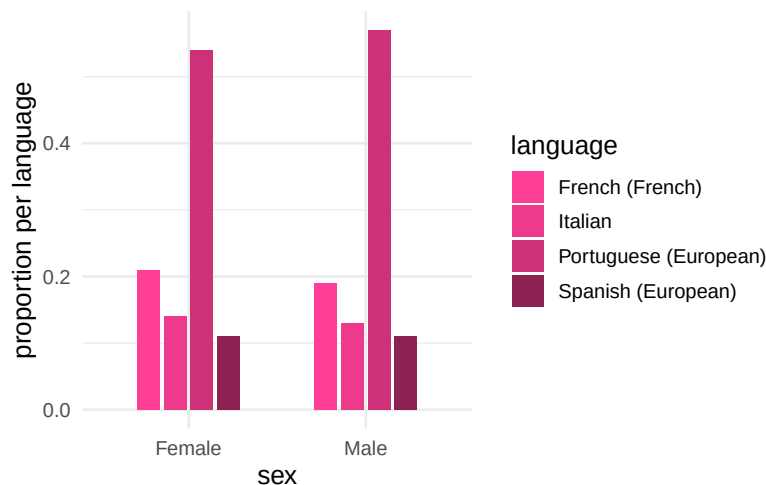
We hebben net kleuren gebruikt om de verschillende levels van een variabele binnen een andere variabele zichtbaar te maken. Dit biedt ons een mooie gelegenheid om wat dieper in te gaan op kleurgebruik binnen R. Het kiezen van eigen kleuren wordt vaak een leuk onderdeel gevonden van het maken van grafieken. Ook hier zijn de mogelijkheden eindeloos.

Wanneer we een variabele via `aes()` afbeelden op een aesthetic waarin kleur een rol speelt (`color` of `fill`), dan zal R in principe zelf de kleuren bepalen die gebruikt worden, zoals te zien in Figuur 6.22. Je kunt deze kleuren echter ook handmatig aanpassen. Met het commando `colors()` krijg je een overzicht

van alle in R gedefinieerde kleurnamen, voor een visueel overzicht zie <https://sites.stat.columbia.edu/tzheng/files/Rcolor.pdf>.

Je kunt deze kleurnamen gebruiken wanneer je een vaste kleur wilt geven aan alle staven of punten in je grafiek. Dit hebben we aan het begin van het hoofdstuk steeds gedaan. In dit geval wijs je deze kleur toe binnen het geom buiten `aes()` met `color = "<kleurnaam>"`, zie bijvoorbeeld Figuur 6.9 waar we `peachpuff2` als kleurnaam hebben toegepast. Wanneer je een variabele naar een kleurattribuut, zoals `colo(u)r` of `fill`, mapt binnen `aes()` en je wilt de kleuren handmatig aanpassen dan moet je `scale_fill_manual()` toevoegen bij een mapping naar `fill` of `scale_color_manual()` bij een mapping naar `color`. Je geeft dan met de functie `c()` aan het argument `values` evenveel kleurnamen mee als dat er levels zijn die je wilt kleuren. De kleuren worden in volgorde gekoppeld aan de levels van de factor. Hieronder hebben we dat gedaan voor Figuur 6.22. Dit resulteert in Figuur 6.23.

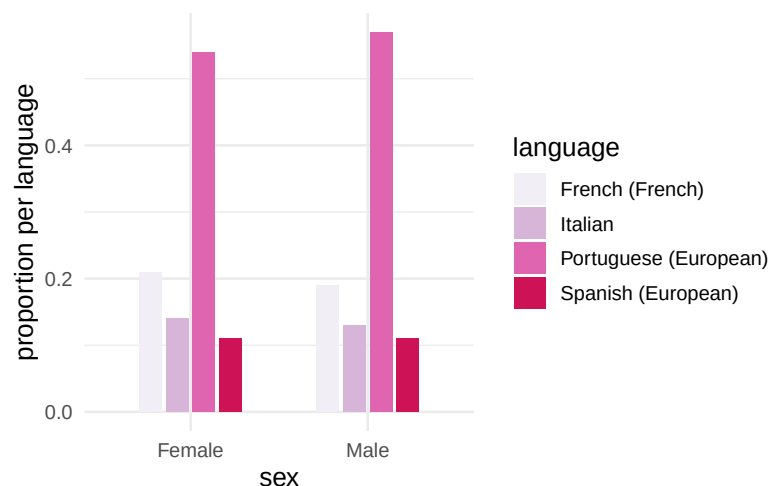
```
demo_romance %>%  
  ggplot(aes(x = sex, y = proportion_sex, fill = language )) +  
  geom_col(position = position_dodge(.6), width = 0.5) +  
  scale_fill_manual(values = c("violetred1", "violetred2", "violetred3", "violetred4")) +  
  labs(y = "proportion per language") +  
  theme_minimal(13)
```



Figuur 6.23: Handmatig de kleuren van de staven aangepast.

Wanneer je zelf kleuren gaat kiezen is het belangrijk om te zorgen dat de levels voldoende contrast hebben. In bovenstaand voorbeeld is dat niet heel erg duidelijk gelukt. Om je hierbij te helpen, bestaan er *paletten* van kleuren met voldoende contrast, op [deze website](#) vind je een overzicht. Deze paletten voeg je toe met `scale_fill_brewer(palette = "<palettenaam>")` (of `scale_color_brewer()`). In Figuur 6.24 gebruiken we het palette PuRd. Verder is het ook raadzaam om rekening te houden met kleurenblindheid. Hiervoor bestaan speciale paletten.

```
demo_romance %>%
  ggplot(aes(x = sex, y = proportion_sex, fill = language )) +
  geom_col(position = position_dodge(.6), width = 0.5) +
  scale_fill_brewer(palette = "PuRd") +
  labs(y = "proportion per language") +
  theme_minimal(13)
```



Figuur 6.24: Een goed gekozen palet aan kleuren, helpt vaak om contrasten duidelijker te maken. Hier is het palet PuRd van `color brewer` gebruikt.

6.6 Een level highlighten

Onze kennis over kleuren kunnen we toepassen om een level van een variabele te *highlighten*. Stel je voor dat je een verslag schrijft waarin je speciale aandacht hebt voor het Italiaans. Je zou er dan voor kunnen kiezen om dit level van de variabele

`language` een andere kleur te geven dan de overige levels. Op deze manier springt het Italiaans er echt uit in je grafiek.

Aangezien we te maken hebben met een mapping van een variabele naar kleur, kun je hiervoor een nieuwe variabele aanmaken. We geven deze variabele hier de naam `lang_color`. Binnen deze variabele geven we `Italian` een andere waarde dan de overige levels. We doen dat met de functie `fct_other()`. Binnen deze functie kun je aangeven van welk(e) level(s) je de naam wilt bewaren met het argument `keep` en welke je een andere naam wilt geven. Deze levels krijgen standaard de naam `Other`, maar je kunt er ook voor kiezen dit een andere naam te geven.

```
wordbank_demo_romance %>%  
  count(language) %>%  
  mutate(language = fct_reorder(language, n),  
         lang_color = language %>% fct_other(keep = c("Italian")))
```

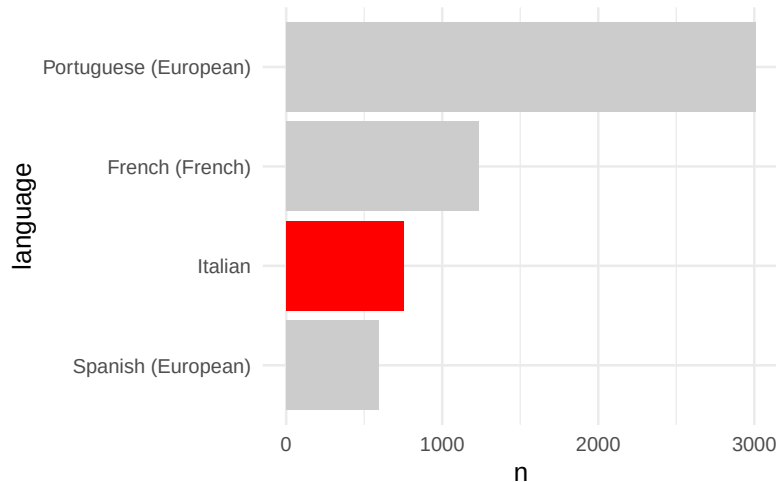
A tibble: 4 x 3

	language	n	lang_color
	<fct>	<int>	<fct>
1	French (French)	1233	Other
2	Italian	752	Italian
3	Portuguese (European)	3012	Other
4	Spanish (European)	593	Other

We gebruiken deze nieuwe variabele `lang_color` vervolgens in de mapping van `fill` op het niveau van `geom_col()` (kan ook binnen `ggplot()`) en we voegen via `scale_fill_manual()` de kleuren toe die we willen gebruiken. Aangezien `Italian` alfabetisch voor `Other` komt, zal het Italiaans de rode kleur krijgen en de andere levels grijs (Figuur 6.25).

```
wordbank_demo_romance %>%  
  count(language) %>%  
  mutate(language = fct_reorder(language, n),  
         lang_color = language %>% fct_other(keep = c("Italian"))) %>%  
  ggplot(aes(x = n, y = language)) +  
  geom_col(aes(fill = lang_color), show.legend = F) +
```

```
scale_fill_manual(values = c("red", "grey80")) +  
theme_minimal(13)
```



Figuur 6.25: Je kunt een of meerdere levels van een variabele highlighten door een aparte variabele aan te maken.

Verder leren

- Lees [hoofdstuk 4](#) en [de eerste sectie van hoofdstuk 5](#) van Healy (2026) voor meer inzicht in de eigenaardigheden van barplots.
- [hoofdstuk 9](#) uit Wickham et al. (2023) geeft een goed beeld van hoe geoms en facets werken in `ggplot2`.
- [Hoofdstuk 16](#) uit Wickham et al. (2023) geeft daarnaast een overzicht van het werken van factoren (zoals het veranderen van de ordening van levels).
- Zie voor een overzicht van wat je allemaal kunt doen met factoren het [forcats cheat sheet](#).
- Alles over kleuren in R vind je op <https://r-graph-gallery.com/ggplot2-color.html>
- Het [ggplot2 cheat sheet](#) geeft een overzicht van de mogelijkheden die dit package biedt.

Healy, K. (2026). *Data Visualisation: A Practical Introduction*. (2de dr.). Princeton University Press. <https://socviz.co/>

Wickham, H., Çetinkaya-Rundel, M., & Grolemund, G. (2023). *R for Data Science* (2de dr.). O'Reilly. <https://r4ds.hadley.nz/>

Wickham, H., Çetinkaya-Rundel, M., & Grolemund, G. (2023). *R for Data Science* (2de dr.). O'Reilly. <https://r4ds.hadley.nz/>

Nu zou je in staat moeten zijn Practicum 4 te maken.

7 Numerieke variabelen

Numerieke variabelen bieden meer mogelijkheden voor beschrijvende statistieken dan de categorische variabelen uit het vorige hoofdstuk. In dit hoofdstuk gaan we zien hoe je distributies (verdelingen) van numerieke variabelen zichtbaar kunt maken, hoe je centrum- en spreidingsmaten kunt berekenen en hoe je in een grafiek zowel een algemeen beeld (centrummaat) als de data zelf (spreiding) kunt laten zien.

Na afloop van dit hoofdstuk:

- kun je numerieke variabelen visualiseren met verschillende soorten plots (histogram, density, dot, box) in `ggplot2`;
- kun je meerdere *geoms* combineren om zowel de data zelf als patronen in de data zichtbaar te maken;
- kun je beschrijvende statistieken berekenen met `tidyverse`;
- kun je datatabellen omzetten van *wide* naar *long* formaat.

codeboek

In deze cursus leer je werken met zo'n 100 veelgebruikte functies in R. Het is belangrijk om voor jezelf een goed overzicht te houden van deze functies. Werk hier actief aan door een eigen codeboek aan te maken. Voeg tijdens of na afloop van het bestuderen van dit hoofdstuk de codes toe aan jouw codeboek.

De data in dit hoofdstuk

We gebruiken in dit hoofdstuk de data van Nederlandse kinderen uit *Workbank*. We kijken naar de woordenschat bij iets oudere kinderen (ca. 1.5 tot 2.5 jaar). De data zijn te vinden in het bestand `wordbank_dutch_WS_child.csv`.

```
wordbank_dutch_child <- read_csv("_datasets/wordbank_dutch_WS_child.csv")

glimpse(wordbank_dutch_child)
```

```
Rows: 303
Columns: 22
$ downloaded      <date> 2025-07-16, 2025-07-16, 2025-07-16, 2025-07-16, ~
$ language        <chr> "Dutch", "Dutch", "Dutch", "Dutch", "Dutch", "Dut~
$ form            <chr> "WS", "WS", "WS", "WS", "WS", "WS", "WS", "WS", "~
$ dataset_name    <chr> "Bergman", "Bergman", "Bergman", "Bergman", "Berg~
$ child_id        <dbl> 76526, 76527, 76528, 76529, 76530, 76531, 76532, ~
$ age             <dbl> 18, 18, 19, 18, 18, 18, 18, 18, 18, 18, 17, 17, 1~
$ comprehension   <dbl> 162, 405, 381, 313, 278, 90, 266, 324, 394, 154, ~
$ production      <dbl> 37, 28, 107, 122, 95, 11, 35, 6, 134, 17, 41, 25,~
$ is_norming      <lgl> FALSE, FALSE, FALSE, FALSE, FALSE, FALSE, FALSE, ~
$ birth_order     <lgl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ caregiver_education <lgl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ ethnicity       <lgl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ race            <lgl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ sex             <chr> "Female", "Female", "Female", "Female", "Female", ~
$ birth_weight    <lgl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ born_early_or_late <lgl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ gestational_age <lgl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ zygosity        <lgl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ language_exposures <lgl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ health_conditions <lgl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ monolingual     <lgl> TRUE, TRUE, TRUE, TRUE, TRUE, TRUE, TRUE, TRUE, T~
$ typically_developing <lgl> TRUE, TRUE, TRUE, TRUE, TRUE, TRUE, TRUE, TRUE, T~
```

We hebben 303 observaties en wederom 22 variabelen. We gaan de dataset inperken naar 4 variabelen en zetten de variabele `child_id` om in een factor. De overige drie variabelen `age`

(leeftijd in maanden), `comprehension` (hoeveel woorden van de vragenlijst een kind begrijpt) en `production` (hoeveel woorden van de vragenlijst een kind zelf al gebruikt) zijn numeriek.

```
wordbank_dutch_child <- wordbank_dutch_child %>%  
  select(child_id, age, comprehension, production) %>%  
  mutate(child_id = as.factor(child_id))
```

7.1 Distributies zichtbaar maken

We gaan de variabele `age` verder bekijken. Wat is de verdeling van leeftijden in onze dataset? We kunnen met `count()` tellen hoe vaak elke leeftijd voorkomt. We gebruiken hier het commando `print(n = 20)` om meer rijen zichtbaar te maken dan de standaard 10 rijen.

```
wordbank_dutch_child %>%  
  count(age) %>%  
  print(n = 20)
```

A tibble: 16 x 2

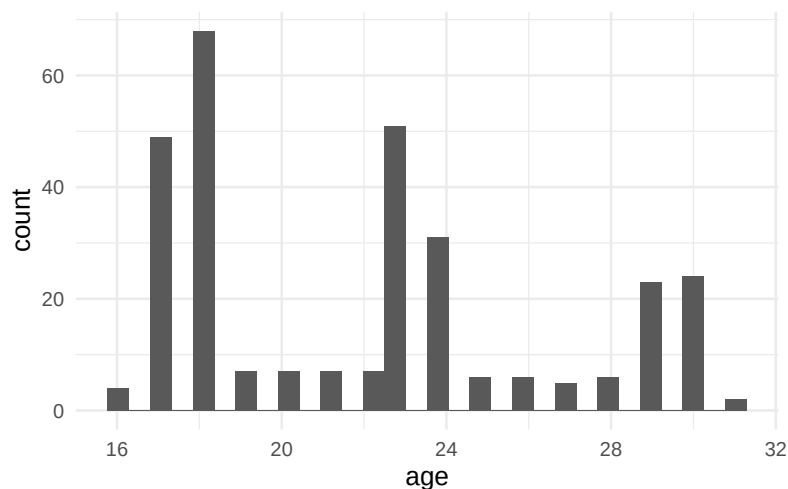
	age	n
	<dbl>	<int>
1	16	4
2	17	49
3	18	68
4	19	7
5	20	7
6	21	7
7	22	7
8	23	51
9	24	31
10	25	6
11	26	6
12	27	5
13	28	6
14	29	23
15	30	24

Voor elke unieke waarde van de variabele `age` krijgen we hier te zien hoe vaak deze voorkomt in de dataset.

Dit is in feite ook wat een histogram voor ons doet: het telt het aantal voorkomens waarbij het de numerieke variabele opdeelt in een aantal groepen. Die groepen worden *bins* genoemd. Elke staaf in het histogram beeldt een bin uit. Het standaard aantal bins voor `geom_histogram()` is 30 en dat levert niet altijd het beste resultaat, zoals R zelf ook aangeeft met de boodschap ‘Pick better value with ‘binwidth’’. Als we goed kijken naar de data in Figuur 7.1 dan zien we dat de staven de data op een vreemde manier indelen en ieder geval niet per maand. Dit zien we bijvoorbeeld als we kijken naar de hele getallen op de x-as. Soms staat het bijbehorende streepje midden in een staaf (bij 16), soms aan het begin (bij 20) en soms aan het eind (bij 24 en 28).

```
wordbank_dutch_child %>%
  ggplot(aes(x = age)) +
  geom_histogram() +
  theme_minimal(13)
```

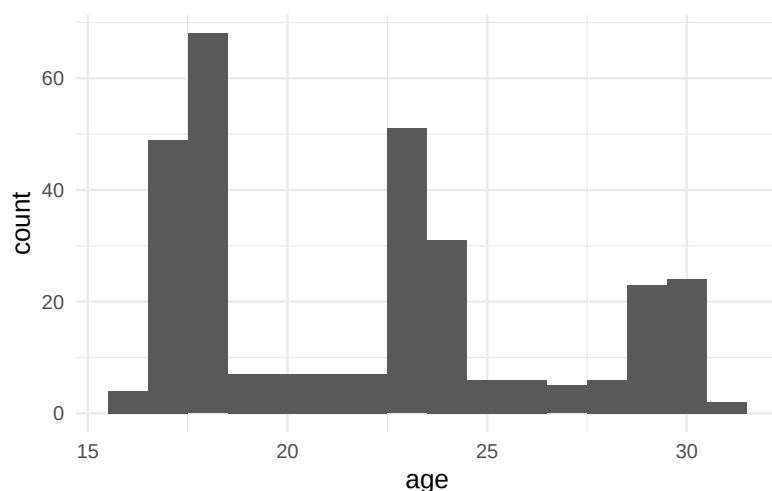
``stat_bin()` using `bins = 30`. Pick better value with `binwidth`.`



Figuur 7.1: Histogram met door R zelf gekozen aantal bins

Het is dus bij een histogram belangrijk om zelf te bedenken hoe je de data wilt opdelen. Met het argument `bins =` binnen `geom_histogram()` kun je aangeven in hoeveel staven je het histogram wilt opdelen. Omdat we bij het tellen hebben gezien dat er 16 verschillende leeftijden voorkomen, ligt het voor deze dataset voor de hand om een getal te nemen waar je 16 door kunt delen om een geheel getal te krijgen. We kiezen hier voor 16. Elke maand krijgt dan zijn eigen staaf.

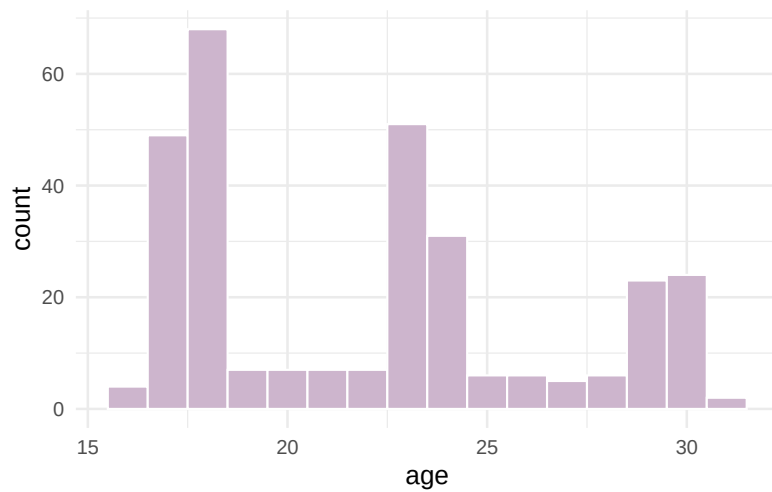
```
wordbank_dutch_child %>%
  ggplot(aes(x = age)) +
  geom_histogram(bins = 16) +
  theme_minimal(13)
```



Figuur 7.2: De staven in dit histogram zijn niet goed van elkaar te onderscheiden.

De staven in Figuur 7.2 zien we alleen nog niet zo duidelijk, omdat zowel de `fill` als de `color` standaard op zwart staan ingesteld. We geven de randen van de staaf de kleur `white` zodat we de bins kunnen zien en geven de staven zelf ook meteen een kleur (Figuur 7.3).

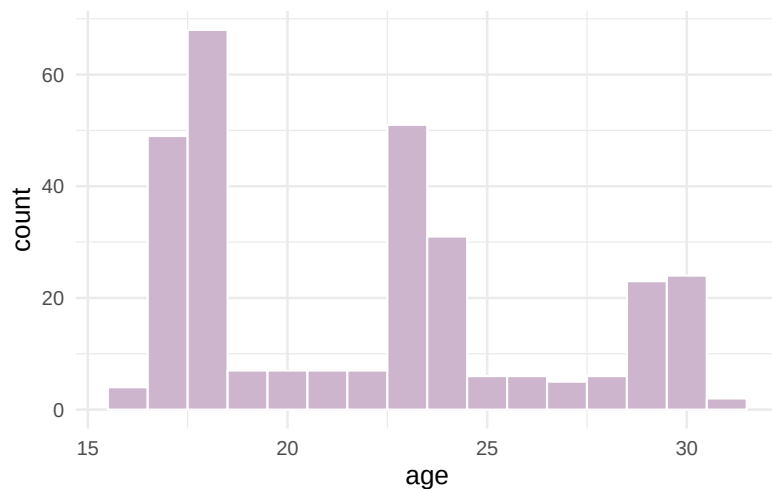
```
wordbank_dutch_child %>%
  ggplot(aes(x = age)) +
  geom_histogram(bins = 16, color = "white", fill = "thistle3") +
  theme_minimal(13)
```



Figuur 7.3: Door de toevoeging `color = white` worden de afzonderlijke staven zichtbaar.

In plaats van het argument `bins` kun je ook `binwidth` gebruiken, zoals R zelf al suggereerde. Met `binwidth` specificeer je de stappen waarin je wilt tellen. Aangezien we hier te maken hebben met gehele getallen kiezen we voor 1, maar je zou bijvoorbeeld ook per 2 of 3 maanden kunnen tellen.

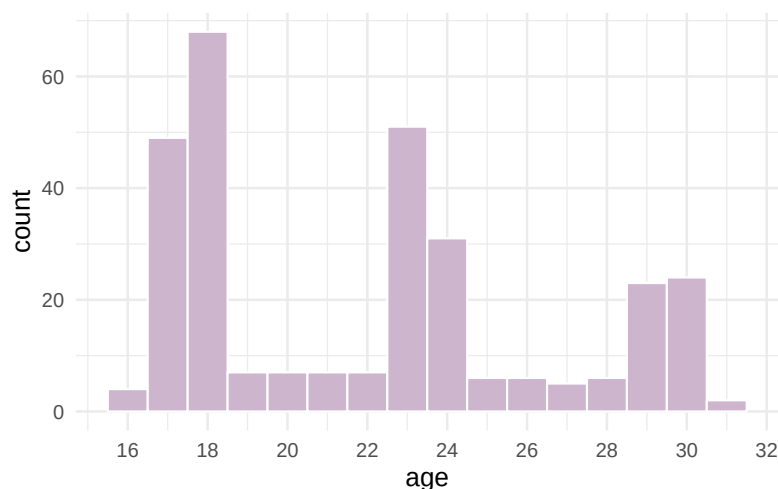
```
wordbank_dutch_child %>%
  ggplot(aes(x = age)) +
  geom_histogram(binwidth = 1, color = "white", fill = "thistle3") +
  theme_minimal(13)
```



Figuur 7.4: Door een logische `binwidth` te kiezen krijg je een beter beeld van de data

Dit histogram in Figuur 7.4 geeft met logische bins een goed beeld van de verdeling van de data. We zien dat een aantal leeftijden veel meer voorkomt in de data dan andere. Door de plaatsing van de getallen op de x-as is het alleen niet meteen duidelijk om welke leeftijden het gaat. Sommige streepjes staan midden in een staaf (bijv. 20 en 25), andere staan tussen staven in (bijv. 17.5). De nummering op de x-as kunnen we aanpassen we met `scale_x_continuous()` (voor de y-as is er `scale_y_continuous` en er zijn ook varianten voor discrete assen). Het argument `breaks` specificeert waar we getallen willen. Dit kun je volledig handmatig doen door een vector van getallen te specificeren met `c()`. Om typewerk te besparen doen we het hier met `seq()`, bekend uit Hoofdstuk 2, waarbij we aangeven dat we de labels bij 16 willen laten beginnen en laten doorlopen tot en met 32 in stapjes van 2 (dus 16, 18, 20, ...). Zie Figuur 7.5.

```
wordbank_dutch_child %>%
  ggplot(aes(x = age)) +
  geom_histogram(binwidth = 1, color = "white", fill = "thistle3") +
  scale_x_continuous(breaks = seq(16, 32, 2)) +
  theme_minimal(13)
```



Figuur 7.5: Door zelf de *breaks* te bepalen kun je de labels van de bins handmatig aanpassen.

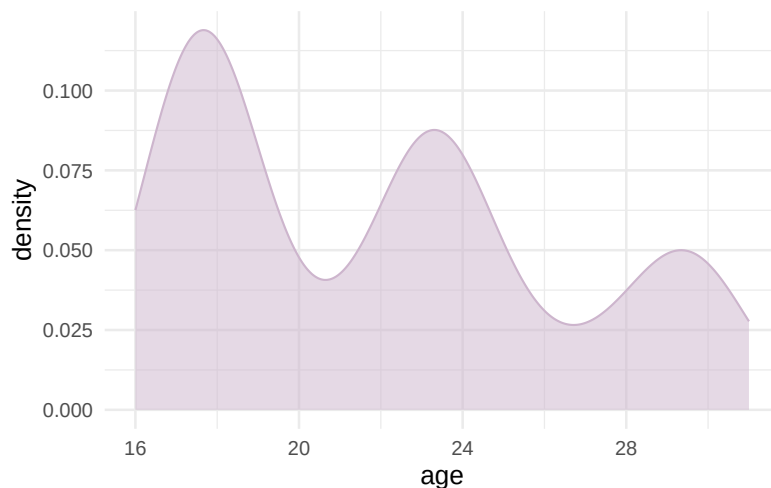
Het blijft bij histogrammen opletten waar elke staaf voor staat. We hebben de waarden in Figuur 7.5 midden in de staaf staan,

maar vaak zie je het ook links of juist rechts van de staaf.

7.1.1 Density

In plaats van een histogram kunnen we de verdeling ook zichtbaar maken met een *density plot* (Figuur 7.6). We hoeven hiervoor alleen een ander geom te specificeren. We gebruiken `geom_density()` en we geven aan dat we de lijn en opvulling van de grafiek in dezelfde kleur willen, met `alpha = .5` geven we aan dat we de opvulling enigszins transparant willen maken.

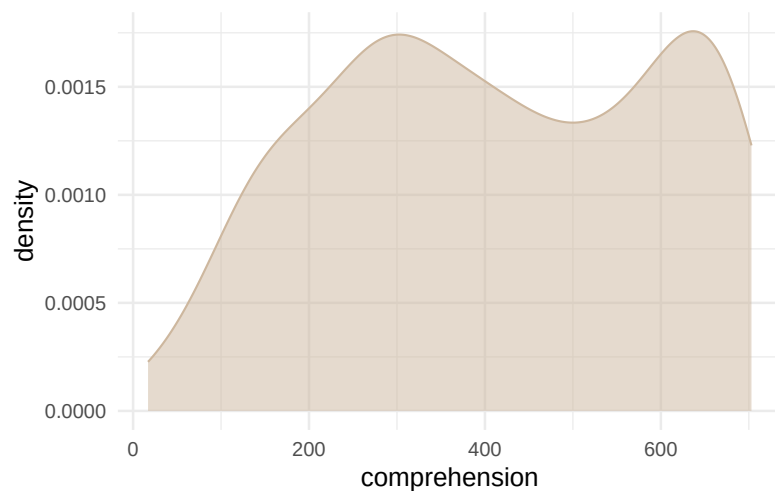
```
wordbank_dutch_child %>%  
  ggplot(aes(x = age)) +  
  geom_density(color = "thistle3", fill = "thistle3", alpha = .5) +  
  theme_minimal(13)
```



Figuur 7.6: Een density plot waarbij de lijn en de oppervlakte onder de grafiek dezelfde kleur hebben gekregen.

Hetzelfde kunnen we doen voor de variabele `comprehension` (Figuur 7.7).

```
wordbank_dutch_child %>%  
  ggplot(aes(x = comprehension)) +  
  geom_density(color = "bisque3", fill = "bisque3", alpha = .5) +  
  theme_minimal(13)
```

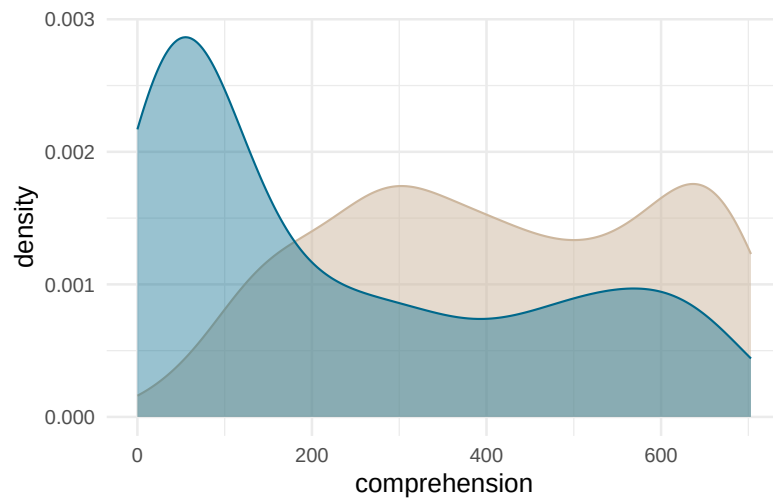


Figuur 7.7: Nog een voorbeeld van een density plot.

De reden dat we de opvulling transparant maken, is dat we dit mooier vinden, maar vooral ook dat dit het mogelijk maakt om meerdere variabelen in een grafiek zichtbaar te maken. Zo kunnen we bijvoorbeeld proberen om ook `production` toe te voegen om zo de verdelingen te vergelijken.

De manier waarop onze dataset is vormgegeven met een aparte kolom voor `comprehension` en een voor `production` maakt dit wel wat uitdagender. Aangezien we in `ggplot()` de variabele `comprehension` al op x hebben afgebeeld, kunnen we daar `production` niet meer toevoegen. De enige optie die we hebben is om gebruik te maken van het feit dat we binnen het geom een nieuwe mapping kunnen maken in de `aes()` van het geom. We voegen dus een nieuw `geom_density()` toe met een eigen kleur (Figuur 7.8. Omdat dit geom later wordt toegevoegd, wordt het over het eerdere heen getekend. De transparantiewaarde met `alpha` komt hier dus goed van pas.

```
wordbank_dutch_child %>%
  ggplot(aes(x = comprehension)) +
  geom_density(color = "bisque3", fill = "bisque3", alpha = .5) +
  geom_density(aes(x = production), color = "deepskyblue4", fill = "deepskyblue4", alpha = .5) +
  theme_minimal(13)
```

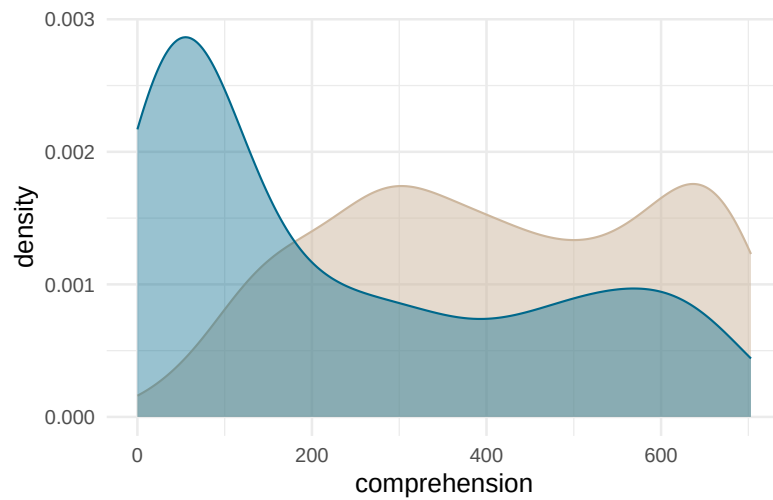


Figuur 7.8: Een eerste poging om twee verdelingen in een density plot zichtbaar te krijgen. Het is nu alleen niet duidelijk welke verdeling bij welke variabele hoort.

Deze grafiek laat de densities van beiden variabelen zien, maar heeft zijn problemen. De x-as zegt nog steeds ‘comprehension’ en het wordt niet duidelijk welke kleur bij welke variabele hoort. Het aanpassen van het label van de x-as via `labs()` gaat dit niet volledig oplossen.

We kunnen proberen om de mapping van `comprehension` naar de x-as te verschuiven van `ggplot()` naar de `aes()` van de eerste aanroep van `geom_density()`, maar ook dit verandert niets (Figuur 7.9).

```
wordbank_dutch_child %>%
  ggplot() +
  geom_density(aes(x = comprehension), color = "bisque3", fill = "bisque3", alpha = .5) +
  geom_density(aes(x = production), color = "deepskyblue4", fill = "deepskyblue4", alpha = .5) +
  theme_minimal(13)
```



Figuur 7.9: Ook met het verschuiven van de `aes`-mapping naar het `geom` wordt het niet duidelijk welke variabelen bij welke verdeling horen.

Het probleem zit dieper. Wanneer we de kleur (of een ander attribuut) willen laten variëren op basis van een variabele, dan moeten de waarden van die variabele in dezelfde kolom staan. Dat is hier niet het geval. We hebben per kind een aparte kolom voor `comprehension` en een voor `production`. Het zijn twee aparte variabelen. De dataset staat in wat we noemen *wide format*.

```
wordbank_dutch_child %>%
  print(n = 2)
```

```
# A tibble: 303 x 4
  child_id   age comprehension production
  <fct>     <dbl>         <dbl>         <dbl>
1 76526      18           162            37
2 76527      18           405            28
# i 301 more rows
```

Dit formaat is handig als we de variabelen apart willen laten zien, maar niet als we ze eigenlijk als waarden van één variabele willen behandelen. Hiervoor moeten we een kolom aanmaken waarin we per kind twee rijen krijgen, een met de waarde voor `comprehension` en een voor `production`. Dit noemen we *long format*. We gaan daarom onze datatabel iets aanpassen door

een nieuwe variabele aan te maken die we `skill` noemen. Dit doen we met de functie `pivot_longer()` (zoals je misschien al had geraden is er ook de tegenhanger `pivot_wider()` voor de tegenstelde omzetting). In het argument `cols` geven we aan welke variabelen we willen samenvoegen tot een nieuwe variabele (de kolomnamen worden de namen van de levels). Met `names_to` geven we de naam van de nieuwe variabele aan en met `values_to` geven we aan in welke variabele de waarden van de oorspronkelijke variabelen terecht gaan komen. Het eerste kind (met id 76526) scoorde 162 op `comprehension` en 37 op `production`. Deze waardes zullen op twee rijen komen in de nieuwe variabele `number_of_words`.

```
dutch_child_long <- wordbank_dutch_child %>%
  pivot_longer(cols = c(comprehension, production),
               names_to = "skill",
               values_to = "number_of_words")

dutch_child_long %>%
  print(n = 4)
```

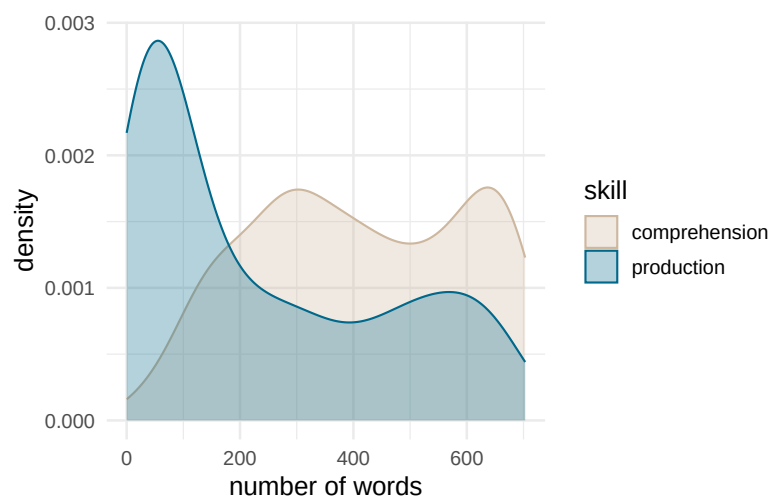
```
# A tibble: 606 x 4
  child_id   age skill      number_of_words
  <fct>     <dbl> <chr>          <dbl>
1 76526      18 comprehension      162
2 76526      18 production         37
3 76527      18 comprehension     405
4 76527      18 production         28
# i 602 more rows
```

We zien nu dat ieder kind twee rijen in de dataset heeft. De lengte van de dataset is dan ook verdubbeld van 303 naar 606. In de eerste twee rijen zien we de scores van 162 en 37 voor kind met id 76526 terug en die zijn in de kolom `skill` gelabelled als `comprehension` en `production`. We zetten de nieuwe variabele `skill` nog even om naar een factor:

```
dutch_child_long <- dutch_child_long %>%
  mutate(skill = as.factor(skill))
```

Nu kunnen we de grafiek maken zoals we hem wilden hebben (Figuur 7.10), waarbij we de nieuwe variabele `skill` afbeelden op de kleur en fill van de density plots. We krijgen hiermee meteen een legenda die aangeeft welke skill bij welke kleur hoort. We hebben nu ook maar één aanroep van `geom_density()` nodig.

```
dutch_child_long %>%
  ggplot(aes(x = number_of_words)) +
  geom_density(aes(color = skill, fill = skill), alpha = .3) +
  scale_fill_manual(values = c("bisque3", "deepskyblue4")) +
  scale_color_manual(values = c("bisque3", "deepskyblue4")) +
  labs(x = "number of words") +
  theme_minimal(13)
```



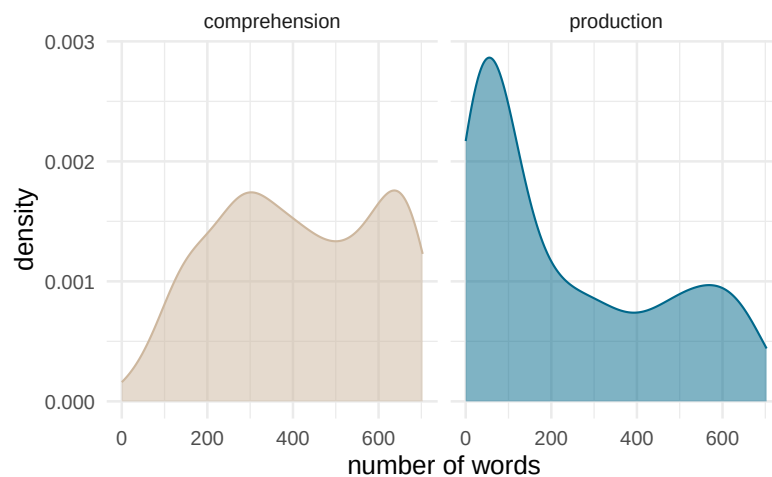
Figuur 7.10: Overlappende density plot op basis van data in *long format*. Nu is wel duidelijk bij welke variabele de verdelingen horen.

De grafiek laat nu duidelijk zien dat er een verschil in verdeling is tussen begrip en productie. Kinderen in deze leeftijd kennen meer woorden dan ze gebruiken.

Als we geen overlap willen van de density plots, maar ze wel in één figuur willen laten zien kunnen we nu ook `facet_wrap()` gebruiken waarbij we de grafiek splitsen op basis van een variabele, in dit geval `skill`. We krijgen dan een aparte grafiek voor `comprehension` en `production` (Figuur 7.11). Omdat deze levels al benoemd zijn in de titels van de aparte plots, is de

legenda overbodig. We halen de legenda weg met `show.legend = F` (waarbij F een afkorting is voor FALSE). De alpha-waarde verhogen we weer iets, aangezien de densities niet meer overlappen. Om voor elke density een andere kleur te krijgen, moeten we twee kleuren meegeven aan `scale_fill_manual()` en `scale_color_manual()`.

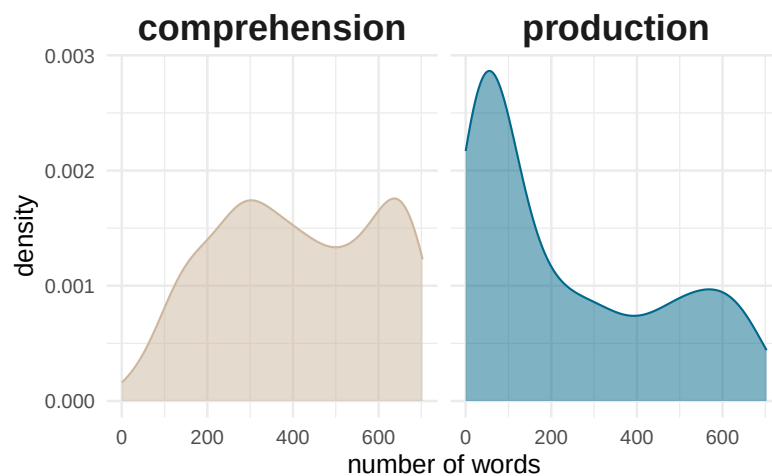
```
dutch_child_long %>%
  ggplot(aes(x = number_of_words)) +
  geom_density(aes(color = skill, fill = skill), alpha = .5, show.legend = F) +
  facet_wrap(~ skill) +
  scale_fill_manual(values = c("bisque3", "deepskyblue4")) +
  scale_color_manual(values = c("bisque3", "deepskyblue4")) +
  labs(x = "number of words") +
  theme_minimal(13)
```



Figuur 7.11: Door met *facets* te werken, krijg je voor ieder level van een variabele een aparte grafiek.

We gebruiken deze grafiek om nog wat extra opties te laten zien om het uiterlijk van grafieken naar je hand te zetten. We gaan hier de titel van de facet wat duidelijker maken door het lettertype te vergroten en dik te drukken. Deze aanpassing doen we weer binnen `theme()`. Met `strip.text` kunnen we de titel aanpassen. We verhogen de grootte naar 18 en zetten de titel in **bold**. Het resultaat vind je in [Figuur 7.12](#)

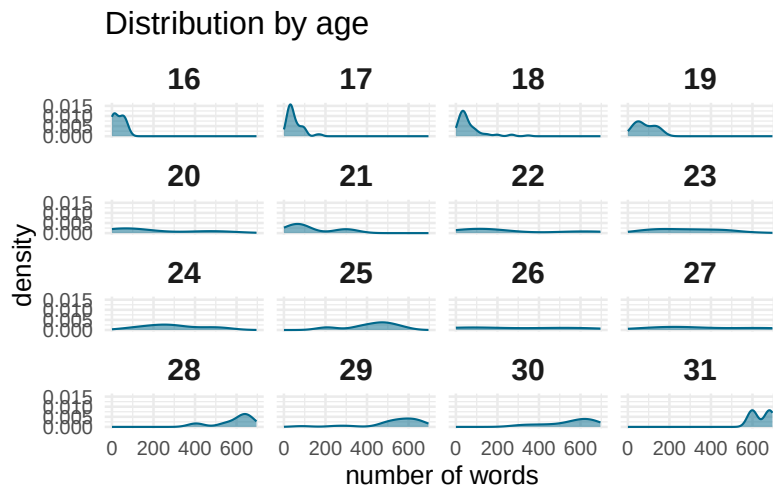
```
dutch_child_long %>%
  ggplot(aes(x = number_of_words)) +
  geom_density(aes(color = skill, fill = skill), alpha = .5, show.legend = F) +
  facet_wrap(~ skill) +
  scale_fill_manual(values = c("bisque3", "deepskyblue4")) +
  scale_color_manual(values = c("bisque3", "deepskyblue4")) +
  labs(x = "number of words") +
  theme_minimal(13) +
  theme(strip.text = element_text(size = 18, face = "bold"))
```



Figuur 7.12: Met wat aanpassingen aan de titels wordt de grafiek beter leesbaar.

Facets kunnen handig zijn om inzicht te krijgen in hoe een variabele zich gedraagt op niveaus van een andere variabele. Aangezien **age** een discrete numerieke variabele is, kunnen we per maand een aparte density plot maken. Bij continue variabelen is dat lastiger, aangezien het dan minder duidelijk is waar je de grenzen moet leggen. We kiezen ervoor om de woordproductie per maand te laten zien (Figuur 7.13). Aangezien we nu in *long format* zitten betekent dat we de data moeten filteren op **skill**. Zo zie je dat het ene formaat handig kan zijn voor de ene soort visualisatie en het andere voor een andere. Het is dus handig als je makkelijk kunt switchen tussen formaten met de **pivot_-**functies.

```
dutch_child_long %>%
  filter(skill == "production") %>%
  ggplot(aes(x = number_of_words)) +
  geom_density(color = "deepskyblue4", fill = "deepskyblue4", alpha = .5, show.legend = F) +
  facet_wrap(~ age) +
  labs(x = "number of words",
       title = "Distribution by age") +
  theme_minimal(13) +
  theme(strip.text = element_text(size = 15, face = "bold"))
```



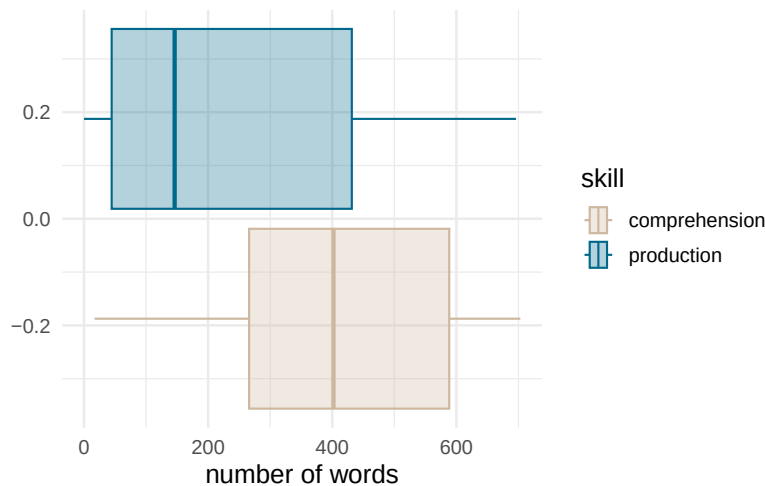
Figuur 7.13: Een facet plot waarbij er voor elke waarde van leeftijd een aparte verdeling wordt weergegeven.

We zien in de figuur een duidelijke ontwikkeling. Aan het begin en het eind zijn er duidelijk pieken en zitten de kinderen allemaal in hetzelfde gebied. Tussen de 20 en 27 maanden is er meer variatie. Sommige kinderen produceren al veel woorden terwijl anderen nog wat lagere aantallen laten zien.

7.1.2 boxplots

Een derde optie om verdelingen zichtbaar te maken, naast histogrammen en density plots, zijn *boxplots* (Figuur 7.14). We zetten onze eerdere density plot Figuur 7.10 om in een boxplot door `geom_density()` te vervangen door `geom_boxplot()`.

```
dutch_child_long %>%
  ggplot(aes(x = number_of_words, color = skill, fill = skill)) +
  geom_boxplot(alpha = .3) +
  scale_fill_manual(values = c("bisque3", "deepskyblue4")) +
  scale_color_manual(values = c("bisque3", "deepskyblue4")) +
  labs(x = "number of words") +
  theme_minimal(13)
```

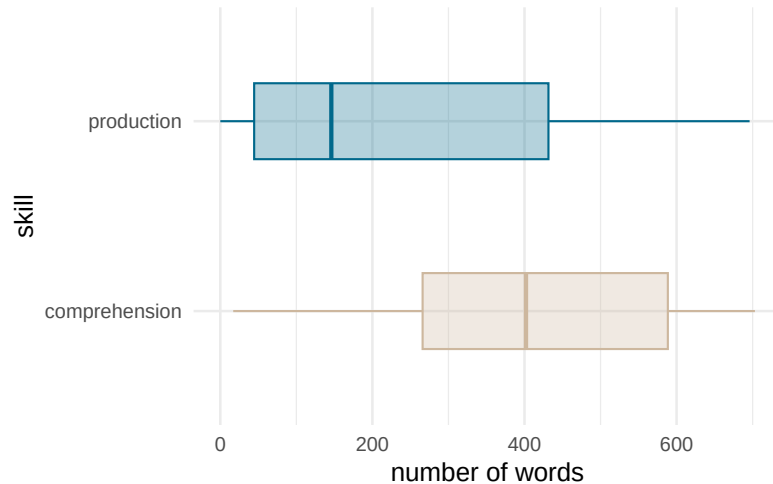


Figuur 7.14: Een eerste versie van een boxplot waarbij de verdeling van de productie en comprehensie van woorden wordt getoond.

Net zoals het histogram komt het maken van een boxplot met een aantal eigenaardigheden. Zo zien we in Figuur 7.14 getallen staan op de y-as. Het lijkt erop dat de levels van de variabele `skill` ieder een waarde hebben gekregen (rond 0.2 en -0.2). Die getallen willen we weghalen, aangezien ze betekenisloos zijn (ze zijn gekoppeld aan de gevraagde breedte van de box door het argument `width = .4`). De oplossing daarvoor is simpel. We voegen `y = skill` toe aan de `aes()`-mapping (hier in `ggplot()`). Om te zorgen dat de grafiek niet redundant wordt, halen we meteen de legenda weg (Figuur 7.15).

```
dutch_child_long %>%
  ggplot(aes(x = number_of_words, y = skill, color = skill, fill = skill)) +
  geom_boxplot(width = .4, alpha = .3, show.legend = F) +
  scale_fill_manual(values = c("bisque3", "deepskyblue4")) +
  scale_color_manual(values = c("bisque3", "deepskyblue4")) +
```

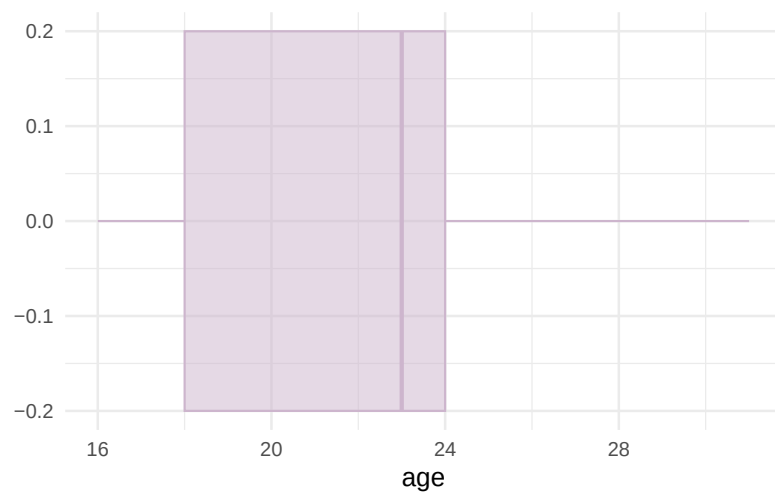
```
labs(x = "number of words") +
theme_minimal(13)
```



Figuur 7.15: Door een kleine aanpassing worden de levels van de variabele *skill* nu getoond als labels op de y-as en is de legenda overbodig geworden.

Het toevoegen van een y-variabele helpt ons ook als we maar één variabele willen afbeelden in een boxplot. Wanneer we dit niet doen krijgen we weer getallen op de y-as, zie Figuur 7.16.

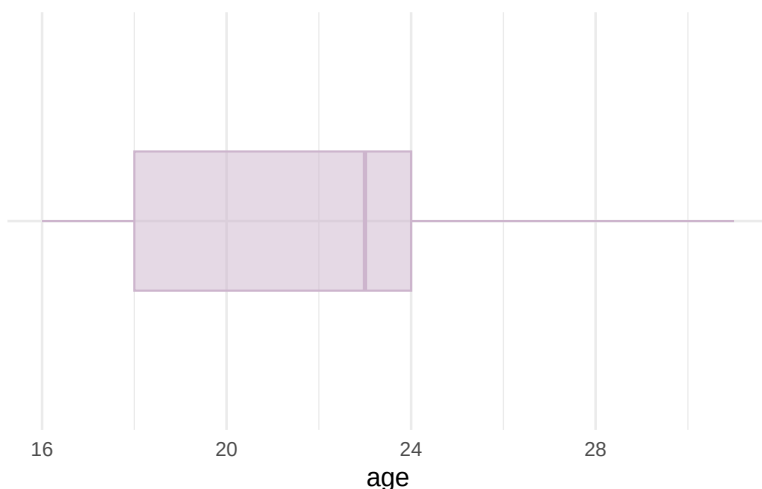
```
dutch_child_long %>%
  filter(skill == "production") %>% #ontdubbelen van de data, elk kind staat er nu 2 keer in
  ggplot(aes(x = age)) +
  geom_boxplot(color = "thistle3", fill = "thistle3", width = .4, alpha = .5) +
  theme_minimal(13)
```



Figuur 7.16: Een boxplot van een variabele waarbij er weer ongewenste getallen verschijnen op de y-as.

Met een omweg kunnen we van deze getallen afkomen (Figuur 7.17). We voegen een *dummy*-variabele toe op `y` met `y = factor(0)` en we specificeren vervolgens in `theme()` met `element_blank()` dat we geen tekst op de y-as willen.

```
dutch_child_long %>%
  filter(skill == "production") %>% #ontdubbelen van de data, elk kind staat er nu 2 keer in
  ggplot(aes(x = age, y = factor(0))) +
  geom_boxplot(color = "thistle3", fill = "thistle3", width = .4, alpha = 0.5) +
  labs(y = NULL) +
  theme_minimal(13) +
  theme(axis.text.y = element_blank())
```



Figuur 7.17: Met een dummy variabele kunnen de ongewenste getallen op de y-as onderdrukt worden.

Hiermee wordt de box in boxplot ook minder breed, in overeenstemming met de door ons gevraagde breedte met het argument `width`.

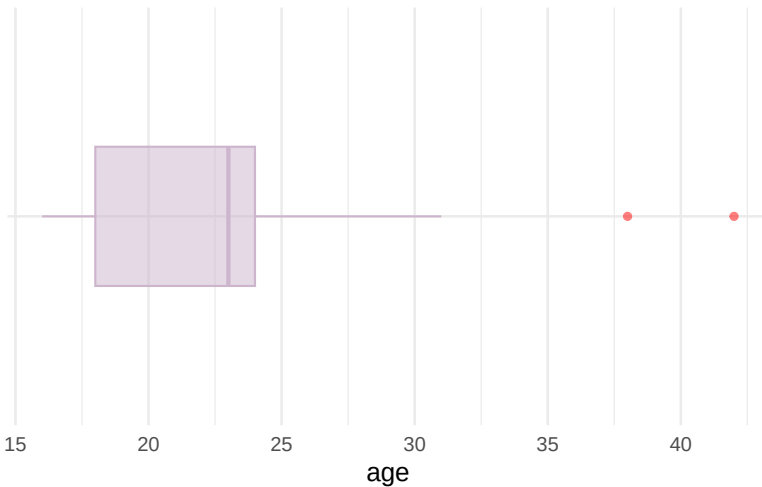
Boxplots zijn ideaal om *outliers* zichtbaar te maken. Dit zijn punten die meer dan $1.5 \times IQR$ onder het eerste kwartiel of boven het derde kwartiel liggen. In de variabelen in onze dataset zitten geen outliers. Voor de oefening voegen we daarom handmatig twee outliers toe. We maken hiervoor eerst een datatabel aan met verzonden data van twee kinderen.

```
outlier_cases <- tribble(~child_id, ~age, ~skill, ~number_of_words,
                        99999, 38, "production", 999,
                        99999, 42, "production", 999)

outlier_cases <- outlier_cases %>%
  mutate(child_id = as.factor(child_id))
```

Met `add_case()` voegen we deze twee observaties toe aan onze datatabel. In `geom_boxplot()` specificeren we met `outlier.color` en `outlier.alpha` de kleur en transparantie van de punten waarmee de outliers worden getoond (Figuur 7.18).

```
dutch_child_long %>%
  filter(skill == "production") %>%
  add_case(outlier_cases) %>%
  ggplot(aes(x = age, y = factor(0))) +
  geom_boxplot(color = "thistle3",
               fill = "thistle3",
               width = .4,
               alpha = 0.5,
               outlier.color = "red",
               outlier.alpha = .5) +
  labs(y = NULL) +
  theme_minimal(13) +
  theme(axis.text.y = element_blank())
```



Figuur 7.18: Outliers worden zichtbaar als punten. De kleur en de transparantie kunnen worden aangepast met `outlier.color` en `outlier.alpha`.

7.2 Beschrijvende statistieken berekenen

R biedt verschillende functies waarmee we beschrijvende statistieken kunnen berekenen. Tabel 7.1 geeft een overzicht van veelgebruikte functies.

functienaamwerking	werkt met <code>summarize()</code>
<code>range()</code> bepaalt het bereik van een variabele (geeft minimale en maximale waarde)	nee
<code>min()</code> bepaalt de minimale waarde van een variabele	ja
<code>max()</code> bepaalt de maximale waarde van een variabele	ja
<code>mean()</code> berekent het gemiddelde van een variabele	ja
<code>median()</code> berekent de mediaan van een variabele	ja
<code>sd()</code> berekent de standaarddeviatie van een variabele	ja
<code>var()</code> berekent de variantie van een variabele	ja
<code>IQR()</code> bepaalt de interkwartielafstand (IQR) van een variabele	ja
<code>quantile()</code> bepaalt de kwantielen van een variabele, hiervoor moet je met het argument <code>probs =</code> aangeven welke kwantielen je wilt	alleen wanneer er maar één kwantiel wordt opgevraagd
<code>summary()</code> geeft een aantal beschrijvende statistieken van een variabele (min, max, 1e en 3e kwartiel, mediaan, gemiddelde)	nee

Tabel 7.1: Overzicht van functies waarmee beschrijvende statistieken kunnen worden berekend

De meeste van deze functies kunnen gecombineerd worden met

`summarize()` (of `summarise()`), de *tidyverse*-functie die je moet gebruiken om beschrijvende statistieken te berekenen. De functies die hier niet mee combineren kun je gebruiken met de base R `$`-notatie om een variabele uit een datatabel te selecteren.

```
summary(wordbank_dutch_child$age)
```

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
16.00	18.00	23.00	22.01	24.00	31.00

Een alternatief hiervoor is de functie `with()` waarin je de naam van de datatabel waarin de variabele zit meegeeft:

```
with(wordbank_dutch_child,
      quantile(age, probs = c(0, .25, .50, .75, 1)))
```

0%	25%	50%	75%	100%
16	18	23	24	31

Met `summarize()` kun je meerdere beschrijvende statistieken tegelijk laten berekenen. Je kunt hierbij zelf een naam geven aan de te berekenen maat. Deze naam komt voor het `=`-teken.

```
wordbank_dutch_child %>%
  summarize(mean_age = mean(age),
            sd_age = sd(age),
            mean_comprehension = mean(comprehension),
            median_comprehension = median(comprehension))
```

```
# A tibble: 1 x 4
  mean_age sd_age mean_comprehension median_comprehension
  <dbl>   <dbl>           <dbl>           <dbl>
1    22.0    4.47             412.             402
```

De functie `summarize()` kun je combineren met `group_by()` om beschrijvende statistieken per level van een (andere) variabele te berekenen.

```
wordbank_dutch_child %>%
  group_by(age) %>%
  summarize(mean_comprehension = mean(comprehension),
            mean_production = mean(production)) %>%
  ungroup() %>%
  print(n = 4)
```

```
# A tibble: 16 x 3
   age mean_comprehension mean_production
<dbl>           <dbl>           <dbl>
1    16             125.             32.5
2    17             243.             50.5
3    18             282.             65.2
4    19             257.             83
# i 12 more rows
```

In *long format* kunnen we `group_by` ook gebruiken om beschrijvende statistieken voor combinaties van variabelen te berekenen:

```
dutch_child_long %>%
  group_by(age, skill) %>%
  summarize(mean_number_words = mean(number_of_words)) %>%
  ungroup() %>%
  print(n = 4)
```

``summarise()`` has grouped output by 'age'. You can override using the ``.groups`` argument.

```
# A tibble: 32 x 3
   age skill      mean_number_words
<dbl> <fct>           <dbl>
1    16 comprehension      125.
2    16 production         32.5
3    17 comprehension      243.
4    17 production         50.5
# i 28 more rows
```

Het package `skimr` biedt een functie `skim()` die je in een keer een overzicht geeft van allerlei statistieken, waaronder ook aantal observaties en missing data, gesorteerd voor categorische en numerieke variabele apart. Hiervoor moet je het package `skimr` natuurlijk wel geïnstalleerd hebben en aanroepen (aan het begin van je script waar je ook alle andere gebruikte packages aanroept).

```
library(skimr)

wordbank_dutch_child %>%
  skim()
```

Name	Piped data
Number of rows	303
Number of columns	4
Column type frequency:	
factor	1
numeric	3
Group variables	None

Tabel 7.2: Data summary

Variable type: factor

skim_variable	missing	complete	ordered	n_unique	top_counts
child_id	0	1	FALSE	210	766: 3, 766: 3, 766: 3, 766: 3

Variable type: numeric

skim_variable	missing	complete	n_unique	mean	sd	p0	p25	p50	p75	p100	hist
age	0	1	22.01	4.47	16	18.0	23	24.0	31		
comprehension	0	1	412.39	188.30	17	266.04	402	588.57	703		
production	0	1	237.49	119.71	10	44.5	146	431.56	696		

De functie produceert zelfs een klein histogram om de verdeling van numerieke variabelen weer te geven. Door de naam van een variabele in te vullen in `skim()` krijg je alleen het overzicht voor die variabele. Ook deze functie kun je gebruiken in combinatie met `group_by()`:

```
dutch_child_long %>%
  group_by(skill) %>%
  skim(number_of_words) %>%
  ungroup()
```

Name	Piped data
Number of rows	606
Number of columns	4
Column type frequency:	
numeric	1
Group variables	skill

Tabel 7.5: Data summary

Variable type: numeric

skim_variable	n_missing	n_complete	mean	sd	p0	p25	p50	p75	p100	hist
number_of_comprehension	1	412.398	88.307	266.040	2588.5703					
number_of_words	1	237.421	919.70	44.5146	431.596					

7.3 Centrummaten zichtbaar maken

Berekende beschrijvende statistieken van numerieke variabelen kunnen we zichtbaar maken met `ggplot`. We gebruiken hiervoor `geom_point()` i.p.v. een staafdiagram. Dit omdat we later ook de ruwe datapunten zichtbaar willen maken. We berekenen als voorbeeld eerst het gemiddelde aantal begrepen en geproduceerde woorden per leeftijd:

```
dutch_means <- dutch_child_long %>%
  group_by(age, skill) %>%
  summarize(mean_number_words = mean(number_of_words)) %>%
  ungroup()
```

``summarise()`` has grouped output by 'age'. You can override using the ``.groups`` argument.

```
dutch_means
```

```
# A tibble: 32 x 3
   age skill      mean_number_words
  <dbl> <fct>          <dbl>
1    16 comprehension          125.
2    16 production             32.5
3    17 comprehension          243.
4    17 production             50.5
5    18 comprehension          282.
6    18 production             65.2
7    19 comprehension          257.
8    19 production              83
9    20 comprehension          414.
10   20 production             188.
# i 22 more rows
```

Vervolgens plotten we deze gemiddelden met behulp van `geom_point()` waarbij we de twee typen skills weer hun eigen kleur geven (Figuur 7.19).

```
dutch_means %>%
  ggplot(aes(x = age, y = mean_number_words, color = skill)) +
  geom_point(size = 3, alpha = .7) +
  labs(y = "mean number of words",
       x = "age in months",
       title = "Word comprehension and production by age") +
  scale_color_manual(values = c("bisque3", "deepskyblue4")) +
  theme_minimal(14)
```



Figuur 7.19: Gemiddelde begrip en productie van woorden in een dotplot.

Figuur 7.19 is niet (of moeilijk) te maken met *wide format*, aangezien we een variabele (`skill`) willen afbeelden op de kleur van de punten. Het *wide format* is wel geschikt om ruwe data te plotten. Die gaan we nu toevoegen.

7.4 De data zichtbaar maken

Centrummaten geven maar een erg beperkt beeld van de data. We willen daarom naast de centrummaat (hier het gemiddelde) ook iets van de spreiding in de data laten zien. We doen dat door van elk kind de geobserveerde waarde te plotten. Op basis van deze geobserveerde waardes per kind hebben we het gemiddelde per leeftijd berekend.

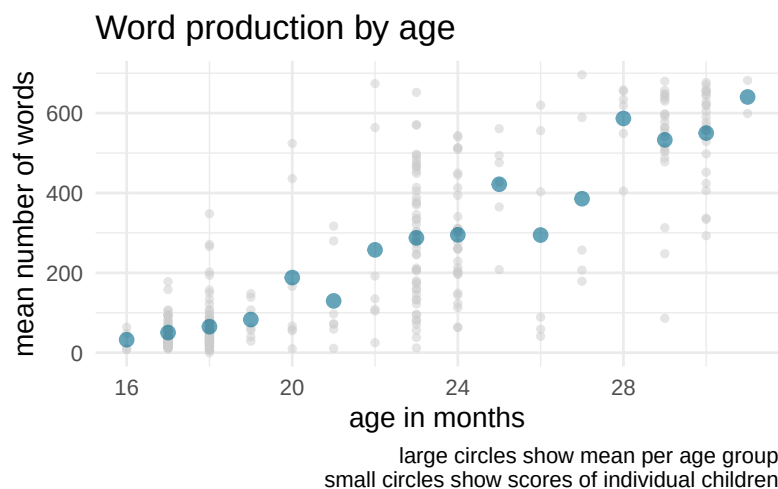
We gebruiken als basis de code voor Figuur 7.19. Om de grafiek overzichtelijk te houden voegen we alleen de datapunten voor `production` toe. We voegen aan deze grafiek de observaties per kind toe via een extra aanroep van `geom_point()` (regel 4 in onderstaand codeblok) waarbij we de waardes uit een andere datatabel (`wordbank_dutch_child`) halen dan waar we de gemiddelden uit halen (`dutch_means`, zie r. 1). De volgorde van de twee aanroepen van `geom_point()` is hier belangrijk. Om de gemiddelden over de individuele punten heen te plotten, gebruiken we de eerste aanroep van `geom_point()` voor de individuele punten en de tweede aanroep voor de gemiddelden.

We voegen aan het eerste `geom_point()` (r. 4) expliciet de dataset toe met het argument `data =` (anders zou de dataset van `ggplot()` worden overgenomen) en we maken ook een nieuwe `aes()` aan (regel 4) om de juiste variabelen uit de dataset zichtbaar te maken. De individuele punten willen we zichtbaar houden en daarom zetten we `alpha` wat lager. Om de punten te onderscheiden van de punten die het gemiddelde weergeven maken we de observaties van individuele kinderen grijs. We lichten de betekenis van de twee typen punten toe door een `caption` toe te voegen aan de figuur onder `labs()` (regel 9).

```

1 dutch_means %>%
2   filter(skill == "production") %>%
3   ggplot(aes(x = age, y = mean_number_words)) +
4   geom_point(data = wordbank_dutch_child, aes(x = age, y = production), color = "grey80", alpha = .6) +
5   geom_point(color = "deepskyblue4", size = 3, alpha = .6) +
6   labs(y = "mean number of words",
7        x = "age in months",
8        title = "Word production by age",
9        caption = "large circles show mean per age group\nsmall circles show scores of individual children")
10  theme_minimal(14)

```



Wat valt op? We zien dat er veel variatie in aantal kinderen per leeftijd zit en dat er veel variatie in de spreiding om

het gemiddelde per leeftijd zit. Bij de leeftijd van 23 is deze spreiding bijvoorbeeld enorm en bij 19 maanden heel beperkt, terwijl bij 26 maanden de individuele observaties erg ver van het gemiddelde af liggen. Het toevoegen van de individuele datapunten maakt meteen duidelijk dat het gemiddelde niet altijd een goede weergave van de onderliggende data is. Je zou nu zelf een vergelijkbare grafiek kunnen maken voor woordbegrip.

Verder leren

- Lees de secties *Histograms and Density Plots* (hoofdstuk 4) en *Continuous Variables by Group or Category* (hoofdstuk 5) van Healy (2026) voor meer inzicht in het visualiseren van numerieke variabelen.
- Hoofdstuk 5 van Wickham et al. (2023) geeft meer informatie over het omzetten van datatabellen van en naar *long/wide* formaat.
- Het [ggplot2 cheat sheet](#) zou inmiddels minder geheimen voor je moeten hebben en geeft een goed overzicht van wat er tot nu besproken is.

Healy, K. (2026). *Data Visualisation: A Practical Introduction*. (2de dr.). Princeton University Press. <https://socviz.co/>

Wickham, H., Çetinkaya-Rundel, M., & Grolemund, G. (2023). *R for Data Science* (2de dr.). O'Reilly. <https://r4ds.hadley.nz/>

Nu zou je in staat moeten zijn Practicum 5 te maken.

Deel I

**Practica {#sec-practica
.unnumbered,}**

Zoals gezegd, coderen leren je alleen door het zelf te doen. Tijdens de cursus is er dan ook elke week een practicum waarin je aan de slag gaat met data en R. Op deze pagina vind je algemene informatie over de werkwijze bij de practica. Lees dit goed door voor het eerste practicum.

Hoe haal je het meeste uit een practicum?

Tijdens de practica lopen de docent en studentassistent rond om vragen te beantwoorden. Je haalt dus het meeste uit een practicum door vragen te stellen. Hiervoor moet je natuurlijk wel vragen hebben. Een goede voorbereiding is hierbij de sleutel. We zien maar al te vaak dat studenten onvoorbereid op het practicum komen en ter plekke de documenten openen. Het resultaat is dat ze aan het eind van het practicum nog met de eerste opdrachten bezig zijn en niet aan de latere opdrachten toekomen, die vaak moeilijker zijn. Die opdrachten moet je dan zonder hulp maken. Verstandiger is het dus om goed voorbereid op het practicum te komen. Hoe bereid je je goed voor?

- Bestudeer de opgegeven hoofdstukken ter voorbereiding op het practicum. Zie hiervoor ook het overzicht in Tabel 7.7;
- Maak voor jezelf een overzicht van de functies die in de hoofdstukken worden besproken. Voeg deze toe aan je eigen codeboek;
- Lees de practicumopdrachten door en bedenk welke functies je bij iedere opdracht zou kunnen gebruiken. Noteer deze bij de opdracht of in een apart document;
- Maak een scriptbestand aan voor het practicum (zie hieronder) en werk al wat code uit. Kijk tegen welke problemen je aanloopt en stel hier vragen over op het practicum.

Practicum	Onderwerp	Voorbereiding
1	Werken met data in R	hoofdstukken 1, 2 en 3

Tabel 7.7: voorbereiding per practicum

Practicum	Onderwerp	Vorbereiding
2	Bewerken van data	Hoofdstuk 4
3	Grafieken maken met <code>ggplot</code>	Hoofdstuk 5
4	Grafieken en tabellen voor categorische data	Hoofdstuk 6
5	Numerieke data visualiseren	Hoofdstuk 7

Wat heb je nodig?

Je werkt binnen `RStudio`. Zorg dat dit programma samen met `R` geïnstalleerd is op je computer (zie sectie 2.1). We werken voor alle practica in hetzelfde `R`-project `OBS_practica.Rproj`. Download hiervoor het bestand `OBS_practica.zip` van Brightspace en pak dat uit op je eigen computer. Zet deze map op een handige locatie op je computer (bijv. in de map voor de cursus `OBS`). Je vindt in de map de volgende bestanden:

- `OBS_practica.Rproj`: hierin staat informatie voor `RStudio` over wat er allemaal bij de practica hoort.
- een map `datasets`: hierin vind je de databestanden die bij de practica in deze cursus gebruikt worden (uitgezonderd die voor de opdrachten). Zie het bestand `ReadMe.txt` voor meer informatie.
- een map `templates_scripts_quarto`: hierin vind je drie templates die je kunt gebruiken bij het uitwerken van je code voor een practicum. Zie het bestand `ReadMe.txt` voor meer informatie.
- `voornaam_achternaam_script_Practicum1.R`: voor Practicum 1 hebben we al een script aangemaakt waarin je de practicumopdracht kunt maken. Dit is het document waarin je verschillende commando's kunt uitwerken en vervolgens laten uitvoeren door `R`. Voor de volgende practica moet je dit zelf doen

door een van de templates te kopiëren uit de map `templates_scripts_quarto`. Verander voor je begint de naam van het bestand door je eigen voor- en achternaam en het cijfer van het betreffende practicum in te vullen (dus bijv. `peter_deswart_script_Practicum1.R`).

Aan de slag

Je begint elk practicum door het bestand `OBS_practica.Rproj` te openen (en dus niet het programma `RStudio`). Wen jezelf dit aan. Op deze manier heb je namelijk meteen toegang tot alle relevante bestanden. Het scheelt je een hoop foutmeldingen (en daarvan komen er nog genoeg). Om aan de slag te gaan, open je dus de aangemaakte folder voor de cursus (bijv. `OBS`) en dubbelklik je op `OBS_practica.Rproj`. `RStudio` opent dan en je kunt gaan werken aan de practicumopdrachten. Klik in het window rechtsonder (met de tab Files open) op `voornaam_achternaam_script_Practicum1.R` (dat je als het goed is al hebt aangepast naar je eigen naam). Let op: zorg dat je voor ieder practicum een nieuw script aanmaakt op basis van het templaet waarin je je eigen naam en het cijfer van het practicum aanpast. Wanneer je het script opent vanuit `RStudio`, zal het zichtbaar worden in een window linksboven in `RStudio`. In het script kun je je code gaan schrijven (denk aan de richtlijnen uit Sectie 3.3). De code kun je uitvoeren door met de cursor op de betreffende regel te gaan staan of een deel ervan te selecteren en vervolgens op 'Run' te klikken. Je kunt hier ook de toetscombinatie `Ctrl + Enter` (`Cmd + enter` op Mac) voor gebruiken (zie ook sectie 1.15 in het boek van Winter (2019) voor andere handige shortcuts die je kunt gebruiken). Het resultaat van de actie wordt dan zichtbaar in de R-console (linksonder in `RStudio`). Je kunt ook commentaar toevoegen in je script: alles op een regel na een `#` wordt niet door R uitgevoerd.

`RStudio` weet dat je met een R-script bezig bent en zal soms

- suggesties geven voor te gebruiken functies;
- dingen die het herkent een kleurtje geven;

Winter, B. (2019). *Statistics for Linguists: An Introduction Using R*. Routledge. <https://doi.org/10.4324/9781315165547>

- spontaan dingen aanvullen, zoals haakjes;
- in de kantlijn een markering zetten als er iets niet lijkt te kloppen.

i Let op

Sla je script regelmatig op, zodat je geen code kwijtraakt. Klik hiervoor op het floppy-disk-symbool of gebruik de sneltoetsen **Ctrl + S**/**Cmd + S**.

Practicum 1

Werken met data in R

Leerdoelen

- Kennismaken met (data in) R
- Inlezen van databestanden voor bewerking in R
- Bewerken van variabelen in R

Voorbereiding

Lees hoofdstukken 1 t/m 3 uit het dictaat. De benodigde R-functies zijn te vinden in het dictaat. Functies die niet in het dictaat besproken worden, worden gegeven in de opdracht. Lees ook de algemene richtlijnen voor practica door.

Introductie

Een taalwetenschapper wil weten of training met een computerprogramma helpt bij het leren van vaste uitdrukkingen door tweedetaalleerders van het Nederlands ten opzichte van een klassikale methode. Het voordeel van het computerprogramma is dat deelnemers gedetailleerde feedback krijgen, terwijl dit klassikaal een stuk moeilijker is. De onderzoeker verwacht daarom dat mensen die met het computerprogramma oefenen meer Nederlandse uitdrukkingen kennen, dan mensen die hebben gewerkt via een klassikale methode.

Om dit te onderzoeken werden Duitse leerders van het Nederlands onderworpen aan een voormeting (pre-test) om hun kennis van Nederlandse uitdrukkingen te testen. Vervolgens werd de groep participanten in tweeën verdeeld.

Groep 1 mocht oefenen met het computerprogramma, terwijl groep 2 klassikaal ging oefenen met uitdrukkingen. Na een week van oefenen werden de deelnemers opnieuw getest op hun kennis van uitdrukkingen. In deze practicumopdracht ga je werken met de data van deze onderzoeker.

i Let op

Sla je script regelmatig op, zodat je geen code kwijtraakt. Klik hiervoor op het floppy-disk-symbool of gebruik de sneltoetsen `Ctrl + S`/`Cmd + S`.

! Probeer het eerst zelf!

Bij de opdrachten vind je naast een beschrijving van de vraag ook een Hint en de Code. Dit om je te helpen als je even niet verder komt. Hoe verleidelijk het ook is om meteen naar het antwoord te gaan, doe dit niet. Houd je aan de volgorde: probeer het eerst zelf, lukt dat niet gebruik de hint, mocht je dan nog vastzitten en er echt niet uitkomen ga dan pas naar het antwoord. Dit gaat je op de lange termijn ontzettend helpen.

Het inlezen van de data

7.5 Opdracht

Zorg het bestand `Dataset.txt` wordt ingelezen in R als datatabel en beschikbaar is voor verwerking onder de naam `uitdrukkingen`. Als het gelukt is, zul je in het venster `Environment`, rechtsboven in RStudio, nu een kopje `Data` vinden met daaronder het dataframe `uitdrukkingen`.

7.6 Hint

Gebruik de functie `read_delim()`, zie sectie 3.2. Zorg wel dat je het package `tidyverse` hebt geladen (en dat het geïnstalleerd is), zie sectie 2.3.

7.7 Code

```
# Aanroepen package tidyverse
library(tidyverse)

# Inlezen van de data uit het bestand Dataset.txt uit de map datasets (N.B. deze code gaat e
uitdrukkingen <- read_delim("datasets/Dataset.txt")
```

Het verkennen van de data

Nu we de data beschikbaar hebben als datatabel in R, kunnen we ons gaan richten op de inhoud: hoe zien de data er precies uit?

Eerste zes rijen zichtbaar maken

7.8 Opdracht

Maak de eerste zes rijen van het databestand zichtbaar om te kijken hoe de data eruitzien.

7.9 Hint

Gebruik de functie `head()`, zie sectie [3.2.1](#).

7.10 Code

```
# Eerste zes rijen zichtbaar maken
head(uitdrukkingen)
```

```
# A tibble: 6 x 6
  Proefpersoon Geslacht Groep Geboortejaar PercentageCorrect_Voormeting
    <dbl>    <chr>    <dbl>      <dbl>                <dbl>
1         1      M      1        1988                 20
2         2      V      2        1992                 36
3         3      M      1        1995                 15
4         4      M      2        1997                 55
```

5	5 V	1	1987	43
6	6 V	2	1999	11

```
# i 1 more variable: PercentageCorrect_Nameting <dbl>
```

Aantal rijen en kolommen

7.11 Opdracht

Hoeveel rijen (observaties) en kolommen (variabelen) telt de dataset? Gebruik hiervoor de relevante R-functies

7.12 Hint

Gebruik de functies `nrow()` en `ncol()`, zie sectie [3.1](#).

7.13 Code

```
# Aantal rijen en kolommen vaststellen
ncol(uitdrukkingen)
```

```
[1] 6
```

```
nrow(uitdrukkingen)
```

```
[1] 10
```

De functie View()

7.14 Opdracht

Een andere manier om de dataset zichtbaar te maken is via de functie `View()`. Deze functie is echter alleen te gebruiken in RStudio. Maak de dataset zichtbaar met deze functie.

7.15 Code

```
#Dataset zichtbaar maken  
View(uitdrukkingen)
```

Namen van de variabelen

7.16 Opdracht

De namen van de variabelen in de aangeleverde dataset voldoen niet aan onze richtlijnen. Wat is het probleem?

7.17 Hint

Zie sectie [3.2.1](#).

7.18 Antwoord

De namen maken geen gebruik van *snake_case*. Er worden hoofdletters gebruikt en soms worden inhoudswoorden aan elkaar geschreven.

7.19 Opdracht

Pas de namen van de variabelen de datatabel aan zodat ze voldoen aan onze richtlijnen.

7.20 Hint

Je kunt dit handmatig doen met de functie `rename()` of met de functie `clean_names()` uit het package `janitor`. Zie sectie [3.2.1](#).

7.21 Code

```
# Aanpassen namen variabelen
uitdrukkingen <- janitor::clean_names(uitdrukkingen)

# alternatief handmatig
uitdrukkingen <- rename(.data = uitdrukkingen,
                        proefpersoon = Proefpersoon,
                        geslacht = Geslacht,
                        groep = Groep,
                        geboortejaar = Geboortejaar,
                        percentage_correct_voormeting = PercentageCorrect_Voormeting,
                        percentage_correct_nameting = PercentageCorrect_Nameting
                        )
```

Meetniveaus van de variabelen

7.22 Opdracht

Beschrijf het type van alle variabelen in het databestand en geef kort aan waarom je voor de betreffende type gekozen hebt.

7.23 Hint

Zie [sectie 1.2](#) uit IMS.

Details van variabelen opvragen

7.24 Opdracht

Gebruik de functie `str()` om de details van de variabelen in het databestand op te vragen. Op deze manier kun je zien van welk type de variabelen volgens R zijn. Komen deze door R

toegekende typen overeen met je eigen antwoorden uit de vorige vraag? Zo niet, wie heeft er gelijk?

7.25 Hint

Zie Tabel 3.1. Zie ook secties 1.5-1.9 uit SfL voor de verschillende type variabelen in R.

7.26 Code

```
# Details variabelen opvragen
str(uitdrukkingen)

spec_tbl_ [10 x 6] (S3: spec_tbl_df/tbl_df/tbl/data.frame)
 $ proefpersoon      : num [1:10] 1 2 3 4 5 6 7 8 9 10
 $ geslacht          : chr [1:10] "M" "V" "M" "M" ...
 $ groep             : num [1:10] 1 2 1 2 1 2 1 2 1 2
 $ geboortejaar      : num [1:10] 1988 1992 1995 1997 1987 ...
 $ percentage_correct_voormeting: num [1:10] 20 36 15 55 43 11 18 29 25 33
 $ percentage_correct_nameting  : num [1:10] 60 46 88 75 81 55 69 45 83 78
- attr(*, "spec")=
 .. cols(
 ..   Proefpersoon = col_double(),
 ..   Geslacht = col_character(),
 ..   Groep = col_double(),
 ..   Geboortejaar = col_double(),
 ..   PercentageCorrect_Voormeting = col_double(),
 ..   PercentageCorrect_Nameting = col_double()
 .. )
- attr(*, "problems")=<externalptr>
```

Bewerken van de data

Soms worden variabelen door R niet herkend als het juiste type, of soms ontbreekt er informatie of wil je zelf nieuwe variabelen toevoegen. Gelukkig heb je de mogelijkheid om deze informatie in R aan te passen of toe te voegen. Let wel op dat dit goed gaat, want anders kun je bij latere analyses in de problemen komen.

Aanpassen van het type van een variabele

7.27 Opdracht

Verander de variabele `proefpersoon` in een `factor`, zodat het door R als een categorische variabele herkend wordt. Doe hetzelfde voor de variabelen `groep` en `geslacht`.

7.28 Hint

Gebruik de functie `mutate()` in combinatie met `as.factor()`, zie sectie [3.2.2](#).

7.29 Code

```
# Aanpassen datatype variabelen
uitdrukkingen <- mutate(
  .data = uitdrukkingen,
  proefpersoon = as.factor(proefpersoon),
  groep = as.factor(groep),
```

```
    geslacht = as.factor(geslacht)
  )
```

```
# Controleren
str(uitdrukkingen)
```

```
tibble [10 x 6] (S3: tbl_df/tbl/data.frame)
 $ proefpersoon      : Factor w/ 10 levels "1","2","3","4",...: 1 2 3 4 5 6 7 8 9 10
 $ geslacht          : Factor w/ 2 levels "M","V": 1 2 1 1 2 2 2 2 1 2
 $ groep             : Factor w/ 2 levels "1","2": 1 2 1 2 1 2 1 2 1 2
 $ geboortejaar      : num [1:10] 1988 1992 1995 1997 1987 ...
 $ percentage_correct_voormeting: num [1:10] 20 36 15 55 43 11 18 29 25 33
 $ percentage_correct_nameting  : num [1:10] 60 46 88 75 81 55 69 45 83 78
```

Toevoegen namen van levels

7.30 Opdracht

Voeg nu de bijbehorende namen van de levels aan de variabele groep toe: computer en klassikaal.

7.31 Hint

Gebruik de functie `fct_recode()`, zie sectie [3.2.2](#).

7.32 Code

```
# Toevoegen van namen aan levels
uitdrukkingen <- mutate(uitdrukkingen,
  groep = fct_recode(groep,
    computer = "1",
    klassikaal = "2"
  )
)
```

```
# Controleren  
str(uitdrukkingen)
```

```
tibble [10 x 6] (S3: tbl_df/tbl/data.frame)  
$ proefpersoon      : Factor w/ 10 levels "1","2","3","4",...: 1 2 3 4 5 6 7 8 9 10  
$ geslacht          : Factor w/ 2 levels "M","V": 1 2 1 1 2 2 2 1 2  
$ groep             : Factor w/ 2 levels "computer","klassikaal": 1 2 1 2 1 2 1 2  
$ geboortejaar      : num [1:10] 1988 1992 1995 1997 1987 ...  
$ percentage_correct_voormeting: num [1:10] 20 36 15 55 43 11 18 29 25 33  
$ percentage_correct_nameting  : num [1:10] 60 46 88 75 81 55 69 45 83 78
```

Nieuwe variabele proportie_correct_voormeting toevoegen

7.33 Opdracht

De variabele `percentage_correct_voormeting` geeft het aantal goede antwoorden op de voormeting weer als een percentage. Maak een nieuwe variabele `proportie_correct_voormeting` waarin het aantal goede antwoorden wordt weergegeven als een proportie. Wat is volgens R het type van deze nieuwe variabele? Leg uit waarom R voor dit type kiest.

7.34 Hint

Zowel percentage als proportie geven het deel van een geheel weer. Het verschil is dat een percentage loopt van 0-100 en een proportie van 0-1. Voor het aanmaken van nieuwe variabelen gebruiken we de functie `mutate()`, zie sectie [3.2.2](#).

7.35 Code

```
# Toevoegen variabele met proportie
uitdrukkingen <- mutate(uitdrukkingen,
                        proportie_correct_voormeting = percentage_correct_voormeting/100
                        )

# Controleren type van de variabele
str(uitdrukkingen)
```

```
tibble [10 x 7] (S3: tbl_df/tbl/data.frame)
 $ proefpersoon      : Factor w/ 10 levels "1","2","3","4",...: 1 2 3 4 5 6 7 8 9 10
 $ geslacht          : Factor w/ 2 levels "M","V": 1 2 1 1 2 2 2 2 1 2
 $ groep             : Factor w/ 2 levels "computer","klassikaal": 1 2 1 2 1 2 1 2 1 2
 $ geboortejaar      : num [1:10] 1988 1992 1995 1997 1987 ...
 $ percentage_correct_voormeting: num [1:10] 20 36 15 55 43 11 18 29 25 33
 $ percentage_correct_nameting  : num [1:10] 60 46 88 75 81 55 69 45 83 78
 $ proportie_correct_voormeting : num [1:10] 0.2 0.36 0.15 0.55 0.43 0.11 0.18 0.29 0.25 0.33
```

Nieuwe variabele verschil_metingen toevoegen

7.36 Opdracht

Maak nu een nieuwe variabele in het databestand met de naam `verschil_metingen`. Deze variabele moet het verschil weergeven tussen de nameting en de voormeting.

7.37 Hint

Ook hierbij kun je de functie `mutate()` gebruiken, zie sectie [3.2.2](#).

7.38 Code

```
# Variabele verschil_metingen aanmaken
uitdrukkingen <- mutate(uitdrukkingen,
                        verschil_metingen = percentage_correct_nameting - percentage_correct_
)
```

Nieuwe variabele leeftijd toevoegen

7.39 Opdracht

Een van de variabelen in het databestand is `geboortejaar`. Voeg een nieuwe variabele `leeftijd` toe aan het databestand. Het experiment is afgenomen in 2017. Bereken automatisch de leeftijd op basis van het geboortejaar van de participanten met behulp van R.

7.40 Hint

En ook dit kan met de functie `mutate()`, zie Sectie [3.2.2](#).

7.41 Code

```
# Toevoegen variabele leeftijd
uitdrukkingen <- mutate(uitdrukkingen,
                        leeftijd = 2017 - geboortejaar
)
```

Een grafiek maken

Laten we de functionaliteit van het `tidyverse` meteen ook maar gebruiken om een grafiek te maken. We maken een grafiek waarin we het percentage correct op de nameting uitzetten

tegen dat van de voormeting en we geven elk datapunt een kleur afhankelijk van de groep waartoe deze behoort. We gebruiken hiervoor `ggplot2`, het populairste onderdeel van `tidyverse` waar je in de rest van de cursus nog veel over gaat leren (vanaf Hoofdstuk 5).

7.42 Opdracht

Hieronder vind je een deel van de code om de grafiek te maken, alleen de variabelen ontbreken nog. Kopieer onderstaande code, voeg de variabelen op de juiste puntjes (...) in en voer de code uit. Bekijk het resultaat: zie je verschil tussen de twee groepen?

```
#deel van de code voor een grafiek waarin percentage_correct_nameting wordt uitgezet tegen p

uitdrukkingen %>%
  ggplot(aes(x = ...,
             y = ...,
             colour = ...)) +
  geom_point(size = 3) +
  theme_minimal(13)
```

7.43 Hint

In deze grafiek zetten we variabele Y uit tegen variabele X. De kleur willen we laten variëren per groep waartoe de leerder behoort.

7.44 Code

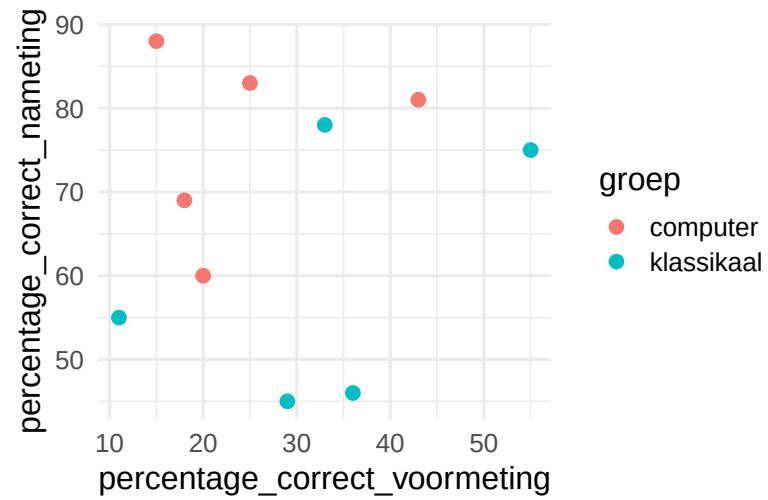
```
#code voor een grafiek waarin percentage_correct_nameting wordt uitgezet tegen percentage_co

uitdrukkingen %>%
  ggplot(aes(x = percentage_correct_voormeting,
```

```

    y = percentage_correct_nameting,
    colour = groep)) +
  geom_point(size = 3) +
  theme_minimal(16)

```



Je hebt Practicum 1 afgerond!

Practicum 2

Bewerken van data

Leerdoelen

- Bekend raken met de belangrijkste ‘werkwoorden’ uit het tidyverse: `filter()`, `arrange()`, `select()`, `mutate()`, `summarize()`, `group_by()` en de pipe operator `%>%`.
- Filteren, splitsen en sorteren van data
- Samenvoegen van datatabellen
- Maken van simpele numerieke samenvattingen van data
- Aanmaken van een steekproef uit een grotere dataset

Vorbereiding

Lees hoofdstuk 4 uit het dictaat. De benodigde R-functies zijn te vinden in het dictaat (zie ook eerdere hoofdstukken en practica). Functies die niet in het dictaat besproken worden, worden gegeven in de opdracht. Het [dplyr cheatsheet](#) geeft een overzicht van relevante functies. Lees ook de [algemene richtlijnen voor practica](#) door.

Introductie

Een taalwetenschapper doet onderzoek naar de *(on)regelmaticheid* van werkwoorden in het Nederlands. In dit onderzoek is de taalwetenschapper geïnteresseerd in de relatie met *etymologische leeftijd* (wanneer een werkwoord is ontstaan) en met *frequentie van voorkomen*. Aangezien er een tendens lijkt te zijn om taal regelmatig te maken, vermoedt de onderzoeker dat onregelmatige werkwoorden (bijv. *blijken* –

bleek) etymologisch gezien vaker terug te voeren zijn op oudere taalstadia. Regelmatige werkwoorden (bijv. *groeien* – *groeide*) zullen daarentegen vaker recentelijker ontstaan zijn. Gezien de tendens om woorden regelmatig te maken, vermoedt de onderzoeker daarnaast dat onregelmatige werkwoorden een hogere woordfrequentie zullen hebben. Deze hebben ze nodig om onregelmatig te blijven: wanneer taalgebruikers een werkwoord vaak horen in onregelmatige vorm, dan zal het werkwoord niet zo snel worden omgezet naar een regelmatige vorm.

Om deze hypothesen te onderzoeken heeft de taalwetenschapper van een grote groep Nederlandse werkwoorden informatie verzameld, waaronder de etymologische leeftijd en de woordfrequentie. Voor elk werkwoord heeft hij bepaald in welk deel van de taalgeschiedenis ze ontstaan zijn. Hiertoe maakt hij onderscheid in 5 taalstadia (van oud naar jong): IndoEuropees, Germaans, WestGermaans, Duits/Nederlands, Nederlands. Van elk woord heeft hij ook de woordfrequentie bepaald door te tellen hoe vaak ze voorkomen in een groot corpus van geschreven taal. Daarnaast heeft hij van elk werkwoord de lengte in letters geteld. Deze data heeft hij opgeslagen in het databestand `werkwoorden.csv`. De onderzoeker is vervolgens ook voor elk werkwoord nagegaan welke type hulpwerkwoord het werkwoord neemt in de voltooide tijd. Deze informatie heeft hij opgeslagen in het databestand `hulpwerkwoorden.txt`.⁹

Let op

Sla je script regelmatig op, zodat je geen code kwijtraakt. Klik hiervoor op het floppy-disk-symbool of gebruik de sneltoetsen `Ctrl + S`/`Cmd + S`.

Probeer het eerst zelf!

Bij de opdrachten vind je naast een beschrijving van de vraag ook een Hint en de Code. Dit om je te helpen als je even niet verder komt. Hoe verleidelijk het ook is om meteen naar het antwoord te gaan, doe dit niet. Houd

⁹ De data zijn afkomstig uit de dataset `etymology` in het package `languageR`.

je aan de volgorde: probeer het eerst zelf, lukt dat niet gebruik de hint, mocht je dan nog vastzitten en er echt niet uitkomen ga dan pas naar het antwoord. Dit gaat je op de lange termijn ontzettend helpen.

Het inlezen van de data

Let op

Zorg dat je het package `tidyverse` hebt geladen. Zie het practicum van vorige week als je bent vergeten hoe dat ook alweer moet.

Databestand inlezen

7.45 Opdracht

Lees het databestand `werkwoorden.csv` in met de bijpassende `tidyverse`-functie, geef de tibble de naam `ww` en maak de eerste zes rijen van de data zichtbaar om te kijken of het inlezen goed is gegaan.

7.46 Hint

Het databestand `werkwoorden.csv` gebruikt een puntkomma als scheidingsteken (open het maar eens). Zie verder Hoofdstuk 3. Het zichtbaar maken van de eerste zes rijen heb je vorige week ook geoefend.

7.47 Code

```
#N.B. in het eerste deel van de uitwerkingen maken we nog geen gebruik van de pipe-operator
# Aanroepen package tidyverse (indien nog niet gedaan)
library(tidyverse)

# Inlezen data onder naam 'ww'
ww <- read_csv2("datasets/werkwoorden.csv")

# N.B. hierboven wordt een relatief pad gebruikt naar het bestand.
# Een absoluut pad zou ook werken (zie hoofdstuk 2)

# Eerste zes rijen zichtbaar maken
head(ww)
```

```
# A tibble: 6 x 5
  Werkwoord Frequentie Regelmatigheid Lengte EtyLeeftijd
  <chr>         <dbl> <chr>         <dbl> <chr>
1 blijken      9.88 irregular      5 Germanic
2 gloeien      6.91 regular      5 Germanic
3 glimmen      7.03 irregular      4 Germanic
4 rijzen       7.37 irregular      4 Germanic
5 werpen       8.49 irregular      4 Germanic
6 delven       6.17 irregular      4 WestGermanic
```

7.48 Opdracht

Pas de namen van de variabelen in de datatabel aan zodat ze voldoen aan onze richtlijnen.

7.49 Hint

Je kunt dit handmatig doen met de functie `rename()` of met de functie `clean_names()` uit het package `janitor`. Zie sectie [3.2.1](#).

7.50 Code

```
# Aanpassen variabelennamen
ww <- janitor::clean_names(ww)

# Controleren of het goed is gegaan
names(ww)
```

```
[1] "werkwoord"      "frequentie"      "regelmatigheid" "lengte"
[5] "ety_leeftijd"
```

Aantal observaties en variabelen

7.51 Opdracht

Hoeveel observaties (rijen) en variabelen (kolommen) telt de dataset?

7.52 Hint

Vorige week hebben we hiervoor de functies `nrow()` en `ncol()` geleerd.

7.53 Code

```
# Aantal observaties (rijen) en variabelen (kolommen)
nrow(ww); ncol(ww)
```

```
[1] 285
```

```
[1] 5
```

```
#alternatief printen van tibble naar de console:
```

```
ww #of:
```

```
# A tibble: 285 x 5
```

	werkwoord	frequentie	regelmatigheid	lengte	ety_leeftijd
	<chr>	<dbl>	<chr>	<dbl>	<chr>
1	blijken	9.88	irregular	5	Germanic
2	gloeien	6.91	regular	5	Germanic
3	glimmen	7.03	irregular	4	Germanic
4	rijzen	7.37	irregular	4	Germanic
5	werpen	8.49	irregular	4	Germanic
6	delven	6.17	irregular	4	WestGermanic
7	snikken	6.72	regular	4	Germanic
8	laten	11.1	irregular	4	Germanic
9	snijden	7.89	irregular	5	Germanic
10	schrikken	8.20	irregular	6	DutchGerman

```
# i 275 more rows
```

```
print(ww)
```

```
# A tibble: 285 x 5
```

	werkwoord	frequentie	regelmatigheid	lengte	ety_leeftijd
	<chr>	<dbl>	<chr>	<dbl>	<chr>
1	blijken	9.88	irregular	5	Germanic
2	gloeien	6.91	regular	5	Germanic
3	glimmen	7.03	irregular	4	Germanic
4	rijzen	7.37	irregular	4	Germanic
5	werpen	8.49	irregular	4	Germanic
6	delven	6.17	irregular	4	WestGermanic
7	snikken	6.72	regular	4	Germanic
8	laten	11.1	irregular	4	Germanic
9	snijden	7.89	irregular	5	Germanic
10	schrikken	8.20	irregular	6	DutchGerman

```
# i 275 more rows
```

Meetniveaus van de variabelen

7.54 Opdracht

Beschrijf de meetniveaus van alle variabelen in het databestand en geef kort aan waarom je voor de betreffende meetniveaus gekozen hebt.

Details van variabelen opvragen

7.55 Opdracht

Van welk type zijn de variabelen in de `tibble` volgens R? Komen deze door R toegekende typen overeen met jouw meetniveaus uit de vorige vraag?

7.56 Hint

Om het door R toegekende type te achterhalen kun je bij `tibbles` de functies `glimpse()` of `str()` gebruiken. Zie sectie [3.2.2](#).

7.57 Code

```
# Details variabelen opvragen. N.B. resultaten kunnen afhangen van je computerinstellingen
glimpse(ww)
```

```
Rows: 285
```

```
Columns: 5
```

```
$ werkwoord      <chr> "blijken", "gloeien", "glimmen", "rijzen", "werpen", "d~
$ frequentie     <dbl> 9.883183, 6.908755, 7.025538, 7.373374, 8.493515, 6.173~
$ regelmatigheid <chr> "irregular", "regular", "irregular", "irregular", "irre~
$ lengte         <dbl> 5, 5, 4, 4, 4, 4, 4, 4, 5, 6, 3, 5, 5, 6, 4, 4, 4, 6~
$ ety_leeftijd   <chr> "Germanic", "Germanic", "Germanic", "Germanic", "German~
```

```
#alternatief:  
str(wv)
```

```
spc_tbl_ [285 x 5] (S3: spec_tbl_df/tbl_df/tbl/data.frame)  
$ werkwoord      : chr [1:285] "blijken" "gloeien" "glimmen" "rijzen" ...  
$ frequentie     : num [1:285] 9.88 6.91 7.03 7.37 8.49 ...  
$ regelmatigheid: chr [1:285] "irregular" "regular" "irregular" "irregular" ...  
$ lengte         : num [1:285] 5 5 4 4 4 4 4 4 5 6 ...  
$ ety_leeftijd   : chr [1:285] "Germanic" "Germanic" "Germanic" "Germanic" ...  
- attr(*, "spec")=  
.. cols(  
..   Werkwoord = col_character(),  
..   Frequentie = col_double(),  
..   Regelmatigheid = col_character(),  
..   Lengte = col_double(),  
..   EtyLeeftijd = col_character()  
.. )  
- attr(*, "problems")=<externalptr>
```

Aanpassen van het type van een variabele

7.58 Opdracht

Pas de variabelen in de datatabel, indien nodig, zodanig aan dat ze overeenkomen met de meetniveaus zoals je die zelf eerder beschreven hebt. Let op: zorg ervoor dat de levels van etymologische leeftijd chronologisch geordend zijn (van oud naar jong): IndoEuropean, Germanic, WestGermanic DutchGerman, Dutch. Maak hier voor de veiligheid een nieuwe variabele van in je datatabel met de naam `ety_age`.

7.59 Hint

Om een nieuwe variabele te maken gebruik je de functie `mutate()`. Zie Hoofdstuk 3. Zie ook practicum 1 voor het omzetten van een variabele naar een factor. Het herordenen

van de levels van een factor kan met de functie `fct_relevel()`. Deze neemt als eerste argument de naam van de variabele waarvan je de levels wilt aanpassen en als tweede argument een vector met de namen van de levels in de juiste volgorde. Zet de namen van de levels tussen aanhalingstekens. Zie `?fct_relevel()` voor meer informatie.

7.60 Code

```
# Eventueel aanpassen van variabelen, bijvoorbeeld in geval werkwoord niet herkend is als fa
ww <- mutate(ww,
              werkwoord = as.factor(werkwoord),
              regelmatigheid = as.factor(regelmatigheid)
            )

# Omzetten ety_leeftijd naar chronologische volgorde in nieuwe variable EtyAge:
ww <- mutate(ww,
              ety_age = as.factor(ety_leeftijd),
              ety_age = fct_relevel(ety_age, c("IndoEuropean", "Germanic", "WestGermanic"))
            )

# Controleren of het goed is gegaan
glimpse(ww) #of:
```

```
Rows: 285
Columns: 6
$ werkwoord      <fct> blijken, gloeien, glimmen, rijzen, werpen, delven, snik~
$ frequentie     <dbl> 9.883183, 6.908755, 7.025538, 7.373374, 8.493515, 6.173~
$ regelmatigheid <fct> irregular, regular, irregular, irregular, irregular, ir~
$ lengte         <dbl> 5, 5, 4, 4, 4, 4, 4, 4, 5, 6, 3, 5, 5, 6, 4, 4, 4, 4, 6~
$ ety_leeftijd   <chr> "Germanic", "Germanic", "Germanic", "Germanic", "German~
$ ety_age        <fct> Germanic, Germanic, Germanic, Germanic, Germanic, WestG~
```

```
str(ww)
```

```
tibble [285 x 6] (S3: tbl_df/tbl/data.frame)
 $ werkwoord      : Factor w/ 285 levels "barsten","bellen",...: 9 50 49 129 248 22 169 93 168 1~
```

```
$ frequentie      : num [1:285] 9.88 6.91 7.03 7.37 8.49 ...
$ regelmatigheid: Factor w/ 2 levels "irregular","regular": 1 2 1 1 1 1 2 1 1 1 ...
$ lengte         : num [1:285] 5 5 4 4 4 4 4 4 5 6 ...
$ ety_leeftijd   : chr [1:285] "Germanic" "Germanic" "Germanic" "Germanic" ...
$ ety_age        : Factor w/ 5 levels "IndoEuropean",...: 2 2 2 2 2 3 2 2 2 4 ...
```

Tibbles bewerken

In de volgende opgaven ga je de belangrijke **tidyverse**-functies `filter()`, `select()`, `mutate()` en `arrange()` uitproberen en andere handige functies beschreven in Hoofdstuk 4. N.B. Elke opgave begint met de tibble `ww` zoals je dat aan het eind van de vorige opdracht hebt gemaakt. Zorg er dus voor dat je deze tibble niet verandert, tenzij dit nadrukkelijk wordt gevraagd. Voor de zekerheid kun je deze tibble ook nog onder een andere naam opslaan (bijv. `ww_backup <- ww`). Je kunt dan altijd terug naar je oorspronkelijke dataset (via `ww_backup -> ww`).

Observaties behouden op basis van waarden

7.61 Opdracht

Beperk de tibble `ww` achtereenvolgens tot:

- een tibble die alleen regelmatige werkwoorden bevat.
- een tibble die alleen onregelmatige werkwoorden bevat die als etymologische leeftijd **Germanic** hebben.
- een tibble die alleen werkwoorden bevat met een frequentie lager dan 5 of hoger dan 10.
- een tibble die alleen werkwoorden bevat met een lengte tussen de 4 en 6 letters (dus 4, 5 of 6).

7.62 Hint

Gebruik hiervoor de functie `filter()`. Zie sectie 4.2.

7.63 Code

```
#a.  
filter(wv,  
       regelmatigheid == "regular")
```

```
# A tibble: 143 x 6  
  woord frequentie regelmatigheid lengte ety_leeftijd ety_age  
  <fct>      <dbl> <fct>          <dbl> <chr>         <fct>  
1 gloeien      6.91 regular          5 Germanic    Germanic  
2 snikken      6.72 regular          4 Germanic    Germanic  
3 zetten       9.78 regular          3 WestGermanic WestGermanic  
4 spelen       9.69 regular          5 WestGermanic WestGermanic  
5 krabben       6.56 regular          4 Germanic    Germanic  
6 scheiden      7.62 regular          6 IndoEuropean IndoEuropean  
7 voegen        8.26 regular          4 Germanic    Germanic  
8 doemen        6.57 regular          4 Germanic    Germanic  
9 snoeren       5.16 regular          5 Germanic    Germanic  
10 schikken      7.61 regular          5 DutchGerman DutchGerman  
# i 133 more rows
```

```
#b.  
filter(wv,  
       regelmatigheid == "irregular",  
       ety_age == "Germanic")
```

```
# A tibble: 92 x 6  
  woord frequentie regelmatigheid lengte ety_leeftijd ety_age  
  <fct>      <dbl> <fct>          <dbl> <chr>         <fct>  
1 blijken      9.88 irregular          5 Germanic    Germanic  
2 glimmen      7.03 irregular          4 Germanic    Germanic  
3 rijzen       7.37 irregular          4 Germanic    Germanic  
4 werpen       8.49 irregular          4 Germanic    Germanic  
5 laten       11.1 irregular          4 Germanic    Germanic  
6 snijden      7.89 irregular          5 Germanic    Germanic  
7 blazen       7.46 irregular          5 Germanic    Germanic  
8 worden      12.7 irregular          4 Germanic    Germanic  
9 smijten       6.85 irregular          5 Germanic    Germanic
```

```
10 meten          7.37 irregular          4 Germanic      Germanic
# i 82 more rows
```

```
#c.
filter(wv,
  frequentie < 5 | frequentie > 10)
```

```
# A tibble: 45 x 6
  werkwoord frequentie regelmatigheid lengte ety_leeftijd ety_age
  <fct>          <dbl> <fct>          <dbl> <chr>          <fct>
1 laten          11.1 irregular          4 Germanic      Germanic
2 spreken         10.1 irregular          6 WestGermanic WestGermanic
3 rijgen          4.92 irregular          4 DutchGerman   DutchGerman
4 worden         12.7 irregular          4 Germanic      Germanic
5 zien           11.5 irregular          3 Germanic      Germanic
6 pluizen         4.03 irregular          5 DutchGerman   DutchGerman
7 hebben         13.1 irregular          3 IndoEuropean IndoEuropean
8 zeggen          11.8 irregular          3 Germanic      Germanic
9 gaan           11.8 irregular          2 Germanic      Germanic
10 zwieren         4.86 regular          5 Dutch         Dutch
# i 35 more rows
```

```
#alternatief:
filter(wv,
  xor(frequentie < 5, frequentie > 10))
```

```
# A tibble: 45 x 6
  werkwoord frequentie regelmatigheid lengte ety_leeftijd ety_age
  <fct>          <dbl> <fct>          <dbl> <chr>          <fct>
1 laten          11.1 irregular          4 Germanic      Germanic
2 spreken         10.1 irregular          6 WestGermanic WestGermanic
3 rijgen          4.92 irregular          4 DutchGerman   DutchGerman
4 worden         12.7 irregular          4 Germanic      Germanic
5 zien           11.5 irregular          3 Germanic      Germanic
6 pluizen         4.03 irregular          5 DutchGerman   DutchGerman
7 hebben         13.1 irregular          3 IndoEuropean IndoEuropean
8 zeggen          11.8 irregular          3 Germanic      Germanic
9 gaan           11.8 irregular          2 Germanic      Germanic
```

```
10 zwieren          4.86 regular          5 Dutch          Dutch
# i 35 more rows
```

```
#d.
filter(wv,
  lengte > 3 & lengte < 7)
```

```
# A tibble: 239 x 6
```

	werkwoord	frequentie	regelmatigheid	lengte	ety_leeftijd	ety_age
	<fct>	<dbl>	<fct>	<dbl>	<chr>	<fct>
1	blijken	9.88	irregular	5	Germanic	Germanic
2	gloeien	6.91	regular	5	Germanic	Germanic
3	glimmen	7.03	irregular	4	Germanic	Germanic
4	rijzen	7.37	irregular	4	Germanic	Germanic
5	werpen	8.49	irregular	4	Germanic	Germanic
6	delven	6.17	irregular	4	WestGermanic	WestGermanic
7	snikken	6.72	regular	4	Germanic	Germanic
8	laten	11.1	irregular	4	Germanic	Germanic
9	snijden	7.89	irregular	5	Germanic	Germanic
10	schrikken	8.20	irregular	6	DutchGerman	DutchGerman

```
# i 229 more rows
```

```
#alternatief:
filter(wv,
  between(lengte,4,6)) #zie dplyer cheat sheet
```

```
# A tibble: 239 x 6
```

	werkwoord	frequentie	regelmatigheid	lengte	ety_leeftijd	ety_age
	<fct>	<dbl>	<fct>	<dbl>	<chr>	<fct>
1	blijken	9.88	irregular	5	Germanic	Germanic
2	gloeien	6.91	regular	5	Germanic	Germanic
3	glimmen	7.03	irregular	4	Germanic	Germanic
4	rijzen	7.37	irregular	4	Germanic	Germanic
5	werpen	8.49	irregular	4	Germanic	Germanic
6	delven	6.17	irregular	4	WestGermanic	WestGermanic
7	snikken	6.72	regular	4	Germanic	Germanic
8	laten	11.1	irregular	4	Germanic	Germanic
9	snijden	7.89	irregular	5	Germanic	Germanic
10	schrikken	8.20	irregular	6	DutchGerman	DutchGerman

```
# i 229 more rows
```

Selecteren van variabelen

7.64 Opdracht

Beperk de tibble `ww` achtereenvolgens tot:

- een tibble met alleen de variabelen `werkwoord`, `regelmatigheid` en `ety_age`.
- een tibble met alle variabelen behalve `lengte`.

7.65 Hint

Gebruik hiervoor de functie `select()`. Zie sectie [4.3](#)

7.66 Code

```
#a.  
select(ww, werkwoord, ety_age, regelmatigheid)  
  
# A tibble: 285 x 3  
  werkwoord ety_age      regelmatigheid  
  <fct>      <fct>      <fct>  
1 blijken   Germanic   irregular  
2 gloeien   Germanic     regular  
3 glimmen   Germanic   irregular  
4 rijzen     Germanic   irregular  
5 werpen     Germanic   irregular  
6 delven     WestGermanic irregular  
7 snikken    Germanic     regular  
8 laten      Germanic   irregular  
9 snijden    Germanic   irregular  
10 schrikken DutchGerman irregular  
# i 275 more rows
```

```
#b.
select(ww, -lengte)
```

```
# A tibble: 285 x 5
  woord frequentie regelmatigheid ety_leeftijd ety_age
  <fct>         <dbl> <fct>         <chr>      <fct>
1 blijken      9.88 irregular   Germanic   Germanic
2 gloeien      6.91 regular    Germanic   Germanic
3 glimmen      7.03 irregular   Germanic   Germanic
4 rijzen       7.37 irregular   Germanic   Germanic
5 werpen       8.49 irregular   Germanic   Germanic
6 delven       6.17 irregular   WestGermanic WestGermanic
7 snikken      6.72 regular    Germanic   Germanic
8 laten       11.1 irregular   Germanic   Germanic
9 snijden      7.89 irregular   Germanic   Germanic
10 schrikken    8.20 irregular   DutchGerman DutchGerman
# i 275 more rows
```

Toevoegen van een nieuwe variabele

7.67 Opdracht

Voeg een nieuwe variabele `freq_int` toe aan de tibble `ww` met daarin de frequentie van de werkwoorden afgerond op gehele getallen. Gebruik hierbij de functie `round()`. Zorg dat de nieuwe variabele direct volgt op de variabele `frequentie`.

7.68 Hint

Om variabelen op een specifieke plek in de tibble in te voegen kun je het argument `.before` of `.after` gebruiken bij de functie `mutate()`. Zie sectie [3.2.2](#).

7.69 Code

```
# Toevoegen van nieuwe variabele met mutate()
ww <- mutate(ww,
              freq_int = round(frequentie),
              .after = frequentie)

# Controleren of het goed is gegaan
glimpse(ww)
```

Rows: 285

Columns: 7

```
$ werkwoord      <fct> blijken, gloeien, glimmen, rijzen, werpen, delven, snik~
$ frequentie     <dbl> 9.883183, 6.908755, 7.025538, 7.373374, 8.493515, 6.173~
$ freq_int       <dbl> 10, 7, 7, 7, 8, 6, 7, 11, 8, 8, 10, 7, 10, 10, 5, 7, 8,~
$ regelmatigheid <fct> irregular, regular, irregular, irregular, irregular, ir~
$ lengte         <dbl> 5, 5, 4, 4, 4, 4, 4, 4, 5, 6, 3, 5, 5, 6, 4, 4, 4, 4, 6~
$ ety_leeftijd   <chr> "Germanic", "Germanic", "Germanic", "Germanic", "German~
$ ety_age        <fct> Germanic, Germanic, Germanic, Germanic, Germanic, WestG~
```

Variabelen van plaats verwisselen

7.70 Opdracht

Verwissel de positie van de variabelen `frequentie` en `regelmatigheid`

7.71 Hint

Gebruik hiervoor de functie `relocate()`. Zie sectie [3.2.2](#).

7.72 Code

```
# Verwisselen van positie
relocate(wv,
          frequentie,
          .after = regelmatigheid)

# A tibble: 285 x 7
  werkwoord freq_int regelmatigheid frequentie lengte ety_leeftijd ety_age
  <fct>      <dbl> <fct>                <dbl>  <dbl> <chr>          <fct>
1 blijken      10 irregular            9.88    5 Germanic    Germanic
2 gloeien       7 regular              6.91    5 Germanic    Germanic
3 glimmen       7 irregular            7.03    4 Germanic    Germanic
4 rijzen        7 irregular            7.37    4 Germanic    Germanic
5 werpen        8 irregular            8.49    4 Germanic    Germanic
6 delven        6 irregular            6.17    4 WestGermanic WestGermanic
7 snikken       7 regular              6.72    4 Germanic    Germanic
8 laten       11 irregular           11.1    4 Germanic    Germanic
9 snijden       8 irregular            7.89    5 Germanic    Germanic
10 schrikken    8 irregular            8.20    6 DutchGerman DutchGerman
# i 275 more rows

# alternatief
relocate(wv,
          regelmatigheid,
          .before = frequentie)
```

De pipe

7.73 Opdracht

Maak een nieuwe tibble `dutch_verbs` met daarin alleen de werkwoorden die Dutch als etymologische leeftijd hebben, maar met behoud van alle andere variabelen behalve `ety_leeftijd`. Maak hierbij gebruik van de pipe operator `%>%`. Hoeveel werkwoorden met deze etymologische leeftijd zijn onregelmatig?

7.74 Hint

Dit is een combinatie van dingen die je in twee eerdere opgaven al hebt gedaan. Je kunt ze combineren met de pipe operator %>%.

7.75 Code

```
# Maken van tibble op basis van ww
dutch_verbs <- ww %>%
  filter(ety_leeftijd == "Dutch") %>%
  select(-ety_leeftijd)

#zichtbaar maken van tibble
print(dutch_verbs)
```

```
# A tibble: 8 x 6
  werkwoord frequentie freq_int regelmatigheid lengte ety_age
  <fct>          <dbl>    <dbl> <fct>          <dbl> <fct>
1 zeuren         6.38        6 regular         4 Dutch
2 zwieren         4.86        5 regular         5 Dutch
3 zwaaien         7.76        8 regular         5 Dutch
4 spugen          6.08        6 irregular        5 Dutch
5 dreigen         7.97        8 regular         5 Dutch
6 hollen          6.95        7 regular         3 Dutch
7 duwen           8.10        8 regular         3 Dutch
8 splitsen        6.06        6 regular         6 Dutch
```

Ordenen van een dataset

7.76 Opdracht

Orden je nieuwe tibble `dutch_verbs` uit de vorige vraag op basis van de frequentie van de werkwoorden, een keer met oplopende en een keer met aflopende frequentie. Wat is de hoogste frequentie bij deze groep werkwoorden?

7.77 Hint

Gebruik de functie `arrange()` eventueel in combinatie met `desc()`. Zie sectie 4.5.

7.78 Code

```
# Ordenen van data frame oplopend  
dutch_verbs %>% arrange(frequentie)
```

```
# A tibble: 8 x 6  
  werkwoord frequentie freq_int regelmatigheid lengte ety_age  
  <fct>         <dbl>    <dbl> <fct>         <dbl> <fct>  
1 zwieren         4.86        5 regular         5 Dutch  
2 splitsen        6.06        6 regular         6 Dutch  
3 spugen          6.08        6 irregular        5 Dutch  
4 zeuren          6.38        6 regular         4 Dutch  
5 hollen          6.95        7 regular         3 Dutch  
6 zwaaien         7.76        8 regular         5 Dutch  
7 dreigen         7.97        8 regular         5 Dutch  
8 duwen           8.10        8 regular         3 Dutch
```

```
# Ordenen van data frame aflopend  
dutch_verbs %>% arrange(desc(frequentie))
```

```
# A tibble: 8 x 6  
  werkwoord frequentie freq_int regelmatigheid lengte ety_age  
  <fct>         <dbl>    <dbl> <fct>         <dbl> <fct>  
1 duwen           8.10        8 regular         3 Dutch  
2 dreigen         7.97        8 regular         5 Dutch  
3 zwaaien         7.76        8 regular         5 Dutch  
4 hollen          6.95        7 regular         3 Dutch  
5 zeuren          6.38        6 regular         4 Dutch  
6 spugen          6.08        6 irregular        5 Dutch  
7 splitsen        6.06        6 regular         6 Dutch  
8 zwieren         4.86        5 regular         5 Dutch
```

Datasets samenvoegen en opslaan

We gaan nu data toevoegen aan onze oorspronkelijke dataset `ww`. Deze informatie halen we uit het databestand `hulpwerkwoorden.txt`. Dit databestand zullen we eerst moeten inlezen en vervolgens moeten we controleren of de gegevens goed gecodeerd zijn in R. Net zoals we eerder hebben gedaan voor de dataset `werkwoorden.csv`.

Inlezen van databestand `hulpwerkwoorden.txt`

7.79 Opdracht

Lees het databestand `hulpwerkwoorden.txt` in met de juiste `read_`-functie en geef de tibble de naam `hww`. Controleer of alle variabelen goed gecodeerd zijn en pas aan indien nodig.

7.80 Hint

Open het databestand `hulpwerkwoorden.txt` om te zien hoe de data zijn opgeslagen (in dit geval gescheiden door een spatie). Bedenk welke functie daarbij hoort.

7.81 Code

```
#inlezen hulpwerkwoorden.txt, delimiter spatie
hww <- read_delim("datasets/hulpwerkwoorden.txt", delim = " ")
```

Rows: 285 Columns: 2

-- Column specification -----

Delimiter: " "

chr (2): Werkwoord, Hulpwerkwoord

i Use `spec()` to retrieve the full column specification for this data.

i Specify the column types or set `show\_col\_types = FALSE` to quiet this message.

```
#Controleren of inlezen goed is gegaan
head(hww)
```

A tibble: 6 x 2

Werkwoord Hulpwerkwoord

<chr> <chr>

1	blijken	zijn
2	gloeien	hebben
3	glimmen	zijnheb
4	rijzen	zijn
5	werpen	hebben
6	delven	hebben

```
glimpse(hww)
```

Rows: 285

Columns: 2

\$ Werkwoord <chr> "blijken", "gloeien", "glimmen", "rijzen", "werpen", "de~

\$ Hulpwerkwoord <chr> "zijn", "hebben", "zijnheb", "zijn", "hebben", "hebben",~

```
# Aanpassen namen
hww <- janitor::clean_names(hww)
```

```
# Beide variabelen omzetten naar factor
hww <- hww %>%
  mutate(werkwoord = as.factor(werkwoord),
         hulpwerkwoord = as.factor(hulpwerkwoord)
  )

#Controleren of het goed is gegaan
glimpse(hww)
```

Rows: 285

Columns: 2

```
$ werkwoord      <fct> blijken, gloeien, glimmen, rijzen, werpen, delven, snikk~
$ hulpwerkwoord <fct> zijn, hebben, zijnheb, zijn, hebben, hebben, hebben, heb~
```

Samenvoegen van de datatabelen

7.82 Opdracht

We gaan de informatie uit `hww` toevoegen aan `ww` met behulp van de functie `left_join()`. Deze functie wordt in SfL (Winter, 2019) beschreven onderaan p. 41. Meer achtergrondinformatie over ‘joins’ vind je in [hoofdstuk 19 van R4DS](#) of het `dplyr` cheatsheet. Geef deze nieuwe datatabel de naam `ww_compleet`.

Winter, B. (2019). *Statistics for Linguists: An Introduction Using R*. Routledge. <https://doi.org/10.4324/9781315165547>

7.83 Hint

De datatabel waar je informatie aan gaat toevoegen komt eerst bij het aanroepen van `left_join()`. Om te controleren of alle informatie aanwezig is, kun je de eerste 6 rijen bekijken en kijken of het totaal aantal rijen van de nieuwe datatabel gelijk is aan de oorspronkelijke datatabel `ww`.

7.84 Code

```
# Samenvoegen ww met hww en controleren of alle informatie er is
ww_compleet <- left_join(ww, hww)
```

Joining with `by = join\_by(werkwoord)`

```
head(ww_compleet)
```

```
# A tibble: 6 x 8
  werkwoord frequentie freq_int regelmatigheid lengte ety_leeftijd ety_age
  <fct>         <dbl>    <dbl> <fct>          <dbl> <chr>      <fct>
1 blijken      9.88      10 irregular      5 Germanic Germanic
2 gloeien      6.91       7 regular        5 Germanic Germanic
3 glimmen      7.03       7 irregular      4 Germanic Germanic
4 rijzen       7.37       7 irregular      4 Germanic Germanic
5 werpen       8.49       8 irregular      4 Germanic Germanic
6 delven       6.17       6 irregular      4 WestGermanic WestGermanic
# i 1 more variable: hulpwerkwoord <fct>
```

```
nrow(ww_compleet)
```

```
[1] 285
```

Opslaan van de nieuwe datatabel

7.85 Opdracht

Sla de datatabel `ww_compleet` op in een csv-bestand met de naam `werkwoorden_compleet.csv` waarbij de waardes worden gescheiden door een puntkomma.

7.86 Hint

Zie sectie [3.2](#) voor informatie over het wegschrijven van data naar een bestand. Net zoals er voor het inlezen van csv-bestanden twee `read_`-functies bestaan, zijn er ook twee functies voor het wegschrijven naar een csv-bestand.

7.87 Code

```
# Wegschrijven data in ww_compleet naar werkwoorden_compleet.csv
write_csv2(ww_compleet, "datasets/werkwoorden_compleet.csv")
```

Numerieke samenvattingen maken

Laten we nu eens proberen of we relevante informatie uit onze data kunnen halen voor het nagaan van de vermoedens van de onderzoeker (zie hiervoor de introductie). Je gebruikt hierbij o.a. de functies `group_by()`, `summarize()` en `count()`. Zie sectie 4.5 of het `dplyr` cheatsheet. Werk op basis van het data frame `ww_compleet`.

Tellen van voorkomens

7.88 Opdracht

Laten we eerst eens wat beter kijken naar de etymologische leeftijd van de werkwoorden. Maak een telling van de categorieën van deze variabele. Hoe vaak komt de categorie `WestGermanic` voor in de dataset?

7.89 Hint

Gebruik hiervoor de functie `count()`. Zie sectie 4.5.

7.90 Code

```
# Telling van etymologische leeftijd
ww_compleet %>% count(ety_leeftijd)
```

```
# A tibble: 5 x 2
  ety_leeftijd      n
  <chr>          <int>
1 Dutch           8
2 DutchGerman    28
3 Germanic      173
4 IndoEuropean   33
5 WestGermanic   43
```

Een telling per groep

7.91 Opdracht

Maak nu een telling van de etymologische leeftijd maar weergegeven per level van de factor `regelmatigheid`. Hoe vaak komt elke categorie van `regelmatigheid` voor in de tijds categorie `DutchGerman`? Komt het vermoeden van de onderzoeker uit dat onregelmatige werkwoorden vaker terug te voeren zijn op oudere taalstadia dan regelmatige werkwoorden?

7.92 Hint

Combineer de functies `group_by()`, `summarise()` en `n()`. Zie sectie [4.5](#).

7.93 Code

```
# Aantal voorkomens per leeftijdscategorie per regelmatigheidstype
ww_compleet %>%
  group_by(regelmatigheid, ety_age) %>%
  summarise(
    aantal = n()
  )
```

`summarise()` has grouped output by 'regelmatigheid'. You can override using the `.groups` argument.

```
# A tibble: 10 x 3
# Groups:   regelmatigheid [2]
  regelmatigheid ety_age      aantal
  <fct>          <fct>      <int>
1 irregular     IndoEuropean    21
2 irregular     Germanic         92
3 irregular     WestGermanic    17
4 irregular     DutchGerman     11
5 irregular     Dutch             1
6 regular       IndoEuropean    12
7 regular       Germanic         81
8 regular       WestGermanic    26
9 regular       DutchGerman     17
10 regular      Dutch             7
```

```
#Alternatief met count()
ww_compleet %>% count(regelmatigheid, ety_age)
```

```
# A tibble: 10 x 3
  regelmatigheid ety_age      n
  <fct>          <fct>    <int>
1 irregular     IndoEuropean    21
2 irregular     Germanic         92
3 irregular     WestGermanic    17
4 irregular     DutchGerman     11
5 irregular     Dutch             1
6 regular       IndoEuropean    12
7 regular       Germanic         81
8 regular       WestGermanic    26
9 regular       DutchGerman     17
10 regular      Dutch             7
```

Gemiddelde waarde per groep berekenen

7.94 Opdracht

Hoe zit het met het vermoeden van de onderzoeker over de frequentie van de regelmatige werkwoorden t.o.v. onregelmatige werkwoorden? Bereken hiervoor de gemiddelde frequentie voor iedere groep.

7.95 Hint

Met de functie `mean()` kun je het gemiddelde van een variabele berekenen.

7.96 Code

```
# Gemiddelde frequentie per regelmatigheidstype
ww_compleet %>%
  group_by(regelmatigheid) %>%
  summarise(
    aantal = n(),
    mean_freq = mean(frequentie)
  )
```

```
# A tibble: 2 x 3
  regelmatigheid aantal mean_freq
<fct>          <int>     <dbl>
1 irregular         142       8.19
2 regular          143       7.79
```

7.97 Opdracht

Verandert het antwoord op de vorige vraag als je alleen kijkt naar werkwoorden die uit meer dan 5 letters bestaan?

7.98 Hint

Perk je de dataset eerst in tot alle werkwoorden die uit meer dan 5 letters bestaan voor je gaat groeperen.

7.99 Code

```
#Gemiddelde frequentie per regelmatigheidstype voor werkwoorden langer dan 5 letters
ww_compleet %>%
  filter(lengte > 5) %>%
  group_by(regelmatigheid) %>%
  summarise(
    aantal = n(),
    mean_freq = mean(frequentie)
  )
```

```
# A tibble: 2 x 3
  regelmatigheid aantal mean_freq
  <fct>          <int>     <dbl>
1 irregular         18       7.29
2 regular          17       6.84
```

Tellingen en proporties

7.100 Opdracht

Nu we toch ook informatie over de hulpwerkwoorden beschikbaar hebben: is er eigenlijk een verschil tussen regelmatige en onregelmatige werkwoorden in hun keuze voor hulpwerkwoord? Maak hiervoor een telling per categorie **regelmatigheid**. Voor de uitdaging: probeer eens of je aan die telling ook een proportie per categorie **regelmatigheid**, afgerond op twee decimalen kunt toevoegen. Je output zou er dan bijvoorbeeld zo uitzien:

regelmatigheid	hulpwerkwoord	n	prop
irregular	hebben	94	0.66
irregular	zijn	12	0.08
irregular	zijnheb	36	0.25
regular	hebben	118	0.83
regular	zijn	3	0.02
regular	zijnheb	22	0.15

De kolom `prop` drukt hier uit hoe vaak er gekozen wordt voor een type hulpwerkwoord binnen een level van de factor `regelmatigheid`. Binnen een categorie tellen de proporties (door afronding) op tot (ongeveer) 1. Om een proportie te berekenen heb je het totaal aantal voorkomens binnen een level van `regelmatigheid` nodig. Hiervoor kun je de functie `sum()` gebruiken. Deze functie telt getallen bij elkaar op (berekent de som van getallen).

7.101 Hint

Bereken eerst hoe vaak elk type hulpwerkwoord voorkomt per categorie van `regelmatigheid`. Dit kan met `count()` of een combinatie van `group_by` en `summarise()`, zoals je hierboven al hebt gedaan. Voor proporties moet je die uitkomsten groeperen per level van `regelmatigheid` om de proporties te kunnen berekenen. Proporties kun je berekenen met de functie `sum()`.

7.102 Code

```
#Gemiddelde frequentie per regelmatigheidstype met count()
ww_tabel <- ww_compleet %>%
  count(regelmatigheid, hulpwerkwoord) %>%
  group_by(regelmatigheid) %>%
  mutate(prop = round(n/sum(n), 2)
  )
ww_tabel
```

```
# A tibble: 6 x 4
# Groups:   regelmatigheid [2]
  regelmatigheid hulpwerkwoord      n  prop
  <fct>          <fct>        <int> <dbl>
1 irregular      hebben         94  0.66
2 irregular      zijn          12  0.08
3 irregular      zijnheb        36  0.25
4 regular        hebben       118  0.83
5 regular        zijn           3  0.02
6 regular        zijnheb       22  0.15
```

```
#Alternatief: gemiddelde frequentie per regelmatigheidstype met dubbel gebruik van group_by()
ww_compleet %>%
  group_by(regelmatigheid, hulpwerkwoord) %>%
  summarise(
    aantal = n()
  ) %>%
  group_by(regelmatigheid) %>%
  mutate(prop = round(aantal/sum(aantal), 2)
  )
```

`summarise()` has grouped output by 'regelmatigheid'. You can override using the `.groups` argument.

```
# A tibble: 6 x 4
# Groups:   regelmatigheid [2]
  regelmatigheid hulpwerkwoord aantal  prop
  <fct>          <fct>        <int> <dbl>
1 irregular      hebben         94  0.66
2 irregular      zijn          12  0.08
3 irregular      zijnheb        36  0.25
4 regular        hebben       118  0.83
5 regular        zijn           3  0.02
6 regular        zijnheb       22  0.15
```

Sampling

Vaak wil je een aantal regels van je data zichtbaar maken. Bijvoorbeeld om te controleren of het inlezen van de data goed is gegaan, zoals we aan het begin van dit practicum hebben gedaan. Je kunt hier de functies `head()` en `tail()` gebruiken voor de eerste of de laatste regels van je bestand. Soms wil je echter juist in het midden van je dataset kijken of een random aantal observaties bekijken. In dat geval kun je gebruik maken van de (varianten van de) functie `slice_`, zie sectie [4.2](#).

Data zichtbaar maken

7.103 Opdracht

Maak alleen rij 16 t/m 27 van de tibble `ww_compleet` zichtbaar in de console met behulp van de functie `slice()`.

7.104 Hint

De functie `slice(x, n)` neemt naast het databestand `x` ook een waarde of range van waarden `n` als argument. Een range geef je aan met `:`.

7.105 Code

```
# Zichtbaar maken van rij 16 t/m 27 uit ww_compleet
ww_compleet %>% slice(16:27)
```

```
# A tibble: 12 x 8
  woord frequentie freq_int regelmatigheid lengte ety_leeftijd ety_age
  <fct>      <dbl>    <dbl> <fct>          <dbl> <chr>      <fct>
1 krabben      6.56        7 regular          4 Germanic  Germanic
2 binden       7.59        8 irregular        4 IndoEuropean IndoEuropean
3 worden     12.7       13 irregular        4 Germanic  Germanic
4 scheiden     7.62        8 regular          6 IndoEuropean IndoEuropean
5 smijten      6.85        7 irregular        5 Germanic  Germanic
6 voegen       8.26        8 regular          4 Germanic  Germanic
7 doemen       6.57        7 regular          4 Germanic  Germanic
8 meten        7.37        7 irregular        4 Germanic  Germanic
9 snoeren     5.16        5 regular          5 Germanic  Germanic
10 schikken    7.61        8 regular          5 DutchGerman DutchGerman
11 stinken     6.97        7 irregular        5 Germanic  Germanic
12 snappen     7.03        7 regular          4 DutchGerman DutchGerman
# i 1 more variable: hulpwoord <fct>
```

Een observatie met maximale of minimale waarde zichtbaar maken

7.106 Opdracht

Wat zijn de drie werkwoorden met de hoogste frequentie in de dataset? En welke drie werkwoorden hebben de laagste frequentie?

7.107 Hint

Gebruik hiervoor de functie `slice_max()` (of `slice_min()`). Zie sectie [4.2](#).

7.108 Code

```
# Vinden van de drie werkwoorden met de hoogste frequentie
```

```
ww_compleet %>% slice_max(frequentie, n = 3)
```

```
# A tibble: 3 x 8
```

	werkwoord	frequentie	freq_int	regelmatigheid	lengte	ety_leeftijd	ety_age
	<fct>	<dbl>	<dbl>	<fct>	<dbl>	<chr>	<fct>
1	hebben	13.1	13	irregular	3	IndoEuropean	IndoEuropean
2	worden	12.7	13	irregular	4	Germanic	Germanic
3	kunnen	12.3	12	irregular	3	Germanic	Germanic

```
# i 1 more variable: hulpwerkwoord <fct>
```

```
# Vinden van de drie werkwoorden met de laagste frequentie
```

```
ww_compleet %>% slice_min(frequentie, n = 3)
```

```
# A tibble: 3 x 8
```

	werkwoord	frequentie	freq_int	regelmatigheid	lengte	ety_leeftijd	ety_age
	<fct>	<dbl>	<dbl>	<fct>	<dbl>	<chr>	<fct>
1	pluizen	4.03	4	irregular	5	DutchGerman	DutchGerman
2	prijzen	4.16	4	regular	5	WestGermanic	WestGermanic
3	spruiten	4.42	4	irregular	6	WestGermanic	WestGermanic

```
# i 1 more variable: hulpwerkwoord <fct>
```

Een simple random sample maken

7.109 Opdracht

Maak een simple random sample van tien werkwoorden uit de dataset `ww_compleet` en schrijf deze weg in een tibble met de naam `ww_sample`.

7.110 Hint

Gebruik hiervoor de functie `slice_sample()`. Zie sectie [4.2](#).

7.111 Code

```
# Een random sample van 10 werkwoorden uit ww_compleet
ww_sample <- ww_compleet %>% slice_sample(n = 10)

# N.B. deze functie zal iedere keer een ander resultaat geven. Als je steeds hetzelfde sample wilt
set.seed(329)
ww_compleet %>% slice_sample(n = 10)
```

A tibble: 10 x 8

	werkwoord	frequentie	freq_int	regelmatigheid	lengte	ety_leeftijd	ety_age
	<fct>	<dbl>	<dbl>	<fct>	<dbl>	<chr>	<fct>
1	voelen	10.3	10	regular	4	WestGermanic	WestGermanic
2	breien	6.56	7	regular	4	Germanic	Germanic
3	gaan	11.8	12	irregular	2	Germanic	Germanic
4	mijden	5.61	6	irregular	4	WestGermanic	WestGermanic
5	krimpen	6.35	6	irregular	5	Germanic	Germanic
6	leven	10.2	10	regular	4	Germanic	Germanic
7	spruiten	4.42	4	irregular	6	WestGermanic	WestGermanic
8	stelen	7.39	7	irregular	5	Germanic	Germanic
9	gooien	8.43	8	regular	4	DutchGerman	DutchGerman
10	zien	11.5	11	irregular	3	Germanic	Germanic

i 1 more variable: hulpwerkwoord <fct>

Een stratified sample maken

7.112 Opdracht A

Maak een *stratified sample* waarbij je uit elke groep van de variabele `regelmatigheid` (de strata) random tien werkwoorden trekt. Rangschik de samples op basis van de frequentie van de werkwoorden.

7.113 Hint

Je kunt de strata maken met behulp van de functie `group_by()`.
Zie sectie [4.5](#).

7.114 Code

```
# Een stratified sample maken met 10 werkwoorden per groep
set.seed(18537) #N.B. om het reproduceerbaar te maken kun je beter weer eerst een random se
ww_compleet %>%
  group_by(regelmatigheid) %>%
  slice_sample(n = 10) %>%
  arrange(frequentie)
```

A tibble: 20 x 8

Groups: regelmatigheid [2]

	ww_compleet	frequentie	freq_int	regelmatigheid	lengte	ety_leeftijd	ety_age
	<fct>	<dbl>	<dbl>	<fct>	<dbl>	<chr>	<fct>
1	hikken	4.76	5	regular	3	Germanic	Germanic
2	schaven	4.80	5	regular	6	Germanic	Germanic
3	snuiten	5.16	5	irregular	5	Germanic	Germanic
4	splijten	5.52	6	irregular	6	Germanic	Germanic
5	slijten	5.74	6	irregular	5	Germanic	Germanic
6	vreten	6.32	6	irregular	5	WestGermanic	WestGermanic
7	schelden	6.49	6	irregular	6	Germanic	Germanic
8	breien	6.56	7	regular	4	Germanic	Germanic
9	zonen	6.65	7	regular	4	DutchGerman	DutchGerman
10	zenden	7.47	7	irregular	4	Germanic	Germanic
11	danken	7.71	8	regular	4	Germanic	Germanic
12	eisen	7.73	8	regular	3	IndoEuropean	IndoEuropean
13	varen	7.84	8	irregular	4	Germanic	Germanic
14	stoppen	8.45	8	regular	4	WestGermanic	WestGermanic
15	drukken	8.79	9	regular	4	Germanic	Germanic
16	hoeven	9.23	9	regular	4	Germanic	Germanic
17	willen	11.2	11	irregular	3	IndoEuropean	IndoEuropean
18	staan	11.3	11	irregular	3	Germanic	Germanic
19	maken	11.4	11	regular	4	WestGermanic	WestGermanic
20	kunnen	12.3	12	irregular	3	Germanic	Germanic

```
# i 1 more variable: hulpwerkwoord <fct>
```

7.115 Opdracht B

Bereken voor je sample uit de vorige opdracht de gemiddelde frequentie voor iedere groep. Hoe representatief is jouw sample voor de hele dataset?

7.116 Hint

Je hebt eerder dezelfde berekening gemaakt voor de hele dataset. Vergelijk je resultaten met die gegevens.

7.117 Code

```
# Eerst sample maken, indien je dat nog niet had opgeslagen onder een andere naam
set.seed(18537)
ww_compleet %>%
  group_by(regelmatigheid) %>%
  slice_sample(n = 10) %>% #en vervolgens de gemiddelden berekenen
  summarise(
    aantal = n(),
    mean_freq = mean(frequentie)
  )
```

```
# A tibble: 2 x 3
  regelmatigheid aantal mean_freq
  <fct>          <int>     <dbl>
1 irregular         10       7.93
2 regular           10       7.60
```

Dat was het voor deze keer. De functies die je hebt geleerd zullen nog vaak van pas komen. Volgende week gaan we ons verdiepen in het maken van grafieken in R.

Practicum 3

Grafieken maken met ggplot2

Leerdoelen

- Begrijpen van de opbouw van grafieken gemaakt met `ggplot2`;
- Aanpassen van bestaande grafieken met `ggplot2`;
- Maken van nieuwe grafieken met `ggplot2`;
- Kennismaken met de mogelijkheden van Quarto.

Vorbereiding

Lees hoofdstuk 5 uit het dictaat. De benodigde R-functies zijn te vinden in het dictaat (zie ook eerdere hoofdstukken en practica). Functies die niet in het dictaat besproken worden, worden gegeven in de opdracht. Het [ggplot2 cheatsheet](#) geeft een overzicht van de mogelijkheden die `ggplot2` biedt.

De werking van Quarto zullen we toelichten in een video op Brightspace. Zie hiervoor ook [hoofdstuk 28](#) uit Wickham et al. (2023). Bekijk voor je begint ook nog een keer de [algemene richtlijnen voor practica](#).

Wickham, H., Çetinkaya-Rundel, M., & Grolemund, G. (2023). *R for Data Science* (2de dr.). O'Reilly. <https://r4ds.hadley.nz/>

Introductie

In de opdrachten van dit practicum ga je een aantal grafieken uit het boek IMS aanpassen of vanaf nul proberen na te maken. Hierdoor raak je bekend met de (on)mogelijkheden van `ggplot2`. Bij elke grafiek hoort een aparte dataset. Bij elke opdracht is aangegeven of de data bijgeleverd zijn in een databestand of dat je een package moet installeren en

aanroepen. Probeer bij elke dataset/grafiek eerst goed voor jezelf helder te hebben met wat voor soort data je te maken hebt (wat is het onderwerp van de dataset, welke variabelen zijn er, wat zijn de meetniveaus).

i Let op

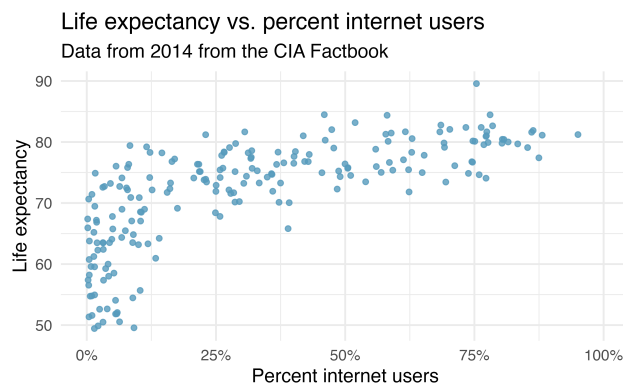
Sla je script regelmatig op, zodat je geen code kwijtraakt. Klik hiervoor op het floppy-disk-symbool of gebruik de sneltoetsen `Ctrl + S`/`Cmd + S`.

! Probeer het eerst zelf!

Bij sommige opdrachten vind je naast een beschrijving van de vraag ook een Hint en de Code. Dit om je te helpen als je even niet verder komt. Hoe verleidelijk het ook is om meteen naar het antwoord te gaan, doe dit niet. Houd je aan de volgorde: probeer het eerst zelf, lukt dat niet gebruik de hint, mocht je dan nog vastzitten en er echt niet uitkomen ga dan pas naar het antwoord. Dit gaat je op de lange termijn ontzettend helpen.

Opdracht 1: Internet use and life expectancy

In deze opdracht maken we de grafiek na uit opgave 11 in hoofdstuk 2 van IMS, zie Figuur 7.20. Hierin is voor een groot aantal landen de levensverwachting (*life expectancy*) uitgezet tegen het percentage internetgebruikers in het land (*percent internet users*). De data zijn afkomstig uit een databestand van de CIA en zijn in het R-project bijgeleverd als `cia_factbook.csv`.



Figuur 7.20: Grafiek bij opgave 11 in hoofdstuk 2 van IMS.

Het inlezen van de data

i Let op

Bij dit practicum maken we gebruik van een aantal packages. Installeer de volgende packages (als je dat niet al eerder hebt gedaan) en roep ze aan: `tidyverse`, `scales`, `ukbabynames` en `openintro`.

7.118 Opdracht

Lees het databestand `cia_factbook.csv` in met de juiste `tidyverse`-functie en noem de tibble `cia_factbook`. Bekijk de variabelen en pas indien nodig hun meetniveau aan.

7.119 Hint

Dit heb je in het vorige practicum ook gedaan. Zie dus het practicum van vorige week en Hoofdstuk 3.

7.120 Code

```
# inlezen data
cia_factbook <- read_csv("datasets/cia_factbook.csv")
```

Rows: 234 Columns: 12

-- Column specification -----

Delimiter: ","

chr (2): country, continent

dbl (10): area, birth_rate, death_rate, infant_mortality_rate, internet_user...

i Use ``spec()`` to retrieve the full column specification for this data.

i Specify the column types or set ``show_col_types = FALSE`` to quiet this message.

```
# omzetten relevante variabelen naar factor
cia_factbook <- cia_factbook %>%
  mutate(country = as.factor(country),
         continent = as.factor(continent)
  )
```

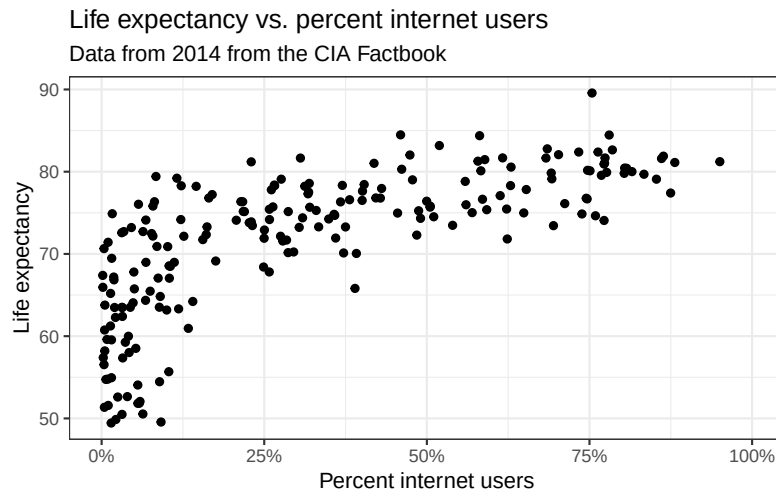
ggplot2-code beter begrijpen

7.121 Opdracht

Met onderstaande code kun je Figuur 7.20 ruwweg namaken (de kleuren zullen nu nog anders zijn). Kopieer de code naar je script en voer de code vervolgens uit. Geef in het script in eigen woorden aan wat elke stap in de code doet. Dit kun je doen door aan het eind van elke regel een comment te plaatsen, voorafgegaan door `#`. Dit is hier al voorgedaan voor regel 10 in de code. Je kunt de hashtag `#` ook gebruiken om te achterhalen wat een deel van de code precies doet. Door `#` voor code te zetten wordt deze niet uitgevoerd. N.B. zorg dan wel dat je de code t/m de laatste bijbehorende `+` uitzet, anders zal R gaan klagen.

```
1 #code om figuur bij opgave 11 van hoofdstuk 2 uit IMS te maken
2 cia_factbook %>%
3   ggplot(aes(y = life_exp_at_birth,
4             x = internet_users/population
5             )) +
6   geom_point() +
7   scale_x_continuous(
8     labels = label_percent(scale = 100),
9     limits = c(0, 1)) +
10  theme_bw() + #past de achtergrond aan naar wit met zwart raster i.p.v. grijs met wit
11  labs(
12    x = "Percent internet users",
13    y = "Life expectancy",
14    title = "Life expectancy vs. percent internet users",
15    subtitle = "Data from 2014 from the CIA Factbook"
16  )
```

Warning: Removed 27 rows containing missing values or values outside the scale range (``geom_point()``).



7.122 Hint

Voer eerst alle code uit en kijk naar grafiek. Schakel dan steeds een stukje code uit met `#` en kijk goed wat er verandert in hoe de grafiek eruit ziet. Zie verder Hoofdstuk 5.

7.123 Code

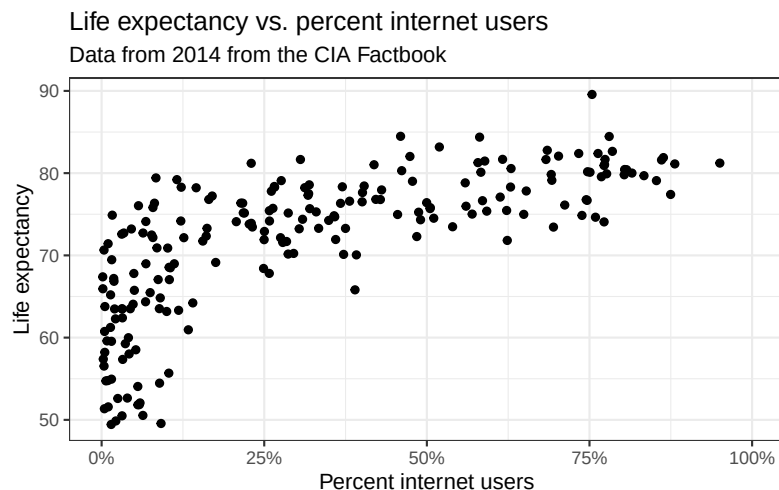
```
1 #code om figuur bij opgave 11 van hoofdstuk 2 uit IMS te maken
2 cia_factbook %>%                                #data naar ggplot sturen
3   ggplot(aes(y = life_exp_at_birth,                #mapt variabele naar y-as
4             x = internet_users/population         #mapt variabele naar x-as, N.B. op basis van t
5             )) +
6   geom_point() +                                  #voegt de punten in de grafiek toe
7   scale_x_continuous(                             #met deze code wordt de x-as aangepast naar ee
8     labels = label_percent(scale = 100),          #scale =100 zorgt dat de percentagens tot 100
9     limits = c(0, 1)) +                          #zorgt dat de x-as loopt van 0 t/m 100 (is 1 x
10  theme_bw() +                                     #past de achtergrond aan naar wit met zwart rast
11  labs(                                             #hier worden namen voor assen gegeven en (onder)
12    x = "Percent internet users",
13    y = "Life expectancy",
14    title = "Life expectancy vs. percent internet users",
```

```

15     subtitle = "Data from 2014 from the CIA Factbook"
16 )

```

Warning: Removed 27 rows containing missing values or values outside the scale range (`geom_point()`).



Interpreten van een waarschuwing

7.124 Opdracht

Toen je de code bij de vorige vraag uitvoerde, kreeg je waarschijnlijk onderstaande waarschuwing. Leg uit wat R je hier vertelt en waarom deze waarschuwing ontstaat.

Warning message: Removed 27 rows containing missing values (`geom_point()`).

7.125 Hint

Bekijk de data van de drie variabelen die in de grafiek worden gebruikt eens goed in de datatabel. Wat valt je op?

7.126 Code

Er zijn 27 landen waarvoor er geen data (NA) beschikbaar zijn voor minstens een van de drie variabelen die gebruikt worden om de grafiek te maken. Dat kun je nagaan met onderstaande code. Deze 27 landen worden door `ggplot2` uit de grafiek gelaten, aangezien de benodigde waarden hiervoor niet bepaald kunnen worden.

```
cia_factbook %>% filter(is.na(life_exp_at_birth) | is.na(internet_users) | is.na(population))
```

country	life_exp_at_birth	internet_users	population
Algeria	76.39	NA	38813722
Angola	55.29	NA	19088106
Nigeria	52.62	NA	177155754
South Sudan	NA	NA	11562695
Western Sahara	62.27	NA	554795
Korea, North	69.81	NA	24851627
Benin	61.07	NA	10160556
Svalbard	NA	NA	1872
Montenegro	NA	280000	650036
Falkland Islands (Islas Malvinas)	NA	2900	3361
Turks and Caicos Islands	79.55	NA	49070
Isle of Man	80.98	NA	86866
Northern Mariana Islands	77.64	NA	51483
Palau	72.60	NA	21186
Curacao	77.98	NA	146836
Niue	NA	1100	1190
Saint Pierre and Miquelon	80.26	NA	5716
American Samoa	74.91	NA	54517
Christmas Island	NA	464	1530
Pitcairn Islands	NA	NA	48
Norfolk Island	NA	NA	2210
Sint Maarten	77.61	NA	39689
Nauru	66.40	NA	9488
Tokelau	NA	800	1337
Holy See (Vatican City)	NA	NA	842

country	life_exp_at_birth	internet_users	population
United States Pacific Island Wildlife Refuges	NA	NA	NA
Saint Barthelemy	NA	NA	7267

Kleur van datapunten aanpassen

7.127 Opdracht

Geef de datapunten in de grafiek een blauwe kleur. Je kunt het commando `colours()` gebruiken om te zien welke kleuren standaard beschikbaar zijn in R.

7.128 Hint

Zie sectie [5.2](#).

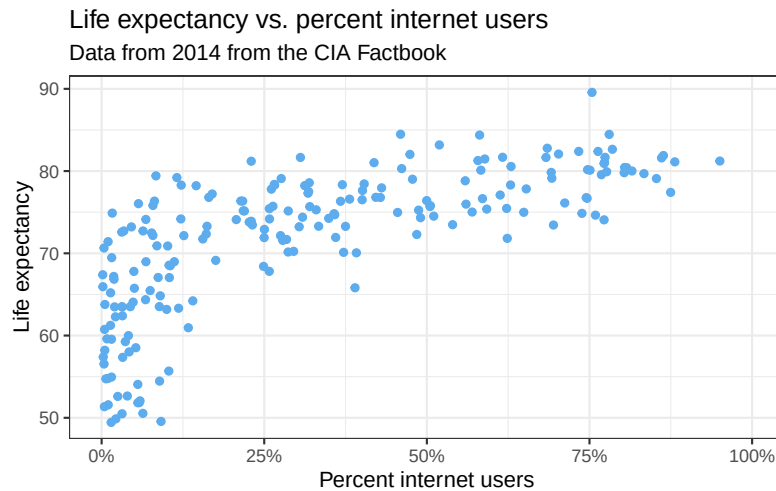
7.129 Code

```

1 #punten blauw maken
2 cia_factbook %>%
3   ggplot(aes(y = life_exp_at_birth,
4             x = internet_users/population #N.B. je kunt NIET hier de mapping colour = "steelblue2"
5             )) +
6   geom_point(colour = "steelblue2") + #kleur van de punten aangepast naar "steelblue2"
7   scale_x_continuous(
8     labels = label_percent(scale = 100),
9     limits = c(0, 1)) +
10  theme_bw() +
11  labs(
12    x = "Percent internet users",
13    y = "Life expectancy",
14    title = "Life expectancy vs. percent internet users",
15    subtitle = "Data from 2014 from the CIA Factbook"

```

Warning: Removed 27 rows containing missing values or values outside the scale range (``geom_point()``).



Datapunten doorzichtig maken

7.130 Opdracht

Een aantal datapunten in de grafiek ligt over elkaar heen. Maak deze datapunten zichtbaar door alle datapunten enigszins transparant te maken.

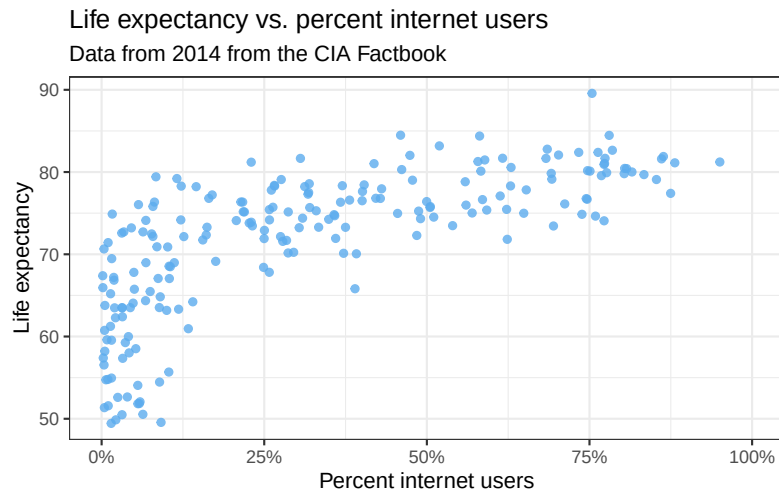
7.131 Hint

Hiervoor gebruik je de setting `alpha`. Zie sectie [5.2](#).

7.132 Code

```
1 #punten transparant maken
2 cia_factbook %>%
3   ggplot(aes(y = life_exp_at_birth,
4             x = internet_users/population
5             )) +
6   geom_point(colour = "steelblue2",
7             alpha = .8) + #transparantie ingesteld met alpha, lager getal is meer transparant
8   scale_x_continuous(
9     labels = label_percent(scale = 100),
10    limits = c(0, 1)) +
11   theme_bw() +
12   labs(
13     x = "Percent internet users",
14     y = "Life expectancy",
15     title = "Life expectancy vs. percent internet users",
16     subtitle = "Data from 2014 from the CIA Factbook"
17   )
```

Warning: Removed 27 rows containing missing values or values outside the scale range (`geom\_point()`).



De kleur van datapunten variëren op basis van een variabele

7.133 Opdracht

Geef de datapunten nu een kleur op basis van het continent waartoe ze behoren. Zie je een patroon in de data?

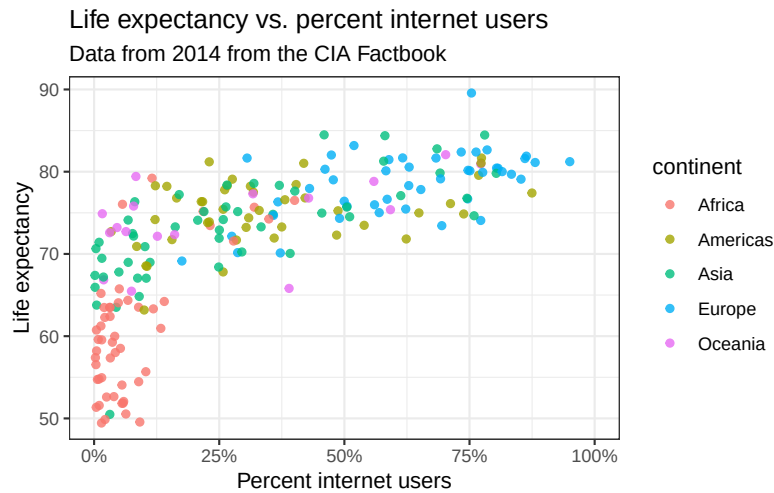
7.134 Hint

Hiervoor zet je de kleur dus niet vast op een waarde, maar maak je een mapping aan van een variabele. Zie sectie 5.1.

7.135 Code

```
1 #punten kleuren op basis van continent
2 cia_factbook %>%
3   ggplot(aes(y = life_exp_at_birth,
4             x = internet_users/population #Je kunt eventueel ook hier de mapping colour =
5             )) +
6   geom_point(aes(colour = continent), #Dit is een mapping naar een variabele, je moet dus ae
7             alpha = .8) +
8   scale_x_continuous(
9     labels = label_percent(scale = 100),
10    limits = c(0, 1)) +
11   theme_bw() +
12   labs(
13     x = "Percent internet users",
14     y = "Life expectancy",
15     title = "Life expectancy vs. percent internet users",
16     subtitle = "Data from 2014 from the CIA Factbook"
17   )
```

Warning: Removed 27 rows containing missing values or values outside the scale range (`geom\_point()`).



Zie je een patroon in de data? De datapunten van sommige continenten clusteren heel duidelijk (bijv. Afrika en Europa). Er lijkt een lichte trend te zijn dat hoe meer internetgebruikers hoe hoger de levensverwachting.

Een trendlijn toevoegen

7.136 Opdracht

Voeg nu voor de hele dataset één zwarte trendlijn toe. Behoud voor de datapunten de kleuren per continent. Wat kun je uit deze trendlijn concluderen?

7.137 Hint

Gebruik hiervoor `geom_smooth()`. Zie sectie [5.2](#).

7.138 Code

```
1 #een trendlijn voor hele dataset
2 cia_factbook %>%
3   ggplot(aes(y = life_exp_at_birth,
4             x = internet_users/population #niet hier de mapping 'colour = continent' toevo
5             )) +
6   geom_point(aes(colour = continent),
7             alpha = .8) +
8   geom_smooth(color = "black") +           #toevoegen van trendlijn met geom_smooth()
9   scale_x_continuous(
10     labels = label_percent(scale = 100),
11     limits = c(0, 1)) +
12   theme_bw() +
13   labs(
14     x = "Percent internet users",
15     y = "Life expectancy",
16     title = "Life expectancy vs. percent internet users",
17     subtitle = "Data from 2014 from the CIA Factbook"
18   )
```

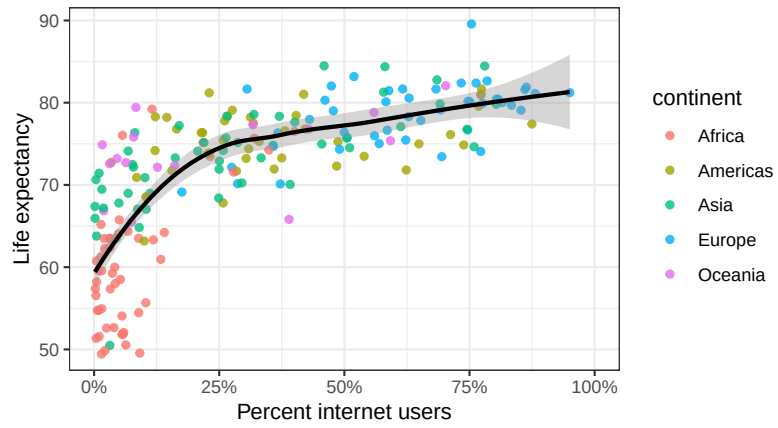
`geom\_smooth()` using method = 'loess' and formula = 'y ~ x'

Warning: Removed 27 rows containing non-finite outside the scale range
(`stat\_smooth()`).

Warning: Removed 27 rows containing missing values or values outside the scale range
(`geom\_point()`).

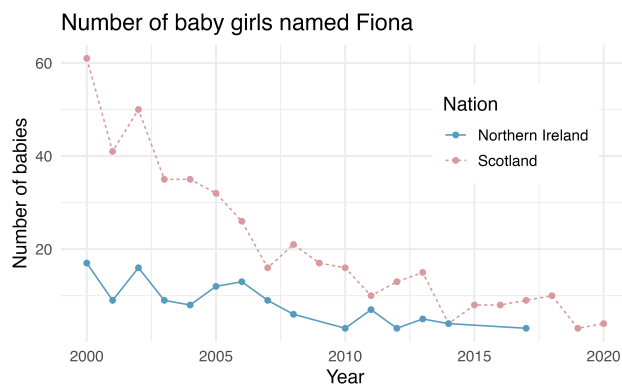
Life expectancy vs. percent internet users

Data from 2014 from the CIA Factbook



Opdracht 2: UK baby names - Fiona

In deze opdracht maken we de grafiek na uit opgave 15 in hoofdstuk 1 van IMS, zie Figuur 7.21. Hierin wordt het aantal meisjes getoond die de naam Fiona hebben gekregen. Het gaat om kinderen geboren in het Verenigd Koninkrijk sinds 2000. De data zijn afkomstig uit het package `ukbabynames`. Dit package moet je geïnstalleerd hebben en aanroepen voor je de opdrachten kunt maken.



Figuur 7.21: Grafiek bij opgave 15 in hoofdstuk 1 van IMS.

Het zichtbaar maken van de datapunten

7.139 Opdracht

Met onderstaande code kun je de aanzet maken voor de grafiek. Als je deze uitvoert, krijg je een lege plot (zie Figuur 7.22). Pas de code nu zodanig aan dat de datapunten zichtbaar worden. De lijnen en kleuren per land voegen we later toe.

```

1 ukbabynames %>%
2   filter(..., ..., ...) %>%
3   ggplot() +
4   labs(
5     title = "Number of baby girls named Fiona",
6     x = "Year",
7     y = "Number of babies",
8     color = "Nation", linetype = "Nation"
9   ) +
10  theme_bw() +
11  theme(
12    legend.position = c(0.8, 0.7),
13    legend.background = element_rect(color = "white")
14  )

```

7.140 Hint

Let op dat je de data eerst inperkt tot meisjes met de naam Fiona die in of na 2000 geboren zijn.

7.141 Code

```

#1. bekijken van de data
head(ukbabynames)

```

```

# A tibble: 6 x 6
  year sex  name      n rank nation
<dbl> <chr> <chr>  <dbl> <dbl> <chr>
1  1996 F    SOPHIE  7087     1 England & Wales
2  1996 F    CHLOE   6824     2 England & Wales
3  1996 F    JESSICA 6711     3 England & Wales
4  1996 F    EMILY   6415     4 England & Wales
5  1996 F    LAUREN  6299     5 England & Wales
6  1996 F    HANNAH  5916     6 England & Wales

```

```
glimpse(ukbabynames)
```

```
Rows: 565,817
```

```
Columns: 6
```

```
$ year   <dbl> 1996, 1996, 1996, 1996, 1996, 1996, 1996, 1996, 1996, 1996, 1996, 199~
$ sex    <chr> "F", "F", "F", "F", "F", "F", "F", "F", "F", "F", "F", "F", "F"~
$ name   <chr> "SOPHIE", "CHLOE", "JESSICA", "EMILY", "LAUREN", "HANNAH", "CHA~
$ n      <dbl> 7087, 6824, 6711, 6415, 6299, 5916, 5866, 5828, 5206, 4948, 464~
$ rank   <dbl> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, ~
$ nation <chr> "England & Wales", "England & Wales", "England & Wales", "Engla~
```

```
#2. aanpassen type naar factor
```

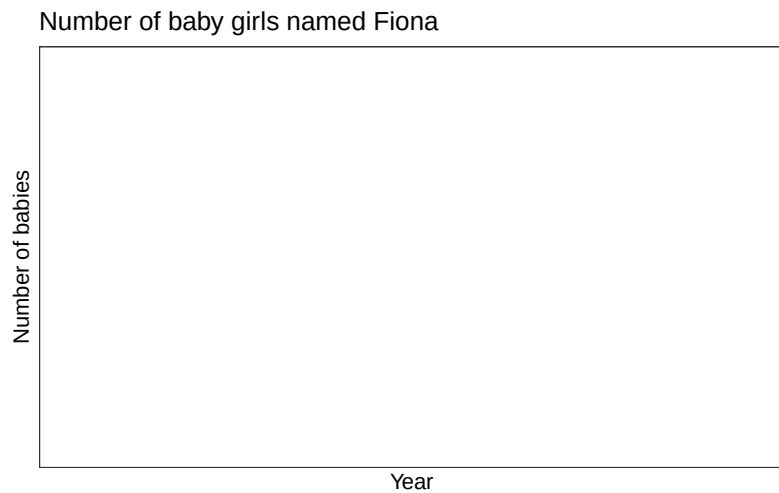
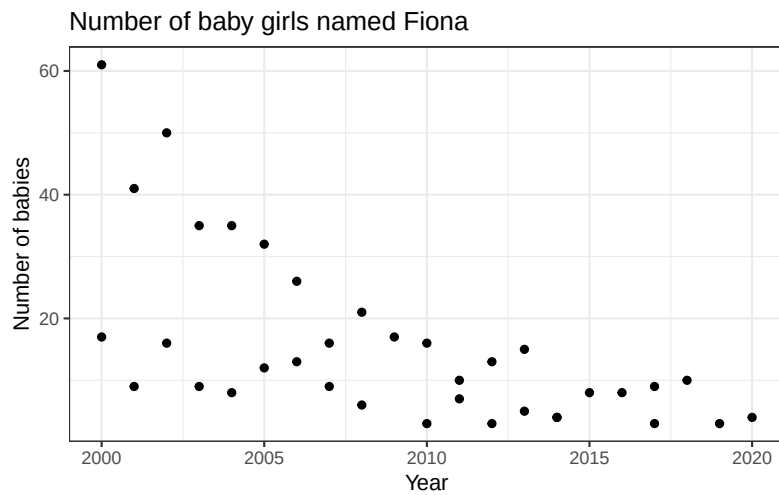
```
ukbabynames <- ukbabynames %>%
  mutate(across(where(is.character), as.factor))
```

```
#3. datapunten filteren en zichtbaar maken: toevoegen mapping en geom_point()
```

```
ukbabynames %>%
  filter(name == "Fiona", sex == "F", year >= 2000) %>%
  ggplot(aes(x = year, y = n)) +
    geom_point() +      #hier worden de punten toegevoegd
    labs(
      title = "Number of baby girls named Fiona",
      x = "Year",
      y = "Number of babies",
      color = "Nation", linetype = "Nation"
    ) +
    theme_bw() +
    theme(
      legend.position = c(0.8, 0.7),
      legend.background = element_rect(color = "white")
    )
```

Warning: A numeric `legend.position` argument in `theme()` was deprecated in ggplot2 3.5.0.

i Please use the `legend.position.inside` argument of `theme()` instead.



Figuur 7.22: een nog lege grafiek

Lijnen toevoegen per groep

7.142 Opdracht

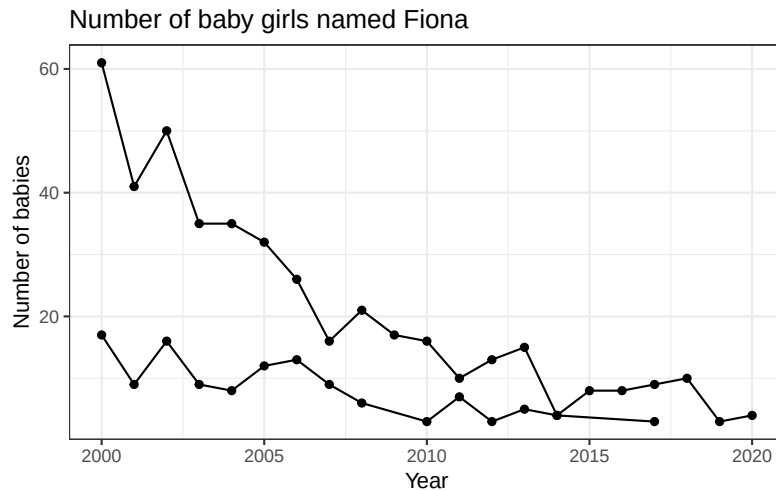
Voeg nu de lijnen toe. Zorg dat er aparte lijnen komen voor ieder land. Hiervoor gebruik je de aes-mapping `group =`.

7.143 Hint

Bedenk welk geom je hiervoor nodig hebt. Zie eventueel het [ggplot2 cheatsheet](#) voor een overzicht.

7.144 Code

```
#lijnen toevoegen via geom_line()
ukbabynames %>%
  filter(name == "Fiona", sex == "F", year >= 2000) %>%
  ggplot(aes(x = year, y = n, group = nation)) +
    geom_line() +                #hier wordt een lijn toegevoegd
    geom_point() +
    labs(
      title = "Number of baby girls named Fiona",
      x = "Year",
      y = "Number of babies",
      color = "Nation", linetype = "Nation"
    ) +
    theme_bw() +
    theme(
      legend.position = c(0.8, 0.7),
      legend.background = element_rect(color = "white")
    )
```



De kleur en lijnsoort aanpassen per groep

7.145 Opdracht

Zorg tot slot dat de datapunten en lijnen per land een kleur krijgen en dat het type lijn per land verschilt. Wat kun je op basis van deze grafiek concluderen?

7.146 Hint

Bedenk om welke *aesthetics* het gaat. Zie hiervoor het [ggplot2 cheatsheet](#).

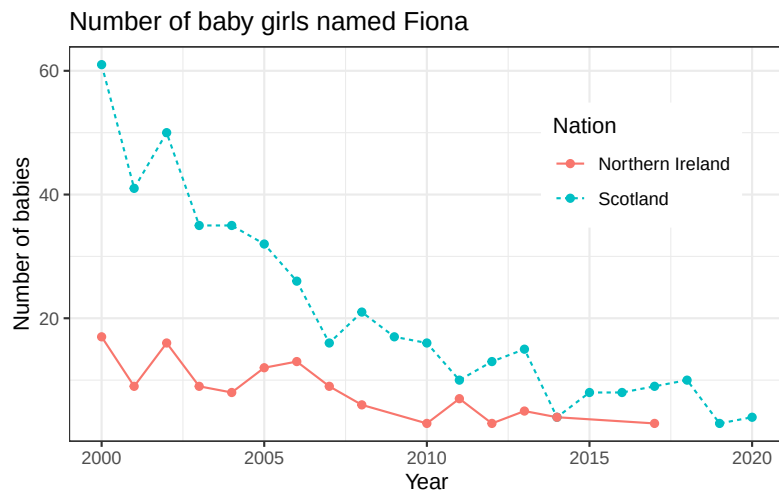
7.147 Code

```
1 #lijnen en punten een kleur geven, lijntype aanpassen
2 #het lijntype pas je aan met de aes-mapping 'linetype'
3 ukbabynames %>%
4   filter(name == "Fiona", sex == "F", year >= 2000) %>%
5   ggplot(aes(x = year, y = n,
```

```

6         group = nation,
7         colour = nation,
8         linetype = nation)) + #door toevoeging van colour en linetype is group = nation
9     geom_line() +
10    geom_point() +
11    labs(
12        title = "Number of baby girls named Fiona",
13        x = "Year",
14        y = "Number of babies",
15        color = "Nation", linetype = "Nation"
16    ) +
17    theme_bw() +
18    theme(
19        legend.position = c(0.8, 0.7),
20        legend.background = element_rect(color = "white")
21    )

```



*Wat kun je op basis van deze grafiek concluderen? Aan het begin van deze eeuw was de naam *Fiona* veel populairder in Schotland dan in Noord-Ierland. De afname is in Schotland veel sterker geweest.*

Code annoteren

7.148 Opdracht

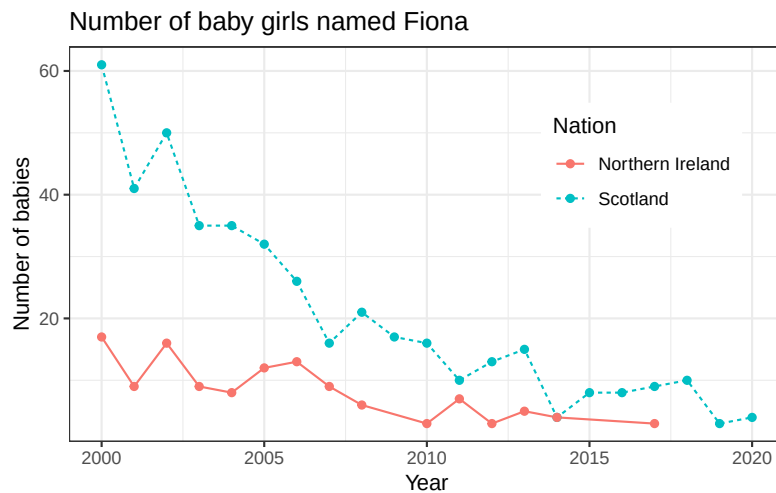
Bekijk de code in de uitwerking van de vorige opdracht en kopieer deze. Geef vervolgens in de code aan wat de bijdrage is van de code in regels 15, 19 en 20.

7.149 Hint

Schakel steeds een stukje code uit met `#` en kijk goed wat er verandert in hoe de grafiek eruit ziet.

7.150 Code

```
1  #lijnen en punten een kleur geven, lijntype aanpassen
2  #het lijntype pas je aan met de aes-mapping 'linetype'
3  ukbabynames %>%
4    filter(name == "Fiona", sex == "F", year >= 2000) %>%
5    ggplot(aes(x = year, y = n,
6              group = nation,
7              colour = nation,
8              linetype = nation)) +
9    geom_line() +
10   geom_point() +
11   labs(
12     title = "Number of baby girls named Fiona",
13     x = "Year",
14     y = "Number of babies",
15     color = "Nation", linetype = "Nation"           #zorgt dat er een legenda verschijnt
16   ) +
17   theme_bw() +
18   theme(
19     legend.position = c(0.8, 0.7),                  #bepaalt positionering van de legenda
20     legend.background = element_rect(color = "white") #zet achtergrondkleur legenda op wit
21   )
```



Meer data zichtbaar maken

7.151 Opdracht

Maak tot slot ook de datapunten van voor 2000 zichtbaar. Wat valt je op?

7.152 Hint

Kijk nog eens terug naar het begin van deze opdracht: hoe heb je toen bepaald welke datapunten zichtbaar werden?

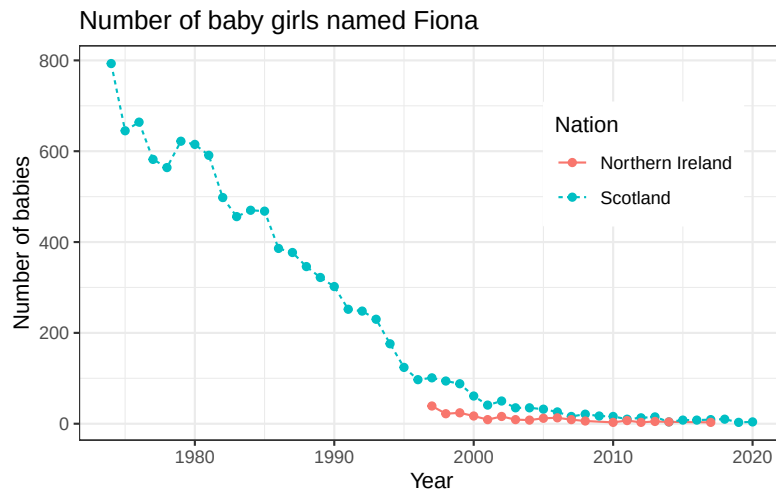
7.153 Code

```
#datapunten van voor 2000 zichtbaar maken via filter, verwijderen van 'year >= 2000'
ukbabynames %>%
  filter(name == "Fiona", sex == "F") %>%
  ggplot(aes(x = year, y = n, group = nation, colour = nation, linetype = nation)) +
  geom_line() +
  geom_point() +
```

```

labs(
  title = "Number of baby girls named Fiona",
  x = "Year",
  y = "Number of babies",
  color = "Nation", linetype = "Nation"
) +
theme_bw() +
theme(
  legend.position = c(0.8, 0.7),
  legend.background = element_rect(color = "white")
)

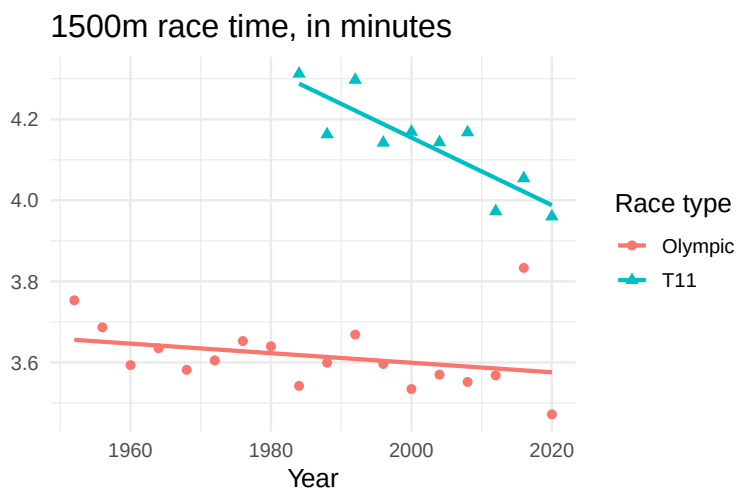
```



Wat valt op? Blijkbaar zijn er geen gegevens voor Noord-Ierland van voor 1997. Voor de Schotse data zien we dat de populariteit van de naam Fiona behoorlijk is teruggelopen sinds halverwege de jaren 70.

Opdracht 3: Nu helemaal zelf: tijden op de 1500m hardlopen

In deze opdracht ga je een variant van figuur 3.4 uit hoofdstuk 3 van IMS namaken. Je vindt de figuur in Figuur 7.23. Hierin zijn de winnende tijden (in minuten) weergegeven op de 1500 meter hardlopen op de Olympische Spelen en de Paralympische Spelen (divisie T11) voor mannen sinds 1950. Aan jou de opdracht om deze grafiek zo goed mogelijk na te maken. De benodigde data vind je in de datatabel `paralympic_1500` uit het package `openintro` (behorend bij het boek IMS). Dit package moet je geïnstalleerd hebben en aanroepen voor je de opdracht kunt maken.



Figuur 7.23: Variant van figuur 3.4 uit IMS.

7.154 Opdracht

Maak Figuur 7.23 na m.b.v. `ggplot2`.

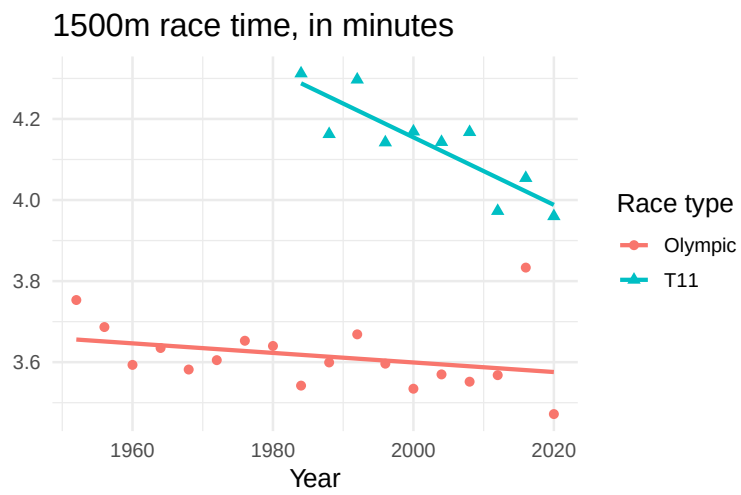
7.155 Hint

Alle stappen die je nodig hebt, heb je al een keer toegepast in dit practicum (of anders in vorige practica). Kijk dus nog eens terug. Zorg dat je voor jezelf helder hebt welke variabele via welke mapping zichtbaar is gemaakt in de grafiek. Bouw de grafiek in stappen op: eerst de datapunten, dan de trendlijn, dan de afwerking. Zorg ervoor dat je de dataset goed ingeperkt hebt.

7.156 Code

```
# zorg dat je het package openintro hebt geladen (bij voorkeur aan het begin van je script)
paralympic_1500 %>%
  filter(division == "Men",
         type == "Olympic" | type == "T11",
         year > 1950) %>%
  ggplot(aes(x = year, y = time_min,
            color = type,
            shape = type)) +
  geom_point(size = 2) +
  geom_smooth(method = "lm", se = FALSE) +
  labs(x = "Year",
       y = "", #alternatief: y = NULL
       color = "Race type",
       shape = "Race type",
       title = "1500m race time, in minutes") +
  theme_minimal(13)
```

```
`geom_smooth()` using formula = 'y ~ x'
```



Deel 2: My first Quarto

Tot nu toe hebben we R-code steeds in een R-script gemaakt. De output werd steeds zichtbaar in R Console of het Plots-window in RStudio. Hoewel dit een handige manier is om analyses uit te werken, is het wat minder handig om analyses te delen met anderen (mede-onderzoekers of je docent). Als je bijvoorbeeld de plaatjes die je hebt gemaakt wilt delen met iemand dan zul je die apart moeten opslaan of naar een ander bestand (bijv. Word) moeten kopiëren. Dit werkt op zich prima, maar het kan wat onhandig zijn wanneer je bijvoorbeeld nog kleine dingetjes wilt veranderen (bijv. de naam van de x-as toch aanpassen). In dat geval moet je het plaatje weer opnieuw kopiëren naar het andere bestand. Er is gelukkig een handiger alternatief en dat is het werken met een Quarto (of R Markdown) document. In zo'n document komt je output (bijv. een grafiek) direct onder je code te staan en kun je daarnaast ook nog gewone tekst typen, wanneer je bijvoorbeeld antwoord op een vraag wilt geven of een toelichting wilt schrijven bij wat je ziet in de data. Het document kun je vervolgens omzetten naar bijvoorbeeld een html-bestand of een pdf-bestand. De practicumopdracht die je nu leest is bijvoorbeeld op deze manier gemaakt. In het tweede deel van dit practicum ga je zelf proberen om zo'n Quarto-document te maken. We gebruiken hiervoor je uitwerkingen van Opdracht 2 van dit practicum over de naam *Fiona*.

Op Brightspace vind je een video met een korte inleiding over het werken met Quarto. Je werkt vervolgens je eigen Quarto-document uit. Download hiervoor het bestand `Practicum3.qmd` van Brightspace en zet het in de map van je R-project. Open het bestand en bewerk het. Doorloop hierbij de volgende stappen:

1. Verander de titel bovenaan het document (nu *My first Quarto*) en voeg je eigen naam toe (nu *Mijn naam*).

2. Maak een sectie *laden van benodigde packages* met daarin een blok met R-code waarin je de benodigde packages aanroept.
3. Maak een sectie *het maken van een grafiek* met daarin een blok met R-code waarin je je uiteindelijke code voor de uiteindelijke grafiek uit Opdracht 2 (met de extra data) toevoegt.
4. Zorg dat de grafiek in je output-document (het uiteindelijke `html`-bestand dat ontstaat nadat je op Render hebt geklikt) een beschrijving (*caption*) krijgt. Om te checken of deze code nog steeds werkt kun je je cursor in het blok met code plaatsen en **Ctrl+Enter** gebruiken net als in een gewoon script.
5. Voeg nu in gewone tekst je antwoord toe op de laatste interpretatievraag uit Opdracht 2 (Wat valt je op aan het patroon in de data?).
6. *Render* je bestand, zodat je een `html`-bestand krijgt. Open dit bestand in een browser.

Gelukt? Mooi! Vanaf nu maken we uitwerkingen niet meer in een script maar in een `Quarto`-bestand.

Je vindt een bestand `Practicum3_uitwerking.qmd` op Brightspace met een voorbeelduitwerking.

Dat was het voor deze keer. Volgende week gaan we ons nog verder verdiepen in het maken van grafieken in R.

Practicum 4

Grafieken en tabellen voor categorische data

Leerdoelen

- visualiseren van categorische variabelen (factoren) met tabellen en staafdiagrammen (barplots) in `ggplot2`;
- de levels van categorische variabelen (factoren) herordenen en van naam veranderen;
- naar wens aanpassen van de kleuren van je grafieken.

Vorbereiding

Lees hoofdstuk 6 uit het dictaat. De benodigde R-functies zijn te vinden in het dictaat (zie ook eerdere hoofdstukken en practica). Functies die niet in het dictaat besproken worden, worden gegeven in de opdracht. Het [ggplot2 cheatsheet](#) geeft een overzicht van de mogelijkheden die `ggplot2` biedt. Relevante *cheat sheets* vind je ook in een map op Brightspace (Content > Practica > R cheat sheets) en via de `help`-tab in RStudio.

Quarto

Vanaf deze week maak je de uitwerkingen in Quarto. Dit betekent dat je werkt in een `qmd`-bestand dat je in RStudio via de knop **Render** kunt omzetten naar een `html`-bestand.

Kopieer om te beginnen het bestand `voornaam_achternaam_PracticumX.qmd` uit de map `templates_scripts_quarto` in je Rproject naar de hoofdmap. Hernoem het bestand door je eigen naam in

te vullen en het juiste nummer van het practicum. Open het bestand vervolgens vanuit **RStudio**. Maak voor afzonderlijke taken steeds een nieuw codeblokje aan en test de werking door je **qmd**-bestand tussendoor te renderen.

Meer informatie over Quarto kun je vinden in [hoofdstuk 28](#) van Wickham et al. (2023) en op de [Quarto website](#). Zie ook deel 2 van Practicum 3.

Wickham, H., Çetinkaya-Rundel, M., & Grolemund, G. (2023). *R for Data Science* (2de dr.). O'Reilly. <https://r4ds.hadley.nz/>

Talen van de wereld

De Ethnologue is een database met alle talen die op dit moment gesproken worden in de wereld. In dit practicum ga je op basis van data uit de Ethnologue tellingen en grafieken maken van het aantal talen in verschillende delen van de wereld.

Let op

Sla je script regelmatig op, zodat je geen code kwijtraakt. Klik hiervoor op het floppy-disk-symbool of gebruik de sneltoetsen **Ctrl + S**/**Cmd + S**. Render je script regelmatig, als een extra check voor de goede werking van je code.

Probeer het eerst zelf

Vanaf deze week vind je geen Hint en Code meer bij de opdrachten. Het is nu aan jou om het eerst helemaal zelf te proberen. Maak hierbij gebruik van de code uit de eerdere practica. Een voorbeelduitwerking verschijnt een dag na het practicum op Brightspace.

Vorbereiding van de data

We gaan werken met twee databestanden die je kunt vinden op de website van de Ethnologue: <https://www.ethnologue.com/codes/>. Het gaat om de volgende databestanden:

1. **LanguageCodes.tab** Dit bestand bevat voor alle talen in de Ethnologue de volgende informatie: een identificeerder (**LangID**), code voor het land waar de taal (primair) wordt gesproken (**CountryID**), status van de taal (**LangStatus**, L = levend, X = uitgestorven), de naam van de taal (**Name**);
2. **CountryCodes.tab** Dit bestand geeft uitleg bij de codes voor landen (**CountryID**) uit het vorige bestand en bevat de volgende informatie: de code van het land (**CountryID**), de naam van het land (**Name**), en de regio waar het land in ligt (**Area**).

We gaan deze bestanden inlezen en ze samenvoegen tot een datatabel (dit heb je al een keer gedaan in Practicum 2).

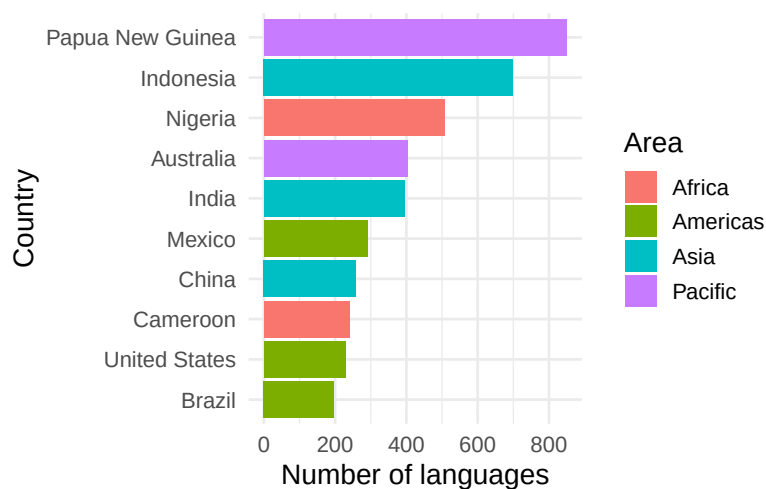
1. Download beide bestanden, voeg ze toe aan de map **datasets** in de map **OBS_practica** en lees ze vervolgens in in R. Dit zijn de urls voor beide bestanden: <https://www.ethnologue.com/codes/LanguageCodes.tab>, <https://www.ethnologue.com/codes/CountryCodes.tab>
2. Bekijk de variabelen in beide datatables, pas hun namen aan aan onze richtlijnen en pas indien nodig hun meetniveau/type aan. *Let op:* zorg dat de variabele **Name** uit het bestand **CountryCodes.tab** uiteindelijk de naam **country_name** krijgt. Dit om dubbele namen te voorkomen.
3. Voeg de twee datatables samen tot een datatabel met de naam **ethnologue**

4. Sla de nieuwe datatabel `ethnologue` op als een databestand met de naam `ethnologue.txt`.

Opdrachten

Beantwoord op basis van de datatabel `ethnologue` onderstaande vragen. Onderbouw elk antwoord met R-code. Probeer er bij de eerste 4 opdrachten voor te zorgen dat de output van je code het antwoord geeft op de vraag en dus zo min mogelijk informatie bevat die irrelevant is voor de beantwoording van de vraag.

1. Hoeveel talen worden er volgens Ethnologue gesproken in Nederland?
2. Om welke talen gaat het?
3. Hoeveel verschillende talen worden er gesproken in Europa?
4. In welk land in Azië worden de meeste talen gesproken?
5. In Figuur 7.24 zie je een grafiek met daarin de top 10 van landen in de wereld waar de meeste talen worden gesproken. Maak deze grafiek zo goed mogelijk na. Het kan hierbij helpen om de taak in stukjes op te delen: eerst de juiste data weergeven, dan in de juiste volgorde en tot slot alle afwerking (zoals kleuren en namen van assen).
6. Maak voor een regio (`area`) naar keuze een tabel waarin je voor elk land aangeeft wat het aandeel (proportie) is van dat land in het totaal aantal talen gesproken in die regio (afronden op 2 decimalen). Dus bijv. Brazilië is verantwoordelijk voor een proportie van 0.17 van de talen gesproken in de regio *Americas*.
7. Maak nu een liggend staafdiagram van de data uit de vorige opdracht. Zorg ervoor dat je landen met een proportie van 0 niet weergeeft. Geef de staaf van een land naar keuze een andere kleur dan de rest. Zorg dat je ergens in de grafiek ook het totaal aantal talen gesproken in de regio vermeldt net als de bron van de data.



Figuur 7.24: Top 10 landen van de wereld met de meeste talen

8. Bereken per regio het percentage talen met een status L en X (afronden op gehele getallen).
9. Maak een grafiek van de data uit de vorige vraag waarmee je laat zien hoe het percentage uitgestorven talen verschilt per werelddeel. Zorg voor een logische ordening van de data.
10. Pas de grafiek uit de vorige vraag aan, zodat deze opgenomen kan worden in een Nederlandstalig verslag. Zorg ervoor dat de status van de talen voluit wordt geschreven als 'levend' en 'uitgestorven'.

Dit is het einde van practicum 4.